



SACA 编程指南

v0.62

2021 年 04 月 12 日

目 录

目 录.....	II
图目录.....	IV
表目录.....	VI
第一章 简介	1
1.1 神威系统	1
1.2 SACA: 神威加速计算架构.....	2
1.3 文档结构	3
第二章 编程模型.....	4
2.1 异构加速编程.....	4
2.2 并行层次	5
2.3 存储层次	6
第三章 编程接口	10
3.1 SWGCC 编译器	10
3.2 语法扩展	15
3.3 SACA 运行时-ATHREAD.....	19
3.4 SIMD 向量编程	106
3.5 辅助工具	111
第四章 性能指导.....	117
4.1 循环展开优化.....	117
4.2 乘加融合优化.....	117
4.3 自动向量化.....	117
4.4 结构体重组优化.....	118
4.5 从核 CACHE 预取优化	118
4.6 从核 CACHE 写暗示优化	118
4.7 从核 CACHE 属性动态配置	119
4.8 从核代码链接时优化.....	121
4.9 从核代码分支预测优化.....	121
4.10 FORTRAN MODULE 访存优化	121
4.11 功耗相关优化.....	121
4.12 快速数学库.....	122
4.13 数学库函数细粒度多版本切换.....	122
4.14 SWPROF 热点函数分析.....	123
附录 A: SWGCC 编译器常用编译选项.....	125
A.1 功能选项.....	125
A.2 性能选项.....	136
A.3 调试选项.....	146

A.4 其它选项.....	148
附录 B: SW39000 众核处理器 SIMD 编程接口—C 语言	152
B.1 概述.....	152
B.2 快速入门.....	155
B.3 SIMD 对 C 语言的扩展	158
B.4 串行程序的向量化	174
B.5 C 内部函数接口	183
附录 C: SW39000 众核处理器 SIMD 编程接口—FORTRAN 语言	278
C.1 简要说明	278
C.2 快速入门	278
C.3 SIMD 对 FORTRAN 语言的扩展	280
C.4 FORTRAN 内部函数接口	286
附录 D: FORTRAN 语言支持的运行时 AThread 接口	338
修订记录.....	341

图目录

图 1	神威·太湖之光”超级计算机系统	1
图 2	SW26010 众核处理器.....	1
图 3	基于 SACA 的神威众核平台应用开发	2
图 4	SACA 异构加速编程	4
图 5	SACA 编程模型并行层次	5
图 6	SACA 编程模型存储层次	6
图 7	编译过程示意图	11
图 8	DMA 各模式示意图	52
图 9	向量块大小示意图	52
图 10	RMA 各模式示意图	66
图 11	向量数据与标量数据对比	106
图 12	ldmreport 使用例图	111
图 13	ldmreport 输出结果	112
图 14	ldmreport 报警信息	112
图 15	ldmreport 错误信息	112
图 16	checklen 工具使用示例	113
图 17	从核栈深度分析例图	113
图 18	mymapfile 中信息	114
图 19	version-spy 版本检测打印例图	116
图 20	从核 cache 属性动态配置编译指示字例图	120
图 21	swprof 热点函数统计例图	123
图 22	热点函数重排信息例图	124
图 23	不可 cache 属性的动态配置代码例图	146
图 24	标量与向量基本概念	152
图 25	主核 simd_loadu 和 simd_load_u 示意图	165
图 26	主核 simd_storeu 和 simd_store_u 示意图	165
图 27	主核 simd_storeul 和 simd_store_uh 示意图	165
图 28	intv8 向量存储位置例图	166
图 29	从核 simd_loadu 和 simd_load_u 示意图	167
图 30	从核 simd_storeu 和 simd_store_u 示意图	167
图 31	intv16 向量存储位置例图	168
图 32	intv8 的移位操作示意图	169
图 33	int256 的移位操作示意图	169
图 34	int 到 intv8 值扩展示意图	169
图 35	float 到 floatv4 值扩展示意图	169
图 36	long 到 int256 值扩展示意图	170
图 37	intv16 移位操作示意图	171
图 38	int512 移位操作示意图	171
图 39	变换前向量化示意图	180
图 40	变换后向量化示意图	180
图 41	数据跨 32 字节的不对界访存	181

图 42 数据跨 64 字节的不对界访存.....	181
---------------------------	-----

表目录

表 1	基础编译器使用说明.....	10
表 2	简要编译选项.....	12
表 3	编译器对界支持表.....	12
表 4	数据类型扩展.....	16
表 5	主存的空间编制和关键字.....	17
表 6	LDM 的空间编制和关键字.....	18
表 7	全归约支持的操作类型 optype.....	85
表 8	全归约支持的数据类型参数 dtype.....	85
表 9	主核分配主存接口.....	88
表 10	从核分配主存接口.....	89
表 11	静态特征参数含义.....	134
表 12	SIMD 主核扩展数据类型.....	158
表 13	SIMD 从核扩展数据类型.....	159
表 14	数组的向量化比较.....	178
表 15	函数体的向量化比较.....	178
表 16	主核装入/存函数接口	184
表 17	主核定点向量运算函数接口.....	189
表 18	主核浮点向量运算函数接口.....	203
表 19	主核数据整理函数接口.....	212
表 20	主核赋值函数接口.....	217
表 21	主核扩展类型打印函数接口.....	219
表 22	主核向量归约接口.....	223
表 23	从核装入/存储函数接口	225
表 24	从核定点向量运算函数接口.....	229
表 25	从核浮点向量运算函数接口.....	239
表 26	从核数据整理函数接口.....	257
表 27	从核赋值函数接口.....	267
表 28	从核扩展类型的打印函数接口.....	270
表 29	从核向量归约接口.....	275
表 30	Fortran 主核扩展数据类型.....	280
表 31	Fortran 从核扩展数据类型.....	280
表 32	Fortran 主核装入/存储函数接口.....	287
表 33	Fortran 主核浮点向量运算函数接口.....	290
表 34	Fortran 主核数据整理函数接口.....	302
表 35	Fortran 从核装入/存储函数接口.....	305
表 36	Fortran 浮点向量运算函数接口.....	308
表 37	Fortran 从核数据整理函数接口.....	330

第一章 简介

1.1 神威系统

近年来,“神威”系列超级计算机一直秉承自主创新的发展方向。2011 年推出的“神威蓝光”超级计算机成为国内首个全部采用国产处理器和系统软件构建的千万亿次计算机系统。2015 年,国家 863 计划支持的“神威·太湖之光”落户国家超算(无锡)中心,如图 1 所示。该系统是世界上首台峰值运行速度超过十亿亿次的超级计算机,也是我国第一台全部采用国产处理器构建的世界第一的超级计算机。2016 年 6 月-2017 年 11 月,在世界超级计算机排名榜单 TOP500 上,“神威·太湖之光”连续四次登顶。



图 1 神威·太湖之光”超级计算机系统

“神威·太湖之光”采用自主研发的 SW26010 申威众核处理器,其结构组成如图 2 所示。申威系列众核处理器均采用类似异构架构,由多个运算核组构成,每个核组由一个运算控制核心(简称主核)加一个运算核心(简称从核)阵列组成。申威系列众核处理器采用自主知识产权的 64 位指令系统,支持向量扩展指令集。

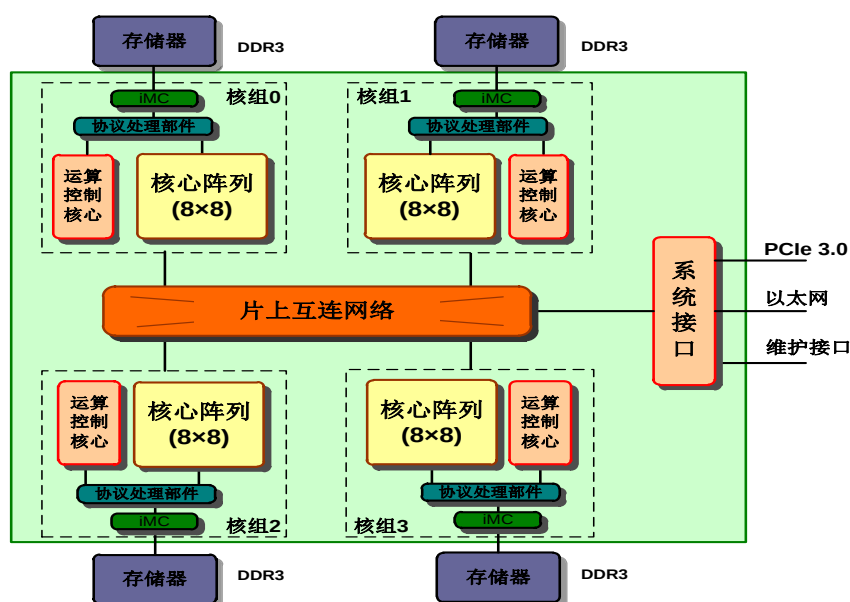


图 2 SW26010 众核处理器

“神威”系列超级计算机提供由操作系统、编译系统、并行语言、运行时系统、存储系统、应用支撑环境、人工智能软件等组成的完备软件栈，高效支撑大规模异构并行应用开发，涉及天气气候、航空航天、海洋环境、生物医药、船舶工程、材料科学、石油勘探、人工智能等众多应用领域，取得了包括戈登贝尔奖在内的一系列应用成果。

1.2 SACA：神威加速计算架构

随着神威平台的发展，其编程环境也不断丰富完善，但功能支撑较为分散，缺乏统一的众核基础编程指导。另一方面，人工智能等新兴应用领域不断涌现，其对众核基础编程的需求与传统科学计算领域不尽相同，需要统一的编程架构来支撑。在这样的背景下，神威平台于 2021 年 1 月正式推出神威加速计算架构：SACA（Sunway Accelerate Computing Architecture）。

SACA 包括面向申威众核架构的并行加速计算平台和编程模型，优化整合了基础编译、运行时系统、动态运行支撑中的编程接口，为用户提供统一的众核基础编程支撑，降低学习成本。SACA 兼容历史代码，进一步提高好用性，为科学计算、人工智能等领域应用提供统一的编程和优化支持。

如图 3 所示，SACA 提供的软件环境集成了众核基础编译器、基础库和运行时，扩展了标准语法。众核基础编译器支持 C、C++、Fortran 等高级编程语言，athread 运行时提供面向申威异构众核结构的高效管理，动态运行支撑为以人工智能为代表的典型应用提供底层服务支持。

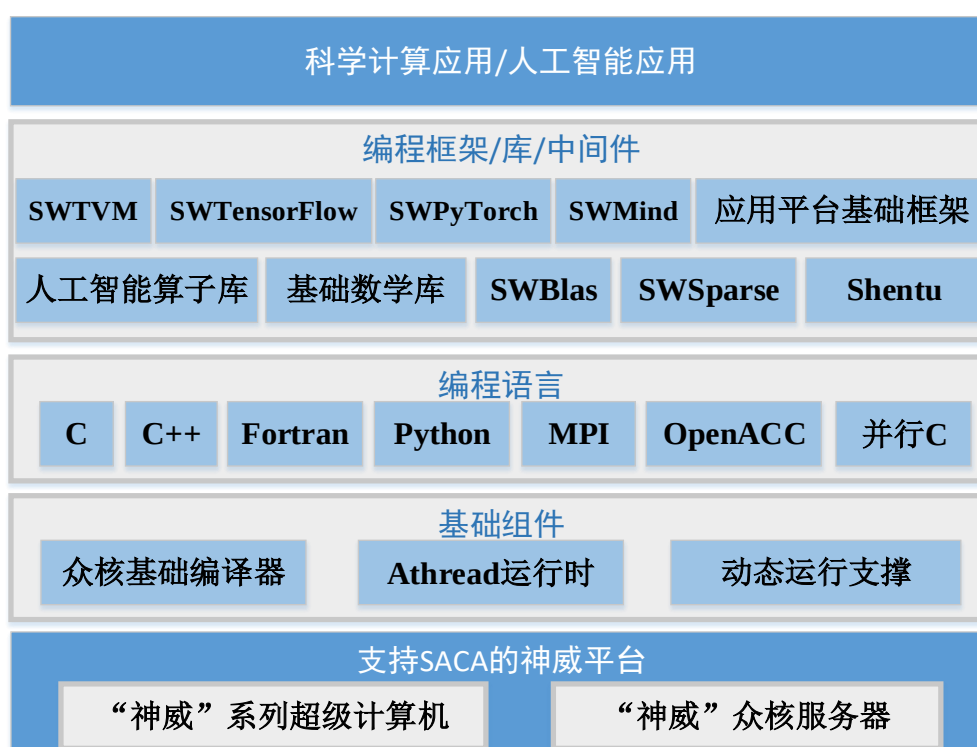


图 3 基于 SACA 的神威众核平台应用开发

1.3 文档结构

本文档主要介绍 SACA 编程和优化方法，为上层应用和软件开发提供神威平台众核基础编程方面的参考和帮助。

本文档的章节安排如下：

- 第一章简要介绍神威系统和 SACA 的基本情况。

- 第二章从异构加速编程、并行层次和存储层次等方面出发介绍 SACA 的编程模型。

- 第三章是本文档的重点，介绍了 SACA 的编程接口，包括国产众核基础编译器 swgcc、SACA 的语法扩展、SACA 运行时接口 athread、SIMD 向量编程接口和一些众核编程辅助工具。

- 第四章针对神威平台上的众核应用开发提供性能提升的指导。

考虑到正文部分的篇幅因素，本文档将部分内容的详细描述整理汇总在附录部分。

- 附录 A 介绍了 swgcc 编译器常用的编译选项。

- 附录 B 列出了 C 语言详细的 SIMD 编程接口。

- 附录 C 列出了 Fortran 语言详细的 SIMD 编程接口。

- 附录 D 列出了 Fortran 语言支持的 SACA 运行时 athread 接口。

注：在未特别注明的情况下，本文档所提及的 CPU 参数基于 SW39000 众核处理器。

第二章 编程模型

2.1 异构加速编程

为适应申威异构众核架构，SACA 采用异构加速编程模式。如图 4 所示，SACA 程序由主核代码和从核代码组成，主核代码运行在运算控制核心（主核）上，负责数据和加速任务的分配和管理；从核代码运行在运算核心（从核）上，实现对核心功能模块的加速。

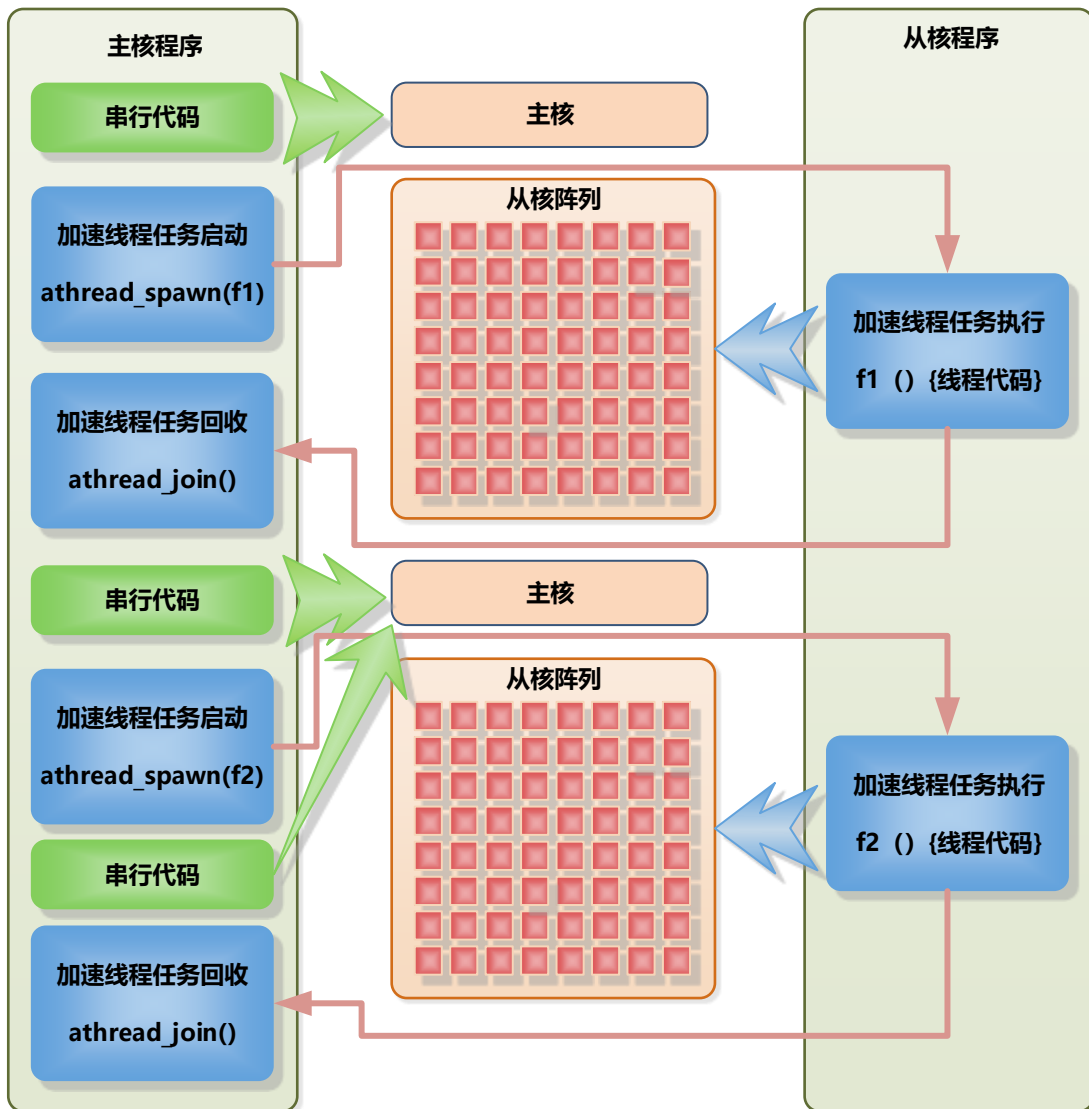


图 4 SACA 异构加速编程

主核程序采用 SACA 运行时 `athread` 的相关接口对加速线程任务进行管理操作，根据接口功能的不同，加速线程任务将会在一个或多个线程阵列上执行。线程任务启动后，主核可以继续运行其他串行代码；而调用加速线程任务回收接口后，就将等待从核阵列完成加速任务的执行。

2.2 并行层次

使用 SACA 编程时，其并行层次主要由任务并行、线程并行、SIMD 数据并行三个层次组成，如图 5 所示。

任务并行主要指核组间的进程级并行，即通过核组的主核上运行的进程来并行完成应用的计算任务。根据应用开发的并行编程框架提供的接口，可以使用 MPI 进程号或并行 C 进程号对不同的进程进行区分。

线程并行主要指通过每个从核上运行的线程来并行完成进程启动的加速任务，进程可以利用 `athread` 接口启动单个或多个核组（仅限于同一个芯片内的计算核组）的从核阵列。SACA 提供预定义变量或接口的形式来获取从核线程的线程号，在进程启动单核组运行的模式下，该线程号为 $0 \sim (N-1)$ 之间（ N 为单个运算核组从核阵列的从核数量）；在进程启动多核组运行的模式下（即大共享模式，见 3.3.4.5 节），该线程号为 $0 \sim (M-1)$ 之间（ M 为进程所启动的从核总量）。

SIMD 数据并行是指从核上运行的操作可以利用 SACA 的 SIMD 编程接口实现数据处理的向量化，以充分利用申威众核芯片的向量指令加速能力。

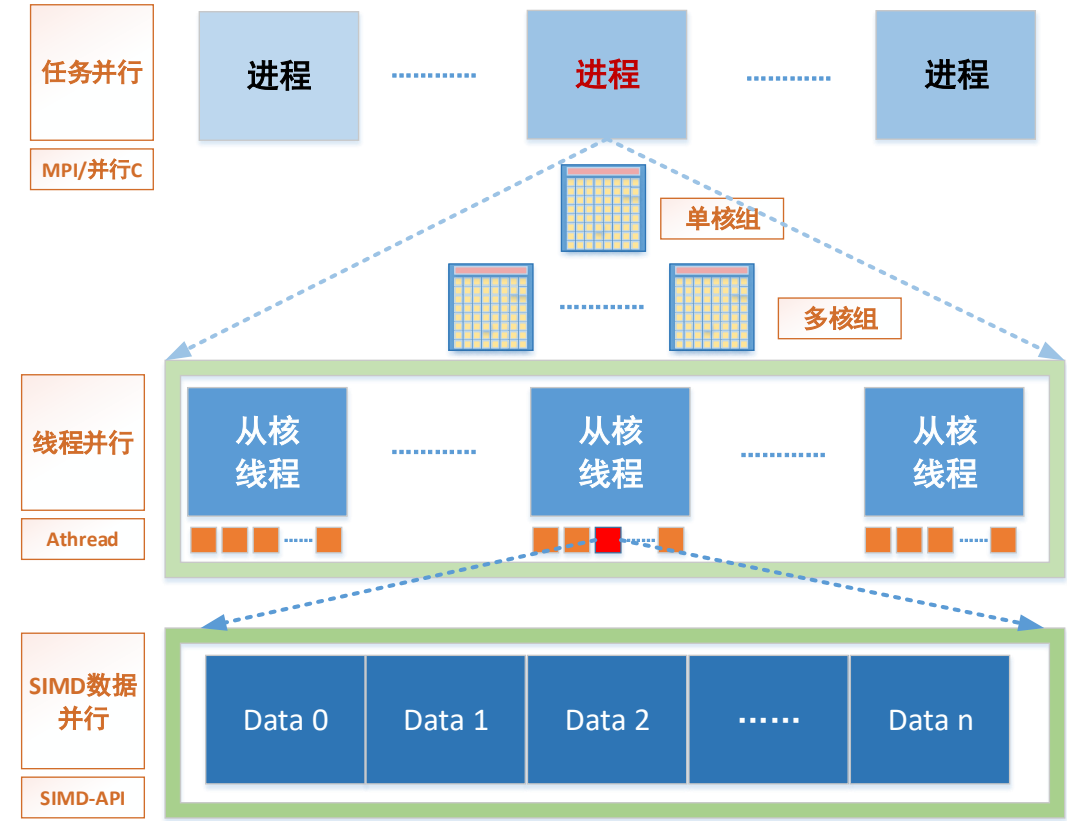


图 5 SACA 编程模型并行层次

2.3 存储层次

2.3.1 主从核存储层次

SACA 编程的存储层次如图 6 所示,主核存储层次包含寄存器、数据 cache、指令 cache、数据和指令共用的二级 cache、主存。从核存储层次包括寄存器、片上数据存储空间（可配置为软件管理的局部数据存储 LDM 或硬件管理的 L1 数据 cache）、指令 cache、主存。

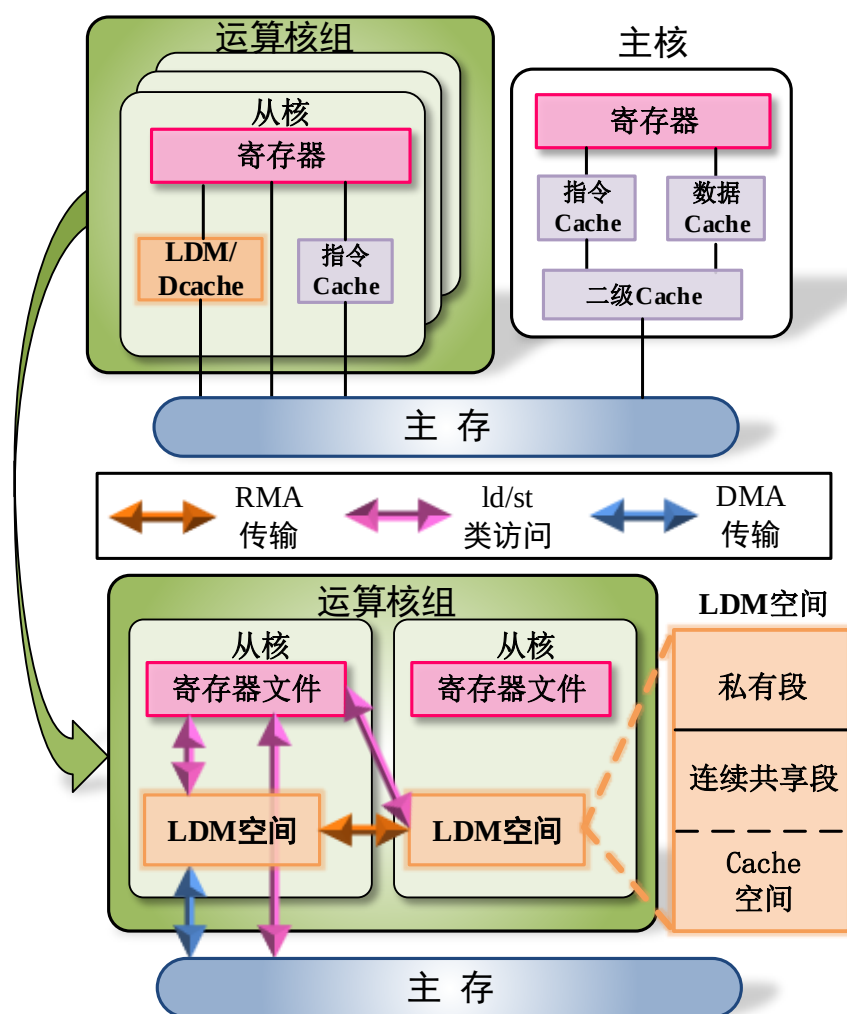


图 6 SACA 编程模型存储层次

主存采用虚地址,分连续段空间和交叉段空间。连续段是每个核组的私有空间,在核组内连续编址,主从核均可直接访问本核组的连续段空间。交叉段是同一芯片内核组间的共享空间,主从核均能够访问全芯片的该分布共享区域。主从核均支持对主存空间的可 cache 访问和不可 cache 访问。

每个从核具有一块高速的本地局部数据存储空间 LDM，其空间容量有限，属于处理器的珍稀资源。LDM 空间访存速度快，但要获得高性能，用户需要显式管理 LDM 的使用，在算法设计时就开始考虑怎么使用 LDM。LDM 空间主要分为私有空间和连续共享空间，LDM 私有空间是本从核快速访问的本地私有空间；LDM 连续共享空间是从核用于阵列内 LDM 共享访问的一种空间，从核可以访问到其他从核的 LDM 空间。在 LDM 连续共享模式下，阵列内每个从核提供相同容量的 LDM 进行连续编址，每个从核提供的用作共享的 LDM 容量分档可调。连续共享机制包含单簇（每个从核簇是一个 2*2 大小的从核阵列）、双簇（两个从核簇组成的 4*2 大小的阵列）、四簇（四个从核簇组成的 8*2 大小的阵列）、全阵列簇（全阵列共享，按从核簇连续编址）、全阵列行（全阵列共享，按行连续编址）等 5 种模式，便于对不同工作集合的就近访问。

从核可以发起 DMA 操作实现 LDM 与主存之间的数据传输，提高从核访问主存的效率。从核可以通过 RMA 操作实现本地 LDM 与其他从核 LDM 之间的数据传输，RMA 操作实现从核间的高效数据交换，可以缓解 LDM 容量受限的问题。

从核的片上数据存储空间在 SW39000 众核处理器中可以被部分配置成硬件管理的数据 cache，具体见 2.3.2 节说明。

2.3.2 存储空间默认大小

核组共享段和只读段默认都是 64M，私有段大小表示是每个启动的从核分配的大小，不指定的默认值为 2M，交叉段是全片 6 个 CG 共享的，不指定的默认值是 0M。

2.3.3 从核数据 cache 相关说明

SW39000 众核处理器提供了从核数据 cache，存储层次关键字、编程接口等方面的设计都与之相关。因此，本节对从核 cache 的相关结构和特征加以特别说明，用以辅助用户理解和使用。

2.3.2.1 从核数据 cache 结构

SW39000 众核处理器的每个从核具有一块高速的本地局部数据存储空间 LDM，LDM 总容量为 256KB。该空间可配置成完全由软件管理，也可将部分容量配置成硬件自动管理的数据 cache，两种管理方式的容量配比分档可调。配置成软件管理的局部存储时，从核可以使用访问 LDM 空间的 Load/Store 的方式加以访问，也可以使用发起异步 DMA 或 RMA 的方式实现 LDM 与外界的数据批量交换；当部分空间配置为一级数据 cache 时，从核可以通过访问主存可 cache 空间的 Load/Store 的方式对该空间进行 cache 访问，cache 行大小为 256 字节。

硬件也支持对主存空间的不可 cache 访问。

硬件支持 0KB、32KB 和 128KB 三档数据 cache 容量配置。用户可以通过作业管理选项或调用 SACA 的配置接口进行配置，若不设置，默认配置从核数据 cache 容量为 32KB。

为避免从核组数据 cache 访问共享主存的假共享问题，硬件支持带 Mask 的写机制，最小粒度为 4B。当一个 cache 行内的数据只有部分内容需要写回时，硬件采用原子读改写的主存访问方式进行处理，以部分访存带宽为代价，保证了在 4B 粒度下从核数据访问的正确性。

为提升从核 cache 的好用性和效率，硬件提供了淘汰、刷新、写暗示、配置私有段区间、预取等指令支持。

淘汰和刷新用于从核 cache 的管理，淘汰将指令确定的数据流虚地址对应的主存数据在本从核数据 cache 中内容淘汰，刷新对本从核数据 cache 中指定位置的 cache 行执行刷新回主存的操作。

写暗示指令提示包含寻址字节的对界 256 字节块的内容将不再有读访问，但不久将有写操作。从核将不读存储器以前的内容而直接分配并占有该 cache 行，cache 行的内容可设为任意值。写暗示避免从主存中载入无用的数据，提高了处理性能。

配置私有段区间指令将映射到该私有段区间内的 cache 行以整 cache 行方式写回，用于改善数据从 cache 写回主存的性能；预取指令将主存中对应地址的数据预取到从核 cache 中，隐藏 cache 访问延迟。

SACA 针对这些硬件指令提供了相应的功能接口或编译优化的支持。

2.3.2.2 cache 一致性

1、单边一致性

从核 cache 不保证与主核 cache 的一致性。主核访存过程中无法从从核 cache 中获得最新数据；从核淘汰或装填时，能够获取主核 cache 中的最新数据。

2、垂直一致性

从核淘汰或装填时，能够获取主核 cache 中的最新数据，但从核 cache 不保证与其它从核 cache 的一致性。从核访存过程中，始终无法从其他从核的 cache 中获得最新数据。

3、cache 和 LDM

从核 cache 不保证与软件管理的 LDM 空间的一致性。从核 DMA 过程中，无法获得从核数据 cache 中的最新数据。使用 DMA 时需要用户自行保证一致性，使用不可 cache 空间或使用刷新淘汰 cache 等操作。

4、屏蔽码机制

为避免由于从核 cache 不保证一致性带来的假共享问题，从核 cache 对共享主存空间的写回支持带屏蔽的细粒度写机制，屏蔽粒度为 4B，一个 cache 行中

只有确实被修改过的数据才会写入主存（读改写方式处理），如果整个 cache 行都被修改，则不使用读改写方式。

为了提升主存写回的效率，避免浪费带宽，在以下情况下硬件不使用带 Mask 的写机制：

1) 对于主存虚空间中属性为“从核私有”的写回操作，硬件采用整 cache 行写回主存的方式；

2) 对于主存虚空间中属性为“从核共享”的写回操作，软件可以将其中部分空间配置成私有段区间，对该区间内的可 cache 访问，硬件采用整 cache 行写回主存的方式；

从核私有空间和共享空间的私有段可以以任意数据粒度访问；而对共享空间，用户必须保证小于 4B 粒度数据访问的正确性。

5、硬件解决假共享

主核和主核之间、主核和从核之间、从核和从核之间，硬件均保证对共享主存的同一个 cache 行的假共享的数据一致性（访问粒度 4B 及以上）。

6、软件保证一致性的操作

对于“从核 cache-从核 cache”、“从核 cache-DMA 操作访问”或“从核 cache-主核 cache”之间存在真共享数据且最新副本在从核/主核 cache 中，或“从核 cache-从核 cache”之间存在独占 cache 行所有权限转移时，用户必须显式进行如下操作序列：

- 1) 对数据所有者的从核/主核的 cache 数据进行刷新；
- 2) 通过 memb 保证刷新完成；
- 3) 需要一致性的双方通过硬件同步或主存空间上的原子操作进行同步。

第三章 编程接口

3.1 swgcc 编译器

swgcc 编译器是基于 GCC 源码为申威众核处理器配备的底层基础编译器，是 SACA 编程环境的重要组成部分，是连接申威众核处理器与应用程序的桥梁，文中称为国产众核基础编译器。swgcc 支持国际标准的编程语言 C、C++ 和 Fortran，支持异构众核的 SIMD 扩展编程，提供循环优化、全局优化、过程间分析与优化等传统编译优化支持，并针对申威众核处理器的体系结构特点，进行高效能的编译优化，从而充分发挥处理器效率。

本章主要介绍 swgcc 编译器的基本使用方法。swgcc 遵守 Unix 和 Linux 编译器标准协议，生成的目标码遵守 Linux ABI 协议。编译生成的目标文件可以链接由其他 Linux 上国产众核基础编译器生成的目标文件。

3.1.1 使用说明

国产众核基础编译器支持 C、C++、Fortran 程序，并且通过选项区别支持主核和从核代码生成，如表 1 所示。编译器新增动态链接功能，用以支持人工智能类应用对动态编译及运行的需求，提升系统的通用性。

语言语法上，国产众核基础编译器支持 C11、C++2014、Fortran2003 标准语法，部分支持 Fortran 2008 标准语法。

表 1 基础编译器使用说明

语言	命令名	主核选项	从核选项	混合链接选项	动态链接选项
C	swgcc	-mhost	-mslave	-mhybrid	-mdynamic
C++	swg++				
Fortran	swgfortran				

备注：目前 C++ 编译器只支持主核代码。

3.1.2 编译示例说明

- 1、独立的主核代码 master.c
swgcc -mhost master.c -o master.out
- 2、独立的从核代码 slave.c
swgcc -mslave slave.c -o slave.out
- 3、主核代码 master.c、从核代码 slave.c，以 -mhybrid 模式混合静态链接
swgcc -mhost -c master.c
swgcc -mslave -c slave.c

swgcc -mhybrid master.o slave.o -o exe.out 生成主从混合代码 exe.out。

4、主核代码 master.c、动态库主核代码 lib_master.c、动态库从核代码 lib_slave.c，以-mdynamic 模式动态链接

swgcc -mhost -fPIC -c lib_master.c

swgcc -mslave -fPIC -c lib_slave.c

swgcc -mdynamic -shared lib_master.o lib_slave.o -o libmix.so 生成主从混合的动态库

swgcc -mhost -c master.c

swgcc -mdynamic master.o -L. -lmix -o mix.out 生成主从混合的动态目标码 mix.out

备注：由于动态库的 LDM 空间相互覆盖，以及从核短 pc24 位的限制，用户需要把代码链接成一个动态库，多个动态库可能会运行出错。用户代码分成多个库的，需要在编译时加-fPIC 选项生成位置无关的.o 文件，ar 压成.a 文件，将所有的.a 文件和其他.o 文件链接成一个动态库。

图 7 描述了 swgcc 编译的主要过程与机制。

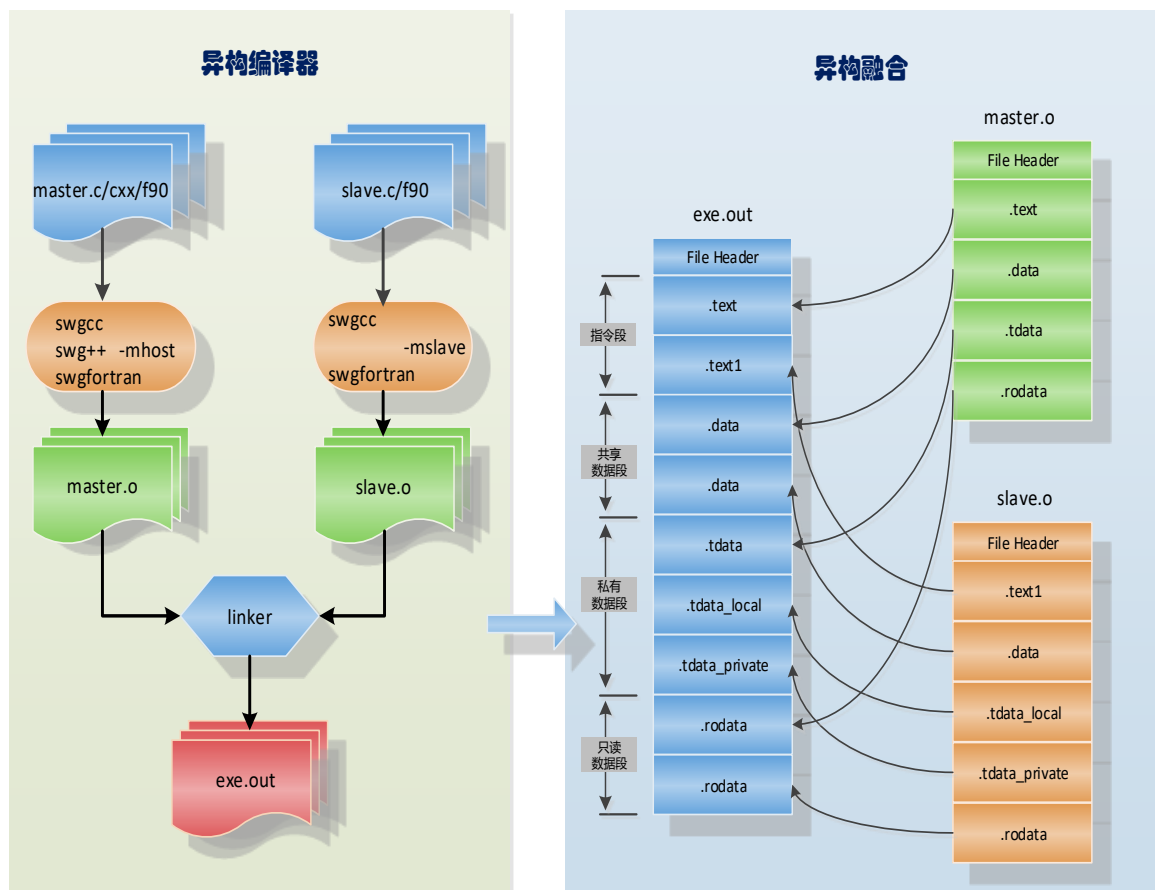


图 7 编译过程示意图

3.1.3 编译选项简要说明

表 2 简要编译选项

选项	作用
-S	生成汇编文件
-c	为每个源文件生成一个中间目标文件，但是不进行链接
-g	为之后的调试生成调试符号信息
-I<dir>	为预处理头文件添加查找<路径>
-l<library>	在链接阶段指定需要链接的库文件
-L<dir>	为链接阶段添加查找<路径>
-o<filename>	指定生成的可执行(库)文件的名字
-O3	生成更高级优化的可执行代码
-O0/-O1	生成低于默认优化的可执行代码（默认为-O2）

注：更多的可用选项以及说明请参考附录 A，或使用“man gcc”帮助文档。

3.1.4 编译器对界支持

3.1.4.1 支持的默认对界

- 1、使用-faddress_align=n 编译，则向量变量、数组、结构体、联合体等的首地址默认对界为 n 字节，其他普通全局变量按照定义变量类型对界；
- 2、不使用-faddress_align=n 编译，则按照定义变量类型对界；
- 3、主核函数初始栈指针保持 32 字节对界；
- 4、从核函数初始栈指针保持 64 字节对界。
- 5、3.1.4.2 节提供的用户自定义属性设置功能，其优先级高于本节的-faddress_align=n 选项。

表 3 编译器对界支持表

变量类型	对界（字节）
向量变量、数组、结构体、联合体首地址 （使用-faddress_align=n）	n
标量全局变量（使用-faddress_align=n）	按照定义类型对界
标量变量、数组、结构体、联合体首地址 （不使用-faddress_align=n）	按照定义类型对界
函数初始栈指针	主核 32，从核 64

3.1.4.2 用户自定义的对界

1、 C 语言对界属性设置

__attribute__ ((aligned(align_size)))

注意：align_size 为 2 的整数次幂，使用如上 attribute 可用于修饰全局变量或者函数内局部变量。

例如：

```
long tt[100] __attribute__ ((aligned(64)));  
int fun(){  
    struct xx xxt __attribute__ ((aligned(512)));  
    .....}
```

2、 Fortran 语言对界属性设置

目前支持对数组类型进行用户自定义对接属性设置，方法如下：

real(kind=8),dimension(16),align(align_size)::aa

或者

real(kind=8),align(align_size)::aa(16)

在数组声明时，增加关键字 align，align_size 应为 2 的整数次幂。

需要注意的是，对于如下的使用方式：

real(kind=8),align(32)::aa(16), b

编译器只保证数组 aa 的首地址为 32B 对界，对于普通变量 b 的地址不做对界调整。

另一方面，鉴于很多历史遗留的 Fortran 课题中使用了 COMMON 块来共享内存空间，编译器对 COMMON 块中包含设置了对界属性的数组的这种情况的对界作如下设置：

```
integer, align(128) :: aa(7), bb, cc(22), dd  
real, align(256) :: ee(13), ff, gg(24), hh  
COMMON / COMM_main1 / aa, cc, dd, ee, hh  
COMMON / COMM_main2 / bb, ff, gg
```

注：使用 fortran 对应的关键字，编译时需要选项 **-fopenmp**。

因为 COMMON 的本意就是其 common 块中的所有元素紧密排列，因此编译器将 COMMON 块的起始地址按照其中所有元素的最大对界值进行对界设置，而 COMMON 块中的各个元素按照起始地址紧密排列。

在上面的例子中，数组 aa, cc 要求 128B 对界，数组 ee, gg 要求 256B 对界，COMM_main1 中最大的对界值是数组 ee 的 256B 对界，因此分配的 COMM_main1 的起始地址是满足 256B 对界的，而因为 aa, cc, dd, ee, hh 紧密排列，可以计算出其中的数组 ee 的起始地址必定无法满足声明时的 256B 对界。同理 COMM_main2 的起始地址满足 256B 对界，数组 gg 的起始地址计算后只有 8B 对界。

3.1.5 编译器版本

如果编译时用选项-v，编译器则会给出自身的版本信息，例如：

```
swgcc -v
gcc version 7.1.0 20170502 (swgcc-1079 Committer: qianh
<qianh@example.com> by comp at localhost.localdomain on Thu Dec 24 21:36:33
CST 2020) (GCC)
```

3.2 语法扩展

3.2.1 编译器预定义宏和变量

3.2.1.1 预定义宏

swgcc 编译器内置了一些宏，灵活的使用这些预定义宏对用户编写异构程序是有用的：

- 1、以下宏在-mhost 选项下有效：

__sw_host__

- 2、以下宏在-mslave 选项下有效：

__sw_slave__

- 3、以下宏在-msimd 选项下有效：

_SIMD_HOST (-mhost 下)

_SIMD_SLAVE (-mslave 下)

- 4、以下宏与选项无关，均有效：

__TARGET_sw_64__

__sw_64

__sw_64__

用户可以使用-dM -E 这两个编译器选项组合来打印所有预定义宏。

3.2.1.2 从核默认局存变量

使用方法：#include<slave.h>

- 1、变量名称：**_PEN**

变量说明：从核逻辑号（0-63）。

- 2、变量名称：**_ROW**

变量说明：从核所在行号（0-7）。

- 3、变量名称：**_COL**

变量说明：从核所在列号（0-7）。

- 4、变量名称：**_CGN**

变量说明：从核所在核组号（0-5）。

- 5、变量名称：**_MYID**

变量说明：从核线程号，介于 0~(M-1)之间（M 为进程所启动的从核总量）。

- 6、变量名称：**_cache_size**

变量说明：从核当前所配置的 cache 大小（0、32、128）KB。

7、变量名称：_ldm_share_mode

变量说明：从核连续共享 LDM 模式（0/单簇、1/双簇、2/四簇、4/全阵列簇、5/全阵列行）。

8、变量名称：_ldm_share_size

变量说明：从核连续共享 LDM 大小（0、4、8、16、32、64、128）KB。

3.2.2 数据类型扩展

基于申威众核处理器的体系结构特点以及指令集，在标准的 C、C++ 和 Fortran 语法标准之外，swgcc 扩展了部分数据类型，主要是向量数据类型以及从核的半精度浮点数等。

从核的半精度浮点数据支持两种格式，即 IEEE-754 标准格式和 BF16 格式。IEEE-754 标准格式 16 位数据包含 1 位符号、5 位指数、10 位尾数，浮点数表示范围为 $6.103515625e-5 \sim 65504$ ；BF16 格式 16 位数据包含 1 位符号、8 位指数、7 位尾数，浮点数表示范围为 $1.17549435e-38 \sim 3.38953139e38$ 。编译器默认使用 IEEE-754 标准格式，编译及链接时加 -mslave-bf16 选项则使用 BF16 格式，在同一程序中建议只使用一种格式，否则会产生不可预知的问题。

表 4 数据类型扩展

	数据类型	关键字	备注
主核	定点整形向量	intv8	8 个 int 类型组成的向量
		uintv8	8 个 unsigned int 类型组成的向量
	定点长整形向量	int256	256 位长整形向量
		uint256	无符号 256 位长整形向量
	单精度浮点向量	floatv4	4 个单精度浮点类型组成的向量
		v4real(kind=4)[Fortran]	
从核	半精度浮点标量	_Float16/float16	若使用 float16，从核程序中需要 #include<slave.h> 默认使用 IEEE-754 标准格式，编译及链接时加 -mslave-bf16 选项则使用 BF16 格式。 打印半精度浮点数据需强制转换成 float 类型。 支持半精度浮点标量的加(+)、减(-)、乘(*)、除(/)、平方根(sqrth)运算。
		real(kind=2)[Fortran]	
	定点整形向量	intv16	16 个 int 类型组成的向量
		uintv16	16 个 unsigned int 类型组成的向量
	定点长整形向量	int512	512 位长整形向量
		uint512	无符号 512 位长整形向量
	单精度浮点向量	floatv8	8 个单精度浮点类型组成的向量

	双精度浮点向量	v8real(kind=4)[Fortran]	8 个双精度浮点类型组成的向量
		doublev8	
	半精度浮点向量	v8real(kind=8)[Fortran]	32 个 _Float16 类型组成的向量
		float16v32	
		v32real(kind=2)[Fortran]	

3.2.3 扩展关键字

3.2.3.1 针对主存的空间编制和关键字

表 5 主存的空间编制和关键字

主存虚空间	空间属性	可 cache	空间编址	使用实例
私有空间 （读写）	从核私有	是	0x4000,0000,0000	从核私有变量 C 语言： __thread int aa[100];
				Fortran： common /private_*/ aa !\$omp threadprivate (private_*/)
阵列共享 （只读）	代 码 段 和 rodata		0x4fff,f000,0000	从核指令
阵列共享 （读写）	数据段		0x5000,0000,0000	默认全局变量 int aa[100];
交叉段 （读写）	段名： .cross_r(初始 化变量) .cross_un(未 初始化变量)		0x5800,0000,0000	C 语言： __cross int aa;
		Fortran： common /cross_*/ aa !\$omp threadprivate (cross_*/)		
私有空间 （读写）	从核私（不可 cache 空间）	否	0x6000,0000,0000	从核私有不可 cache 变量 C 语言： __uncached __thread int aa;
				Fortran： common /uprivate_*/ aa !\$omp threadprivate (uprivate_*/)
阵列共享 （只读）	代 码 段 和 rodata		0x6fff,f000,0000	
阵列共享 （读写）	数据段		0x7000,0000,0000	不可 cache 全局变量 C 语言：

交叉段 (读写)	段名: .cross_r(初始化变量) .cross_un(未初始化变量)		0x7800,0000,0000	__uncached int aa;
				Fortran: common /uncached_*/ aa !\$omp threadprivate (uncached_*/)
				C 语言: __uncached __cross int aa;
				Fortran: common /ucross_*/ aa !\$omp threadprivate (ucross_*/)

备注：使用 fortran 对应的关键字，编译时需要选项 **-fopenmp**

3.2.3.2 针对 LDM 的空间编制和关键字

表 6 LDM 的空间编制和关键字

LDM 虚空间	空间属性	空间编址	对应段名	使用实例
私有 LDM	本从核 LDM 常驻空间	0x0 开始	.ldm_fix	C 语言: __thread_local_fix int aa;
				Fortran: 不支持
	本从核 LDM 空间	LDM 常驻空间之后	.ldm	C 语言: __thread_local int aa[100];
				Fortran: common /local_*/ aa
	本从核 LDM 空间，多个 kernel 间共享	LDM 常驻空间之后	.ldm_reuse_*	C 语言: 不带共享段名的方式（都共享同一片 LDM 空间）: __thread_kernel(kname) int aa[100]; 或带共享段名的方式（同一个共享段名修饰的共享同一片空间）: __thread_kernel(kname,reusename) int xx1 , aa1[100];
				Fortran: 不支持
连续 共享 LDM	本从核阵列 LDM 空间，空间大小由“64* 每	0x3000,0000,0000	.ldm_share	C 语言: __thread_local_share int aa[100];

	个从核共享空间容量”限制			Fortran: common /slocal_*/ aa
--	--------------	--	--	----------------------------------

备注：共享 ldm 空间要求 4 字节对齐访问，编译器-O3 下会把程序里面连续赋值的操作优化成调用 `memset` 函数，地址或长度不是 4 字节倍数时 `memset` 函数里会有字节访问，在这种情况下不希望优化成 `memset`，这时共享 ldm 数组声明时可以使用 `volatile` 修饰。

3.3 SACA 运行时-athread

`athread` 是针对 SACA 异构加速编程模型所设计的运行时支撑，其目的是为了用户能够方便、快捷地对核组内的线程进行控制和调度，从而更好地发挥核组内多从核执行加速性能。

在本编译系统面向的众核架构下，每个线程绑定一个从核资源，所以在使用 `athread` 的时候，需要考虑当前从核资源的分配状况。

3.3.1 头文件说明

使用所有主核接口需要包含如下头文件。

```
#include <athread.h>
```

使用所有从核接口需要包含如下头文件。

```
#include <slave.h>
```

3.3.2 初始化

按照环境变量配置初始化 `athread` 接口功能，接口调用完成后将初始化众核资源，后续所有 `athread` 接口均需要在该接口之后调用。

该节均为主核接口。

函数名：

```
int athread_init(void);
int athread_init_cgs(void);
```

函数说明：

资源描述信息通过环境变量传递进入，环境变量可由用户自行在当前 shell 环境中指定。如果使用作业管理系统提交作业，作业管理程序会将相关资源配置信息转换成环境变量的配置传递给提交的作业进程。

`athread_init()` 用于单进程启动单核组的模式中，可以初始化本核组的众核资源。

`athread_init_cgs()` 用于单进程启动多核组的模式中，可以初始化本芯片内 1/2/3/6 个核组的众核资源。`bsub` 提交作业，可以使用 `-cgsp 64 -mpecg 1` 或者 `-cgsp 64 -mpecg 2` 或 `-cgsp 64 -mpecg 3` 或者 `-cgsp 64 -mpecg 6` 来实现该设

置。

在-mdynamic/-mstatic 模式下，初始化接口会在用户程序开始之前默认调用，可以通过配置环境变量 `ATHREAD_INIT_LATER=1` 来取消默认调用。

参数说明：

无。

返回值：

成功返回 0，为确保成功，失败会导致进程退出。

3.3.3 资源配置信息获取

该小节提供各资源配置的信息获取功能，对应资源的设置由作业管理 `bsub` 的参数指定。接口仅支持-mdynamic/-mstatic 链接，使用-mhybrid 选项链接的程序无法使用这些接口函数。

该节均为主核接口。

3.3.3.1 打印当前资源情况

直接打印当前资源配置情况。

函数名：

`void athread_res_show(void);`

函数说明：

这个接口方便查看当前的资源配置情况，将当前资源配置汇总打印到 stdout。该接口无需使用 3.3.3.2 节中提到的 `athread_res_t` 数据结构。效果示意如下：

```
Athread resource: jobid = 2224575
Athread resource: penum = 128
Athread resource: pemask = 0xffffffffffffff 0xffffffffffffff 0 0 0
Athread resource: pri size = 8 MB
Athread resource: text size = 256 MB
Athread resource: data size = 2048 MB
Athread resource: cross size = 1024 MB
Athread resource: pestack_mode = Private Segment
Athread resource: pestack_size = 64 KB
Athread resource: ldm_shared_mode = 5 ( NONE )
Athread resource: ldm_shared_size = 0 (0KB)
Athread resource: dcache_mode = 0 (0KB)
Athread resource: slave mieee = 0
```

参数说明:

无。

返回值:

无。

3.3.3.2 获取当前资源情况

将当前任务相关资源情况获取到一个 `pthread_res_t` 数据结构中，在使用 `res` 操作接口中的各类 `get` 函数之前需要首先调用该函数。

函数名:

```
int pthread_res_getcurrent(pthread_res_t *res);
```

函数说明:

这个接口主要提供给第三方库使用。有些场景是用户在自己的程序里面执行了 `pthread_init`，初始化了资源。但是第三方的库希望获取到当前资源的状态信息，做一些检查，看是否满足自己的要求，或者做一些其他的配置。此时，可调用本接口。调用完成后会将资源信息放在用户提供一个 `res` 数据结构中，用户可以后面再调用 `res` 操作接口中的各类 `get` 函数得到具体的信息。

参数说明:

用户申请的一个 `pthread_res_t` 数据结构的指针，该数据结构可以未经初始化。

返回值:

成功返回 0。

3.3.3.3 作业号

函数名

```
int pthread_res_getjobid(const pthread_res_t *res, int *jobid);
```

函数说明

`pthread_res_getjobid` 获取 `res` 指向的资源描述数据结构中的 `jobid`，放置在 `jobid` 指针指向的一个缓冲中。

参数说明:

无。

返回值:

返回 0，失败返回非 0 的错误值如下:

EINVAL: 错误的参数。

3.3.3.4 从核数量

函数名:

```
int athread_res_getpenum(const athread_res_t *res, unsigned int *penum);
```

函数说明:

athread_res_getpenum 获取 res 指向的资源描述数据结构中的 penum, 放置在 penum 指针指向的一个缓冲中。

参数说明:

penum: 指示本进程启动的从核任务中配置的从核数量。

返回值:

成功返回 0, 失败返回非 0 的错误值。

EINVAL: 错误的参数。

3.3.3.5 从核位图

函数名:

```
int athread_res_getpemask(const athread_res_t *res, cpumask_t *pemask);
```

函数说明:

athread_res_getpemask 获取 res 指向的资源描述数据结构中的 pemask, 放置在 pemask 指针指向的一个缓冲中。

参数说明:

pemask: 指示从核任务中需要的从核位图, 类型为 cpumask_t 的指针, 具体使用方式参考示例。

返回值:

成功返回 0, 失败返回非 0 的错误值。

EINVAL: 错误的参数。

3.3.3.6 从核私有段空间大小

函数名:

```
int athread_res_getsegpri(const athread_res_t *res, int *segpri);
```

函数说明:

athread_res_getsegpri 获取 res 指向的资源描述数据结构中的 segpri, 放置在 segpri 指针指向的一个缓冲中。

参数说明:

segpri: 指示从核任务中需要的从核私有段大小, 单位为 MB。

返回值:

成功返回 0，失败返回非 0 的错误值。

EINVAL: 错误的参数。

3.3.3.7 主从核共享数据段空间大小

函数名:

`int athread_res_getsegdata(const athread_res_t *res, int *segdata);`

函数说明:

athread_res_getsegdata 获取 res 指向的资源描述数据结构中的 segdata，放置在 segdata 指针指向的一个缓冲中。

参数说明:

segdata: 指示从核任务中需要的从核共享可读写段大小，单位为 MB。

返回值:

成功返回 0，失败返回非 0 的错误值。

EINVAL: 错误的参数。

3.3.3.8 主从核共享指令段空间大小

函数名:

`int athread_res_getsegtext(const athread_res_t *res, int *segtext);`

函数说明:

athread_res_getsegtext 获取 res 指向的资源描述数据结构中的 segtext，放置在 segtext 指针指向的一个缓冲中。

参数说明:

segtext: 指示从核任务中需要的从核共享指令段大小，单位为 MB，该段中存放主从核指令、rodata、动态库。

返回值:

成功返回 0，失败返回非 0 的错误值。

EINVAL: 错误的参数。

3.3.3.9 主从核共享交叉段空间大小

函数名:

`int athread_res_getsegcross(const athread_res_t *res, int *segcross);`

函数说明:

`pthread_res_getsegcross` 获取 `res` 指向的资源描述数据结构中的 `segcross`，放置在 `segcross` 指针指向的一个缓冲中。

参数说明：

`segcross`：指示从核任务中需要的从核交叉段大小，单位为 MB。

返回值：

成功返回 0，失败返回非 0 的错误值。

EINVAL：错误的参数。

3.3.3.10 主核栈空间大小

函数名：

```
#include < sys/time.h >
#include < sys/resource.h >
int getrlimit(int resource, struct rlimit *rlim);
```

函数说明：

获取主核栈空间大小，放置在 `rlim` 指针指向的一个缓冲中，具体用法参考示例。

参数说明：

`resource`：应当设置成 `RLIMIT_STACK`。`rlimit` 原型如下：

```
struct rlimit {
    rlim_t rlim_cur; /* Soft limit */
    rlim_t rlim_max; /* Hard limit (ceiling for rlim_cur) */
};
```

返回值：

成功返回 0，失败返回-1，设置错误 `errno`。

3.3.3.11 从核 L1 dcache 模式

函数名：

```
int pthread_res_getdcache_mode(const pthread_res_t *attr, unsigned int
*dcache_mode);
```

函数说明：

从核支持 3 种 L1 dcache 模式，分别是 0KB，32KB，128KB。

默认的配置是 `SCORE_LDM_DCACHE_32KB`，即从核 dcache 为 32KB。
`pthread_res_getdcache_mode()` 获取 `attr` 中关于 dcache 模式的配置。

参数说明：

***attr:** 资源配置数据结构的指针。

dcache_mode: dcache 配置模式值，具体设置见函数说明。

***dcache_mode:** dcache 配置模式值的指针。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.3.12 从核 LDM 共享模式

函数名:

```
int athread_res_getldmshared_mode(const athread_res_t *attr, unsigned int  
*ldmshared_mode);
```

函数说明:

关于 LDM 共享模式的配置。具体的有:

```
enum score_ldm_shared_mode {  
    SCORE_LDM_SHARED_SINGLE,  
    SCORE_LDM_SHARED_DOUBLE,  
    SCORE_LDM_SHARED_QUAD,  
    SCORE_LDM_SHARED_ALL_CLUSTER,  
    SCORE_LDM_SHARED_ALL_ROW,  
    SCORE_LDM_SHARED_NONE,  
};
```

默认配置为 SCORE_LDM_SHARED_NONE，即不共享。

athread_res_getldmshared_mode 接口获取 attr 中关于 LDM 共享模式的配置。

参数说明:

***attr:** 资源配置数据结构的指针。

ldmshared_mode: LDM 共享模式的配置，配置值见函数说明。

***ldmshared_mode:** LDM 共享模式的配置值的指针。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.3.13 从核 LDM 共享大小

函数名:

```
int athread_res_getldmshared_size(const athread_res_t *attr, unsigned int *ldmshared_size);
```

函数说明:

关于 LDM 共享大小的配置。具体的有:

```
enum score_ldm_shared_size {  
    SCORE_LDM_SHARED_0KB,  
    SCORE_LDM_SHARED_4KB,  
    SCORE_LDM_SHARED_8KB,  
    SCORE_LDM_SHARED_16KB,  
    SCORE_LDM_SHARED_32KB,  
    SCORE_LDM_SHARED_64KB,  
    SCORE_LDM_SHARED_128KB,  
};
```

默认的设置是 SCORE_LDM_SHARED_0KB, 即共享大小为 0KB。
athread_res_getldmshared_size 接口获取 attr 中关于 LDM 共享大小的配置。

参数说明:

*attr: 资源配置数据结构的指针。

ldmshared_size: LDM 共享大小的配置, 配置值见函数说明。

*ldmshared_size: LDM 共享大小的配置值的指针。

返回值:

成功返回 0, 失败返回非 0 的错误编码。

3.3.3.14 从核初始 FPCR MIEEE 配置

函数名:

```
int athread_res_getmieee(const athread_res_t *attr, unsigned int *mode);
```

函数说明:

获取从核 FPCR 模式配置。设置为 1 表示将 fcpr 最后两位设为 00, 允许所有浮点算术异常自陷, 相当于编译器使用-mieee-with-inexact 选项, 设置为 2 时, 将浮点控制状态寄存器 fpcr 的最后两位设为 01, 允许除不精确结果之外的所有浮点算术异常自陷, 相当于编译器使用-mieee 选项。

参数说明:

*attr: 资源配置数据结构的指针。

*mode: 存放配置值的指针。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.3.15 从核初始 FPCR ROUNDING 配置

函数名:

`int athread_res_getrounding(const athread_res_t *attr, unsigned int *mode);`

函数说明:

获取从核 FPCR 动态舍入位设置，有效值只有 0、1、2、3 四个值。

0: 向“0”舍入

1: 向负无穷大舍入

2: 就近舍入

3: 向正无穷大舍入

参数说明:

*attr: 资源配置数据结构的指针。

*mode: 存放配置值的指针。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.3.16 示例

```
#include <athread.h>
#include <sys/time.h>
#include <sys/resource.h>
```

```
static const char ldm_shared_mode_str[6][15] = {
    [SCORE_LDM_SHARED_SINGLE] = "SINGLE",
    [SCORE_LDM_SHARED_DOUBLE] = "DOUBLE",
    [SCORE_LDM_SHARED_QUAD] = "QUAD",
    [SCORE_LDM_SHARED_ALL_CLUSTER] = "ALL_CLUSTER",
    [SCORE_LDM_SHARED_ALL_ROW] = "ALL_ROW",
    [SCORE_LDM_SHARED_NONE] = "NONE",
};
```

```
static const char ldm_shared_size_str[7][7] = {
    [SCORE_LDM_SHARED_0KB] = "0KB",
    [SCORE_LDM_SHARED_4KB] = "4KB",
    [SCORE_LDM_SHARED_8KB] = "8KB",
    [SCORE_LDM_SHARED_16KB] = "16KB",
};
```

```

[SCORE_LDM_SHARED_32KB] = "32KB",
[SCORE_LDM_SHARED_64KB] = "64KB",
[SCORE_LDM_SHARED_128KB] = "128KB",
};

static const char ldm_dcache_mode_str[3][5] = {
[SCORE_LDM_DCACHE_0KB] = "0KB",
[SCORE_LDM_DCACHE_32KB] = "32KB",
[SCORE_LDM_DCACHE_128KB] = "128KB",
};

int main(){
    int jobid,segpri,segdata,segtext,segcross;
    unsigned int penum,dcache_mode,ldmshared_mode,ldmshared_size,mode;
    cpumask_t pmask;
    athread_res_t res;
    struct rlimit rlim;

    athread_init();

    athread_res_show();

    athread_res_getcurrent(&res);
    athread_res_getjobid(&res,&jobid);
    printf("jobid is %d\n",jobid);
    athread_res_getpenum(&res,&penum);
    printf("penum is %d\n",penum);
    athread_res_getpmask(&res,&pmask);
    printf("pmask is %#llx,%#llx,%#llx,%#llx,%#llx,%#llx\n",

penum.bits[0],penum.bits[1],penum.bits[2],penum.bits[3],penum.bits[4],penum.bits[5]);
    athread_res_getsegpri(&res,&segpri);
    printf("segpri is %dMB\n",segpri);
    athread_res_getsegdata(&res,&segdata);
    printf("segdata is %dMB\n",segdata);
    athread_res_getsegtext(&res,&segtext);
    printf("segtext is %dMB\n",segtext);
    athread_res_getsegcross(&res,&segcross);
    printf("segcross is %dMB\n",segcross);
    getrlimit(RLIMIT_STACK,&rlim);
    printf("rlim_cur is %dMB\n",rlim.rlim_cur/1024/1024);
    athread_res_getdcache_mode(&res,&dcache_mode);
    printf("dcache_mode is %s\n",ldm_dcache_mode_str[dcache_mode]);
    athread_res_getldmshared_mode(&res,&ldmshared_mode);

```

```

printf("ldmshared_mode is %s\n",ldm_shared_mode_str[ldmshared_mode]);
athread_res_getldmshared_size(&res,&ldmshared_size);
printf("ldmshared_size is %s\n",ldm_shared_size_str[ldmshared_size]);
athread_res_getmieeee(&res,&mode);
printf("mieeee mode is %d\n",mode);
athread_res_getrounding(&res,&mode);
printf("rounding mode is %d\n",mode);

return 0;
}

```

附加说明：该示例仅支持-mdynamic 与-mstatic 模式。

3.3.4 线程管理

该节均为主核接口。

3.3.4.1 创建/等待从核线程组

函数名：

```

int athread_spawn (start_routine *func, void *arg);
int athread_spawn_noflush (start_routine *func, void *arg);
int athread_join (void);

```

函数说明：

athread_spawn 为本进程创建一组新的从核核心段在从核中运行。使用的从核数量为当前资源属性中申请到的所有从核数量。从核核心段函数运行结束后默认刷新从核 cache。

athread_spawn_noflush 与 athread_spawn 不同的是从核核心段函数运行结束后不刷新从核 cache。如果未配置从核 cache 则两个函数效果相同。athread_join 显式阻塞等待该线程组，直到当前的线程组终止。指定的线程组必须位于当前的进程中。

参数说明：

*func: 从核函数指针

*arg: 从核函数参数指针

返回值：

成功返回 0，失败返回错误编码。

ENODEV: 未执行 athread_init，或调用相关内核接口失败。

EBUSY: 初始化的从核资源中，某些从核正在执行未结束的核心段函数。

ENOMEM: 在初始化 tls 数据时遇到 malloc 失败没内存的情况。

3.3.4.2 按核组位图创建/等待从核线程组

函数名:

```
int athread_spawn_cgmask (int cgmask, start_routine *func, void *arg);  
int athread_spawn_cgmask_noflush (int cgmask, start_routine *func, void  
*arg);  
int athread_join_cgmask (int cgmask);
```

函数说明:

athread_spawn_cgmask 创建一组新的从核核心段在从核中运行。采用默认的资源配置，根据进程的环境变量进行属性配置。

使用的从核为指定的核组位图。需要注意的是，这里的核组位图是物理位图。比如在作业管理模式提交作业使用了-mpecg 接口，本进程申请到了超过 1 个核组的从核资源，如 3 个核组，但这 3 个核组可能是 0-2 核组，也可能是 3-5 核组。在使用该接口前，用户需要获取当前进程能够使用的核组位图。

获取当前可以使用的核组位图可以通过 athread_res_getcurrent 配合 athread_res_getpemask 完成。cgmask 可以由 pemask 计算得到。

需要注意的是，bsub 的 cgsp 和 mpecg 参数配合使用时，可申请 cgsp*mpecg 的从核资源，比如一个阵列 32 个从核，申请 3 个阵列。该接口可以按阵列去 spawn 核心段函数，每个阵列使用的从核数量为 cgsp 的数量。

athread_spawn_cgmask 默认在从核核心线程结束后刷新 cache，athread_spawn_cgmask_noflush 不刷新 cache。athread_join_cgmask 可以按核组位图阻塞等待核组核心段完成。

参数说明:

cgmask: 核组位图，比如为 1 表示 0 核组，为 2 表示 1 核组，为 3f 表示全片所有核组。

*func: 从核函数指针。

*arg: 从核函数参数指针。

返回值:

成功返回 0，失败返回错误编码。

ENODEV: 未执行 athread_init，或调用相关内核接口失败。

EBUSY: 初始化的从核资源中，某些从核正在执行未结束的核心段函数。

ENOMEM: 在初始化 tls 数据时遇到 malloc 失败没内存的情况。

3.3.4.3 进入/退出满阵列快速线程组模式

函数名:

```
int athread_enter64 (void); int athread_leave64 (void);  
int athread_enter64_arg (void); int athread_leave64_arg (void);
```

函数说明:

快速线程组模式采用 LDM 广播方式设置从核函数启动 PC，采用阵列同步机制同步退出函数，比常规模式更加高效，但要求 athread 资源必须为满阵列的倍数。

athread_enter64 使申请到的所有阵列都进入快速线程组模式，对应 athread_spawn64 和 athread_join64，对应无参数 spawn64 模式。

athread_enter64_arg 使申请到的所有阵列都进入快速线程组模式，对应 athread_spawn64_arg 和 athread_join64_arg，对应带参数的 spawn64 模式。athread_leave64 退出快速线程组模式，配合 athread_enter64 使用。athread_leave64_arg 退出快速线程组模式，配合 athread_enter64_arg 使用。

退出快速线程组模式时需要先保证从核没有在运行新的 athread 线程函数。

参数说明:

无

返回值:

成功返回 0。

3.3.4.4 创建/等待满阵列快速模式线程组

函数名:

```
int athread_spawn64 (start_routine *func);  
int athread_join64 (void);  
int athread_spawn64_arg (start_routine *func, void *arg);  
int athread_join64_arg (void);
```

函数说明:

快速线程组模式采用 LDM 广播方式设置从核函数启动 PC，采用阵列同步机制同步退出函数，比常规模式更加高效，但要求 athread 资源必须为满阵列的倍数。athread_spawn64 创建一组新的从核核心段在从核中运行。采用默认的资源配置，根据进程的环境变量进行属性配置。使用的从核数量为当前资源属性中申请到的所有从核数量。athread_join64 显式阻塞等待该线程组，直到当前的线程组终止。指定的线程组必须位于当前的进程中。

快速模式接口不会刷新从核 cache，需要用户自行保证 cache 正确性。

参数说明:

*func: 从核核心段函数指针

*arg: 从核核心段函数参数指针

返回值:

成功返回 0，失败返回错误编码。

ENODEV: 未执行 `athread_init`，或调用相关内核接口失败。

EBUSY: 初始化的从核资源中，某些从核正在执行未结束的核心段函数。

ENOMEM: 在初始化 tls 数据时遇到 `malloc` 失败没内存的情况。

3.3.4.5 创建/等待大共享模式从核线程组

函数名:

```
int athread_spawn_cgs (void* func , void * arg);  
int athread_spawn_cgs_noflush (void* func , void * arg);  
int athread_join_cgs (void);
```

函数说明:

在当前进程中添加新的受控线程组。`athread_spawn_cgs` 会根据 `bsub` 参数自动选择启动 1/2/3/6 核组，线程执行的任务由函数 `fpc` 指定；函数 `fpc` 的参数由 `arg` 提供。

`bsub` 时搭配 `-mpecg 1/2/3/6` 使用。

参数说明:

*func: 从核函数指针。

*arg: 从核函数参数指针。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.4.6 设置/获取并行区线程总数

函数名:

```
int athread_set_num_threads (int num); int athread_get_num_threads (void);
```

函数说明:

用该接口设置或获取当前进程后续并行区启动的所有核组线程总数。

用在 `athread_spawn` 接口前可以影响后续的所有 `spawn` 启动的线程总数。

参数说明:

num: 线程总数。取值范围为 1 到最大总线程数量。

返回值:

成功返回 0，失败返回非 0 的错误编码。

-EINVAL: 并行区线程数设置失败，比如上个并行区任务未完成；设置线程数目超过了可用线程资源。

3.3.4.7 绑定主核线程到指定核组

函数名:

`int SET_TO_SWCG (int cgid);`

函数说明:

依靠 pthread_setaffinity_np 将线程函数绑定到指定 CG 上运行。

参数说明:

int cgid: CG 号，合法值为 0-5。

返回值:

成功返回 0，失败返回非 0 的错误编码。

3.3.4.8 示例

3.3.4.8.1 创建线程组

```
[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    athread_init();
    athread_spawn(fun,0);
    athread_join();
    athread_halt();
    for(int j=0;j<64;j++)
        if(flag [j] !=123 +j) printf("%d err\n",j);
    printf("All done\n");
    return 0;
}

[slave.c]
#include<slave.h>
extern int flag [64];
```

```

__thread int xx=123;
int fun(){
    xx = xx + _PEN;
    flag [_PEN] = xx;
}

```

附加说明：bsub 时需要-cgsp 64。

3.3.4.8.2 按核组位图创建线程组

```

[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag [6][64];
int main(){
    for(int i=0;i<6;i++){
        for(int j=0;j<64;j++){
            flag [i][j] = 0;
        }
    }
    athread_init();
    athread_spawn_cgmask(0x3f,fun,0);
    athread_join_cgmask(0x3f);
    athread_halt();
    for(int i=0;i<6;i++){
        for(int j=0;j<64;j++){
            if(flag [i][j] !=123 +j) printf("%d err\n",j);
        }
    }
    printf("All done\n");
    return 0;
}

[slave.c]
#include<slave.h>
extern int flag [6][64];
__thread int xx=123;
int fun(){
    xx = xx + _PEN;
    flag [_CGN][_PEN] = xx;
}

```

附加说明：该示例仅支持-mdynamic 与-mstatic 模式，bsub 时需要-cgsp 64 –mpecg 6。

3.3.4.8.3 创建快速线程组

```
[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    athread_init();
    athread_enter64();
    athread_spawn64(fun);
    athread_join64();
    athread_leave64();
    athread_halt();
    for(int j=0;j<64;j++)
        if(flag [j] !=123 +j) printf("%d err\n",j);
    printf("All done\n");
    return 0;
}

[slave.c]
#include<slave.h>
extern __uncached int flag [64];
__thread int xx=123;
int fun(){
    xx = xx +_PEN;
    flag [_PEN] = xx;
}
```

3.3.4.8.4 创建带参数快速线程组

```
[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    int arg=123;
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    athread_init();
    athread_enter64_arg();
    athread_spawn64_arg(fun,&arg);
    athread_join64_arg();
    athread_leave64_arg();
    athread_halt();
}
```

```

        for(int j=0;j<64;j++)
            if(flag [j] !=123 +j) printf("%d err\n",j);
        printf("All done\n");
        return 0;
    }
[slave.c]
#include<slave.h>
extern __uncached int flag [64];
int fun(int *arg){
    int xx=*arg;
    xx = xx + _PEN;
    flag [_PEN] = xx;
}

```

3.3.4.8.5 创建大共享模式从核线程组

- 1) 1 主核+384 从核

```

[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag[6][64];
int main(){
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            flag[i][j] = 0;
    athread_init_cgs();
    athread_spawn_cgs(fun,0);
    athread_join_cgs();
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            if(flag[i][j] !=123+i+j) printf("%d: %d err\n",i,j);
    printf("All done\n");
    return 0;
}
[slave.c]
#include<slave.h>
extern int flag[6][64];
__thread int xx=123;
int fun(){
    xx = xx + _CGN+_PEN;
    flag[_CGN][_PEN] = xx;
}

```

附加说明：运行时需加 bsub 参数 “-mpecg 6”。

- 2) 1 主核+128 从核/1 主核+192 从核(以 1+128 为例)

[host.c]

```

#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag[2][64];
int main(){
    for(int i=0;i<2;i++)
        for(int j=0;j<64;j++)
            flag[i][j] = 0;
    int cgid = current_array_id();
    pthread_init_cgs();
    pthread_spawn_cgs(fun,0);
    pthread_join_cgs();
    for(int i=0;i<2;i++)
        for(int j=0;j<64;j++)
            if(flag[i][j] !=123+j) printf("%d: %d err\n",i,j);
    printf("CG%d all done\n",cgid);
    return 0;
}
[slave.c]
#include<slave.h>
extern int flag[2][64];
__thread int xx=123;
int fun(){
    xx = xx + _PEN;
    flag[_CGN%2][_PEN] = xx;
}

```

附加说明：运行时需加 bsub 参数 “-mpeccg 2”。

3) 6 主核+384 从核

```

[host.c]
#include<pthread.h>
#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag[6][64];
void* mythread(int arg){
    int cgid = (int)arg;
    /*绑定该线程到指定核组*/
    SET_TO_SWCG(cgid);
    printf("I am CG%d\n",current_array_id());
    pthread_spawn_cgmask((0x1<<cgid),fun,0);
    pthread_join_cgmask(0x1<<cgid);
}
int main(){
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            flag[i][j] = 0;
}

```

```

    athread_init_all();
    pthread_t pt[6];
    int ret;
    for(int i=0;i<6;i++){
        ret = pthread_create(&pt[i],NULL,mythread,i);
        if(ret!=-1) printf("pthread_create %d err\n ",i);
    }
    for(int i=0;i<6;i++){
        printf("pthread_join %d \n ",i);
        pthread_join(pt[i],NULL);
    }
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            if(flag[i][j] !=123+i+j) printf("%d: %d err\n",i,j);
    printf("All done\n");
    return 0;
}

```

[slave.c]

```

#include<slave.h>
extern int flag[6][64];
__thread int xx=123;
int fun(){
    xx = xx + _CGN+_PEN;
    flag[_CGN][_PEN] = xx;
}

```

3.3.5 辅助接口

该节除 `current_array_id` 为主核接口外，其余均为从核接口。

3.3.5.1 获取当前核组号

函数名：

`int current_array_id(void);`

函数说明：

获取当前核组编号。

参数说明：

无

返回值：

返回当前核组编号。

3.3.5.2 获取核组内从核号

函数名：

`char athread_get_tid (void);`

函数说明：

获取核组内从核编号（0-63）。

参数说明：

无。

返回值：

成功：从核编号（0-63）；

失败：-1。

3.3.5.3 获取核组内从核行号

函数名：

`char athread_get_rid (void);`

函数说明：

获取核组内从核行号（0-7）。

参数说明：

无。

返回值：

成功：从核行号（0-7）；

失败：-1。

3.3.5.4 获取核组内从核列号

函数名：

`char athread_get_cid (void);`

函数说明：

获取核组内从核列号（0-7）。

参数说明：

无。

返回值：

成功：从核列号（0-7）；

失败：-1。

3.3.5.5 获取核组号

函数名:

`char athread_get_cgn (void);`

函数说明:

获取核组编号 (0-5)。

参数说明:

无。

返回值:

成功: 核组编号 (0-5);

失败: -1。

3.3.5.6 示例

[host.c]

```
#include<athread.h>
extern void SLAVE_FUN(fun());
int main(){
    athread_init();
    int cgid = current_array_id();
    printf("cgid is %d\n",cgid);
    athread_spawn(fun,0);
    athread_join();
    athread_halt();
    return 0;
}
```

[slave.c]

```
#include<slave.h>
int fun(){
    char tid,rid,cid,cgn;

    tid = athread_get_tid();
    rid = athread_get_rid();
    cid = athread_get_cid();
    cgn = athread_get_cgn();
    printf("tid is %d\n",tid);
    printf("rid is %d\n",rid);
    printf("cid is %d\n",cid);
    printf("cgn is %d\n",cgn);
}
```

3.3.6 同步

3.3.6.1 节点内所有主核同步

函数名：

`void athread_sync_node (void);`

函数说明：

本节点内所有主核同步，为主核接口。

参数说明：

无。

返回值：

无。

3.3.6.2 主核与单从核同步

函数名：

`void athread_sync_master_spe (int tid);`

函数说明：

该函数为主核接口。主核与本地核组内由 tid 指定的从核同步。当主核调用该函数时，相应从核必须调用 `athread_ssync_master_spe()` 与主核匹配。

参数说明：

tid: 本地从核编号。

返回值：

无。

3.3.6.3 主核与核组同步

函数名：

`void athread_sync_master_array (void);`

函数说明：

该函数为主核接口。主核与本地核组同步。主核必须与本核组内从核都调用主从同步函数，相关同步操作才能正常完成，但调用该函数的从核范围为创建线程时由对应的 `athread_spawn`，`athread_spawn_mask` 或者 `athread_spawn_create` 所指定。

参数说明：

无。

返回值：

无。

3.3.6.4 从核阵列内同步

函数名：

`int athread_ssync (scope scp, int mask);`

函数说明：

从核接口。单核组从核阵列内同步的三种模式：行同步、列同步与点同步。

参数说明：

scope scp: 从核同步模式，支持：

ROW_SCOPE	行同步模式
COL_SCOPE	列同步模式
PP_SCOPE	点同步模式

int mask: 位图参数。

行同步、列同步时，mask 合法范围为 0x1 - 0xff，寄存器中低 8 位有效，[7:0]分别指示同行（或同列）从核 7 到从核 0 是否同步，为 1 时指示该从核参与同步，为 0 时屏蔽该从核。例如 athread_ssync (ROW_SCOPE, 0xf)表示每一行的前 4 个点同步（0-3 号从核同步,8-11 号从核同步,16-19 号从核同步.....），但行与行之间不同步。点同步时，mask 合法范围为 0-63，表示与当前从核同步的对应从核号。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

同步时，进行同步的从核号必须与同步向量相匹配，否则产生异常。

3.3.6.5 从核阵列内全同步

函数名：

`void athread_ssync_array (void);`

函数说明：

从核接口。单核组从核阵列内全同步，同步该核组 64 个从核。

参数说明：

无

返回值：

无

3.3.6.6 共享从核同步

函数名:

`void athread_ssync_sldm (void);`

函数说明:

从核接口。共享 ldm 的从核同步。

参数说明:

无。

返回值:

无。

3.3.6.7 单从核与主核同步

函数名:

`void athread_ssync_master_spe (void);`

函数说明:

从核接口。本从核与本地主核同步。当从核调用该函数时，本地相应的主核必须调用 `athread_sync_master_spe()` 与从核匹配，函数实参为本从核编号。

参数说明:

无。

返回值:

无。

3.3.6.8 核组与主核同步

函数名:

`void athread_ssync_master_array (void);`

函数说明:

从核接口。本核组与主核同步，需本核组内所有从核、相应主核都调用该函数。

参数说明:

无。

返回值:

无。

3.3.6.9 节点内从核同步

函数名:

`void athread_ssync_node (void);`

函数说明:

从核接口。节点内所有从核同步，需节点（即单个 CPU）内所有线程（从核）都调用该函数。

参数说明:

无。

返回值:

无。

3.3.6.10 示例

1) 示例 1

[host.c]

```
#include<athread.h>
__cross int a[6];
extern void SLAVE_FUN(fun());
__cross int flag,i;
volatile __cross long lock;
int main(){
    flag=0;
    athread_init();
    int cgid = athread_get_arrayid();
    a[cgid]=cgid;
    athread_sync_node ();
    if(cgid==0){
        for(int i=0;i<6;i++){
            if(a[i]!=i)
                flag+=1;
        }
    }
    athread_spawn(fun,0);
    athread_join();
    if(cgid==0){
        if(flag==0)
            printf("result ok!\n");
        else
            printf("result error!\n");
    }
    return 0;
}
```

```

    }
[slave.c]
#include<slave.h>
extern __cross int flag;
__uncached __cross int tmp[384];
void fun(){
    int tid = _CGN*64+_PEN;
    tmp[tid]=tid;
    athread_ssync_node ();
    if(tid==0){
        for(int i=0;i<384;i++){
            if(tmp[i]!=i)
                flag+=1;
        }
    }
}

```

2) 示例 2

```

[host.c]
#include<athread.h>
int a[6];
int b[64];
int c[64];
extern void SLAVE_FUN(fun());
int flag=0;
void main(){
    athread_init();
    athread_spawn(fun,0);
    athread_sync_master_spe(0);
    if(a[0]!=11)
        flag+=1;
    athread_sync_master_array ();
    for(int i=0;i<64;i++){
        if(b[i]!=i)
            flag+=1;
    }
    athread_join();
    if(flag==0)
        printf("result ok!\n");
    else
        printf("result fail!\n");
}

[slave.c]
#include<slave.h>
extern __uncached int a[6];

```

```

extern __uncached int b[64];
extern __uncached int c[64];
extern int flag;
__thread_local_share int d[64];
void fun()
{
    if(_PEN == 0) {
        a[0] = 11;
        athread_ssync_master_spe();
    }
    b[_PEN] = _PEN;
    athread_ssync_master_array();
    c[_PEN] = _PEN;
    athread_ssync_array();
    if(_PEN==0){
        for(int i=0;i<64;i++){
            if(c[i]!=i)
                flag+=1;
        }
    }
    d[_PEN] = _PEN;
    athread_ssync_sldm();
    if(_PEN==0){
        for(int i=0;i<64;i++){
            if(d[i]!=i)
                flag+=1;
        }
    }
}

```

3) 示例 3

[host.c]

```

#include<athread.h>
#include<stdio.h>
#include<stdlib.h>
int a[64] = {0};
int he = 0;
extern int SLAVE_FUN(fun);
int main(){
    athread_init();
    athread_spawn(fun,0);
    athread_join();
    return 0;
}

```

[slave.c]

```

#include<slave.h>
extern int a[64];
extern int he;
volatile __thread_local int in_num = 0;
int fun(){
    if( _ROW < 4 ){
        for( int i = 0; i < 100; i++ ){
            in_num++;
        }
        athread_dma_put((void *)&a[_PEN],(void *)&in_num,4);
        athread_ssync (COL_SCOPE, 0xf);
        athread_ssync (ROW_SCOPE, 0xff);
    }
    if( _PEN == 0){
        for( int i = 0; i < 32; i++ ){
            he += a[i];
        }
        if( he != 3200 )
            printf("he error = %d\n",he);
        else
            printf("he ok!\n");
    }
    return 0;
}

```

3.3.7 锁

3.3.7.1 节点默认锁

函数名：

```

int athread_mutex_lock_node (void);
int athread_mutex_trylock_node (void);
int athread_mutex_unlock_node (void);

```

函数说明：

主核接口。节点默认锁加锁、尝试加锁和解锁，用于节点（即单个 CPU）内所有进程的互斥。

参数说明：

无。

返回值：

成功： 0；

失败： 非 0。

3.3.7.2 自定义锁

函数名:

```
int athread_mutex_lock (volatile int64_t *lock);  
int athread_mutex_trylock (volatile int64_t *lock);  
int athread_mutex_unlock (volatile int64_t *lock);
```

函数说明:

主核接口。主核进程对指定地址进行锁操作，使用主存地址可实现本地锁，使用交叉段地址可实现节点锁。

参数说明:

lock: 自定义锁的地址。

返回值:

成功: 0;

失败: 非 0。

3.3.7.3 节点锁

函数名:

主核:

```
int athread_mutex_lock_node_all (void);  
int athread_mutex_unlock_node_all (void);
```

从核:

```
int athread_smutex_lock_node_all (void);  
int athread_smutex_unlock_node_all (void);
```

函数说明:

节点加锁和解锁，用于节点（即单个 CPU）内所有主、从核的互斥，需要主、从核分别调用对应接口。使用该接口时需要 bsub 配置交叉段大小。

参数说明:

无。

返回值:

成功: 0;

失败: 非 0。

3.3.7.4 从核锁

函数名:

```
int athread_lock (long * lock_var);
```

函数说明：

从核接口。加锁，用于从核组线程互斥。

参数说明：

lock_var: 加锁变量，必须加锁方可见。

返回值：

成功：0；

失败：非 0。

3.3.7.5 从核解锁

函数名：

```
int athread_unlock (long * lock_var);
```

函数说明：

从核接口。解锁，用于从核组线程互斥。

参数说明：

lock_var: 解锁变量，必须解锁方可见。

返回值：

成功：0；

失败：非 0。

3.3.7.6 示例

1) 示例 1

[host.c]

```
#include<athread.h>
__cross int a;
__cross int flag;
volatile __cross long lock;
extern void SLAVE_FUN(fun());
int main(){
    athread_init();
    int cgid = athread_get_arrayid();
    athread_mutex_lock_node();
    a+=1;
    athread_mutex_unlock_node();
    athread_mutex_trylock_node();
    a+=1;
    athread_mutex_unlock_node();
    athread_mutex_lock(&lock);
```

```

a+=1;
pthread_mutex_unlock(&lock);
pthread_mutex_trylock(&lock);
a+=1;
pthread_mutex_unlock(&lock);
pthread_sync_node ();
if(cgid==0){
    if(a!=24)
        flag+=1;
}
pthread_spawn(fun,0);
pthread_join();
if(cgid==0){
    if(flag==0)
        printf("result ok!\n");
    else
        printf("result error!\n");
}
return 0;
}

```

[slave.c]

```

#include<slave.h>
__uncached int b;
__uncached long lock_s;
extern __cross int flag;
void fun(){
    pthread_lock(&lock_s);
    b+=1;
    pthread_unlock(&lock_s);
    pthread_ssync_array();
    if(_PEN==0){
        if(b!=64)
            flag+=1;
    }
}

```

附加说明：bsub 时需要 -n 6 -cgsp 64 并设置交叉段大小

2) 示例 2

[host.c]

```

#include<pthread.h>
extern void SLAVE_FUN(fun());
__uncached __cross int flag;
int main(){

```



```

        athread_init();
        athread_spawn(fun,0);
        athread_mutex_lock_node_all();
        flag+=1;
        athread_mutex_unlock_node_all();
        athread_join();
        athread_halt();
        athread_sync_node();
        if(current_array_id()==0)
            if(flag==390)
                printf("result ok!\n");
            else
                printf("result error!\n");
        return 0;
    }
}
[slave.c]
#include<slave.h>
extern __uncached __cross int flag;
int fun(){
    athread_smutex_lock_node_all();
    flag+=1;
    athread_smutex_unlock_node_all();
}

```

附加说明：bsub 时需要 -n 6 -cgsp 64 并设置交叉段大小

3.3.8 DMA

该节均为从核接口。

DMA 提供主存和 LDM 之间进行批量数据传输的功能。DMA 只能由从核发起，支持单从核模式、行多播模式和列多播模式。各模式含义如图 8 所示：

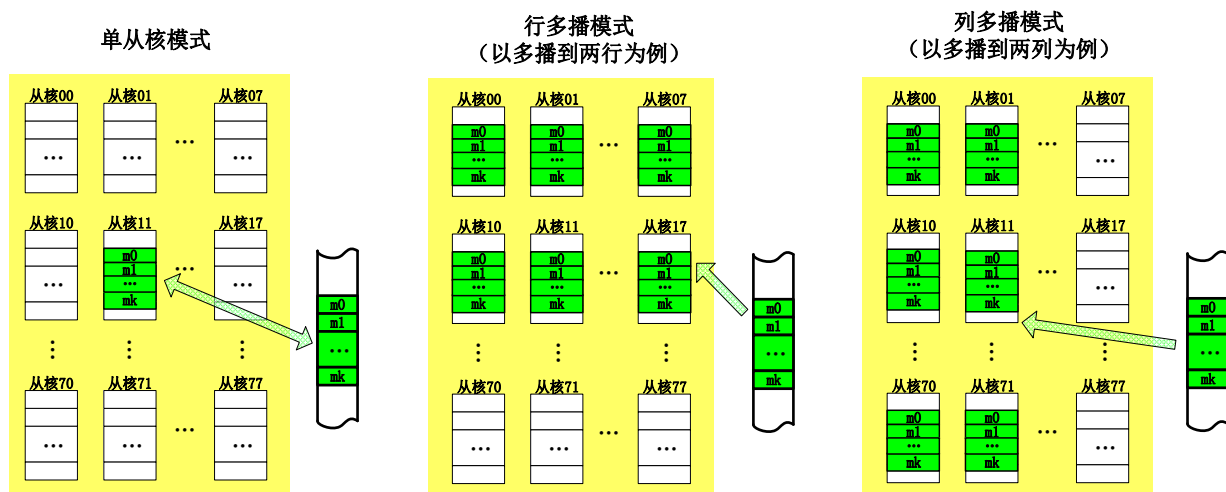


图 8 DMA 各模式示意图

单从核模式/广播模式主存跨步和跨步向量块大小含义如图 9 所示：

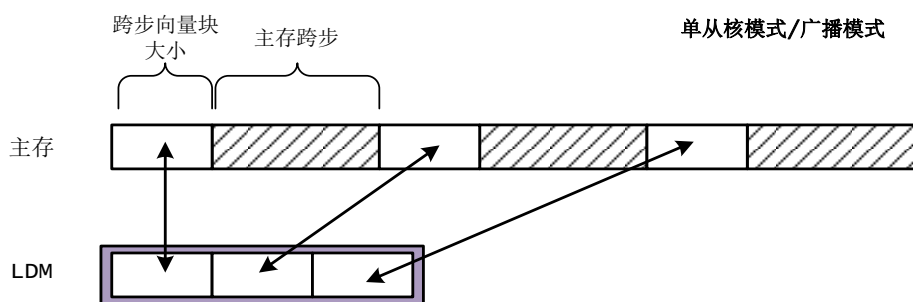


图 9 向量块大小示意图

主存跨步和跨步向量块大小描述了对主存带跨步的读写信息，表示对主存跨步访问时的跳跃间隔。主存跨步为 0 时，表示对主存的连续访问，此时跨步向量块大小的内容无意义。

根据实际使用场景，本系统将 DMA 接口封装为以下几类：

- DMA 读：单从核模式，从主存批量读取数据到 LDM。
- DMA 写：单从核模式，将 LDM 数据批量写入主存。
- DMA 行广播：行多播模式，从主存批量读取数据到同一行上所有从核的 LDM。
- DMA 列广播：列多播模式，从主存批量读取数据到同一列上所有从核的 LDM。
- DMA 广播：行多播模式，从主存批量读取数据到从核阵列所有从核的 LDM。

上述几类 DMA，SACA 均提供阻塞/非阻塞、有跨步/无跨步等多种接口；同时对于行广播、列广播、广播等，还提供集合方式的接口（即要求所有需获取数据的从核都调用）。需特别注意的是，DMA 所有的数据地址、数据长度，都必须为 4 字节的整数倍，即 4 字节对界（对齐）。

DMA 处理部件会把 DMA 请求拆分成 128B 的多次请求，访问主存。非 128B 对界的地址，会拆出更多的请求。因此主存地址建议 128 字节对界。

3.3.8.1 DMA 读

函数名：

1、阻塞接口

```
int athread_dma_get(void *dst, void *src, int len);
```

2、阻塞带跨步的接口

```
int athread_dma_get_stride(void *dst, void *src, int len, int bsize, int stride);
```

函数说明：

单个从核发起从主存到本地局存的 DMA 数据传输，阻塞方式。

参数说明：

dst: 本地局存目标地址（4B 对界/对齐）；

src: 主存源地址（4B 对界/对齐）；

len: 数据传输量（4B 整数倍）；

bsize: 跨步向量块大小（4B 整数倍）；

stride: 主存跨步（4B 整数倍）。

返回值：

成功：0；

失败：非 0。

3.3.8.2 非阻塞 DMA 读

函数名：

1、非阻塞接口

```
int athread_dma_iget(void *dst, void *src, int len, athread_rply_t *rply);
```

2、非阻塞带跨步的接口

```
int athread_dma_iget_stride(void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply);
```

3、用户定义接口

```
int athread_get(dma_mode mode, void *src, void *dest, int len, void *rply, char mask, int stride, int bsize);
```

函数说明：

单个从核发起从主存到本地局存的 DMA 数据传输，非阻塞方式，通常与函数 `athread_dma_wait_value()` 配合使用。

参数说明：

dst: 本地局存目标地址（4B 对界/对齐）；

src: 主存源地址 (4B 对界/对齐);
len: 数据传输量 (4B 整数倍);
bsize: 跨步向量块大小 (4B 整数倍);
stride: 主存跨步 (4B 整数倍);
rply: 回答字地址, 必须 LDM 地址。

返回值:

成功: 0;
失败: 非 0。

3.3.8.3 DMA 等待完成

函数名:

`int athread_dma_wait_value (athread_rply_t *rply, int value);`

函数说明:

该函数用于判断相应的 DMA 操作是否完成。当(*rply)的值大于等于 value 时, 表明相应操作已经完成, 即:

若调用该函数的从核为数据获取方, 表明数据已被写入到本从核的 LDM;

若调用该函数的从核为数据供应方, 表明数据已从本从核的 LDM 取走。

参数说明:

rply: DMA 传输回答字地址, 必须局存地址;
value: 需等待完成的非阻塞 DMA 次数, 为自然数。

返回值:

成功: 0;
失败: 非 0。

3.3.8.4 DMA 写

函数名:

1、阻塞接口

`int athread_dma_put (void *dst, void *src, int len);`

2、阻塞带跨步的接口

`int athread_dma_put_stride (void *dst, void *src, int len, int bsize, int stride);`

函数说明:

单个从核发起从本地 LDM 到主存的 DMA 数据传输, 阻塞方式。

参数说明:

dst: 主存目标地址 (4B 对界/对齐);
src: 本地局存源地址 (4B 对界/对齐);

len: 数据传输量 (4B 整数倍);
bsize: 跨步向量块大小 (4B 整数倍);
stride: 主存跨步 (4B 整数倍)。

返回值:

成功: 0;
失败: 非 0。

3.3.8.5 非阻塞 DMA 写

函数名:

1、非阻塞接口

`int athread_dma_iput(void *dst, void *src, int len, athread_rply_t *rply);`

2、非阻塞带跨步的接口

`int athread_dma_iput_stride(void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply);`

3、用户定义接口

`int athread_put(dma_mode mode, void *src, void *dst, int len, void *rply, int stride, int bsize);`

函数说明:

单个从核发起从本地局存到主存的 DMA 数据传输, 非阻塞方式, 通常与函数 `athread_dma_wait_value()` 配合使用。

参数说明:

dst: 主存目标地址 (4B 对界/对齐);
src: 本地局存源地址 (4B 对界/对齐);
len: 数据传输量 (4B 整数倍);
rply: 回答字地址, 必须局存地址;
bsize: 跨步向量块大小 (4B 整数倍);
stride: 主存跨步 (4B 整数倍)。

返回值:

返回值:

成功: 0;
失败: 非 0。

3.3.8.6 非阻塞 DMA 广播

函数名:

1、非阻塞接口

`int athread_dma_ibcast(void *dst, void *src, int len, athread_rply_t *rply);`

2、非阻塞带跨步的接口

```
int athread_dma_ibcast_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply);
```

函数说明:

DMA 广播，非阻塞方式。只有读（从主存读到局存）。该函数不是集合操作，即只能由一个从核调用。通常与函数 `athread_dma_wait_value()` 配合使用。

参数说明:

`dst`: 本地局存目标地址（4B 对界/对齐）；
`src`: 主存源地址（4B 对界/对齐）；
`len`: 数据传输量（4B 整数倍）；
`rply`: 回答字地址，必须局存地址；
`bsize`: 跨步向量块大小（4B 整数倍）；
`stride`: 主存跨步（4B 整数倍）。

返回值:

成功: 0;
失败: 非 0。

示例:

```
rply = 0;      //rply 为 athread_rply_t 类型，用户需保证其位于 LDM
athread_ssync_array (); //保证回答字的初始化不与 DMA 广播冲突
if (athread_tid == 0)    //单个从核发起 DMA 广播
{
    athread_dma_ibcast (dst, src, len, &rply);
    .....                //其它可并发的操作

} else {
    .....                //其它线程: 可与 DMA 并发的操作
}
athread_dma_wait_value (&rply, 1); //保证从核可见 DMA 广播取到的数据
```

3.3.8.7 集合方式的 DMA 广播

函数名:

1、阻塞接口

```
int athread_dma_bcast_coll (void *dst, void *src, int len);
```

2、阻塞带跨步的接口

```
int athread_dma_bcast_stride_coll (void *dst, void *src, int len, int bsize, int stride);
```

函数说明:

DMA 广播，只有读（从主存读到局存）。该函数是集合操作，需要核组

内所有从核都调用。阻塞方式。

参数说明：

dst: 本地局存目标地址（4B 对界/对齐）；
src: 主存源地址（4B 对界/对齐）；
len: 数据传输量（4B 整数倍）；
bsize: 跨步向量块大小（4B 整数倍）；
stride: 主存跨步（4B 整数倍）。

返回值：

成功：0；
失败：非 0。

3.3.8.8 非阻塞 DMA 行广播

函数名：

1、非阻塞行广播接口

```
int athread_dma_row_ibcast (void *dst, void *src, int len, athread_rply_t *rply);
```

2、非阻塞带跨步的行广播接口

```
int athread_dma_row_ibcast_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply);
```

函数说明：

DMA 行广播，只有读（从主存读到局存），没有写。非阻塞方式。

参数说明：

dst: 本地局存目标地址（4B 对界/对齐）；
src: 主存源地址（4B 对界/对齐）；
len: 每个从核传输的数据量（4B 整数倍）；
bsize: 跨步向量块大小（4B 整数倍）；
stride: 主存跨步（4B 整数倍）；
rply: 回答字地址，必须局存地址。

返回值：

成功：0；
失败：非 0。

示例：

```
rply = 0;    //rply 为 athread_rply_t 类型，用户需保证其位于 LDM  
athread_ssync_row (); //保证回答字的初始化不与 DMA 广播冲突  
if (athread_cid == 0)    //单个从核发起 DMA 广播  
{  
    athread_dma_row_ibcast (dst, src, len, &rply);  
    .....                //其它可并发的操作
```

```

    } else {
        ..... //其它线程：可与 DMA 并发的操作
    }
    athread_dma_wait_value (&rply, 1); //保证行上其他从核看见最新数据

```

3.3.8.9 集合方式的 DMA 行广播

函数名：

1、阻塞接口

```
int athread_dma_row_bcast_coll (void *dst, void *src, int len);
```

2、阻塞带跨步接口

```
int athread_dma_row_bcast_stride_coll (void *dst, void *src, int len, int bsize,
int stride);
```

函数说明：

DMA 行广播，只有读（从主存读到局存）。该函数是集合操作，需要从核阵列同一行上所有从核都调用。阻塞方式。

参数说明：

dst：本地局存目标地址（4B 对界/对齐）；

src：主存源地址（4B 对界/对齐）；

len：数据传输量（4B 整数倍）；

bsize：跨步向量块大小（4B 整数倍）；

stride：主存跨步（4B 整数倍）。

返回值：

成功：0；

失败：非 0。

3.3.8.10 非阻塞 DMA 列广播

函数名：

1、非阻塞列广播接口

```
int athread_dma_col_ibcast (void *dst, void *src, int len, athread_rply_t
*rply);
```

2、非阻塞带跨步的列广播接口

```
int athread_dma_col_ibcast_stride (void *dst, void *src, int len, int bsize, int
stride, athread_rply_t *rply);
```

函数说明：

DMA 列广播，只有读（从主存读到局存），没有写。非阻塞方式。

参数说明：

dst: 本地局存目标地址 (4B 对界/对齐);
src: 主存源地址 (4B 对界/对齐);
len: 每个从核传输的数据量 (4B 整数倍);
stride: 主存跨步 (4B 整数倍);
bsize: 跨步向量块大小 (4B 整数倍);
rply: 回答字地址, 必须局存地址。

返回值:

成功: 0;
失败: 非 0。

示例:

```
rply = 0;      //rply 为 athread_rply_t 类型, 用户需保证其位于 LDM
athread_ssync_col ();
if (athread_rid == 0)      //单个从核发起 DMA 广播
{
    athread_dma_col_ibcast (dst, src, len, &rply);
    .....                //其它可并发的操作

} else {
    .....                //其它线程: 可与 DMA 并发的操作
}
athread_dma_wait_value (&rply, 1); //保证列上其他从核看见最新数据
```

3.3.8.11 集合方式的 DMA 列广播

函数名:

1、阻塞接口

`int athread_dma_col_bcast_coll (void *dst, void *src, int len);`

2、阻塞带跨步接口

`int athread_dma_col_bcast_stride_coll (void *dst, void *src, int len, int bsize, int stride);`

函数说明:

DMA 行广播, 只有读 (从主存读到局存)。该函数是集合操作, 需要从核阵列同一行上所有从核都调用。阻塞方式。

参数说明:

dst: 本地局存目标地址 (4B 对界/对齐);
src: 主存源地址 (4B 对界/对齐);
len: 数据传输量 (4B 整数倍);
bsize: 跨步向量块大小 (4B 整数倍);
stride: 主存跨步 (4B 整数倍)。

返回值:

成功：0；
失败：非 0。

3.3.8.12 共享 ldm 空间 DMA

函数名：

`int athread_memcpy_sldm(void *dst, void *src, int len, int direction);`

函数说明：

从核发起共享 ldm 空间的 DMA 数据传输，需要 LDM 共享空间内的所有从核调用。

参数说明：

dst：目标地址（4B 对界/对齐）；

src：源地址（4B 对界/对齐）；

len：数据传输量（4B 整数倍）；

direction：数据传输的方向，可选项有 MEM_TO_LDM 和 LDM_TO_MEM。

返回值：

成功：0；
失败：非 0。

3.3.8.13 DMA 栏栅

函数名：

`int athread_dma_barrier(void);`

函数说明：

DMA 栏栅。用于本核心发出的 DMA 的保序操作。组内栏栅保证此命令前本从核发出的所有 DMA 命令执行完毕，才执行后续的 DMA 操作。

参数说明：

无。

返回值：

成功：0；
失败：非 0。

3.3.8.14 DMA 全栏栅

函数名：

`int athread_dma_all_barrier(void);`

函数说明：

DMA 全栏栅。兼具 DMA 栏栅和 RDMA 栏栅的语义，从而保证同一个从核发出的先后 DMA/RMA 操作的序。

参数说明：

无。

返回值：

成功：0；

失败：非 0。

3.3.8.15 示例

1) 示例 1

[host.c]

```
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag =0;
void main(){
    athread_init();
    athread_spawn(fun,0);
    athread_join();
    if(flag==0)
        printf("result ok!\n");
    else
        printf("result fail\n");
}
```

[slave.c]

```
#include<slave.h>
int a[10]={10,9,8,7,6,5,4,3,2,1};
__thread_local int b[10];
int c[10];
volatile __thread_local int reply;
extern int flag;
void fun(){
    athread_dma_get(b,a,sizeof(int));
    if(b[0]!=10){
        flag+=1;
        printf("athread_dma_get fail!\n");
    }
    athread_dma_put(c,b,sizeof(int));
    if(c[0]!=10){
        flag+=1;
        printf("athread_dma_put fail!\n");
    }
    athread_dma_get_stride(b,a,sizeof(int)*5,sizeof(int),sizeof(int));
}
```

```

if(b[0]!=10||b[1]!=8||b[2]!=6||b[3]!=4||b[4]!=2){
    flag+=1;
    printf("athread_dma_get_stride fail!\n");
}
athread_dma_put_stride(c,b,sizeof(int)*5,sizeof(int),sizeof(int));
if(c[0]!=10||c[2]!=8||c[4]!=6||c[6]!=4||c[8]!=2){
    flag+=1;
    printf("athread_dma_put_stride fail!\n");
}
memset(b,0,sizeof(int)*10);
reply=0;
athread_get(PE_MODE,a,b,sizeof(int),&reply,0,0,0);
while(reply!=1);
if(b[0]!=10){
    flag+=1;
    printf("athread_get fail!\n");
}
reply=0;
athread_get(PE_MODE,a,b,sizeof(int)*5,&reply,0,sizeof(int),sizeof(int));
while(reply!=1);
if(b[0]!=10||b[1]!=8||b[2]!=6||b[3]!=4||b[4]!=2){
    flag+=1;
    printf("athread_get1 fail!\n");
}
reply=0;
athread_put(PE_MODE,b,c,sizeof(int),&reply,0,0);
while(reply!=1);
if(c[0]!=10){
    flag+=1;
    printf("athread_put fail!\n");
}
reply=0;
athread_put(PE_MODE,b,c,sizeof(int)*5,&reply,sizeof(int),sizeof(int));
while(reply!=1);
if(c[0]!=10||c[2]!=8||c[4]!=6||c[6]!=4||c[8]!=2){
    flag+=1;
    printf("athread_put1 fail!\n");
}
athread_dma_barrier();
memset(b,0,sizeof(int)*10);
reply=0;
athread_dma_iget(b,a,sizeof(int),&reply);
athread_dma_wait_value(&reply,1);
if(b[0]!=10){

```

```

        flag+=1;
        printf("pthread_dma_iget fail!\n");
    }
    pthread_dma_all_barrier();
    reply=0;
    pthread_dma_iget_stride(b,a,sizeof(int)*5,sizeof(int),sizeof(int),&reply);
    pthread_dma_wait_value(&reply,1);
    if(b[0]!=10||b[1]!=8||b[2]!=6||b[3]!=4||b[4]!=2){
        flag+=1;
        printf("pthread_dma_iget_stride fail!\n");
    }
    reply=0;
    pthread_dma_iput(c,b,sizeof(int),&reply);
    pthread_dma_wait_value(&reply,1);
    if(c[0]!=10){
        flag+=1;
        printf("pthread_dma_iput fail!\n");
    }
    reply=0;
    pthread_dma_iput_stride(c,b,sizeof(int)*5,sizeof(int),sizeof(int),&reply);
    pthread_dma_wait_value(&reply,1);
    if(c[0]!=10||c[2]!=8||c[4]!=6||c[6]!=4||c[8]!=2){
        flag+=1;
        printf("pthread_dma_iput_stride fail!\n");
    }
}

```

2) 示例 2

[host.c]

```
#include<pthread.h>
```

```
extern void SLAVE_FUN(fun());
```

```
int flag =0;
```

```
void main(){
```

```
    pthread_init();
```

```
    pthread_spawn(fun,0);
```

```
    pthread_join();
```

```
    if(flag==0)
```

```
        printf("result ok!\n");
```

```
    else
```

```
        printf("result fail\n");
```

```
}
```

[slave.c]

```
#include<slave.h>
```

```
int a[10]={10,9,8,7,6,5,4,3,2,1};
```

```

__thread_local int b[10];
int c[10];
volatile __thread_local int reply;
__thread_local_share int d;
int e;
extern int flag;
void fun(){

    pthread_dma_bcast_coll(b,a,sizeof(int));
    if(b[0]!=10){
        flag+=1;
        printf("pthread_dma_bcast_coll error!\n");
    }
    pthread_dma_bcast_stride_coll(b,a,sizeof(int)*5,sizeof(int),sizeof(int));
    if(b[0]!=10||b[1]!=8||b[2]!=6||b[3]!=4||b[4]!=2){
        flag+=1;
        printf("pthread_dma_bcast_stride_coll error!\n");
    }
    if(_PEN==0){
        reply=0;
        pthread_dma_row_ibcast (b, &a[1], sizeof(int), &reply);
        pthread_dma_wait_value(&reply,1);
    }
    pthread_ssync_array();
    if(_ROW==0&&b[0]!=9){
        flag+=1;
        printf("pthread_dma_row_ibcast error!\n");
    }
    if(_PEN==0){
        reply=0;

pthread_dma_row_ibcast_stride(b,a,sizeof(int)*2,sizeof(int),sizeof(int),&reply);
        pthread_dma_wait_value(&reply,1);
    }
    pthread_ssync_array();
    if(_ROW==0&&(b[0]!=10||b[1]!=8)){
        flag+=1;
        printf("pthread_dma_row_ibcast_stride error!\n");
    }
    b[0]=0;
    pthread_ssync_array();
        pthread_dma_row_bcast_coll(b,a,sizeof(int));
    pthread_ssync_array();
    if(b[0]!=10){

```

```

        flag+=1;
        printf("athread_dma_row_bcast_coll error!\n");
    }
    athread_dma_row_bcast_stride_coll(b,a,sizeof(int)*2,sizeof(int),sizeof(int));
    athread_ssync_array();
    if((b[0]!=10||b[1]!=8)){
        flag+=1;
        printf("athread_dma_row_bcast_stride_coll error!\n");
    }

    b[0]=0;
    athread_ssync_array();
    athread_dma_col_bcast_coll(b,a,sizeof(int));
    athread_ssync_array();
    if(b[0]!=10){
        flag+=1;
        printf("athread_dma_col_bcast_coll error!\n");
    }
    athread_ssync_array();

    athread_dma_col_bcast_stride_coll(b,a,sizeof(int)*2,sizeof(int),sizeof(int));
    athread_ssync_array();
    if((b[0]!=10||b[1]!=8)){
        flag+=1;
        printf("athread_dma_col_bcast_stride_coll error!\n");
    }

    if(_PEN==0){
        reply=0;
        athread_dma_col_ibcast (b, &a[1], sizeof(int), &reply);
        athread_dma_wait_value(&reply,1);
    }
    athread_ssync_array();
    if(_COL==0&&b[0]!=9){
        flag+=1;
        printf("athread_dma_col_ibcast error!\n");
    }
    if(_PEN==0){
        reply=0;
        athread_dma_col_ibcast_stride(b,a,sizeof(int)*2,sizeof(int),sizeof(int),&reply);
        athread_dma_wait_value(&reply,1);
    }
    athread_ssync_sldm();
    if(_COL==0&&(b[0]!=10||b[1]!=8)){

```

```

    flag+=1;
    printf("athread_dma_col_ibcast_stride error!\n");
}

athread_memcpy_sldm(&d,&a,sizeof(int),MEM_TO_LDM);
athread_memcpy_sldm(&e,&d,sizeof(int),LDM_TO_MEM);
if(d!=10||e!=10){
    flag+=1;
    printf("athread_memcpy_sldm error!\n");
}

```

3.3.9 RMA

该节均为从核接口。

RMA 是在核组内从核 LDM 之间进行的远程数据传输操作。RMA 支持单从核模式（包括 put 和 get）、行多播模式和列多播模式。各模式含义如图 10 所示：

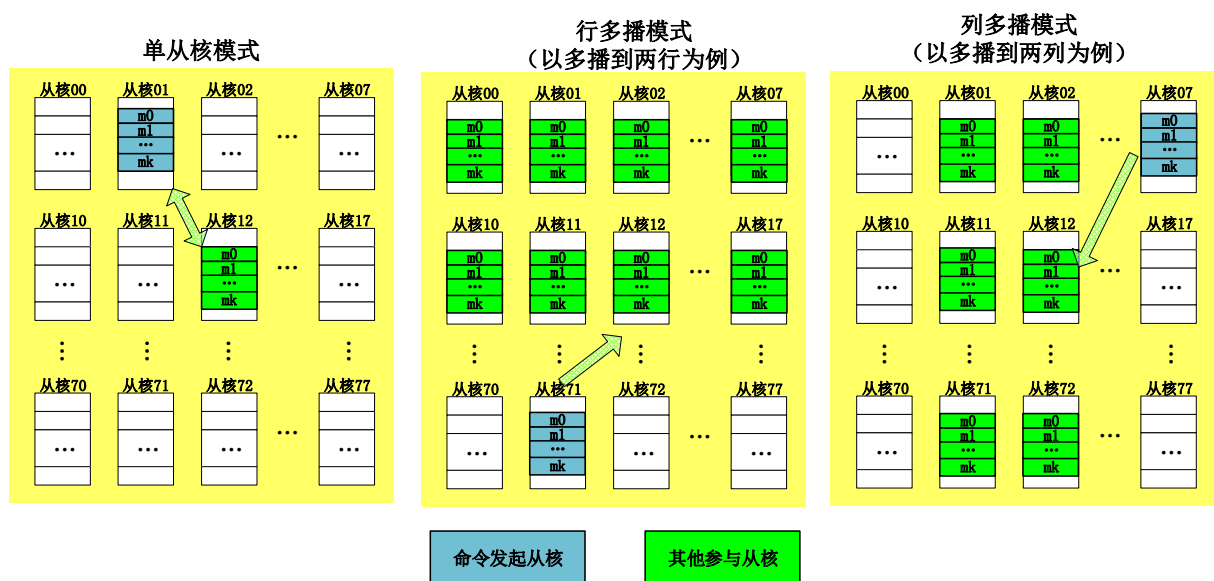


图 10 RMA 各模式示意图

与 DMA 不同的是，RMA 传输不支持跨步访问。根据硬件特征和实际使用场景，本系统将 RMA 封装为单边接口类型：由单个从核主动发起 RMA 数据传输，其他相关从核通过判断相应回答字的值确定 RMA 是否完成。RMA 接口分为以下几类：

- RMA 读：单从核模式，从其他从核 LDM 读取数据到本从核 LDM。
- RMA 写：单从核模式，将本从核 LDM 数据写入其他从核 LDM。
- RMA 行广播/多播：行多播模式，将本从核 LDM 数据写入某些从核行的 LDM。
- RMA 列广播/多播：列多播模式，将本从核 LDM 数据写入某些从核

行的 LDM。

- RMA 广播：行多播模式，将本从核 LDM 数据写入核组所有从核的 LDM。

上述几类 RMA，SACA 均提供阻塞、非阻塞接口；同时对于行广播、列广播、广播等，还提供集合方式的接口（即要求所有需获取数据的从核都调用）。需特别注意的是，RMA 所有的数据地址、数据长度，都必须为 4 字节的整数倍，即 4 字节对界（对齐）。

3.3.9.1 RMA 读

函数名：

```
int athread_rma_get(void *l_addr, int len, int r_tid,  
void *r_addr, athread_rply_t *r_rply);
```

函数说明：

以阻塞方式发起 RMA 读操作。该函数返回时表明数据已经到达本从核 LDM；从核号为 r_tid 的数据源从核，则可通过调用 athread_rma_wait_value()，判断 rply 的值是否增加 1 来确定该 RMA 操作是否完成，通常只有当 rply 的值增加 1 后方可修改 r_addr 指向的 LDM 空间。

参数说明：

l_addr：本地接收地址，必须为局存地址（4B 对界/对齐）；

len：数据传输量，单位为字节（4B 的整数倍）；

r_tid：远程从核号；

r_addr：远程源地址，必须为局存地址（4B 对界/对齐）；

r_rply：远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

示例：

```
rply = 0;  
athread_ssync_array(); /*防止偶数从核 rply 的值先被奇数从核  
发起的 RMA 操作修改，而无法判断完成*/  
if(athread_tid & 0x1) /*奇数从核从偶数从核号从核读取数据*/  
    athread_rma_get(l_addr, len, r_tid, r_addr, &rply);  
else  
    athread_rma_wait_value(&rply, 1);
```

3.3.9.2 非阻塞 RMA 读

函数名:

```
int athread_rma_iget (void *l_addr, athread_rply_t *l_rply, int len,int r_tid,  
void *r_addr, athread_rply_t *r_rply);
```

函数说明:

以非阻塞方式发起 RMA 读操作，通常与函数 `athread_rma_wait_value()` 配合使用。

参数说明:

`l_addr`: 本地接收地址，必须为局存地址（4B 对界/对齐）；

`l_rply`: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

`len`: 数据传输量，单位为字节（4B 的整数倍）；

`r_tid`: 远程从核号；

`r_addr`: 远程源地址，必须为局存地址（4B 对界/对齐）；

`r_rply`: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

3.3.9.3 RMA 等待完成

函数名:

```
int athread_rma_wait_value (athread_rply_t *rply, int value);
```

函数说明:

该函数用于判断相应的 RMA 操作是否完成。当`(*rply)`的值大于等于 `value` 时，表明相应操作已经完成，即：

若调用该函数的从核为数据获取方，表明数据已被写入到本从核的 LDM；

若调用该函数的从核为数据供应方，表明数据已从本从核的 LDM 取走。

参数说明:

`rply`: 回答字地址，必须局存地址；

`value`: 需等待完成的非阻塞 RMA 次数，为自然数。

返回值:

成功: 0;

失败: 非 0。

3.3.9.4 RMA 写

函数名:

```
int athread_rma_put (void *l_addr, int len,int r_tid, void *r_addr,  
athread_rply_t *r_rply);
```

函数说明:

以阻塞方式发起 RMA 写操作。

参数说明:

l_addr: 本地接收地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，单位为字节（4B 的整数倍）；

r_tid: 远程从核号；

r_addr: 远程源地址，必须为局存地址（4B 对界/对齐）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功：0；

失败：非 0。

3.3.9.5 非阻塞 RMA 写

函数名:

```
int athread_rma_iput (void *l_addr, athread_rply_t *l_rply, int len,int r_tid,  
void *r_addr, athread_rply_t *r_rply);
```

函数说明:

以非阻塞方式发起 RMA 写操作，通常与函数 `athread_rma_wait_value()` 配合使用。

参数说明:

l_addr: 本地接收地址，必须为局存地址（4B 对界/对齐）；

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

len: 数据传输量，单位为字节（4B 的整数倍）；

r_tid: 远程从核号；

r_addr: 远程源地址，必须为局存地址（4B 对界/对齐）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功：0；

失败：非 0。

3.3.9.6 RMA 广播

函数名:

```
int athread_rma_bcast (void *dst, void *src, int len,athread_rply_t *r_rply);
```

函数说明:

单个从核发起（非集合方式）的 RMA 广播，阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）;

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）;

len: 数据传输量，以字节为单位（4B 整数倍）;

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

3.3.9.7 非阻塞 RMA 广播

函数名:

```
int athread_rma_ibcast (void *dst, void *src, athread_rply_t *l_rply,int len,athread_rply_t *r_rply);
```

函数说明:

单个从核发起（非集合方式）的 RMA 广播，非阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）;

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）;

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成;

len: 数据传输量，以字节为单位（4B 整数倍）;

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

3.3.9.8 集合方式的 RMA 广播

函数名:

```
int athread_rma_bcast_coll (void *dst, void *src, int len,int root);
```

函数说明:

集合方式（需参与广播的从核都调用）的 RMA 广播，阻塞方式。

参数说明:

dst: 接收地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

root: 发起广播的从核号。

返回值:

成功: 0；

失败: 非 0。

3.3.9.9 RMA 行广播

函数名:

```
int athread_rma_row_bcast (void *dst, void *src, int len, athread_rply_t *r_rply);
```

函数说明:

单个从核发起（非集合方式）的 RMA 行广播，同一从核行上的所有从核均接受数据。阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0；

失败: 非 0。

示例:

```
r_rply = 0;
athread_ssync_row ();           //确保 r_rply 的值不会先被 RMA 修改
if (athread_cid == 6)           //每行由 1 个从核发起 RMA 行广播
    athread_dma_row_bcast (dst, src, len, &r_rply);
athread_rma_wait_value (&r_rply, 1);
```

3.3.9.10 非阻塞 RMA 行广播

函数名:

```
int athread_rma_row_ibcast (void *dst, void *src, int len, athread_rply_t
```

`*l_rply, athread_rply_t *r_rply);`

函数说明：

单个从核发起（非集合方式）的 RMA 行广播，同一从核行上的所有从核均接受数据。非阻塞方式。

参数说明：

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

len: 数据传输量，以字节为单位（4B 整数倍）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

3.3.9.11 集合方式的 RMA 行广播

函数名：

`int athread_rma_row_bcast_coll (void *dst, void *src, int len, int root);`

函数说明：

集合方式（需参与同一从核行上的所有从核都调用）的 RMA 行广播，列号为 root 的从核，向同一从核行上的所有从核广播数据。阻塞方式。

参数说明：

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起行广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

root: 发起行广播的从核在从核行上的相对编号（即列号 athread_cid）。

返回值：

成功：0；

失败：非 0。

示例：

```
/* 以每个从核行的 5 号从核为根的 RMA 行广播，每个从核行同时进行 */
athread_rma_row_bcast_coll (dst, src, len, 5);
```

3.3.9.12 RMA 行多播

函数名：

`int athread_rma_row_mcast (void *dst, void *src, int len, unsigned char mask, athread_rply_t *r_rply);`

函数说明：

单个从核发起（非集合方式）的 RMA 行多播，阻塞方式。发起从核的目标地址也接受多播数据，无论该从核是否位于 **mask** 指定的从核行。

参数说明：

dst: 接受多播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

mask: 从核列掩码，为 1 的位所对应的从核行所有从核均接收数据；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

3.3.9.13 非阻塞 RMA 行多播

函数名：

```
int athread_rma_row_imcast (void *dst, void *src, int len, athread_rply_t *l_rply, unsigned char mask, athread_rply_t *r_rply);
```

函数说明：

单个从核发起（非集合方式）的 RMA 行多播，非阻塞方式。发起从核的目标地址也接受多播数据，无论该从核是否位于 **mask** 指定的从核行。

参数说明：

dst: 参与多播的接收缓冲地址，必须为局存地址（4B 对界/对齐）；

src: 发起多播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

mask: 从核列掩码，为 1 的位所对应的从核行所有从核均接收数据；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值：

成功：0；

失败：非 0。

3.3.9.14 RMA 列广播

函数名：

```
int athread_rma_col_bcast (void *dst, void *src, int len, athread_rply_t
```

`*r_rply);`

函数说明:

单个从核发起（非集合方式）的 RMA 列广播，同一从核列上的所有从核均接受数据。阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

示例:

```
r_rply = 0;
athread_ssync_row ();      //确保 r_rply 的值不会先被 RMA 修改
if (athread_cid == 6)      //每行由 1 个从核发起 RMA 行广播
    athread_dma_row_bcast (dst, src, len, &r_rply);
athread_rma_wait_value (&r_rply, 1);
```

3.3.9.15 非阻塞 RMA 列广播

函数名:

`int athread_rma_col_ibcast (void *dst, void *src, int len,athread_rply_t *l_rply, athread_rply_t *r_rply);`

函数说明:

单个从核发起（非集合方式）的 RMA 列广播，同一从核列上的所有从核均接受数据。非阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

len: 数据传输量，以字节为单位（4B 整数倍）；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

3.3.9.16 集合方式的 RMA 列广播

函数名:

```
int athread_rma_col_bcast_coll(void *dst, void *src, int len, int root);
```

函数说明:

集合方式（需参与同一从核行上的所有从核都调用）的 RMA 行广播，行号为 root 的从核，向同一从核列上的所有从核广播数据。阻塞方式。

参数说明:

dst: 接受广播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起行广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

root: 发起行广播的从核在从核行上的相对编号（即列号 athread_rid）。

返回值:

成功: 0;

失败: 非 0。

示例:

```
/* 以每个从核列的 3 号从核为根的 RMA 列广播，每个从核行同时进行 */  
athread_rma_col_bcast_coll(dst, src, len, 3);
```

3.3.9.17 RMA 列多播

函数名:

```
int athread_rma_col_mcast(void *dst, void *src, int len, unsigned char mask,  
athread_rply_t *r_rply);
```

函数说明:

单个从核发起（非集合方式）的 RMA 列多播，阻塞方式。发起从核的目标地址也接受多播数据，无论该从核是否位于 mask 指定的从核列。

参数说明:

dst: 接受多播数据的目标地址，必须为局存地址（4B 对界/对齐）；

src: 发起广播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

mask: 从核列掩码，为 1 的位所对应的从核列所有从核均接收数据；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0;

失败: 非 0。

3.3.9.18 非阻塞 RMA 列多播

函数名:

```
int athread_rma_col_imcast (void *dst, void *src, int len,athread_rply_t *l_rply, unsigned char mask, athread_rply_t *r_rply);
```

函数说明:

单个从核发起（非集合方式）的 RMA 列多播，非阻塞方式。发起从核的目标地址也接受多播数据，无论该从核是否位于 mask 指定的从核列。

参数说明:

dst: 参与多播的接收缓冲地址，必须为局存地址（4B 对界/对齐）；

src: 发起多播的从核源地址，必须为局存地址（4B 对界/对齐）；

len: 数据传输量，以字节为单位（4B 整数倍）；

l_rply: 本地回答字，用于发起 RMA 的从核判断 RMA 操作是否完成；

mask: 从核列掩码，为 1 的位所对应的从核列所有从核均接收数据；

r_rply: 远程回答字地址，必须局存地址，用于远程从核判断该 RMA 操作是否完成。

返回值:

成功: 0；

失败: 非 0。

3.3.9.19 共享 ldm 空间 RMA

函数名:

```
int athread_sldm_get (void *dst, void *src, int len);
```

函数说明:

从核发起共享 ldm 空间的 RMA 数据传输，需要 LDM 共享空间内的所有从核调用。

参数说明:

dst: 目标地址（4B 对界/对齐）；

src: 源地址（4B 对界/对齐）；

len: 数据传输量（4B 整数倍）；

返回值:

成功: 0；

失败: 非 0。

3.3.9.20 RMA 栏栅

函数名:

`int athread_rma_barrier(void);`

函数说明：

RMA 栏栅。用于本核心发出的 DMA 的保序操作。组内栏栅保证此命令前本从核发出的所有 DMA 命令执行完毕，才执行后续的 DMA 操作。

参数说明：

无。

返回值：

成功：0；

失败：非 0。

3.3.9.21 RMA 全栏栅

函数名：

`int athread_rma_all_barrier(void);`

函数说明：

RMA 全栏栅。兼具 DMA 栏栅和 RDMA 栏栅的语义，从而保证同一个从核发出的先后 DMA/RMA 操作的序。

参数说明：

无。

返回值：

成功：0；

失败：非 0。

3.3.9.22 示例

```
[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag;
int main(){
    flag=0;
    athread_init();
    athread_spawn(fun,0);
    athread_join();
    athread_halt();
    if(flag!=0)
        printf("Result error!\n");
    else
        printf("Result ok!\n");
    return 0;
}
```

```

[slave.c]
#include<slave.h>
extern int flag;
__thread_local int a;
__thread_local int reply;
int fun(){
    //阻塞 RMA 读
    reply=0;
    if(_PEN==0)
        a=1;
    athread_ssync_array();
    athread_rma_get(&a,4,0,&a,&reply);
    if(a!=1)
        flag+=1;
    //非阻塞 RMA 读
    athread_ssync_array();
    reply=0;
    if(_PEN==0)
        a=2;
    athread_ssync_array();
    athread_rma_iget(&a,&reply,4,0,&a,&reply);
    athread_rma_wait_value(&reply,1);
    if(a!=2)
        flag+=1;
    //阻塞 RMA 写
    athread_ssync_array();
    reply=0;
    athread_ssync_array();
    if(_PEN==0){
        a=3;
        athread_rma_put(&a,4,1,&a,&reply);
    }else if(_PEN==1){
        athread_rma_wait_value(&reply,1);
        if(a!=3)
            flag+=1;
    }
    //非阻塞 RMA 写
    athread_ssync_array();
    reply=0;
    if(_PEN==0){
        a=4;
        athread_rma_iput(&a,&reply,4,1,&a,&reply);
    }else if(_PEN==1){
        athread_rma_wait_value(&reply,1);
    }
}

```

```

        if(a!=4)
            flag+=1;
    }
    //阻塞 RMA 广播
    athread_ssync_array();
    reply=0;
    athread_ssync_array();
    if(_PEN==0){
        a=5;
        athread_rma_bcast(&a,&a,4,&reply);
    }
    athread_ssync_array();
    if(a!=5)
        flag+=1;
    //非阻塞 RMA 广播
    athread_ssync_array();
    reply=0;
    athread_ssync_array();
    if(_PEN==0){
        a=6;
        athread_rma_ibcast(&a,&a,&reply,4,&reply);
        athread_rma_wait_value(&reply,1);
    }
    athread_ssync_array();
    if(a!=6)
        flag+=1;
    //集合方式的 RMA 广播
    athread_ssync_array();
    reply=0;
    if(_PEN==0){
        a=7;
    }
    athread_ssync_array();
    athread_rma_bcast_coll(&a,&a,4,0);
    if(a!=7)
        flag+=1;
    //RMA 行广播
    athread_ssync_array();
    reply=0;
    athread_ssync_array();
    if(_PEN==0){
        a=8;
        athread_rma_row_bcast(&a,&a,4,&reply);
    }

```

```

athread_ssync_array();
if(_ROW==0&&a!=8)
    flag+=1;
//非阻塞 RMA 行广播
athread_ssync_array();    reply=0;
athread_ssync_array();
if(_PEN==0){
    a=9;
    athread_rma_row_ibcast(&a,&a,4,&reply,&reply);
    athread_rma_wait_value(&reply,1);
}
athread_ssync_array();
if(_ROW==0&&a!=9)
    flag+=1;
//集合方式的 RMA 行广播
athread_ssync_array();
reply=0;
a=_PEN;
athread_ssync_array();
athread_rma_row_bcast_coll(&a,&a,4,0);
athread_ssync_array();
if(a!=(_PEN/8)*8)
    flag+=1;
//RMA 行多播
athread_ssync_array();
reply=0;
if(_PEN==0)
    a=11;
athread_ssync_array();
if(_PEN==0)
    athread_rma_row_mcast(&a,&a,4,0xff,&reply);
athread_ssync_array();
if(a!=11)
    flag+=1;
//非阻塞 RMA 行多播
athread_ssync_array();
reply=0;
if(_PEN==0)
    a=12;
athread_ssync_array();
if(_PEN==0)
    athread_rma_row_imcast(&a,&a,4,&reply,0xff,&reply);
athread_rma_wait_value(&reply,1);
athread_ssync_array();

```

```

if(a!=12)
    flag+=1;
//RMA 列广播
athread_ssync_array();
reply=0;
athread_ssync_array();
if(_PEN==0){
    a=13;
    athread_rma_col_bcast(&a,&a,4,&reply);
}
athread_ssync_array();
if(_COL==0&&a!=13)
    flag+=1;
//非阻塞 RMA 列广播
athread_ssync_array();
reply=0;
athread_ssync_array();
if(_PEN==0){
    a=14;
    athread_rma_col_ibcast(&a,&a,4,&reply,&reply);
    athread_rma_wait_value(&reply,1);
}
athread_ssync_array();
if(_COL==0&&a!=14)
    flag+=1;
//集合方式的 RMA 列广播
athread_ssync_array();
reply=0;
a=_PEN;
athread_ssync_array();
athread_rma_col_bcast_coll(&a,&a,4,0);
athread_ssync_array();
if(a!=(_PEN%8))
    flag+=1;
//RMA 列多播
athread_ssync_array();
reply=0;
if(_PEN==0)
    a=16;
athread_ssync_array();
if(_PEN==0)
    athread_rma_col_mcast(&a,&a,4,0xff,&reply);
athread_ssync_array();
if(a!=16)

```

```

        flag+=1;
//非阻塞 RMA 列多播
athread_ssync_array();
reply=0;
if(_PEN==0)
    a=17;
athread_ssync_array();
if(_PEN==0)
athread_rma_col_imcast(&a,&a,4,&reply,0xff,&reply);
athread_rma_wait_value(&reply,1);
athread_ssync_array();
if(a!=17)
    flag+=1;
}

```

附加说明：bsub 需指定-mpegc 64

3.3.10 计时

3.3.10.1 主核获取节拍数

函数名：

`unsigned long athread_time_cycle (void);`

函数说明：

主核接口。返回主核当前的周期计数器的值（节拍数）。

参数说明：

无。

返回值：

节拍计数。

3.3.10.2 从核获取节拍计数

函数名：

`unsigned long athread_stime_cycle (void);`

函数说明：

从核接口。返回本从核当前的周期计数器的值（节拍数）。

参数说明：

无。

返回值：

节拍计数。

3.3.10.3 示例

```
[host.c]
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    unsigned long start,end;
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    athread_init();
    start=athread_time_cycle();
    athread_spawn(fun,0);
    athread_join();
    end=athread_time_cycle();
    athread_halt();
    for(int j=0;j<64;j++)
        if(flag [j] !=123 +j) printf("%d err\n",j);
    printf("All done\n");
    printf("host clock is %ld\n",end-start);
    return 0;
}

[slave.c]
#include<slave.h>
extern int flag [64];
__thread int xx=123;
int fun(){
    unsigned long start,end;
    start = athread_stime_cycle();
    xx = xx +_PEN;
    flag [_PEN] = xx;
    end = athread_stime_cycle();
    printf("slave clock is %ld\n",end-start);
}
```

3.3.11 集合通信

该节均为从核接口。

3.3.11.1 全交换

函数名：

```
int athread_alltoall(void *src_addr, void *dest_addr, int units_size);
```

函数说明：

提供从核阵列内的数据全交换接口，注意需要源地址和目标地址不同。每个线程的源地址上保存需要交换的 `units_size*线程总数` 的数据量。

参数说明：

`src_addr`: 数据源地址，必须为局存地址（4B 对界/对齐）；
`dest_addr`: 数据目标地址，必须为局存地址（4B 的整数倍）；
`units_size`: 数据交换单元的大小，单位为字节（4B 的整数倍）。

返回值：

成功：0；
失败：非 0。

3.3.11.2 相同地址全交换

函数名：

```
int athread_alltoall_inplace(void *src_addr, int units_size, void *buf, int buf_item);
```

函数说明：

提供从核阵列内的数据全交换接口，源地址和目标地址相同，需要在接口中指定用于中间计算的缓存地址 `buf` 与大小，更适用于局存地址不足的情况。每个线程的源地址上保存需要交换的 `units_size*线程总数` 的数据量。

参数说明：

`src_addr`: 数据源地址，同时也是目标地址，必须为局存地址（4B 对界/对齐）；
`units_size`: 数据交换单元的大小，单位为字节（4B 的整数倍）；
`buf`: 可用临时空间的初始地址，必须为局存地址（4B 对界/对齐）；
`buf_item`: 可用临时空间可存放的数据交换单元数目。

返回值：

成功：0；
失败：非 0。

3.3.11.3 全规约

函数名：

```
int athread_reduart(void *src_addr, void *dest_addr, int units, int dtype, int optype, void *buf, int buf_item);
```

函数说明：

提供从核阵列内的数据全规约接口，源地址和目标地址可相同或不同，

需要在接口中指定规约缓存地址 buf 与大小。全归约支持的操作类型 optype 如表 7 所示；支持的数据类型参数 dtype 如表 8 所示：

表 7 全归约支持的操作类型 optype

OP_add	OP_and	OP_or	OP_xor	OP_eqv	OP_min	OP_max
--------	--------	-------	--------	--------	--------	--------

表 8 全归约支持的数据类型参数 dtype

athread_int	athread_uint	athread_long	athread_ulong
athread_float	athread_double	athread_intv16	athread_uintv16
athread_int512	athread_uint512	athread_floatv8	athread_doublev8

参数说明：

- src_addr:数据源地址，必须为局存地址（4B 对界/对齐）；
- dest_addr:数据目标地址，必须为局存地址，单位为字节（4B 的整数倍）；
- int units:规约数据单元个数；
- int dtype:规约数据类型；
- int optype:规约操作类型；
- void * buf:规约缓存起始地址；
- int buf_item:规约缓存地址包含的数据单元个数。

返回值：

- 成功：0；
- 失败：非 0。

3.3.11.4 示例

1) 全交换

```
[host.c]
#include<stdio.h>
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag;
int main()
{
    athread_init();
    athread_spawn(fun, 0);
    athread_join();
    if (flag == 0)
        printf("result ok!\n");
    else
        printf("result error!%d\n", flag);
    return 0;
}
```

```

[slave.c]
#include<slave.h>
extern int flag;
__thread_local int a[64], b[64];
__thread_local int c;
void fun()
{
    for (int i = 0; i < 64; i++)
        a[i] = _PEN;
    athread_await_array();
    athread_alltoall(a, b, sizeof(int));
    athread_await_array();
    for (int i = 0; i < 64; i++) {
        if (b[i] != i)
            flag += 1;
    }
    athread_alltoall_inplace(a, sizeof(int), &c, 1);
    athread_await_array();
    for (int i = 0; i < 64; i++) {
        if (a[i] != i)
            flag += 1;
    }
}
}

```

附加说明：bsub 时需指定-cgsp 64 并设置交叉段大小

2) 全规约

```

[host.c]
#include<stdio.h>
#include<athread.h>
extern void SLAVE_FUN(fun());
int flag;
int main(){
    athread_init();
    athread_spawn(fun,0);
    athread_join();
    if(flag==0)
        printf("result ok!\n");
    else
        printf("result error!%d\n",flag);
    return 0;
}

[slave.c]
#include<slave.h>
#include<simd.h>
extern int flag;

```

```

__thread_local int a[64], b[64];
__thread_local int c[64];
__thread_local floatv8 va = { 1, 1, 1, 1, 1, 1, 1, 1 };
__thread_local floatv8 vb;
__thread_local int512 vc = { 0, 1, 2, 3, 4, 5, 6, 7 };

void fun()
{
    for (int i = 0; i < 64; i++)
        a[i] = 1;
    athread_await_array();
    athread_redundant(a, b, 64, athread_int, OP_add, c, 64);
    athread_await_array();
    for (int i = 0; i < 64; i++)
        if (b[i] != 64)
            flag += 1;
    athread_await_array();
    athread_redundant(a, b, 64, athread_int, OP_and, c, 64);
    athread_await_array();
    for (int i = 0; i < 64; i++)
        if (b[i] != 1)
            flag += 1;
    athread_await_array();
    athread_redundant(&va, &vb, 1, athread_floatv8, OP_add, c, 64);
    athread_await_array();
    for (int i = 0; i < 8; i++)
        if (vb[i] != 64)
            flag += 1;
    athread_await_array();
    athread_redundant(&vc, &vc, 1, athread_int512, OP_max, c, 64);
    athread_await_array();
    for (int i = 0; i < 8; i++)
        if (vc[i] != i)
            flag += 1;
}

```

附加说明： bsub 时需指定-cgsp 64

3.3.12 空间分配

3.3.12.1 主存分配

主核接口：

表 9 主核分配主存接口

接口名称	功能	从核可 cache 属性	空间说明
malloc	申请主存连续共享段 或者交叉段	是	返回 0x5000,0000,0000 或 0x5800,0000,0000为起始地址的空 间，具体见详细说明
_sw_xmalloc	申请交叉段	是	返回0x5800,0000,0000为起始地址的 空间，具体见详细说明
_sw_xfree	释放交叉段		与_sw_xmalloc配合使用
libc_aligned_malloc	申请主存连续共享段	是	返回 0x5000,0000,0000为起始地址 的空间，返回地址64字节对齐
libc_aligned_free	释放主存连续共享段		与libc_aligned_malloc配合使用
libc_set_aligned_malloc(size_t __size,int align)	申请主存连续共享段	是	返回 0x5000,0000,0000为起始地址 的空间，返回地址align字节对齐
libc_set_aligned_free	释放主存连续共享段		与libc_set_aligned_malloc配合使用
libc_uncached_malloc	申请主存连续共享段	否	返回 0x7000,0000,0000为起始地址 的空间
libc_uncached_free	释放主存连续共享段		与libc_uncached_malloc配合使用
libc_uncached_aligned_malloc	申请主存连续共享段	是	返回 0x7000,0000,0000为起始地址 的空间，返回地址64字节对齐
libc_uncached_aligned_free	释放主存连续共享段		与 libc_uncached_aligned_malloc 配 合使用

详细说明：

-mhybrid 模式：

1、非大共享模式下，主核 malloc 返回连续共享段 0x5000 为起始地址的空间地址，_sw_xmalloc 返回交叉段 0x5800 为起始地址的空间地址，只允许一个主核调用_sw_xmalloc。

2、大共享模式下，主核 malloc 返回连续共享段 0x5000 为起始地址的空间地址，_sw_xmalloc 按照下面的描述返回交叉段 0x5800 为起始地址的空间地址：

1) -mpeccg 2：每个 CPU 起 3 个进程，每个进程使用 2 个从核阵列资源，进程绑定在 CG0/CG2/CG4，动态交叉段空间平均分成 3 份，CG0/CG2/CG4

_sw_xmalloc 返回独立的交叉段空间。

2) -mpecg 3: 每个 CPU 起 2 个进程，每个进程使用 3 个从核阵列资源，进程绑定在 CG0/CG3，动态交叉段空间平均分成 2 份，CG0/CG3 _sw_xmalloc 返回独立的交叉段空间。

3) -mpecg 6: 每个 CPU 起 1 个进程，使用 6 个从核阵列资源，_sw_xmalloc 返回交叉段空间。

3、非大共享模式下，bsub 加-xmalloc 选项，主核 malloc 返回交叉段空间，动态交叉段空间平均分成 6 份，每个主核 malloc 返回独立的交叉段空间，不能再使用_sw_xmalloc。

大共享模式下，bsub 加-xmalloc 选项，主核 malloc 返回交叉段空间，按照上述第 2 点的描述返回交叉段 0x5800 为起始地址的空间地址，不能再使用_sw_xmalloc。

-mdynamic 模式:

主核 malloc 默认返回连续共享段 0x5000 为起始地址的空间地址，bsub 加-crossmalloc 选项时，主核 malloc 返回交叉段 0x5800 为起始地址的空间地址。

备注：cross_size 分为静态交叉段空间和动态交叉段空间，静态交叉段地址在前，静态交叉段空间存放程序中__cross 关键字声明的数据，可动态分配的动态交叉段空间大小为“cross_size-静态交叉段空间大小”。

从核接口:

表 10 从核分配主存接口

接口名称	功能	从核可 cache 属性	空间说明
malloc	申请主存从核私有空间	是	返回0x4000,0000,0000为起始地址的空间
libc_aligned_malloc	申请主存从核私有空间	是	返回0x4000,0000,0000为起始地址的空间，返回地址64字节对齐
libc_aligned_free	释放主存从核私有空间		与libc_aligned_malloc配合使用
libc_set_aligned_malloc(size_t __size,int align)	申请主存从核私有空间	是	返回0x4000,0000,0000为起始地址的空间，返回地址align字节对齐
libc_set_aligned_free	释放主存从核私有空间		与libc_set_aligned_malloc配合使用
libc_uncached_malloc	申请主存从核私有空间	否	返回0x6000,0000,0000为起始地址的空间
libc_uncached_free	释放主存从核私有空间		与 libc_uncached_malloc 配合使

			用
libc_uncached_aligned_malloc	申请主存从核私有空间	是	返回0x6000,0000,0000为起始地址的空间，返回地址64字节对齐
libc_uncached_aligned_free	释放主存从核私有空间		与 libc_uncached_aligned_malloc 配合使用

3.3.12.2 从核局存分配

该节均为从核接口。从核 LDM 支持精简快速的动态分配，具体接口如下：

3.3.12.2.1 get_allocatable_size

函数名：

`unsigned int get_allocatable_size(void);`

函数说明：

得到当前可动态分配的 LDM 空间大小，也即局存动态分配运行时库管理的 LDM 空间大小。

参数说明：

无

返回值：

当前可分配的 LDM 空间大小，单位为字节。

3.3.12.2.2 ldm_malloc

函数名：

`void* ldm_malloc(size_t size);`

函数说明：

局存空间申请，支持 Fortran 中调用。

参数说明：

size: 以字节为单位的空间大小。

返回值：

成功返回分配 LDM 空间的起始地址；
失败返回 NULL。

3.3.12.2.3 ldm_free

函数名：

`void ldm_free(void *addr, size_t size);`

函数说明：

局存空间释放，释放空间起始地址为 addr。支持 Fortran 中调用。

参数说明:

`void * addr`: 必须是前面某一次 `ldm_malloc` 返回的分配成功的局存地址;

`size_t size`: 显式指定释放空间的大小, 一般必须与前面某一次 `ldm_malloc` 的大小相匹配。

返回值:

无

3.3.12.3 示例

[host.c]

```
#include<athread.h>
extern void SLAVE_FUN(fun());
int main(){
    athread_init();
    int *p;
    p=(int *)malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==5){
        printf("malloc addr is %p\n",p);
    }
    else{
        printf("malloc error!\n");
        exit(-1);
    }
    free(p);
    p=(int *)libc_aligned_malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==5&&((unsigned long)p%64)==0){
        printf("libc_aligned_malloc addr is %p\n",p);
    }
    else{
        printf("libc_aligned_malloc error!\n");
        exit(-1);
    }
    libc_aligned_free(p);
    p=(int *)libc_uncached_malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==7){
        printf("libc_uncached_malloc addr is %p\n",p);
    }
    else{
        printf("libc_uncached_malloc error!\n");
        exit(-1);
    }
    libc_uncached_free(p);
    p=(int *)libc_uncached_aligned_malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==7&&((unsigned long)p%64)==0){
```

```

printf("libc_uncached_aligned_malloc addr is %p\n",p);
}
else{
    printf("libc_uncached_aligned_malloc error!\n");
    exit(-1);
}
libc_uncached_aligned_free(p);
pthread_spawn(&fun,0);
pthread_join();
pthread_halt();
printf("All done\n")
return 0;
};

```

[slave.c]

```

#include<slave.h>
int fun(){
    int *p;
    p=(int *)malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==4){
        printf("malloc addr is %p\n",p);
    }
    else{
        printf("malloc error!\n");
        exit(-1);
    }
    free(p);
    p=(int *)libc_aligned_malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==4&&((unsigned long)p%64)==0){
        printf("libc_aligned_malloc addr is %p\n",p);
    }
    else{
        printf("libc_aligned_malloc error!\n");
        exit(-1);
    }
    libc_aligned_free(p);
    p=(int *)libc_uncached_malloc(4);
    if(p!=NULL&&((unsigned long)p>>44)==6){
        printf("libc_uncached_malloc addr is %p\n",p);
    }
    else{
        printf("libc_uncached_malloc error!\n");
        exit(-1);
    }
    libc_uncached_free(p);
}

```

```

p=(int *)libc_uncached_aligned_malloc(4);
if(p!=NULL&&((unsigned long)p>>44)==6&&((unsigned long)p%64)==0){
    printf("libc_uncached_aligned_malloc addr is %p\n",p);
}
else{
    printf("libc_uncached_aligned_malloc error!\n");
    exit(-1);
}
libc_uncached_aligned_free(p);
unsigned int size = get_allocatable_size();
printf("allocatable_size is %d\n",size);
p=(int *)ldm_malloc(4);
if(p!=NULL){
    printf("ldm_malloc addr is %p\n",p);
}
else{
    printf("ldm_malloc error!\n");
    exit(-1);
}
ldm_free(p,4);
}

```

3.3.13 浮点寄存器配置

3.3.13.1 主核读写 fpcr 相关接口

3.3.13.1.1 RFPCR

函数名：

`void RFPCR (unsigned long val);`

函数说明：

读主核浮点控制寄存器 fpcr，将寄存器值存到 val 变量中。

参数说明：

val：用于存储 fpcr 寄存器值的变量。

返回值：

无

3.3.13.1.2 WFPCR

函数名：

`void WFPCR(unsigned long val);`

函数说明：

将 val 变量值写到主核浮点控制寄存器 fpcr。

参数说明：

val: 用于写入 fpcr 寄存器的值。

返回值：

无

3.3.13.2 从核读写 fpcr 相关接口

3.3.13.2.1 RFPCR0

函数名：

`void RFPCR0 (unsigned long val);`

函数说明：

读从核浮点控制寄存器 fpcr 低 64 位值到 val 变量。

参数说明：

val: 用于存储 fpcr 寄存器低 64 位值的变量。

返回值：

无

3.3.13.2.2 RFPCR1

函数名：

`void RFPCR1 (unsigned long val);`

函数说明：

读从核浮点控制寄存器 fpcr 高 64 位值到 val 变量。

参数说明：

val: 用于存储 fpcr 寄存器高 64 位值的变量。

返回值：

无

3.3.13.2.3 RFPCR

函数名：

`void RFPCR (unsigned long val0, unsigned long val1);`

函数说明：

读从核浮点控制寄存器 fpcr，将低 64 位值存到 val0 变量，高 64 位值存到 val1 变量。

参数说明：

val0: 用于存储 fpcr 寄存器低 64 位值的变量

val1: 用于存储 fpcr 寄存器高 64 位值的变量

返回值:

无

3.3.13.2.4 WFPCR

函数名:

`void WFPCR (unsigned long val0, unsigned long val1);`

函数说明:

将 val0、val1 变量拼成 128 位值写到从核浮点控制寄存器, val0 为低 64 位值, val1 为高 64 位值。

参数说明:

val0: 用于写 fpcr 寄存器低 64 位值的变量

val1: 用于写 fpcr 寄存器高 64 位值的变量

返回值:

无

3.3.13.3 设置浮点运算舍入模式相关接口

函数名:

`void SET_round_mode (int val);`

函数说明:

主核接口。将浮点控制寄存器 fpcr 中的动态舍入位设置为用户定义的值。

参数说明:

val: 舍入模式, 有效值只有 0、1、2、3 四个值, 其他无效且会报错;

0: 向“0”舍入

1: 向负无穷大舍入

2: 就近舍入

3: 向正无穷大舍入

返回值:

无

3.3.13.4 示例

1) 主核示例

```
#include<athread.h>
```

```
int main(){  
    unsigned long val;  
    RFPCR(val);  
    printf("%0#llx\n",val);
```

```

        SET_round_mode(1);
        RFPCR(val);
        printf("%#llx\n",val);
        WFPCR(val);
    }
2) 从核示例
#include<slave.h>

int main(){
    unsigned long val0,val1;
    RFPCR0(val0);
    RFPCR1(val1);
    printf("%#llx,%#llx\n",val0,val1);
    SET_round_mode(1);
    RFPCR0(val0);
    RFPCR1(val1);
    printf("%#llx,%#llx\n",val0,val1);
    RFPCR(val0,val1);
    printf("%#llx,%#llx\n",val0,val1);
    WFPCR(val0,val1);
}

```

3.3.14 主核 IO 空间操作

该节均为主核接口。

3.3.14.1 SPEIO_REG_ADDR

函数名：

`unsigned long * SPEIO_REG_ADDR (int cgid, int speid, unsigned long reg);`

函数说明：

获取指定从核的内部寄存器 IO 访问地址。

参数说明：

cgid: 核组号 (0-5)

speid: 从核号 (0-63)

reg: 从核内部寄存器的 IO 地址

预定义的 reg 包括有：

#define SPE_EBXIO_FPCR	0x100800	//浮点控制状态寄存器
#define SPE_EBXIO_DEF0	0x100400	//用户自定义项 0
#define SPE_EBXIO_CTRL	0x100900	//EBOX 控制寄存器
#define SPE_IBXIO_GPC	0x8000	//IBOX 全局 PC

```
#define SPE_IBXIO_RUN          0x8100      //IBOX 运行控制
```

返回值：
从核内部寄存器 IO 访问地址。

3.3.14.2 SPEIO_READ_REG

函数名：

```
unsigned long SPEIO_READ_REG (int cgid, int speid, unsigned long reg);
```

函数说明：

通过 IO 访问获取指定从核的内部寄存器内容。

参数说明：

cgid: 核组号 (0-5)

speid: 从核号 (0-63)

reg: 从核内部寄存器的 IO 地址

预定义的 reg 包括有：

```
#define SPE_EBXIO_FPCR          0x100800    //浮点控制状态寄存器
#define SPE_EBXIO_DEF0          0x100400    //用户自定义项 0
#define SPE_EBXIO_CTRL          0x100900    //EBOX 控制寄存器
#define SPE_IBXIO_GPC           0x8000      //IBOX 全局 PC
#define SPE_IBXIO_RUN           0x8100      //IBOX 运行控制
```

返回值：

从核内部寄存器的 64 位内容。

3.3.14.3 SPEIO_WRITE_REG

函数名：

```
void SPEIO_WRITE_REG (int cgid, int speid, unsigned long reg, unsigned long val);
```

函数说明：

通过 IO 访问写指定从核的内部寄存器。

参数说明：

cgid: 核组号 (0-5)

speid: 从核号 (0-63)

reg: 从核内部寄存器的 IO 地址

val: 要写入内部寄存器的值

预定义的 reg 包括有：

```
#define SPE_EBXIO_FPCR          0x100800    //浮点控制状态寄存器
#define SPE_EBXIO_DEF0          0x100400    //用户自定义项 0
```

```

#define SPE_EBXIO_CTRL      0x100900    //EBOX 控制寄存器
#define SPE_IBXIO_GPC       0x8000      //IBOX 全局 PC
#define SPE_IBXIO_RUN       0x8100      //IBOX 运行控制

```

返回值:

无

3.3.14.4 SPEIO_LDM_ADDR

函数名:

```

unsigned long * SPEIO_LDM_ADDR (int cgid, int speid, unsigned long
disp);

```

函数说明:

获取指定从核 LDM 的 IO 访存地址。

参数说明:

cgid: 核组号 (0-5)

speid: 从核号 (0-63)

disp: 从核内部 LDM 地址 (0~256KB)

返回值:

指定从核的 LDM 的 IO 访存地址。

附加说明:

跨核组访问从核 LDM 的 IO 访存地址时,需要调用核心接口获取权限。

3.3.14.5 示例

1) 示例 1

```

#include<athread.h>
extern void SLAVE_FUN(fun());
int main(){
    athread_init();
    unsigned long *p = SPEIO_REG_ADDR(0,0,SPE_EBXIO_FPCR);
    printf("%#llx\n",*p);
    unsigned long f = SPEIO_READ_REG(0,0,SPE_EBXIO_FPCR);
    printf("%#llx\n",f);
    SPEIO_WRITE_REG(0,0,SPE_EBXIO_FPCR,f);
    f = SPEIO_READ_REG(0,0,SPE_EBXIO_FPCR);
    printf("%#llx\n",f);
    athread_spawn(fun,0);
    athread_join();
    athread_halt();
    printf("All done\n");
    return 0;
}

```



```

    }
2) 示例 2
#include<athread.h>
extern void slave_fun();
main()
{
    athread_init();
    int cgid=current_array_id();
    if(cgid==0)
    {
        int stask_fd=athread_get_slave_fd();
        int val=ioctl(stask_fd, SLAVE_MAP_SPE_IO, 1); //获取 1 核组 IO 权限
        if(val != 0) {
            printf("map stask spe failed %d, %d\n",val,errno);
        }
        unsigned long destptr = (void *)SPEIO_LDM_ADDR(1,1,32); //写 1 核组 1 从
核地址为 32 的 LDM 空间
        *(long *)destptr=2;
    }
    athread_sync_node (); //主核同步等待 IO 操作完成
    athread_spawn(fun,0);
    athread_join();
    athread_halt();    }

```

3.3.15 从核 ldmcache

3.3.15.1 获取从核连续共享 LDM 大小

函数名：

`long get_slave_ldmshare_size (void);`

函数说明：

得到当前从核的连续共享 LDM 空间大小

参数说明：

无

返回值：

返回值单位为字节。有 7 种可能的返回值：分别为 0、4096、8192、16384、32768、65536、131072，即代表连续共享 LDM 大小为 0、4KB、8KB、16KB、32KB、64KB、128KB。

附加说明：

也可在从核程序中直接使用 `_ldm_share_size` 局存变量获得连续共享 LDM 空间大小。

3.3.15.2 获取从核 cache 大小

函数名:

`long get_slave_cache_size(void);`

函数说明:

得到当前从核 cache 空间大小。

参数说明:

无

返回值:

返回值单位为字节。有 3 种可能的返回值：分别为 0、32768、131072，即代表 cache 大小为 0、32KB、128KB。

附加说明:

也可在从核程序中直接使用 `_cache_size` 局存变量获得连续从核 cache 空间大小。

3.3.15.3 配置从核连续共享 LDM 模式与大小

函数名:

`int set_ldm_mode_size(int ldm_shared_mode, int ldm_shared_size);`

函数说明:

配置从核的连续共享 LDM 模式及大小。

参数说明:

`ldm_shared_mode`: 从核连续共享 LDM 模式，支持:

<code>LDMS_SINGLE</code>	单簇
<code>LDMS_DOUBLE</code>	双簇
<code>LDMS_QUAD</code>	四簇
<code>LDMS_ALL_CLUSTER</code>	全阵列簇
<code>LDMS_ALL_ROW</code>	全阵列行

`ldm_shared_size`: 从核连续共享 LDM 大小，支持:

`LDMS_0KB | LDMS_4KB | LDMS_8KB | LDMS_16KB |
LDMS_32KB | LDMS_64KB | LDMS_128KB`

返回值:

0 成功; 1 失败。

附加说明:

1、硬件要求所有从核的连续共享 LDM 模式以及容量必须一致，这一点需要用户在配置时保证;

2、为保证程序运行的正确性，一般用户对连续共享 LDM 模式及容量以及下一节的 cache 容量进行配置时，应当遵循以下的流程:

- 1) 通过判回答字，确认从核之前参与的 DMA/RMA 请求都已完成；
- 2) 插入 memb 指令，确认从核之前发出的访存操作都已完成；
- 3) 进行核组内从核的全同步；
- 4) 调用配置接口进行配置；
- 5) 再次进行核组内从核的全同步；

3、上述流程会带来较大的时间开销，对性能要求比较高的用户在能够熟练掌握连续共享 LDM 以及 cache 运行使用机制的基础上，如果能够保证多个从核的运行速度基本同步的情况下，可以省却上面的 MEMB 指令以及从核同步的步骤。

4、-b 选项用于将从核程序栈指针指向 LDM 空间，这样可以使对栈变量的访问转为对 LDM 空间上的访问，提升程序性能。需要注意的是，如果栈空间太大，程序运行过程中可能会对原有用户的 LDM 空间的使用造成影响，产生不可预知的错误。尤其是动态调整从核连续共享 LDM 模式/容量以及从核 cache 容量时，会额外动态申请与栈空间大小相同的 LDM 空间进行栈空间的转移工作。

3.3.15.4 配置从核 cache 容量

函数名：

```
int set_cache_size (int cache_size);
```

函数说明：

配置从核的 cache 容量。

参数说明：

cache_size: 从核 cache 大小，支持：

CACHE_0KB | CACHE_32KB | CACHE_128KB

返回值：

0 成功；1 失败。

附加说明：

1、调整从核 cache 容量过程中会对整个从核 cache 进行刷新，用户需要注意数据的一致性问题。

2、一般情况下要求所有从核的 cache 容量必须一致。如果用户不配置连续共享 LDM 空间，允许针对不同的从核配置不同的 cache 容量，一旦配置了连续共享 LDM 空间，则必须保证所有从核的 cache 容量相同。这一点需要用户在配置时保证。

3、为保证程序运行的正确性，用户配置遵循的流程与 3.3.15.3 节相同。

4、-b 选项用于将从核程序栈指针指向 LDM 空间，这样可以使对栈变量的访问转为对 LDM 空间上的访问，提升程序性能。需要注意的是，如果栈空间太大，程序运行过程中可能会对原有用户的 LDM 空间的使用造成影

响，产生不可预知的错误。尤其是动态调整从核连续共享 LDM 模式/容量以及从核 cache 容量时，会额外动态申请与栈空间大小相同的 LDM 空间进行栈空间的转移工作。

3.3.15.5 淘汰连续主存空间对应 cache 行

函数名：

`void evict_slave_cache_cont (void* start, void* end);`

函数说明：

淘汰连续主存空间 [start, end] 对应 cache 行。

参数说明：

void * start: 主存空间起始地址

void * end: 主存空间结束地址

返回值：

无

附加说明：

start 和 end 为主存空间地址，与数据类型无关。对于结构体数组，需要用户自行保证起始地址与结束地址的正确性。

3.3.15.6 刷新整个从核 cache

函数名：

`void flush_slave_cache (void);`

函数说明：

刷新整个从核 cache。

参数说明：

无

返回值：

无

3.3.15.7 从核 cache 写暗示

函数名：

`void sw_slave_write_hint (void * mem);`

函数说明：

将主存地址 mem 对应的 cache 行置写暗示。

参数说明：

void * mem: 主存地址。

返回值:

无

附加说明:

写暗示指令提示包含寻址字节的对界 256 字节块的内容将不再有读访问,但不久将有写操作。从核将不读存储器以前的内容而直接分配并占有该 cache 行,cache 行的内容可设为任意值。写暗示避免从主存中载入无用的数据,提高了处理性能。

3.3.15.8 从核 cache 私有段设置

函数名:

```
void sw_slave_private_section0 (void * start, void * end);  
void sw_slave_private_section1 (void * start, void * end);  
void sw_slave_private_section2 (void * start, void * end);  
void sw_slave_private_section3 (void * start, void * end);
```

函数说明:

将主存空间 [start, end] 对应 cache 行设置为私有段,目前支持 4 套私有空间的配置。

参数说明:

void * start: 主存空间起始地址;

void * end: 主存空间结束地址。

返回值:

无

附加说明:

映射到该私有段区间内的 cache 行写回主存时以独占方式写回,用于改善写回主存性能。

start 和 end 为主存空间地址,与数据类型无关。对于结构体数组,需要用户自行保证起始地址与结束地址的正确性。

软件必须保证设置私有段区间指令发射后,对该地址有 store 操作。

区间内的 cache 行被淘汰后私有属性仍生效,置无效的方法是设置私有段的起始地址=结束地址。

3.3.15.9 从核指针设成不可 cache 访问

函数名:

```
void set_ptr_uncached(void **addr);
```

函数说明:

将指针 addr 指向的地址设为不可 cache 访问。

参数说明:

void **addr

返回值:

无

3.3.15.10 从核指针设成可 cache 访问

函数名:

void set_ptr_cached(void **addr);

函数说明:

将指针 addr 指向的地址设为可 cache 访问。

参数说明:

void **addr

返回值:

无

示例:

```
#include<stdio.h>
int a[100]; //可 cache 空间
void main()
{
    int *p1=a;
    set_ptr_uncached(&p1); //通过 p1 访问数组 a 为不可 cache 访问
    int *p2=p1;
    set_ptr_cached(&p2); //通过 p2 访问数组 a 为可 cache 访问
    printf("%p,%p,%p\n",a,p1,p2);
}
```

3.3.15.11 示例

```
#include<slave.h>
int a[64] __attribute__((aligned(256)));
int main(){
    long ldm_sharesize,cache_size;
    ldm_sharesize=get_slave_ldmshare_size();
    printf("ldm_sharesize is %ldKB\n",ldm_sharesize/1024);
    cache_size=get_slave_cache_size();
    printf("cache_size is %ldKB\n",cache_size/1024);
    set_ldm_mode_size(LDMS_ALL_ROW,LDMS_32KB);
    ldm_sharesize=get_slave_ldmshare_size();
    printf("ldm_sharesize is %ldKB\n",ldm_sharesize/1024);
    set_cache_size(CACHE_128KB);
    cache_size=get_slave_cache_size();
```

```

printf("cache_size is %ldKB\n",cache_size/1024);
for(int i=0;i<64;i++){
a[i]=0;
}
evict_slave_cache_cont(&a[0],&a[63]);
flush_slave_cache();
sw_slave_write_hint(a);
sw_slave_private_section0(&a[0],&a[63]);
int *p=a;
printf("%p\n",p);
set_ptr_uncached(&p);
printf("%p\n",p);
set_ptr_cached(&p);
printf("%p\n",p);
}

```

3.4 SIMD 向量编程

3.4.1 概述

SIMD（Single Instruction Multiple Data）是单指令流多数据流的缩写。SW39000 众核处理器主核支持 256 位 SIMD 向量扩展指令，从核支持 512 位 SIMD 向量扩展指令。该功能的增加，不仅能够降低功耗，而且在一定程度上提升了指令级并行的能力。本节给出简单的 SIMD 向量编程介绍，详细的接口描述请参见附录 B。

对于标量与向量的区别，如图 11 所示，标量的运算粒度为单个元素，向量的运算粒度为一组有序的标量元素。一个向量中各个元素是互不相关的，对当前向量元素的操作结果不影响到其他向量元素。比如有一个向量，我们要使向量中的每一个元素 A_i 都乘以一个标量 b ，那么 b 与 A_1 相乘的结果不影响 A_2 的结果，各自独立。

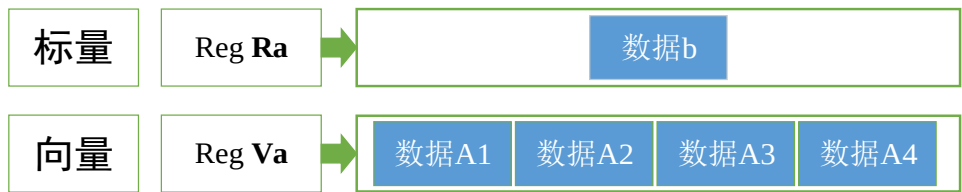


图 11 向量数据与标量数据对比

一条 SIMD 指令相当于一个小的循环，所以可以减少指令数，从而可以降低对指令访问带宽的要求，并且减少了由循环引起的控制相关。例如对于主核：

```
int a (N),b (N),c (N)
for(i=0; i < N; i++) {
    a[i]=b[i]+c[i]
}
```

上面的程序如果采用 SIMD 技术，则最终的实现相当于：

```
for(i=0; i<N; i=i+8){
    a[i+7:i]=b[i+7:i]+c[i+7:i]
}
```

可以发现，循环迭代次数减少为原来的八分之一，该循环的总指令数减少到原来的八分之一左右。

使用 SACA 的 SIMD 向量编程功能时，需要 `#include <simd.h>` 头文件，并在编译时加上 `-msimd` 选项。

3.4.2 串行程序向量化示例

一般情况下，串行程序向量化的着力点是程序中的循环。

```
int vertex[n], sum;
for (i = 0; i < n; i++){
    sum=sum+vertex[i];
};
```

对于上面的循环体，非常简洁规整，可以进行向量化。向量化一个循环一般可以分以下步骤：循环展开和分裂、变量替换、操作替换。

3.4.2.1 循环展开与分裂

对上面的循环根据向量宽度进行循环展开与分裂，以在主核上的实现为例，变化为如下的形式：

```
int sum_temp[8], vertex[n], sum;
for (i = 0; i < n/8; i += 8){
    for(j = 0; j < 8; j++){
        sum_temp[j] = sum_temp[j] + vertex[8*i+j];
    }
};
for(j = 0; j < 8; j++)
    sum = sum + sum_temp[j];
for (i = (n/8)*8; i < n; i++){
    sum=sum+vertex[i];
};
```

该例子中如果 n 不能被 8 整除，将最后不够一次向量操作的部分分裂出来，前面的部分就可以向量化了。

3.4.2.2 变量替换

声明扩展向量类型变量有两种策略，一种是在算法一级把关键变量都替换成扩展向量类型变量，这种方法改造程序彻底，但对程序改动较大；一种方法是在需要向量化的地方才引入扩展向量类型变量用于 SIMD 操作，该方法牵涉的地方较小，有利于快速向量化和调试。

```
int sum_temp[8]; → intv8 vsum;
int vertex[n]; → intv8 tx;
intv8 为主核的定点整形向量。
```

3.4.2.3 操作替换

若不是全局替换，在用 SIMD 操作替换普通操作之前，需要把标准类型变量映射（或复制）到扩展向量类型变量上，操作完之后，再把扩展类型向量变量映射（或复制）回标准类型变量。如：

```
simd_load (tx, &(vertex[8*i])); //将标准类型映射到扩展向量类型
simd_store (tx, &(vertex[8*i])); //将扩展向量类型映射到标准类型
```

通过上述步骤，对于示例的标量循环，最终的向量化程序如下：

```
int vertex[n], sum;
//扩展向量数据类型声明
intv8 vsum, tx;
//对能被 8 整除的部分进行向量化
for (i = 0; i < n/8; i += 8){
    //将 vertex[8*i]~ vertex[8*i+7]的连续 8 个标量从内存映射加载到向量中
    simd_load (tx, &(vertex[8*i]));
    //向量加法
    vsum = vsum + tx;
};
//调用扩展向量函数接口对向量中的 8 个标量进行规约求和
sum = simd_reduc_plusw(vsum);
//对于不能被 8 整除的部分仍然采用标量进行运算
for (i = (n/8)*8; i < n; i++){
    sum=sum+vertex[i];
};
```

3.4.3 对 C 语言的扩展

申威众核处理器的主从核都提供了较为丰富的 SIMD 指令。为了用户更好的使用 SIMD，SACA 在编译器中提供了一整套方案，通过直接定义数据类型、调用内部函数，或者通过编译器自动的向量识别，来满足用户的需求。

3.4.3.1 数据类型扩展

编译器所支持的向量数据类型及相关说明可参见 3.2.2 节内容。

3.4.3.2 对界

使用 SIMD 进行向量编程时，向量的访存操作对于内存地址有一定的对界要求，用户需要知道这些限制，否则程序容易出错，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

对于主核而言，向量类型的数据项在内存中除了 `floatv4` 类型是 16 字节对界，其余都是 32 字节对界。编译器保证用向量类型定义的变量在内存中是 32 字节（16 字节）对界的。包含向量类型的结构体（`struct`）/联合（`union`），编译器保证它们是 32 字节（或 16 字节）对界的（必要的时候在内部进行了垫塞）。对于从核而言，向量类型的数据项在内存中除了 `floatv8` 类型是 32 字节对界，其余都是 64 字节对界。编译器保证从核向量类型定义的变量在内存中是 64 字节（32 字节）对界的。包含向量类型的结构体（`struct`）/联合（`union`），编译器保证它们是 64 字节（或 32 字节）对界的（必要的时候在内部进行了垫塞）。

在主核架构中，不对界的向量 Load/Store 会引发异常，操作系统收到异常信号后就会将这些 Load/Store 拆分成标准类型的 Load/Store，从而大大降低性能。在从核架构中，不对界的向量 Load/Store 会引发不对界访存异常，程序会直接报错退出。因此在向量编程时，编译程序可以通过添加 `-faddress_align=n` 选项，使得所有类型数组、结构体、联合体等的首地址是 `n` 字节对界的。

3.4.3.3 对 C 操作与内部函数的扩展

对于 C 语言中针对标量数据类型的操作，绝大部分都可以直接应用于向量数据类型上，例如取大小的 `sizeof()`、赋值操作、取地址符号 `&x`，指针等。此外对于常用的运算符操作，也支持从标量到向量数据类型的扩展，例如基本的算术操作（`+`、`-`、`*`、`/`）、逻辑操作（`&`、`|`、`^`等）、移位操作（`>>`、`<<`）、比较操作（`==`、`<=`、`<`等）等，当然，针对主从核不同的处理器结构所支持的向量指令的不同，部分运算符并不是对所有的向量数据类型都支持，具体情况请参附录 B。

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。例如上面示例程序中的 `vsum = vsum + tx` 也可以通过 `vsum = simd_vaddw(vsum, tx)` 来实现，其效果是一样的。但是对于 `sum = simd_reduc_plusw(vsum)` 这样的向量规约求和操作，就没有办法通过通用的运算操作符来实现。运算符之外扩充的内部函数主要用于以下功能的实现上：

标准类型数据与扩展向量类型数据的映射存取，主要是针对硬件提供的向量访存指令；

向量数据类型的初始赋值操作；

向量数据类型的条件选择操作，整理操作等；

向量数据类型的打印功能。

3.4.3.4 数据类型转换

C 语言中支持标准数据类型之间的互相转换，扩展到向量数据类型之后，编译器同样支持向量数据类型之间以及向量数据类型与标量数据类型之间的转换。需要注意的是，与扩展向量数据类型相关的数据类型转换，每一种转换都有不同

的语义，用户需要加以区分。具体的不同的转换的含义请参见附录 B。

3.4.4 程序向量化限制

3.4.2 节给出了一个循环向量化的示例，但是并非所有的循环都可以向量化。循环的向量化会受到很多方面的限制。

1、硬件限制

向量化受限于硬件的限制。在 SIMD 扩展中，向量化访存操作必须是地址连续的，且要求一定的对界要求。这意味着虽然有些循环是可向量化的，但具体针对国产众核体系结构，需要进行程序变换后才能向量化。另外，由于定点向量类型的指令集是不完备的，例如不支持定点向量的乘法、除法等，这样的循环就没有办法向量化。

2、程序相关性限制

向量化的基础是相关性分析，循环要能够向量化，要求将要被向量化的几次循环迭代之间不能存在相关性。下面这样的循环是无法进行向量化的。

```
for (i=1; i<N-1; i++){  
    data[i] = data[i-1]*0.25 + data[i]*0.5 + data[i+1]*0.25;  
}
```

3、循环结构限制

循环可以是常用的 for，while-do 或 repeat-until 结构，也可以是使用 goto 和标号组成的。然而，循环必须是只有一个入口和一个出口时才能被向量化。

4、循环退出条件限制

循环退出条件决定循环执行的迭代次数。向量化需要迭代次数是可以计算的，也就是说迭代次数必须是下述表达式：

- 1) 常数；
- 2) 循环不变量；
- 3) 外层循环控制变量的线性函数。

3.5 辅助工具

本节介绍 SACA 编程环境的辅助工具，为用户调试、分析和优化代码提供帮助。

3.5.1 混合程序反汇编器

SACA 面向的异构众核架构主从核指令集不兼容，对于主从核混合程序，单纯使用主核反汇编器或者从核反汇编器都不能正确反汇编混合程序，因此 SACA 专门提供了针对混合程序的反汇编器 `swobjdump`。

`swobjdump` 能够区分混合程序中的主核代码与从核代码，并分别调用主核反汇编器与从核反汇编器进行反汇编处理，从而得到混合程序正确的反汇编代码。

`swobjdump` 只支持 `-D` 和 `-d` 选项，一次只能处理一个混合程序目标码，其中 `-D` 反汇编所有段，`-d` 只反汇编代码段。用法：

`swobjdump <-D/-d> <file>`，或者：

`swobjdump <file> <-D/-d>`

3.5.2 ldmreport

在 SW39000 众核处理器上，LDM 空间不允许超过 256k。`ldmreport` 工具可以检测 LDM 空间的使用量，并在 LDM 空间溢出 256k 时报错，超过 128k 时报警。

具体使用如图 12 所示：

这是一个 `ldm` 用例 `c.c`，里面定义了六个静态变量：

```
#include "slave.h"

#define LOOP 100

__thread_local volatile int local_rply;
__thread_local volatile int remote_rply;
//__thread_local long long a[16384 + 32];
__thread_local long long a[32];

__thread_local unsigned long long reg_tmp[2];
__thread_local unsigned long long la[LOOP];
__thread_local unsigned long Rb[LOOP], Rc[LOOP];
```

图 12 ldmreport 使用例图

首先对其进行编译，可执行 `swgcc -mslave -msimd -faddress_align=64 -c c.c`，将会得到 `c.o`。

然后执行 `ldmreport c.o`，将会输出如图 13 所示结果：

```

LDM local variable list is:
indx:0      addr:0x40      size:4      name:local_rply
indx:1      addr:0x44      size:4      name:remote_rply
indx:2      addr:0x80      size:256    name:a
indx:3      addr:0x180     size:16     name:reg_tmp
indx:4      addr:0x1c0     size:800    name:la
indx:5      addr:0x500     size:800    name:Rb
indx:6      addr:0x840     size:800    name:Rc
==== LDM local max align: 64B, data num: 7, total size: 2944B

```

图 13 ldmreport 输出结果

即输出每个变量所用 LDM 空间大小。如果总的 LDM 空间占用很大（如超过 128k），将会报警，如图 14 所示：

```

LDM local variable list is:
indx:0      addr:0x40      size:4      name:local_rply
indx:1      addr:0x44      size:4      name:remote_rply
indx:2      addr:0x80      size:131328  name:a
indx:3      addr:0x20180   size:16     name:reg_tmp
warning!!!the ldmsym.addr overflow 128k!!!
indx:4      addr:0x201c0   size:800    name:la
warning!!!the ldmsym.addr overflow 128k!!!
indx:5      addr:0x20500   size:800    name:Rb
warning!!!the ldmsym.addr overflow 128k!!!
indx:6      addr:0x20840   size:800    name:Rc
warning!!!the ldmsym.addr overflow 128k!!!
==== LDM local max align: 64B, data num: 7, total size: 134016B

```

图 14 ldmreport 报警信息

如果总的 LDM 空间占用过大，超过了 256k，将会有如错误!未找到引用源。所示的提示：

```

=====LDM INFO begin=====
LDM local variable list is:
indx:0      addr:0x40      size:800000  name:Rc
indx:1      addr:0xc3540   size:800000  name:Rb
warning!!!the ldmsym.addr overflow 128k!!!
indx:2      addr:0x186a40  size:800000  name:la
warning!!!the ldmsym.addr overflow 128k!!!
indx:3      addr:0x249f40  size:16      name:reg_tmp
warning!!!the ldmsym.addr overflow 128k!!!
indx:4      addr:0x249f80  size:256     name:a
warning!!!the ldmsym.addr overflow 128k!!!
indx:5      addr:0x24a080  size:4       name:remote_rply
warning!!!the ldmsym.addr overflow 128k!!!
indx:6      addr:0x24a084  size:4       name:local_rply
warning!!!the ldmsym.addr overflow 128k!!!

==== LDM local max align: 64B, data num: 7, total size: 2400448B
==== ldm_malloc start addr: 0x24a0c0
==== ldm_share start addr: 0x300000000000 align: 0B size: 0B
=====ALL LDM SIZE: 2400448B(private),0B(shared)]=====
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Your program's static ldm data overflow 256K, it cannot be loaded TO SW
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```

图 15 ldmreport 错误信息

3.5.3 checklen

从核 L1 指令 cache 典型配置为 32KB，当被调用函数的指令段大小超过该

容量时，会发生指令 **cache** 替换而导致性能下降。因此 SACA 提供了查看目标码中某从核函数及所有其调用函数指令段总大小的 **checklen** 工具。

checklen 工具用法如下：

checklen <a.out> <func_name>

checklen 能够自动识别<a.out>中的所有从核函数，从中递归地找出<func_name>函数及所有其调用函数，计算所有调用函数的起始地址及指令段大小，并在最后计算出所有调用函数指令段的总大小。用户可以根据上述输出判断<func_name>函数执行时是否发生指令 **cache** 替换，从而指导对程序性能进行调优。

checklen 工具使用实例如图 16 所示：

```
[swcomp@psn01 test]$ checklen a.out
-----
checklen a.out func_name
-----
[swcomp@psn01 test]$ checklen a.out main
-----
calldepth      func_name      start_addr      length
-----
0              main          00004ffff0012c00 64
1              th_printf     00004ffff0013ed0 1024
2              __GI_memset   00004ffff0013630 416
2              __GI_strcpy   00004ffff0014370 48
2              __GI_strlen  00004ffff0013b10 48
2              CRTS_ssig_queue 00004ffff00142d0 160
-----
The ICache occupation is: 1760 B
-----
```

图 16 **checklen** 工具使用示例

```

=====slave-core func stack depth analysis=====
stack depth for call graph root nodes:
slave___uclibc_main: 384
slave___GI_abort: 384
slave___libc_sigaction: 384
slave_th_printf: 320
slave___GI___libc_fcntl: 256
slave___GI___libc_open: 256
slave___GI_getgid: 128
slave___GI_getuid: 128
slave___syscall_open: 128
slave___GI_sigprocmask: 128
slave___GI_exit: 128
slave___raise: 128
slave___libc_getpid: 128
slave___GI_kill: 128
slave_CRTS_$$sig_queue: 128
slave_main: 64
slave___check_one_fd: 64
slave___GI___uclibc_init: 64
slave___GI___uclibc_fini: 64
slave___GI_getegid: 64
slave___GI_geteuid: 64
slave___GI_getenv: 64
slave___GI_exit: 64
slave___syscall_error: 64
slave_start: 16
slave___syscall_rt_sigaction: 16
slave___libc_creat: 0
slave_memcpy: 0
slave_memset: 0
slave_fortran_init: 0
slave_fortran_cleanup: 0
slave___GI_memcmp: 0
slave___sigismember: 0
slave___sigaddset: 0
slave___sigdelset: 0
___syscall_sigreturn: 0
___syscall_rt_sigreturn: 0
slave___GI_strcpy: 0
Maximum stack required is 384
=====

```

图 17 从核栈深度分析例图

3.5.4 从核栈深度分析选项

从核栈深度分析可以显示出用户程序从核函数的调用关系图，以及各个函数的最大栈深度，为用户优化程序提供参考。

支持单从核程序及主从混合程序。

使用方式（链接时使用选项）：

-Wl,--stack-analysis

将会按照从大到小顺序打印出所有函数调用关系图中根节点的栈深度以及整个程序的最大栈深度需求，如图 17 所示。

如果想查看函数调用关系及其他函数的详细栈深度信息，可以使用如下选项：

-Wl,-Map -Wl,mymapfile

在生成的 mymapfile 文件中可以看到如图 18 所示信息：


```

slave____stdio_transzw
slave__GI_printf: 320 1344
calls:
* slave__GI_vfprintf
slave_test: 400128 401472
calls:
* slave__GI_printf
slave_fun3: 64 64
slave_fun1: 64 64
slave_fun2: 64 128
calls:
* slave_fun1
slave_main: 64 401536
calls:
* slave_test
  slave_fun3
  slave_fun2
  slave_fun1

```

图 18 mymapfile 中信息

无法识别的栈深度情况包括：

- 1、函数指针；
- 2、嵌套函数调用将被忽略；
- 3、其他只有在运行时才能得知的动态栈深度调整。

3.5.5 从核栈在 LDM 上的溢出检查选项

当从核栈空间置于 LDM 上时，也即是程序运行时使用了作业管理 `bsub` 选项 `-b` 时，提供检查机制，为用户报告哪个函数的栈空间溢出到了 `ldm_malloc` 的动态空间或 LDM 的静态空间。

使用方式：

编译时使用选项 `-fcheck-ldm-stack`，只针对从核程序

单从核课题链接时增加 `-lslave`

作业管理 `bsub` 使用 `-b` 选项

程序运行时首先检查从核函数的栈空间是否溢出到 `ldm_malloc` 的动态空间，没有则继续检查从核函数的栈空间是否溢出到 LDM 的静态空间。

如果当程序调用函数 `fun1` 时发现 `fun1` 的栈空间已经溢出到 `ldm_malloc` 的动态空间内了，则会在 `fun1` 函数执行前就跳转到异常报错函数，打印如下：

```
Ldm_malloc space will be overflowed by stack on ldm [function pc: 4ffff0410488]
```

```
See function name : swaddr2line -f -e a.out [function pc]
```

以上打印说明程序在 `pc=0x4ffff0410488` 的地方开辟栈空间，栈空间溢出到了 `ldm_malloc` 动态空间内，使用命令 `swaddr2line -f -e a.out 0x4ffff0410488` 可以查到该 `pc` 对应的函数为 `slave_fun1`。

如果当程序调用函数 `fun1` 时发现 `fun1` 的栈空间已经溢出到 LDM 静态空间内了，则会在 `fun1` 函数执行前就跳转到异常报错函数，打印如下：

```
Stack on ldm will be overflowed [function pc: 4ffff0012ec4]
```

```
See function name : swaddr2line -f -e a.out [function pc]
```

以上打印说明程序在 `pc=0x4ffff0012ec4` 的地方开辟栈空间，栈空间溢出到

了 LDM 静态空间内，使用命令 `swaddr2line -f -e a.out 0x4fff0012ec4` 可以查到该 pc 对应的函数为 `slave_fun1`。

3.5.6 从核汇编器特殊选项

-Wa,-slave_disp_noalign

从核访存指令对偏移有对界要求，如 `ldl` 的偏移地址低 3 位[2:0]默认为 0，之前的处理是从核汇编器遇到偏移不满足对界要求的就报错退出，如“`ldl $1,4($2)`”，汇编器会报“Fatal error: disp[2:0] of ‘ldl’ is not 0”错。

实际应用中会出现下面这种情况，如对于“`ldl $1,4($2)`”，其中（`$2+4`）地址是满足 8 字节对界要求的，但是汇编器还是会报错。针对这种情况，即访存指令偏移不对界，但是用户保证基址+偏移满足对界要求的情况下，使用 `-Wa,-slave_disp_noalign` 选项，汇编器会处理成 `ldi`+访存两条指令。

如“`ldl $1,4($2)`”，使用 `-Wa,-slave_disp_noalign` 选项，汇编器处理成“`ldi $63, 4($2); ldl $1, 0($63)`”，其中 `$63` 为编译器保留的内部寄存器。

该选项默认打开。

3.5.7 基于控制流图的代码静态特征提取选项

SACA 支持代码静态特征提取。通过对代码控制流图的分析，以函数为单位提取代码中的静态特征。

编译选项：

-fextract-feature

示例：

\$ swgcc -fextract-feature example.c

代码静态特征的具体意义请参见附录 A。

3.5.8 version-spy 版本检测

SACA 中 `swgcc` 编译器版本更新较快，版本更新提供新功能或解决用户问题的同时，也可能会引入其他方面的问题，给用户使用带来困扰。为此我们提供 `version-spy` 版本检测工具，用来检测目标码中.o 与.a 文件是由哪个版本的编译器生成的，在用户遇到编译器版本相关的问题时，方便进行错误或者异常定位。

工具使用方法：

`version-spy <a.out>`

打印示例如图 19 所示：

```

=====
Library info:
[0]:/usr/sw/swgcc/swgcc710-tools-SEA-1079/usr/bin/../../../../swgcc710-to
=>size:928770 time:Thu Dec 24 21:58:37 2020

[1]:/usr/sw/swgcc/swgcc710-tools-SEA-1079/usr/bin/../../../../swgcc710-to
=>size:480580 time:Thu Dec 24 21:58:37 2020

[2]:/usr/sw/swgcc/swgcc710-tools-SEA-1079/usr/bin/../../../../swgcc710-to
=>size:1596048 time:Thu Dec 24 21:53:35 2020

[3]:/usr/sw/swgcc/swgcc710-tools-SEA-1079/usr/bin/../../../../swgcc710-to
=>size:441956 time:Thu Dec 24 21:55:35 2020

=====

swgcc version:
==>.o s were made by different compiler:
[0]GCC: (GNU) 3.3.2
[1]GCC: (GNU) 7.1.0 20170502 (swgcc-1079 Committer: qianh <qianh@examp]
=====

```

图 19 version-spy 版本检测打印例图

第四章 性能指导

本章介绍 SACA 中常用的优化支持，为用户在神威平台开发众核应用提供性能指导，包括循环展开、乘加融合等传统优化，也包括针对申威众核处理器架构开发的编译优化。更为详细的编译选项和优化支持可参见附录 A。

4.1 循环展开优化

编译器支持多种场景的循环展开优化，常用优化选项包括以下几种：

1、**-funroll-loops** 循环展开优化

示例：

```
$ swgcc -funroll-loops example.c
```

2、**-funroll-all-loops** 激进的循环展开优化

示例：

```
$ swgcc -funroll-all-loops example.c
```

3、**-funroll-nest-loops** 面向多重循环结构的循环展开优化

示例：

```
$ swgcc -funroll-nest-loops example.c
```

4、**--param max-unroll-times=n** 设定最大的循环展开次数，默认为 8

示例：

```
$ swgcc -funroll-loops --param max-unroll-times=16 example.c
```

4.2 乘加融合优化

编译器支持对乘加操作进行指令融合优化，利用硬件乘加单元，将乘法和加法两条指令合并为一条乘加指令。

编译选项：

-mfma

示例：

```
$ swgcc -mfma example.c
```

4.3 自动向量化

编译器支持主从核的自动向量化功能，使用方式如下：

编译时在-O2 选项优化的基础上加上-msimd -ftree-vectorize 选项或者直接使用-O3 -msimd 选项即可打开编译器的自动向量化功能。

编译选项：

-msimd -ftree-vectorize/ -O3 -msimd

示例：

- 1、\$ swgcc -msimd -ftree-vectorize example.c
- 2、\$ swgcc -O3 -msimd example.c

4.4 结构体重组优化

编译器支持结构体重组优化，将访问次数较多的热点结构体域剥离，组成新的结构体，增强数据访问的局部性。“访问次数较多”的阈值可配置，默认为 90%。

编译选项：

- 1、-fipa-type-escape -fipa-struct-peeling
- 2、--param struct-reorg-cold-struct-ratio=<value>

注：<value>取值介于 0-100，默认为 10

示例：

- 1、\$ swgcc -fipa-type-escape -fipa-struct-peeling example.c
- 2、\$ swgcc -fipa-type-escape -fipa-struct-peeling --param struct-reorg-cold-struct-ratio=20 example.c

4.5 从核 cache 预取优化

编译器支持对连续数组访问进行预取，将主存中数据预取到从核 cache 中。

编译选项：

- fprefetch-loop-arrays

示例：

- \$ swgcc -mslave -fprefetch-loop-arrays example.c

附加说明：

- 1、预取优化默认打开循环展开优化，可以通过调整循环展开次数影响预取效果。
- 2、预取参数可以通过以下编译参数进行调整：
 - --param prefetch-latency-slave=n 设定预取延迟为 n 条指令，默认为 800。该参数影响预取距离。
 - --param simultaneous-prefetches-slave=n 同时预取的指令数为 n，默认为 15。
 - --param l1-cache-slave-size=n 从核 L1cache 大小，单位为 KB，默认为 32。

4.6 从核 cache 写暗示优化

编译器支持从核 cache 的写暗示优化，对连续大块数据的 cache 写操作进行

优化，提高 cache 访问性能。

编译选项：

-fcache-hint

示例：

\$ swgcc -mslave -fcache-hint slave1.c slave2.c

4.7 从核 cache 属性动态配置

编译器支持从核变量的可 cache 属性和不可 cache 属性的动态配置，通过编译选项加编译指导语句配合使用。

编译选项：

-fdynamic-uncache-symbols

编译指导语句：区分数据默认是 cache 和 uncache 的两种情况，设置两组编译指导语句。

1、数据默认为 cached

#pragma uncache “xxx” ----将符号 xxx 访问变换为 uncache 访问

#pragma resume_cache “xxx” ----将符号 xxx 访问恢复为 cache 访问

2、数据默认为 uncached

#pragma cache “xxx” ---将符号 xxx 访问变换为 cache 访问

#pragma resume_uncache “xxx” ---将符号 xxx 访问恢复为 uncache 访问

3、编译指导语句参数说明

#pragma uncache “aa” 将符号 aa 的访问替换为 uncache 访问

#pragma uncache “aa@@foo” 将函数 foo 内符号 aa 的访问替换为 uncache 访问

示例：

编译命令

\$ swgcc -mslave -fdynamic-uncache-symbols slave.c

编译指示字

```
1 #include "stdio.h"
2 #include <slave.h>
3
4 int aa=3;
5 __uncached int dd=4;
6 __uncached int bb=5;
7 __uncached int cc[20];
8
9 int foo(){
10     #pragma cache "dd@@foo"
11     flush_slave_cache();
```

```

12      int ret = aa+dd;
13      #pragma resume_uncache "dd@@foo"
14      flush_slave_cache();
15      return ret;
16 }
17
18 int main()
19 {
20     printf("dd is %d\taa is 0x%x\n", dd, &aa);
21     aa = dd+4;
22     #pragma uncache "aa@@main"
23     flush_slave_cache();
24     printf("dd is %d\taa is 0x%x\n", dd, &aa);
25     aa = bb+5;
26     cc[10] += foo();
27     #pragma resume_cache "aa@@main"
28     #pragma cache "bb"
29     flush_slave_cache();
30     printf("bb is %d\tbb addr is 0x%x\n", bb, &bb);
31
32     #pragma cache "cc"
33     flush_slave_cache();
34     for(int i=0; i<20; i++)
35         printf("%d\n", cc[i]);
36 }

```

图 20 从核 cache 属性动态配置编译指示字例图

附加说明：

- 该机制面向对处理器体系结构较为熟悉的高级用户。
- 该机制面向从核 cache 结构。
- 该机制不保证数据访问的正确性，需要在属性变化前后添加 cache 刷新语句或者 cache 淘汰语句来保证 cache 与主存之间的数据一致性。
- 该机制采用基于 pragma 实现的文本分析策略，一组编译指导语句确定了变换的空间，在此空间内的符号访问被转换为 uncache/uncached 访问，在此空间外，不做任何变换。
- “#prama resume_xxx”为可选语句，如果不添加，则表示对应的”#prama xxx”作用范围将持续到文本结尾；若为“#prama xxx@@yyy”，则作用范围将持续到函数 yyy 文本结尾。（此处 xxx 表示 cache/uncache）。

4.8 从核代码链接时优化

编译器支持从核代码链接时优化，需要在编译从核文件及-mhybrid 链接时使用。

编译选项：

-mslave-lto

示例：

\$ swgcc -mhost -c host.c

\$ swgcc -mslave -mslave-lto -c slave1.c slave2.c

\$ swgcc -mhybrid -mslave-lto host.o slave1.o slave2.o

4.9 从核代码分支预测优化

编译器支持从核代码分支预测优化，通过自动代码插桩，改变条件转移指令的预测方向，降低因分支预测失败导致的性能损失。

基于硬件支持，实现了两套分支预测优化机制：一是基于 hint 指令的分支预测优化，一是基于 pws 指令的分支预测优化。

编译选项：

1、-fbranch-predict-hint

2、-mpws

示例：

1、\$ swgcc -mslave -fbranch-predict-hint slave1.c slave2.c

2、\$ swgcc -mslave -mpws slave1.c slave2.c

4.10 Fortran module 访存优化

编译器支持将 Fortran 语言以 ldm__开头的 module 中的变量放在 LDM 中，加速变量访问。

编译选项：

-fmodule-ldm

示例：

\$ swfortran -fmodule-ldm example.F90

4.11 功耗相关优化

编译器支持面向 SW39000 众核处理器特定结构的功耗优化，包括以下几种功耗优化选项：

1、-freg-bypass

面向寄存器旁路的指令调度优化。只支持从核代码，在-O2、-O3 情况下使能。

示例：**\$ swgcc -mslave -freg-bypass slave.c**

2、-mslave-l0cache

基于 L0 指令 cache 的指令调度优化。只支持从核代码，在循环展开优化遍（例如-funroll-loops）使能的情况下有效。

示例：**\$ swgcc -mslave -mslave-l0cache slave.c**

3、-flow-power

面向控制总线翻转的指令调度优化。只支持从核代码，在指令调度优化遍（例如-fschedule-insns、-fschedule-insns2）使能的情况下有效。

示例：**\$ swgcc -mslave -flow-power slave.c**

4.12 快速数学库

SACA 提供面向 SW39000 处理器体系结构特征优化的高性能快速数学库，使用方式如下：

链接时使用-lm（主核）、-lm_slave（从核）默认使用高性能数学库。

链接选项：

-lm（主核）、-lm_slave（从核）

示例：

\$ swgcc -mhost host1.o host2.o -lm

\$ swgcc -mslave slave1.o slave2.o -lm_slave

\$ swgcc -mhybrid host.o slave.o -lm -lm_slave

4.13 数学库函数细粒度多版本切换

SACA 同时提供 GNU 标准数学库和面向申威众核处理器优化的高性能数学库，受限于二进制浮点数据表示的精度误差，在特定条件下，二者的计算结果无法实现完全一致。通过编译选项配置，用户可以自定义配置代码中调用的部分数学函数的实现版本，辅助用户在代码移植过程中对程序正确性进行验证。

该功能只支持从核代码。在-fmath-opt 不使能情况（默认情况）下，从核链接高性能数学库；在-fmath-opt 使能情况下，从核链接标准数学库，-fmath-opt=<str> 指定的数学函数链接高性能数学库中的函数实现。

编译选项：

-fmath-opt=<str>

示例：

\$ swgcc -mslave -fmath-opt=sin,exp,cos slave1.c slave2.c -lm_slave

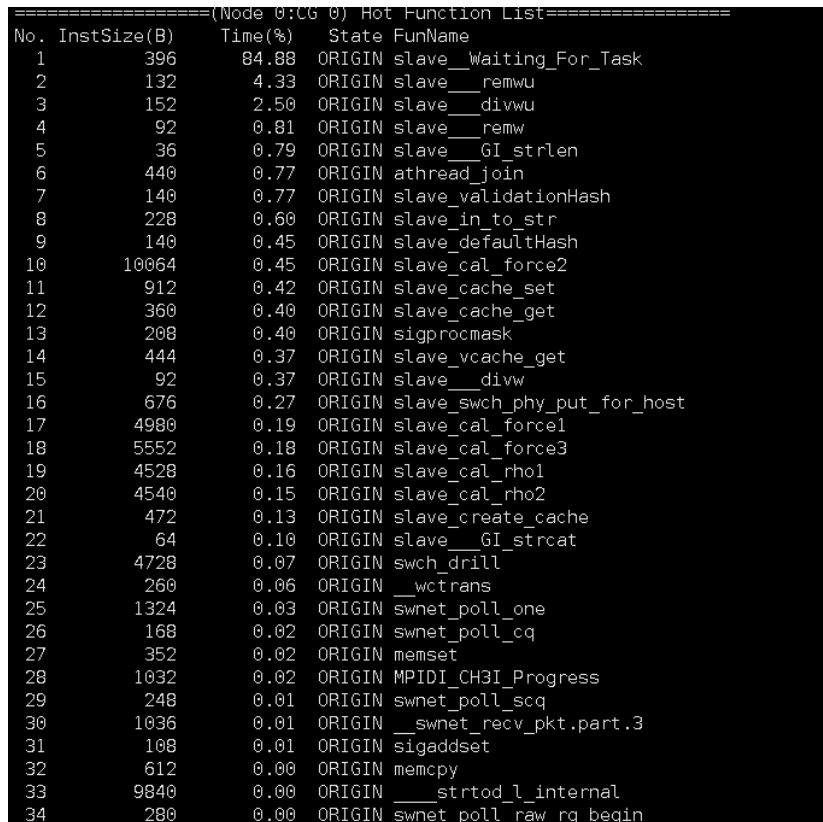
4.14 swprof 热点函数分析

SACA 提供一种基于可执行文件的热点函数分析工具 **swprof**，实现类似 **-pg** 选项的功能，但用户程序却不需要重新加选项编译。**swprof** 工作模式由作业管理 **bsub** 的参数 **-swrunarg “-P[arg]”** 或者 **-swrunarg “-p”** 指定，**-P (--prof)** 选项支持三个参数：“all”、“master”、“slave”，指示加载器进入不同的程序剖析模式，获取对应的热点函数。

- **all**: 对所有主从代码进行剖析，获取主从热点函数。
- **master**: 只对主核代码进行剖析，获取主核热点函数。
- **slave**: 只对从核代码进行剖析，获取从核热点函数。

-p (--pg) 该选项用于对程序进行热点函数分析，**-p** 等价于 **-P all**，对所有主从代码进行剖析。

使能 **swprof** 后，将在用户程序运行结束后在屏幕输出热点函数统计信息，如图 21 所示：



No.	InstSize(B)	Time(%)	State FunName
1	396	84.88	ORIGIN slave__Waiting_For_Task
2	132	4.33	ORIGIN slave__remwu
3	152	2.50	ORIGIN slave__divwu
4	92	0.81	ORIGIN slave__remw
5	36	0.79	ORIGIN slave__GI_strlen
6	440	0.77	ORIGIN athread_join
7	140	0.77	ORIGIN slave_validationHash
8	228	0.60	ORIGIN slave_in_to_str
9	140	0.45	ORIGIN slave_defaultHash
10	10064	0.45	ORIGIN slave_cal_force2
11	912	0.42	ORIGIN slave_cache_set
12	360	0.40	ORIGIN slave_cache_get
13	208	0.40	ORIGIN sigprocmask
14	444	0.37	ORIGIN slave_vcache_get
15	92	0.37	ORIGIN slave__divw
16	676	0.27	ORIGIN slave_swch_phy_put_for_host
17	4980	0.19	ORIGIN slave_cal_force1
18	5552	0.18	ORIGIN slave_cal_force3
19	4528	0.16	ORIGIN slave_cal_rho1
20	4540	0.15	ORIGIN slave_cal_rho2
21	472	0.13	ORIGIN slave_create_cache
22	64	0.10	ORIGIN slave__GI_strcat
23	4728	0.07	ORIGIN swch_drill
24	260	0.06	ORIGIN __wctrans
25	1324	0.03	ORIGIN swnet_poll_one
26	168	0.02	ORIGIN swnet_poll_cq
27	352	0.02	ORIGIN memset
28	1032	0.02	ORIGIN MPIDI_CH3I_Progress
29	248	0.01	ORIGIN swnet_poll_scq
30	1036	0.01	ORIGIN __swnet_recv_pkt.part.3
31	108	0.01	ORIGIN sigaddset
32	612	0.00	ORIGIN memcpy
33	9840	0.00	ORIGIN __strtod_l_internal
34	280	0.00	ORIGIN swnet_poll_raw_rq_begin

图 21 swprof 热点函数统计例图

除了 **-P** 和 **-p** 外，**swprof** 还提供以下选项：

- **-n (--hf_number)** 指定统计的热点函数数目，将只统计前 **n+1** 个热点函数。
- **-N (--profnode)** 指定参与统计热点函数的节点列表（逻辑节点号从小到大），例如‘0,2,4-5’。
- **-f (--hf_reorder)** 针对从核热点函数，进行热点函数代码段重排优化以缓解指令 **cache** 冲突，该优化需要和 **-p** 或 **-P** 选项同时使用才有效

果。加载器会将重排后的代码段重新写回原目标码，用户重新提交运行程序（无需再指定热点函数相关选项），可以得到优化后的执行情况。使用该选项后，将在热点函数统计信息之后打印重排的函数信息，如图 22 所示：

```
===== (Node 0:CG 0) End of HF List =====  
  
Reorder function slave_Waiting_For_Task, move from 0x4ffff041b6ec to 0x4ffff03f7390  
Reorder function slave__remwu, move from 0x4ffff041cd80 to 0x4ffff03f7520  
Reorder function slave__divwu, move from 0x4ffff041cb90 to 0x4ffff03f75b0  
Reorder function slave__GI_strlen, move from 0x4ffff041f030 to 0x4ffff03f7650  
Reorder function slave_validationHash, move from 0x4ffff0418660 to 0x4ffff03f7680  
Reorder function slave__remw, move from 0x4ffff041ce08 to 0x4ffff03f7710  
Reorder function slave_in_to_str, move from 0x4ffff04188d0 to 0x4ffff03f7770  
Reorder function slave_defaultHash, move from 0x4ffff04185d0 to 0x4ffff03f7860  
Reorder function slave_cal_force2, move from 0x4ffff04146b0 to 0x4ffff03f78f0  
Reorder function slave_cache_set, move from 0x4ffff04189c0 to 0x4ffff03fa040  
Reorder function slave_vcache_get, move from 0x4ffff0418410 to 0x4ffff03fa3d0  
Reorder function slave__divw, move from 0x4ffff041cc28 to 0x4ffff03fa590  
Reorder function slave_cache_get, move from 0x4ffff0418d50 to 0x4ffff03fa5f0  
Reorder function slave_swch_phy_put_for_host, move from 0x4ffff0418ec0 to 0x4ffff03fa760  
Reorder function slave_cal_force1, move from 0x4ffff0413330 to 0x4ffff03faa10  
Reorder function slave_cal_force3, move from 0x4ffff0416e00 to 0x4ffff03fbd90  
Reorder function slave_cal_rho1, move from 0x4ffff0410890 to 0x4ffff03fd340  
Reorder function slave_cal_rho2, move from 0x4ffff0411a40 to 0x4ffff03fe4f0  
Reorder function slave_create_cache, move from 0x4ffff04186f0 to 0x4ffff03ff6b0  
Reorder function slave__GI_strcat, move from 0x4ffff041eff0 to 0x4ffff03ff890  
Reorder function slave_cal_df, move from 0x4ffff0412c00 to 0x4ffff03ff8d0  
=====
```

图 22 热点函数重排信息例图

- **-D(--ipad)** 在从核热点函数重排优化过程中，进行指令垫塞优化以进一步缓解指令 cache 冲突，该优化需要和 **-p/-P** 以及 **-f** 选项一起使用才有效果。该选项需指定进行指令垫塞拷贝的份数，取值范围 0~8，超出范围将使用默认值 0。使用该选项后，与 **-f** 选项类似，将在热点函数统计信息之后打印多次拷贝重排的热点函数信息。
- **-U(--user_hf_reorder)** 该选项执行用户自定义热点函数重排优化，不通过热点函数统计，用户自行指定需要重排的从核热点函数名。需要提供函数名列表参数，例如‘slave_k1,slave_k2’。该选项将在用户程序运行之前执行热点函数重排，重写目标码，并打印出相关重排信息，用户无需重新运行目标码，本次运行就将得到优化后的执行情况。
- **-Q(--swprof_quiet)** 该选项打开后将不在屏幕输出关于 **swprof** 的打印信息，而是将其输出到以 **swprof_nodeid_cgid.log** 方式命名的文件中。

附加说明：

- 使用 **-mdynamic** 选项链接生成的目标码无法使用 **swprof** 热点函数分析功能。
- 不建议将用户指定的热点函数重排优化 **-U** 和自动的重排或垫塞优化 **-f/-D** 一块使用，因为互相影响将导致无法预知的结果。
- 所有的相关优化只会执行一次，即优化过的目标码再次执行 **swprof** 相关优化将不起作用。如需再次尝试，请重新编译生成目标码。

附录 A: swgcc 编译器常用编译选项

本附录描述 swgcc 编译器支持的常用编译选项。按照章节顺序分为功能选项、性能选项、调试选项以及其他选项。更全的编译选项可通过 `man gcc` 命令查看。

本附录还对“神威·太湖之光”系统编译器（swcc、swf90）的部分选项进行了比对。

A.1 功能选项

A.1.1 -mhost

描述:

指示编译器生出 SW39000 众核处理器主核代码。

备注:

- 1、打开预定义宏 `_SIMD_HOST`;
- 2、swgcc 缺省使用 `-mhost` 生成主核代码;
- 3、对应 swcc 编译器的 `-host` 选项。

A.1.2 -mslave

描述:

指示编译器生出 SW39000 众核处理器从核代码。

备注:

- 1、打开预定义宏 `_SIMD_SLAVE`;
- 2、禁止同时打开 `-mhost` 和 `-mslave` 选项，否则编译器报错退出;
- 3、对应 swcc 编译器 `-slave` 选项。

A.1.3 -mhybrid

描述:

指示链接器进行主从核代码混合链接，链接对象是目标代码.o 文件。

备注:

对应 swcc 编译器 `-hybrid` 选项。

示例:

```
swgcc -mhybrid master.o slave.o -o ./a.out
```

A.1.4 -msimd

描述:

指示编译器支持 SIMD 扩展编程接口，生成相应的 SIMD 扩展指令。

备注:

对应 swcc 编译器-simd 选项，不再包含地址对界功能，对界功能由下面的-faddress_align=n 选项完成。

A.1.5 -mdynamic

描述:

编译器支持动态链接功能，用以支持人工智能类应用对动态编译及运行的需求，提升系统的通用性。

备注:

由于动态库的 LDM 空间相互覆盖，以及从核短 pc24 位的限制，用户需要把代码链接成一个动态库，多个动态库可能会运行出错。用户代码分成多个库的，需要在编译时加-fPIC 选项生成位置无关的.o 文件，ar 压成.a 文件，将所有的.a 文件和其他.o 文件链接成一个动态库。

示例:

主核代码 master.c、动态库主核代码 lib_master.c、动态库从核代码 lib_slave.c，以-mdynamic 模式动态链接。

```
swgcc -mhost -fPIC -c lib_master.c
```

```
swgcc -mslave -fPIC -c lib_slave.c
```

```
swgcc -mdynamic -shared lib_master.o lib_slave.o -o libmix.so
```

生成主从混合的动态库

```
swgcc -mhost -c master.c
```

```
swgcc -mdynamic master.o -L. -lmix -o mix.out
```

生成主从混合的动态目标码 mix.out

A.1.6 -faddress_align=n

描述:

指示编译器将所有数组、向量、结构体、联合体等的首地址按照 nB 对界。

备注:

该选项剥离出原来的-msimd 选项的对界功能，一般情况下，主核程序设置 n 为 32，从核程序设置 n 为 64。

A.1.7 -c

描述:

指示编译器只对源文件进行编译和汇编，生成目标文件，不进行链接。

备注:

- 1、源文件格式为‘.c’，‘.i’，‘.s’等，目标文件格式为‘.o’；
- 2、对应 swcc 编译器-c 选项。

A.1.8 -S

描述:

指示编译器只对源文件进行编译，生成汇编文件，不进行汇编和链接。

备注:

- 1、源文件格式为‘.c’，‘.i’等，汇编文件格式为‘.s’；
- 2、对应 swcc 编译器-S 选项。

A.1.9 -E

描述:

指示编译器仅对源文件进行预处理，不进行编译、汇编和链接。

备注:

- 1、源文件格式为‘.c’等，预处理后的源文件格式为‘.i’；
- 2、对应 swcc 编译器-cpp 或 swf90 编译器-ftpp 选项。

A.1.10 -o *file*

描述:

指示编译器将输出内容存放在命名为 *file* 的文件中。

备注:

- 1、该选项在生成预处理的源文件、汇编文件、目标文件及可执行文件的过程均可使用；
- 2、对应 swcc 编译器-o 选项。

A.1.11 -fcheck-lbm-stack

描述:

当从核栈空间放在 LDM 上时，运行过程中每个从核函数一开始先检测栈空间是否会溢出到用户 LDM 空间，如果溢出，则进入报错函数，否则正

常执行。

A.1.12 -fstack-protector

描述:

启用堆栈保护功能，但是只为局部变量包含 `char` 类型数组的函数或者含有 `alloca` 调用的函数打开。

备注:

该选项包含以下变体:

- 1、`-fstack-protector-all`: 为所有函数打开堆栈保护功能;
- 2、`-fstack-protector-strong`: 使用更加聪明的策略为某些函数打开堆栈保护功能;
- 3、`-fstack-protector-explicit`: 仅为含有 “`stack_protect`” attribute 的函数打开堆栈保护功能，如: `void test() __attribute__((stack_protect))`。

A.1.13 -fmax-stack-var-size=*n*

描述:

指定可以被放在栈上的静态数据的空间的最大值，单位为 B。

备注:

- 1、该选项为 `swgfortran` 选项;
- 2、不指定的情况下，*n* 的默认大小为 32768。

A.1.14 -fstack-arrays

描述:

指示编译器将所有局部数组存放到栈空间，即使局部数组的大小未知。

备注:

- 1、该选项为 `swgfortran` 选项;
- 2、当打开 ‘`-Ofast`’ 优化选项时，该选项默认打开。

A.1.15 -finit-local-zero

描述:

指示编译器将 `INTEGER`、`REAL` 以及 `COMPLEX` 变量置零，逻辑变量置为 `false`，`CHARACTER` 变量置为一个空字符串。

备注:

- 1、该选项为 `swgfortran` 选项;

2、对应的 swf90 编译器-zerouv 选项。

示例：

swgfortran -S -O0 -finit-local-zero a.f90 -o 1.s 得到 1.s。

A.1.16 -ffree-form

描述：

强制对源文件按照自由格式进行语法分析，默认.f90、.F90、.f95、.F95 文件被认为自由格式。

备注：

- 1、该选项为 swgfortran 选项；
- 2、对应 swf90 编译器-freeform 选项。

示例：

创建一个 b.f 文件，该源文件不是自由格式文件，因此正常情况下不加 -ffreeform 选项无法对其进行语法分析。swgfortran -O0 b.f 得到结果报 error；swgfortran -O0 -ffree-form b.f 能够编译完成，得到可执行文件，且保证正确性。

A.1.17 -fconvert='big-endian'

描述：

输入输出设置为大端格式。

备注：

- 1、该选项为 swgfortran 选项；
- 2、对应 swf90 编译器-convert big_endian 选项；
- 3、该选项的形式为-fconvert=conversion, conversion 可以为 native, swap, big-endian, little-endian；
- 4、-fconvert='big-endian' 这个选项是编译选项，相当于运行时的 GFORTRAN_CONVERT_UNIT='big_endian' ./a.out。

示例：

swgfortran -fconvert='big-endian' -S -O0 b.f90 -o 1.s 得到文件 1.s。

A.1.18 -mieee

描述：

指示编译器允许使用产生精确结果的任何操作。对于非精确结果的情况会影响精度。例如，X*0 可以换成 0，X/X 可以换成 1，虽然当 X 为+inf, -inf 或 NaN 时结果是不准确的。

备注:

- 1、该选项仅适用于主核程序，从核硬件支持非规格化数，不需要使用该选项；
- 2、对应 swf90 编译器-OPT:IEEE_arith=2 选项。

示例:

```
swgfortran -S -O0 -mieee c.f90 -o 1.s  
swgcc -S -O0 -mieee a.c -o 1.s
```

A.1.19 -mieee-with-inexact

描述:

指示编译器严格遵从 IEEE 754 标准。

备注:

- 1、该选项仅适用于主核程序，从核硬件支持非规格化数，不需要使用该选项；
- 2、对应 swf90 编译器-OPT:IEEE_arith=1 选项。

A.1.20 -mftz

描述:

指示编译器在发生下溢时，将真零（64 位全“0”）写入目标寄存器，同时将非规格化的源操作数当做有符号的“0”处理。

备注:

- 1、该选项仅适用于主核程序，从核硬件支持非规格化数，不需要使用该选项；
- 2、该选项仅在打开-mieee 或-mieee-with-inexact 选项时有效；
- 3、对应 swcc 编译器-ftz 选项。

A.1.21 -masm-volatile

描述:

指示编译器将程序中所有的嵌汇编设置 volatile 属性，不允许对其进行优化。

备注:

用户程序员在使用嵌汇编时容易出现低级错误，查错难度大，如果程序中有嵌汇编且没有加上 volatile 属性，运行出错时可以先加上该选项排除错误。

A.1.22 -fgnu89-inline

描述:

指示编译器识别旧标准内联函数关键字“inline”。

备注:

内联函数 `inline` 是旧标准，新标准是 `__inline__`。旧标准可以加 `-fgnu89-inline` 识别。编译器会内部识别函数是否会内联，如果要强制内联需要加 `__attribute__((__always_inline__))`。

A.1.23 -fno-tree-ch

描述:

该选项指示编译器关闭循环头部拷贝优化。循环头部拷贝优化在 `-O1` 选项及以上打开。

备注:

用户在循环体内部写嵌汇编，嵌汇编里使用标号，若打开循环头部拷贝优化，则 `pass_tree_ch` 优化遍会把这个循环体的头部提到循环体外预先执行，导致嵌汇编代码也被复制，报标号重复定义。优化遍本身没有问题，主要是编译器对嵌汇编代码不做任何处理。使用 `-fno-tree-ch` 选项关闭该优化可以解决。

A.1.24 -fstrict-aliasing

描述:

该选项指示编译器使用更严格的别名检查。该选项在 `-O3` 以上优化下打开。

备注:

某些课题中存在不同类型的指针和变量的别名关系，更严格的别名检查机制会认为这些指针和变量不是别名，编译器认为没有关系，值并没有被修改，所以被优化掉，导致 `O3` 结果错误。使用 `-fno-strict-aliasing` 选项关闭该优化可解决。

A.1.25 -fno-common

描述:

该编译指示将 `comm` 或者 `bss` 的未初始化变量强制初始化 0，这样程序中的多重符号定义将直接报错退出。

A.1.26 -funconstrained-commons

描述:

指示编译器在 `common` 块中声明的变量未来可能会被更长的数据覆盖，阻止编译器对超出 `common` 块边界的赋值语句进行优化。

备注:

该选项为 `swgfortran` 选项。

A.1.27 -fdec-math

描述:

指示编译器兼容旧标准的三角函数接口，例如 `sind` 和 `cosd` 等。

备注:

该选项仅适用于 `swgfortran` 编译器。

A.1.28 -Werror=xxx

描述:

指示编译器将 `xxx` 类型的警告信息当做错误信息报出，提示用户修改程序。

备注:

1、支持的警告类型包括:

- Waddress
- Warray-bounds=1 (only with '-O2')
- Wbool-compare
- Wbool-operation
- Wc++11-compat -Wc++14-compat
- Wchar-subscripts
- Wcomment
- Wduplicate-decl-specifier (C and Objective-C only)
- Wenum-compare (in C/ObjC; this is on by default in C++)
- Wformat
- Wint-in-bool-context
- Wimplicit (C and Objective-C only)
- Wimplicit-int (C and Objective-C only)
- Wimplicit-function-declaration (C and Objective-C only)
- Winit-self (only for C++)
- Wlogical-not-parentheses
- Wmain (only for C/ObjC and unless '-ffreestanding')
- Wmaybe-uninitialized

- Wmemset-elt-size
- Wmemset-transposed-args
- Wmisleading-indentation (only for C/C++)
- Wmissing-braces (only for C/ObjC)
- Wnarrowing (only for C++)
- Wnonnull
- Wnonnull-compare
- Wopenmp-simd
- Wparentheses
- Wpointer-sign
- Wreorder
- Wreturn-type
- Wsequence-point
- Wsign-compare (only in C++)
- Wsizeof-pointer-memaccess
- Wstrict-aliasing
- Wstrict-overflow=1
- Wswitch
- Wtautological-compare
- Wtrigraphs
- Wuninitialized
- Wunknown-pragmas
- Wunused-function
- Wunused-label
- Wunused-value
- Wunused-variable
- Wvolatile-register-var

- 2、建议用户程序加上-Werror=implicit-function-declaration, 以防止出现返回值类型匹配错误的问题;
- 3、建议用户程序在-O3 情况下或者-fstrict-aliasing 使能情况下, 开启-Wstrict-aliasing, 以防止代码中违反规则的代码被错误地优化掉, 从而导致程序结果出错。

A.1.29 --params bbss_data_min_size=xxx

描述:

设置放入 bbss 段内数据大小的最小阈值, 默认为 1024G。

备注:

由于 bss 段内数据过大会导致 gp 寻址无法访问部分数据, 新设置 bbss 段用于存在部分体积较大的数据, 数据大小的值通过编译选项配置。

A.1.30 -fextract-feature

描述:

基于控制流图的代码静态特征提取，对代码的静态分析提供支持。

备注:

- 1、以函数为单位进行静态特征收集；
- 2、特征参数含义如下表所示，共 65 项静态特征。

表 11 静态特征参数含义

标号	类型	描述
ft1	CFG(basic blocks)	Number of basic blocks in the method
ft2	CFG(basic blocks)	Number of basic blocks with a single successor
ft3	CFG(basic blocks)	Number of basic blocks with two successors
ft4	CFG(basic blocks)	Number of basic blocks with more then two successors
ft5	CFG(basic blocks)	Number of basic blocks with a single predecessor
ft6	CFG(basic blocks)	Number of basic blocks with two predecessors
ft7	CFG(basic blocks)	Number of basic blocks with more then two predecessors
ft8	CFG(basic blocks)	Number of basic blocks with a single predecessor and a single successor
ft9	CFG(basic blocks)	Number of basic blocks with a single predecessor and two successors
ft10	CFG(basic blocks)	Number of basic blocks with a two predecessors and one successor
ft11	CFG(basic blocks)	Number of basic blocks with two successors and two predecessors
ft12	CFG(basic blocks)	Number of basic blocks with more then two successors and more then two predecessors
ft13	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft14	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft15	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft16	CFG(edges)	Number of edges in the control flow graph
ft17	CFG(edges)	Number of critical edges
ft18	CFG(edges)	Number of abnormal edges
ft24	Instructions	Number of instructions in (method, loops)
ft25	Instructions	Average of number of instructions in basic blocks
ft19	Instructions	Number of direct calls in (method, loops)
ft39	Instructions	Number of indirect calls (i.e. done via pointers) in (method, loops)
ft20	Instructions	Number of conditional branches in (method, loops)
ft56	Instructions	Number of unconditional branches in (method, loops)
ft33	Instructions	Number of switch instructions in (method, loops)
ft21	Instructions	Number of assignments in (method, loops)
ft35	Instructions	Number of instructions that do pointer arithmetic in (method, loops)

ft22	Instructions	Number of binary integer operations in (method, loops)
ft23	Instructions	Number of binary floating point operations in (method, loops)
ft34	Instructions	Number of unary operations in (method, loops)
ft36	Instructions	Number of indirect references via pointers ('*' in C)
ft37	Instructions	Number of times the address of a variables is taken ('&' in C)
ft38	Instructions	Number of times the address of a function is taken ('&' in C)
ft40	Instructions	Number of assignment instructions with the right operand an integer constant in (method, loops)
ft41	Instructions	Number of binary operations with one of the operands an integer constant in (method, loops)
ft42	Instructions	Number of calls where the number of arguments of the callee is in the interval [M, N] ([M,N]=[0, 3], N > 4)
ft43	Instructions	Number of calls with pointers as arguments
ft44	Instructions	Number of calls that return a pointer
ft45	Instructions	Number of calls that return an integer
ft46	Constants	Number of occurrences of integer constant zero
ft48	Constants	Number of occurrences of integer constant one
ft47	Constants	Number of occurrences of 32-bit integer constants
ft49	Constants	Number of occurrences of 64-bit integer constants
ft51	Variables	Number of references (def/use) of a static/extern variables in (method, loops)
ft50	Variables	Number of references of a local variables in (method, loops)
ft52	Variables	Number of local variables referred in (method, loops)
ft53	Variables	Number of static/extern variables referred in (method, loops)
ft54	Variables	Number of local variables that are pointers in (method, loops)
ft55	Variables	Number of static/extern variables that are pointers in (method, loops)
ft26	PHI-NODES	Average of number of phi-nodes at the beginning of a basic block
ft27	PHI-NODES	Average of arguments for a phi-node
ft28	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft29	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft30	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft31	PHI-NODES	Number of basic block where total number of arguments for all phi-nodes is in the interval [M,N] ($[M,N]=[1, 5]$, $N > 5$)
ft32	PHI-NODES	Number of basic block where total number of arguments for all phi-nodes is in the interval [M,N] ($[M,N]=[1, 5]$, $N > 5$)
ft57	Loops	CYCLOMATIC COMPLEXITY
ft58	Loops	HALSTEAD's METRICS - HN2 is total number of operands
ft59	Loops	HALSTEAD's METRICS - Hn2 is number of distinct operands (Halstead n2)
ft60	Loops	HALSTEAD's METRICS - N is num var defs (should be == Halstead n2 or Halstead N2?)
ft61	Loops	HALSTEAD's METRICS - HN1 is total number of operators (Halstead N1)

		(approx due to abstraction)
ft62	Loops	HALSTEAD's METRICS - H_{n1} is number of distinct operators (Halstead $n1$) (approx due to abstraction)
ft63	Loops	HALSTEAD's METRICS - approx of Halstead difficulty $D = H_{n1}/2 * (H_{n2} / H_{n2})$ (NB uses the aver routine to avoid problems with 0 divisor in computation of H_{n2}/H_{n2})
ft64	Loops	HALSTEAD's METRICS - approx of Halstead volume (volume == $H_N * \log_2(H_n)$ where $H_N = H_{n1}+H_{n2}$ and $H_n = H_{n1}+H_{n2}$)
ft65	Loops	HALSTEAD's METRICS - approx of Halstead effort, which == Difficulty * Volume

A.1.31 -ffree-line-length-*<n>*

描述:

Fortran 语句允许每行最长 132 个字符，如果超过 132 个字符，可使用 -ffree-line-length-*<n>* 选项指定最长为 *n* 个字符，如 -ffree-line-length-300 指定每行最长为 300 个字符。

示例:

```

program test
integer ::status
call handle_error(status, __FILE__, __LINE__)
end program

```

如果由于目录过深或者文件名过长导致程序第三行超过 132 个字符，这时可使用 -ffree-line-length-*<n>* 选项。

A.2 性能选项

A.2.1 -O[0/1/2/3/s]

1、-O0

描述:

该选项提供基础级别的优化，减少编译时间，生成调试所需信息。

备注:

1) 该选项使能下列优化选项:

-faddress_align	-fpcc-struct-return
-faggressive-loop-optimizations	-fpeephole
-fauto-inc-dec	-fplt
-fchkp-check-incomplete-type	-fprefetch-loop-arrays
-fchkp-check-read	-fsched-critical-path-heuristic
-fchkp-check-write	-fsched-dep-count-heuristic

-fchkp-instrument-calls	-fsched-group-heuristic
-fchkp-narrow-bounds	-fsched-interblock
-fchkp-optimize	-fsched-last-insn-heuristic
-fchkp-store-bounds	-fsched-rank-heuristic
-fchkp-use-static-bounds	-fsched-spec
-fchkp-use-static-const-bounds	-fsched-spec-insn-heuristic
-fchkp-use-wrappers	-fsched-stalled-insns-dep
-fcommon	-fschedule-fusion
-fdata-sections	-fsemantic-interposition
-fdelete-null-pointer-checks	-fshow-column
-fdwarf2-cfi-asm	-fshrink-wrap-separate
-fearly-inlining	-fsigned-zeros
-feliminate-unused-debug-types	-fsplit-ivs-in-unroller
-ffp-int-builtin-inexact	-fssa-backprop
-ffunction-cse	-fstdarg-opt
-ffunction-sections	-fstrict-volatile-bitfields
-fgcse-lm	-fsync-libcalls
-fgnu-runtime	-ftrapping-math
-fgnu-unique	-ftree-cselim
-fident	-ftree-forwprop
-finline-atomics	-ftree-loop-if-convert
-fira-hoist-pressure	-ftree-loop-im
-fira-share-save-slots	-ftree-loop-ivcanon
-fira-share-spill-slots	-ftree-loop-optimize
-fivopts	-ftree-parallelize-loops=
-fkeep-static-consts	-ftree-phi-prop
-fleading-underscore	-ftree-reassoc
-flifetime-dse	-ftree-scev-cprop
-flto-odr-type-merging	-funit-at-a-time
-fmath-errno	-funroll-innermost-loop
-fmerge-debug-strings	-funwind-tables

2) 对应 swcc 编译器-O0 选项。

2、-O1

描述:

该选项指示编译器会在不引入过多编译时间开销的前提下,生成更快更小的代码。

备注:

1) 该选项在-O0 基础上使能下列优化选项:

-fbranch-count-reg	-fsplit-wide-types
-fcombine-stack-adjustments	-fssa-phiopt
-fcompare-elim	-ftoplevel-reorder
-fcprop-registers	-ftree-bit-ccp
-fdefer-pop	-ftree-builtin-call-dce

-fforward-propagate	-ftree-ccp
-fguess-branch-probability	-ftree-ch
-fif-conversion	-ftree-coalesce-vars
-fif-conversion2	-ftree-copy-prop
-finline	-ftree-dce
-finline-functions-called-once	-ftree-dominator-opts
-fipa-profile	-ftree-dse
-fipa-pure-const	-ftree-fre
-fipa-reference	-ftree-pta
-fmerge-constants	-ftree-sink
-fmove-loop-invariants	-ftree-slsr
-fomit-frame-pointer	-ftree-sra
-freorder-blocks	-ftree-ter
-fshrink-wrap	

2) 对应 swcc 编译器-O1 选项。

3、-O2

描述:

在-O1 的基础上的进一步优化。这是 swgcc 的默认优化等级。-O2 会比-O1 多使能一些优化选项，使能-O2 后，编译器能够在不增大代码体积且不占用大量编译时间的前提下，提高生成代码的执行性能。

备注:

1) 在-O1 基础上，-O2 使能了以下编译选项:

-falign-labels	-fisolate-erroneous-paths-dereference
-fcaller-saves	-flra-remat
-fcode-hoisting	-foptimize-sibling-calls
-fcrossjumping	-foptimize-strlen
-fcse-follow-jumps	-fpartial-inlining
-fdevirtualize	-fpeephole2
-fdevirtualize-speculatively	-free
-fexpensive-optimizations	-freorder-functions
-fgcse-lm	-frerun-cse-after-loop
-fhoist-adjacent-loads	-fschedule-insns
-findirect-inlining	-fschedule-insns2
-finline-small-functions	-fstore-merging
-fipa-bit-cp	-fstrict-overflow
-fipa-cp	-fthread-jumps
-fipa-icf	-ftree-pre
-fipa-icf-functions	-ftree-sra
-fipa-icf-variables	-ftree-switch-conversion
-fipa-ra	-ftree-tail-merge
-fipa-sra	-ftree-vrp
-fipa-vrp	

2) 对应 swcc 编译器-O2 选项。

4、-O3

描述:

在-O2 的基础上进一步提高生成代码的性能。

备注:

- 1) 在-O2 基础上，-O3 使能了以下编译选项：

-fgcse-after-reload	-fsplit-paths
-finline-functions	-fstrict-aliasing
-fipa-cp-clone	-ftree-loop-distribute-patterns
-fpeel-loops	-ftree-loop-vectorize
-fpredictive-commoning	-ftree-partial-pre
-fsplit-loops	-ftree-slp-vectorize
-finline-functions	-funswitch-loops
-fipa-cp-clone	
- 2) 对应 swcc 编译器-O3 选项。
- 3) 在-O3 选项的基础上打开-msimd 选项编译器会进行自动向量化的编译优化。

5、-Os

描述:

在-O2 的基础上对生成代码的体积进行进一步优化。

备注:

- 1) 在-O2 基础上，-Os 使能了以下编译选项：
 - finline-functions
- 2) 与-O2 相比，-Os 关闭了以下编译选项：
 - foptimize-strlen
 - fschedule-insns
- 3) 对应 swcc 编译器-Os 选项。

A.2.2 -Ofast

描述:

该选项指示编译器不用严格遵循语言标准，相比-O3 选项，优化手段更为激进。

备注:

- 1、在-O3 基础上，-Ofast 使能了以下编译选项：
 - fassociative-math
 - fcx-limited-range
 - ffinite-math-only
 - freciprocal-math
- 2、在-O3 基础上，-Ofast 关闭了以下编译选项：
 - fmath-errno

-ftrapping-math
-fsigned-zeros
-funsafe-math-optimizations

3、对应 swcc 编译器-Ofast 选项。

A.2.3 -ffast-math

描述：

该选项指示编译器不用严格遵守 IEEE754 标准，允许编译器对浮点运算进行一定形式的变形，以提高程序运行速度。对于精度要求较高的程序，该优化可能导致程序结果错误。

备注：

- 1、该选项仅在-Ofast 优化等级下打开；
- 2、该选项打开以下优化选项：

-fno-math-errno
-funsafe-math-optimizations
-ffinite-math-only
-fno-rounding-math
-fno-signaling-nans
-fcx-limited-range

A.2.4 -ffp-contract=*style*

描述：

指示编译器是否进行浮点表示缩写，例如浮点乘加指令优化等。

备注：

- 1、*style* 可以是 on/off/fast；
- 2、on 目前不支持，效果同 off；
- 3、fast 指示编译器开启乘加指令优化，默认在-O2 选项以上开启；
- 4、对应 swcc 编译器-TARG:madd=on/off 选项。

A.2.5 -funroll-loops --param max-unroll-times=*n*

描述：

指示编译器进行循环展开，max-unroll-times 参数指示最大的循环展开次数。

备注：

- 1、参数 max-unrolled-insns 定义了循环体中最大可以被展开的指令数；
参数 max-average-unrolled-insns 定义了循环体中最大可以被展开的

指令数的平均值。对规模较大的循环体，或者需要展开很多次的循环体，适当增加这两个参数值，有助于循环展开优化的使能。

2、对应 swcc 编译器-LNO:outer_unroll_max,ou_max=n 选项。

A.2.6 -funroll-all-loops

描述:

指示编译器对循环体进行展开优化。

备注:

-funroll-all-loops 在不保证性能提升的前提下，强行对循环体进行展开优化。

A.2.7 -fprefetch-loop-arrays

描述:

指示编译器对循环中的数组进行预取,将主存中数据预取到从核 cache 中。

预取优化默认打开循环展开优化，可以通过调整循环展开次数影响预取效果。预取参数可以通过以下编译参数进行调整：

- 1、--params -prefetch-latency-slave=n 设定预取延迟为 n 条指令，默认为 800。该参数影响预取距离。
- 2、--params -simultaneous-prefetches-slave=n 同时预取的指令数为 n，默认为 15。
- 3、--params -l1-cache-slave-size=n 从核 L1cache 大小，单位为 KB，默认为 32。

备注:

- 1、对应 swcc 编译器-LNO:prefetch=n 选项；
- 2、只支持从核代码。

A.2.8 -flto

描述:

指示编译器打开链接时优化。

备注:

使用链接时优化，需要在编译和链接阶段都添加-flto 选项。

A.2.9 -mslave-lto

描述:

编译器支持从核代码链接时优化，需要在编译从核文件及-mhybrid 链接时使用。

示例:

```
swgcc -mhost -c host.c
swgcc -mslave -mslave-lto -c slave1.c slave2.c
swgcc -mhybrid -mslave-lto host.o slave1.o slave2.o
```

A.2.10 -fwhole-program

描述:

指示编译器将当前编译单元中的变量设置为外部不可见属性，方便编译器进行更为激进的优化。

备注:

使用该选项可能导致对当前编译单元中变量的外部引用报未定义错误。

A.2.11 -fipa-type-escape -fipa-struct-peeling

描述:

指示编译器寻找程序中的结构体定义、声明及引用信息，将访问次数较多的热点结构体域剥离，组成新的结构体，增强数据访问的局部性。

备注:

- 1、“访问次数较多”的阈值可配置;
- 2、--param struct-reorg-cold-struct-ratio=<value>, <value>取值介于 0-100，默认为 10。

示例:

```
swgcc -fipa-type-escape -fipa-struct-peeling example.c
swgcc -fipa-type-escape -fipa-struct-peeling --param struct-reorg-cold-struct-ratio=20 example.c
```

A.2.12 -Wl,--gc-sections

描述:

编译时加上-ffunction-sections、-fdata-sections 选项，链接时使用-Wl,--gc-sections 选项，编译器会在链接时，取消对未使用到的数据段/代码段的链接。

备注:

目前已设置为编译器的默认选项。

A.2.13 -fbranch-predict-hint

描述:

面向指令预测 hint 指令的编译支持。

备注:

该选项仅对从核有效。由于硬件对于 hint 指令使用比较苛刻，在一些情况下，该选项不会提高转移预测性能；在最坏情况下，优化后也不会对降低程序原始性能。

A.2.14 -mpws

描述:

基于转移预测器方向切换的分支预测优化。

备注:

该选项仅对从核有效。此外该选项对程序分支特征要求比较严格，使用不当甚至可能严重降低程序性能。

A.2.15 -fmodule-ldm

描述:

module 名以 “ldm__”开头的 module 中的变量都放置在 LDM 空间。

备注:

该选项为 swgfortran 选项。

示例:

```
module ldm__modx
  integer aa
  type node_struct
    real x,y,z
  end type
  type(node_struct) sss
end
```

加-fmodule-ldm 选项编译生成的 mod，里面的 aa 和 sss 结构体都放置在从核 LDM 空间。

A.2.16 -mfma

描述:

编译器支持对乘加操作进行优化，利用硬件乘加单元，将乘法和加法两条指令合并为一条乘加指令。

示例：

```
swgcc -mfma example.c
```

A.2.17 -msimd -ftree-vectorize

描述：

编译器支持主从核的自动向量化功能，编译时在-O2 选项优化的基础上加上-msimd -ftree-vectorize 选项或者直接使用-O3 -msimd 选项即可打开编译器的自动向量化功能。

示例：

```
swgcc -msimd -ftree-vectorize example.c  
swgcc -O3 -msimd example.c
```

A.2.18 -fcache-hint

描述：

编译器支持从核 cache 的写暗示优化，对连续大块数据的 cache 写操作进行优化，提高 cache 访问性能。

示例：

```
swgcc -mslave -fcache-hint slave1.c slave2.c
```

A.2.19 -fdynamic-uncache-symbols

描述：

编译器支持从核变量的可 cache 属性和不可 cache 属性的动态配置，通过编译选项加编译指导语句配合使用。

编译指导语句：区分数据默认是 cache 和 uncache 的两种情况，设置两组编译指导语句。

1、数据默认为 cached

#pragma uncache “xxx” ----将符号 xxx 访问变换为 uncache 访问。

#pragma resume_cache “xxx” ----将符号 xxx 访问恢复为 cache 访问。

2、数据默认为 uncached

#pragma cache “xxx” ---将符号 xxx 访问变换为 cache 访问。

#pragma resume_uncache “xxx” ---将符号 xxx 访问恢复为 uncache 访问。

3、编译指导语句参数说明

#pragma uncache “aa” 将符号 aa 的访问替换为 uncache 访问。

#pragma uncache “aa@@foo” 将函数 foo 内符号 aa 的访问替换为 uncache 访问。

备注：

- 1、该机制只面向对处理器体系结构较为熟悉的高级用户；
- 2、该机制只面向从核 cache 结构；
- 3、该机制不保证数据访问的正确性，需要在属性变化前后添加 cache 刷新语句或者 cache 淘汰语句来保证 cache 与主存之间的数据一致性；
- 4、该机制采用基于 pragma 实现的文本分析策略，一组编译指导语句确定了变换的空间，在此空间内的符号访问被转换为 uncache/uncached 访问，在此空间外，不做任何变换；
- 5、”#prama resume_xxx“为可选语句，如果不添加，则表示对应的”#prama xxx”作用范围将持续到文本结尾；若为”#prama xxx@@yyy“，则作用范围将持续到函数 yyy 文本结尾。（此处 xxx 表示 cache/uncache）。

示例：

swgcc -mslave -fdynamic-uncache-symbols slave.c

```
1 #include "stdio.h"
2 #include <slave.h>
3
4 int aa=3;
5 __uncached int dd=4;
6 __uncached int bb=5;
7 __uncached int cc[20];
8
9 int foo(){
10     #pragma cache "dd@@foo"
11     flush_slave_cache();
12     int ret = aa+dd;
13     #pragma resume_uncache "dd@@foo"
14     flush_slave_cache();
15     return ret;
16 }
17
18 int main()
19 {
20     printf("dd is %d\taa is 0x%lx\n", dd, &aa);
21     aa = dd+4;
```



```

22      #pragma uncache "aa@@main"
23      flush_slave_cache();
24      printf("dd is %d\taa is 0x%lx\n", dd, &aa);
25      aa = bb+5;
26      cc[10] += foo();
27      #pragma resume_cache "aa@@main"
28      #pragma cache "bb"
29      flush_slave_cache();
30      printf("bb is %d\tbb addr is 0x%lx\n", bb, &bb);
31
32      #pragma cache "cc"
33      flush_slave_cache();
34      for(int i=0; i<20; i++)
35          printf("%d\n", cc[i]);
36 }

```

图 23 不可 cache 属性的动态配置代码例图

A.2.20 -lm、-lm_slave

描述:

编译器提供面向处理器体系结构特征优化的高性能快速数学库，链接时使用-lm、-lm_slave 默认使用高性能数学库。

备注:

- 1、-lm 针对主核；
- 2、-lm_slave 针对从核。

示例:

```

swgcc -mhost host1.o host2.o -lm
swgcc -mslave slave1.o slave2.o -lm_slave
swgcc -mhybrid host.o slave.o -lm -lm_slave

```

A.3 调试选项

A.3.1 -fverbose-asm

描述:

指示编译器在汇编文件中输出更多的调试信息，包括：

- 1、编译器版本及命令行选项；
- 2、汇编指令对应的源代码行号；

3、汇编指令对应的高级语言表达式信息，例如内联函数信息等。

备注：

该选项仅适用于汇编文件生成过程。

A.3.2 -dp/-dP

描述：

指示编译器在汇编文件中输出指令模板信息。**-dp** 输出简要信息，**-dP** 输出完整信息。

备注：

该选项仅适用于汇编文件生成过程。

A.3.3 -g

描述：

指示编译器在生成的目标码中加入 DWARF 信息，后续可以使用 gdb 工具进行调试。

备注：

使用该选项进行 gdb 调试，优化级别建议设置为-O0。

A.3.4 -pg

描述：

指示编译器对主核程序进行插桩，在运行时生成 profile 文件，后续可以使用 gprof 工具进行热点函数分析。

备注：

用户也可以不加编译选项，在运行时使用 bsub 相关选项调用 SACA 提供的热点函数分析和重排优化功能，并可同时针对主从核进行。

4.3.5 -fswperf

描述：

指示编译器对性能计数器进行插桩。

备注：

在链接时也需要使用该选项以添加相应库文件的支持。

4.3.6 -fdump-tree-all

描述:

指示编译器输出所有 gimple 优化遍信息。

4.3.7 -fdump-rtl-all

描述:

指示编译器输出所有 rtl 优化遍信息。

4.3.8 -fdump-ipa-all

描述:

指示编译器输出所有 ipa 优化遍信息。

A.4 其它选项

A.4.1 为特定函数设置编译优化选项

A.4.1.1 #pragma GCC optimize ("string", ...)

描述:

该编译指导语句指示编译器为后续定义的函数设置新的编译优化选项，新的编译选项由“string”指定，不同的选项通过‘,’分隔。

备注:

- 1、新的编译选项包括编译优化等级（-O[0/1/2/3/s]及-Ofast，需去除开头的‘-’）以及与目标机器无关的编译优化选项（以“-f”开头，需去除开头的“-f”），与平台相关的选项（以“-m”开头）无法通过该指导语句进行设置；
- 2、该指导语句设置的编译优化选项将应用于后续定义的所有函数。

示例:

```
#pragma GCC optimize("O2","cache-hint")
//后续函数的编译选项设置为-O2 -fcache-hint
void fun()
{
    .....
}
```

A.4.1.2 #pragma GCC reset_options

描述:

该编译指导语句指示编译器清除最近通过 `#pragma GCC optimize` 语句设置的编译选项，将编译优化选项恢复为由命令行指定的缺省选项。

示例:

```
#pragma GCC optimize("O2","cache-hint")
//后续函数的编译选项设置为-O2 -fcache-hint
void fun_0()
{
    .....
}
#pragma GCC reset_options
//后续函数的编译选项恢复为通过命令行设置的缺省选项
void fun_1()
{
    .....
}
```

A.4.1.3 __attribute__((optimize("string", ...)))

描述:

该属性通知编译器为本函数设置新的编译选项，新的编译选项由“string”指定，不同的选项通过‘,’分隔。

备注:

- 1、新的编译选项包括编译优化等级（-O[0/1/2/3/s]及-Ofast，需去除开头的‘-’）以及与目标机器无关的编译优化选项（以“-f”开头，需去除开头的“-f”），与平台相关的选项（以“-m”开头）无法通过该指导语句进行设置；
- 2、该属性的作用范围仅为所处的函数定义，其余函数仍采用命令行设置的缺省选项。

示例:

```
__attribute__((optimize("O2","cache-hint")))
//本函数的编译选项设置为-O2 -fcache-hint
void fun_0()
{
    .....
}
```

```
//后续函数的编译选项恢复为通过命令行设置的缺省选项
void fun_1()
{
    .....
}
```

A.4.2 自动向量化相关选项

A.4.2.1 -faggressive-ifcvt-for-vect

描述:

执行自动向量化的循环中含有条件语句时，添加此选项可以执行激进的条件语句转换从而实现自动向量化。

示例:

```
double sq[16];
for(i=0;i<16;i++) {
    sq[i] = (sq[i]>0.0)?sq[i]:0.0;
    sq[i] = 1.0-sq[i];
}
```

A.4.2.2 -funroll-innermost-loop

描述:

当有多层循环嵌套时，便一起在 O3 默认情况下会自动展开最内层循环，最内层循环展开后整个循环体中的语句数目变多，导致再以后的 pass 处理中难以正确分析出可以自动向量化的循环，从而导致不能自动向量化，添加该选项可以解决此问题。

示例:

```
real*8 cosths(30),sinths(30)
do i=1,m
    do j=1,n
        cosths(j) = i + 0.01012
        sinths(j) = i + 0.01015
    enddo
enddo
```

A.4.2.3 -mtrunc

描述:

当自动向量化的过程中需要将 `float` 类型转换为 `double` 类型时使用。

示例：

```
float a[16];  
double b[16];  
for (i = 0; i < 16; i++)  
    b[i] = a[i];
```

A.4.3 功耗相关优化

A.4.3.1 -freg-bypass

描述：

面向寄存器旁路的指令调度优化。只支持从核代码，在 `-O2`、`-O3` 情况下使能。

A.4.3.2 -mslave-l0cache

描述：

基于 L0 指令 `cache` 的指令调度优化。只支持从核代码，在循环展开优化遍（例如 `-funroll-loops`）使能的情况下有效。

A.4.3.3 -flow-power

描述：

面向控制总线翻转的指令调度优化。只支持从核代码，在指令调度优化遍（例如 `-fschedule-insns`、`-fschedule-insns2`）使能的情况下有效。

附录 B：SW39000 众核处理器 SIMD 编程接口—C 语言

B.1 概述

B.1.1 背景

SW39000 众核处理器主核支持 256 位 SIMD 扩展指令，从核支持 512 位 SIMD 扩展指令，该功能的增加，不仅能够降低功耗，而且在一定程度上提升了指令级并行的能力。

B.1.2 有关 SIMD 的几个基本概念

SIMD（Single Instruction Multiple Data）是单指令流多数据流的缩写，SW39000 众核处理器主核支持的 SIMD 处理长度为 256 位，从核支持的 SIMD 处理长度为 512 位。

B.1.2.1 标量与向量

标量：运算粒度为单个元素

向量：运算粒度为一组有序的标量



图 24 标量与向量基本概念

B.1.2.2 SW39000 众核处理器主核支持的几种 SIMD 操作

- 1、64*4 的双精度浮点运算
即一次操作处理 4 个双精度浮点运算
- 2、64*4 的单精度浮点运算
即一次操作处理 4 个单精度浮点运算
- 3、32*8 的定点运算
即一次操作处理 8 个 32 位的定点运算
- 4、256*1 的定点运算

即一次操作处理 1 个 256 位的长整型运算

该运算可在部分情况下支持一次处理 4 个 64 位的长整形运算

B.1.2.3 SW39000 众核处理器从核支持的几种 SIMD 操作

1、64*8 的双精度浮点运算

即一次操作处理 8 个双精度浮点运算

2、64*8 的单精度浮点运算

即一次操作处理 8 个单精度浮点运算

3、32*16 的定点运算

即一次操作处理 16 个 32 位的定点运算

4、512*1 的定点运算

即一次操作处理 1 个 512 位的长整型运算

该运算可在部分情况下支持一次处理 8 个 64 位的长整形运算

5、16*32 的半精度浮点运算

即一次操作处理 32 个 16 位的半精度浮点运算

B.1.2.4 SIMD 的主要特点

我们知道，一个短向量中各个元素是互不相关的，对当前向量元素的操作结果不影响到其他向量元素。比如有一个数组，我们要使数组中的每一个元素 A_i 都乘以一个标量 b ，那么 b 与 A_1 相乘的结果不影响 A_2 的结果，各自独立。

一条 SIMD 指令相当于一个小的循环，所以可以减少指令数，从而可以降低对指令访问带宽的要求。并且减少了由循环引起的控制相关。

例如主核：

```
int a (N),b (N),c (N)
for(i=0; i < N; i++) {
    a[i]=b[i]+c[i]
}
```

如果采用 SIMD 技术，则最终的实现相当于：

```
for(i=0; i<N; i=i+8){
    a[i+7:i]=b[i+7:i]+c[i+7:i]
}
```

例如从核：

```
int a (N),b (N),c (N)
for(i=0; i < N; i++) {
    a[i]=b[i]+c[i]
}
```


如果采用 SIMD 技术，则最终的实现相当于：

```
for(i=0; i<N; i=i+16){  
    a[i+15:i]=b[i+15:i]+c[i+15:i]  
}
```

可以发现，主核循环迭代次数减少为原来的八分之一，该循环的总指令数减少到原来的八分之一左右；从核循环迭代次数减少为原来的十六分之一，该循环的总指令数减少到原来的十六分之一左右，都提高了单机的效率。

然而，SW39000 众核处理器主核上的 SIMD 需要满足以下一些要求：

1. 仅在内存中连续的数据可以被取入向量寄存器进行向量运算；
2. 数据在内存中要求 32 字节对界（64*4 单精度浮点要求 16 字节对界），不对界的向量访存会引起异常，然后由操作系统模拟，性能上有很大的降低；

SW39000 众核处理器从核上的 SIMD 需要满足以下一些要求：

- 1、 仅在内存中连续的数据可以被取入向量寄存器进行向量运算；
- 2、 数据在内存中要求 64 字节对界（64*8 单精度浮点要求 32 字节对界），不对界的向量访存会引起异常，从核会直接报错退出。

B.1.3 SW39000 众核处理器主核、从核 SIMD 实现概述

SW39000 众核处理器主核与从核支持的 SIMD 功能，借鉴了 SIMD 技术及专用机设计理念——主核：首先把浮点流水线设计成为单指令流多数据流部件（SIMD 部件），以增强其浮点运算能力，同时增加 256 位专用部件流水线，提高整数处理能力，增强了对实际应用的能力。

从核：首先把浮点流水线设计成为单指令流多数据流部件（SIMD 部件），以增强其浮点运算能力，同时增加 512 位专用部件流水线，提高整数处理能力，增强了对实际应用的能力。

SW39000 众核处理器主从核各提供了一百余条指令来实现 SIMD 的功能。用户可以直接通过汇编调用这些指令，但十分不方便。为了用户更好的使用 SIMD，我们提供了一整套方案，通过直接定义数据类型、调用内部函数，或者通过编译器自动的向量识别，来满足用户的需求。

B.1.4 编译器对 C 语言的 SIMD 扩展

我们在 SW39000 众核处理器的配套基础编译器 swgcc 上实现了一整套 SIMD 支持，针对 C 语言扩展了一系列 SIMD 数据类型和函数接口，通过在 C 语言中使用这些数据类型和接口进行编程，用户就相当于直接使用了对应的 SIMD 功能部件。

扩展的函数接口与 SIMD 指令一一对应，在编译的时候由 swgcc 编译器 inline

进来，使用这些函数接口编程可以在 C 语言一级获得与汇编编程一样的性能。

B.1.5 SIMD 扩展的适用范围

SIMD 解决的是 SW39000 众核处理器主核与从核上的短向量问题，属于串程序的向量化的范畴。我们知道，神威高性能计算机的并行语言用户可以使用 MPI、OPENACC 等并行语言进行编程，在这些并行程序中也可以使用 SIMD 扩展编程，并且最终这些扩展将交由 swgcc 编译器来处理。

B.1.6 使用 SIMD 可以预期获得多大的性能提升

SIMD 对性能的提升可由程序本身的特性和程序编写技巧等多方面因素决定。对于完全 SIMD 向量化的程序，对主核来说，32*8 的向量运算性能可以达到标量的 8 倍，64*4 的向量运算性能可以达到标量的 4 倍，如果浮点使用向量乘加部件，性能还可以提高到 8 倍。对从核来说，32*16 的向量运算性能可以达到标量的 16 倍，64*8 的向量运算性能可以达到标量的 8 倍，16*32 的半精度浮点向量运算性能可以达到标量的 32 倍，如果半精度浮点使用向量乘加部件，性能还可以提高到 64 倍。

B.2 快速入门

本节以 C 语言为例，给出一个简单的 SIMD 程序，用来说明语言的某些基本特征。本节没有详尽地描述 SIMD 的所有特征，目的在于让你对 SIMD 编程先有一个大致的概念。

B.2.1 两个简单的例子

例 1. 下面是用 C 语言编写的一个 SIMD 程序（主核）：

```
#include <simd.h>          //包含 SIMD 头文件
main()
{
    int arr[8] __attribute__((aligned(32)))= {1,2,3,4,5,6,7,8};
    int i,res[8],t=0;
    intv8 va,vb,vi;
    simd_load(va,arr);
    for (i=16;i>=1;i>-=1) {
        vi = simd_set_intv8(i,i,i,i,i,i,i,i);    //从标准类型向扩展类型赋值
        va ^= simd_vsraw(va,i); //扩展类型的变量使用运算符^
    }
    vb=simd_veqvw(va,vi); //扩展类型的变量使用扩展的内部函数接口
```

```

simd_print_intv8(vb);    //用 intv8 类型的格式打印
simd_print_intv8(va);    //用 intv8 类型的格式打印
simd_store(va, res);     //intv8 类型的存储
for (i=0; i<8; i++)
    t=t+ res[i];
printf("%d\n",t);
}

```

例 2. 下面是用 C 语言编写的一个 SIMD 程序（从核）：

```

#include <simd.h>        //包含 SIMD 头文件
main()
{
    int arr[16] __attribute__((aligned (64)))= {1,2,3,4,5,6,7,8,10,11,12,13,14,15};
    int i,res[16],t=0;
    intv16 va,vb,vi;
    simd_load (va,arr);
    for (i=16;i>=1;i>-=1) {
        vi = simd_set_intv16(i,i,i,i,i,i,i,i,i,i,i,i,i,i,i,i); //从标准类型向扩展类型
赋值
        va ^= simd_vsraw(va,i); //扩展类型的变量使用运算符
    }
    vb=simd_vconw (va,vi,&arr[2]); //扩展类型的变量使用扩展的内部函数接口
    simd_print_intv16(vb);    //用 intv16 类型的格式打印
    simd_print_intv16(va);    //用 intv16 类型的格式打印
    simd_store(va, res);     //intv16 类型的存储
    for (i=0; i<16; i++)
        t=t+ res[i];
    printf("%d\n",t);
}

```

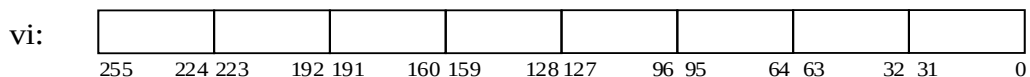
B.2.2 变量声明

在 SIMD 程序中，变量的声明方式与 C 中是一致的，只是在标准 C 的基础上扩展了一些基本类型。

主核：上面的例 1 中使用了：

```
intv8 vi;
```

声明了一个 intv8 类型的 SIMD 变量，表示 vi 是 8 个 32 位整型组成的类型变量。



从核：上面的例 2 中使用了：

```
intv16 vi;
```

声明了一个 intv16 类型的 SIMD 变量，表示 vi 是 16 个 32 位整型组成的类

[illegible]

swgcc 扩展了一些内部函数用于将标准类型的数据赋给扩展类型。

```
vi = simd set intv8 (i, i, i, i, i, i, i, i);
```

vi:

i	i	i	i	i	i	i	i
---	---	---	---	---	---	---	---

255 224 223 192 191 160 159 128 127 96 95 64 63 32 31 0

```
vi = simd set intv16 (i, i, i, i, i, i, i, i, i, i, i, i, i, i, i, i);
```

vi:

i	i	i	i	i	i	i	i	i	i	i	i	i	i	i	i																
511	480	479	448	447	416	415	384	383	352	351	320	319	288	287	256	255	224	223	192	191	160	159	128	127	96	95	64	63	32	31	0

swgcc 还对运算符进行了扩展,可以使用 C 语言中的一些运算符对扩展类型的变量进行运算。

$$va \wedge va > 1;$$

有些运算不能用运算符表示，只能使用扩展的内部函数。例如例子中的“等效”运算：

$$vb = \text{simd_veqvw}(va, vi);$$

如从核:

$$va \wedge va > 1;$$

有些运算不能用运算符表示，只能使用扩展的内部函数。例如例子中的“拼接”运算：

```
vb = simd vconw(va,vi,&arr[2]);
```

它表示:将 va 和 vi 中 16 个 32 位字分别按 arr[2]的地址值进行高低位拼接。

B.2.5 扩展类型的类型转换

在 swgcc 中支持几种扩展类型的转换，swgcc 支持的几种类型转换和它们各自的用法在后面的章节中有详细的介绍。

B.2.6 扩展类型的打印

swgcc 为每一种扩展的数据类型扩展了四个内部函数用于打印，例如 intv8 类型的四个打印函数为：simd_print_intv8(intv8)、simd_fprint_intv8(FILE *, intv8)、simd_print_intv8_X(intv8)、simd_fprint_intv8_X(FILE *, intv8)。

上面的例 1 中：

```
simd_print_intv8(va); //用 intv8 类型的格式打印
```

输出如下：

```
[ 15, 5, 4, 6, 7, 2, 3, 1 ]
```

B.3 SIMD 对 C 语言的扩展

本节主要介绍 swgcc 编译器针对 C 语言进行的 SIMD 扩展。

B.3.1 主核数据类型的扩展

在标准 C 的基础上扩展了 6 种数据类型：

- ◆ intv8: 32*8 有符号整型
- ◆ uintv8: 32*8 无符号整型
- ◆ int256: 256*1 有符号长整型
- ◆ uint256: 256*1 无符号长整型
- ◆ floatv4: 64*4 单精度浮点
- ◆ doublev4: 64*4 双精度浮点

表 12 SIMD 主核扩展数据类型

扩展类型	说明	值的范围
intv8	8 个 int	$-2^{31} \dots 2^{31}-1$
uintv8	8 个 unsigned int	$0 \dots 2^{32}-1$
int256	256 位有符号长整型	—
uint256	256 位有符号长整型	—
floatv4	4 个 float	$1.17549435e-38 \dots 3.40232847e38$
doublev4	4 个 double	$2.2250738585072013e-308 \dots 1.7976931328623158e308$

B.3.2 从核数据类型的扩展

在标准 C 的基础上扩展了 7 种数据类型：

- ◆ intv16: 32*16 有符号整型
- ◆ uintv16: 32*16 无符号整型
- ◆ int512: 512*1 有符号长整型
- ◆ uint512: 512*1 无符号长整型
- ◆ floatv8: 64*8 单精度浮点
- ◆ doublev8: 64*8 双精度浮点
- ◆ float16v32: 16*32 半精度浮点

表 13 SIMD 从核扩展数据类型

扩展类型	说明	值的范围
intv16	16 个 int	$-2^{31} \dots 2^{31}-1$
uintv16	16 个 unsigned int	$0 \dots 2^{32}-1$
int512	512 位有符号长整型	—
uint512	512 位无符号长整型	—
floatv8	8 个 float	$1.17549435\text{e-}38 \dots 3.40232847\text{e}38$
doublev8	8 个 double	$2.2250738585072013\text{e-}308 \dots 1.7976931328623158\text{e}308$
float16v32	32 个 _Float16	$6.103515625\text{e-}5 \dots 65504$

B.3.3 主核对界（Host Alignment）

本节描述主核 SIMD 的对界要求，当使用 SIMD 数据进行编程的时候，用户必须清楚这些对界问题，因为编译器不会在对界方面产生报警或错误信息，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

主核向量类型的数据项在内存中除了 floatv4 类型是 16 字节对界，其余都是 32 字节对界。编译器保证用向量类型定义的变量在内存中是 32 字节（16 字节）对界的。包含主核向量类型的结构体（struct）/联合（union），编译器保证它们是 32 字节（或 16 字节）对界的（必要的时候在内部进行了垫塞）。

例如：

```
struct st { doublev4 vb; int a; }  
sizeof(struct st)返回值是 64。
```

在 SW39000 众核处理器主核架构中，不对界的 Load/Store 会引发异常，操作系统收到异常信号后就会将这些 Load/Store 拆分成标准类型的 Load/Store，从而大大降低性能。因此在向量编程时，编译程序一般需要添加 -faddress_align=n 选项来保证所有类型数组、结构体、联合体等的首地址是 n 字节对界。一般情况下，主核向量程序需要设置 n=32，但是对于主从混合程序，为保证主从程序对界一致，编译主从程序时都需要添加 -faddress_align=64 选项，保证所有类型数组、结

构体、联合体等的首地址是 64 字节对界的。

C 库中的 `malloc` 函数分配的空间首地址为 16 字节对界，为满足主核向量对界要求并与从核向量对界一致，新增 `libc_aligned_malloc` 函数，该函数分配的空间首地址为 64 字节对界，需配合 `libc_aligned_free` 函数使用。

B.3.4 从核对界（Slave Alignment）

本节描述从核 SIMD 的对界要求，当使用 SIMD 数据进行编程的时候，用户必须清楚这些对界问题，因为编译器不会在对界方面产生报警或错误信息，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

从核向量类型的数据项在内存中除了 `floatv8` 类型是 32 字节对界，其余都是 64 字节对界。编译器保证用从核向量类型定义的变量在内存中是 64 字节（32 字节）对界的。包含从核向量类型的结构体（`struct`）/联合（`union`），编译器保证它们是 64 字节（或 32 字节）对界的（必要的时候在内部进行了垫塞）。

例如：

```
struct st { doublev8 vb; int a; }
```

`sizeof(struct st)`返回值是 128。

在 SW39000 众核处理器从核构架中，不对界的 Load/Store 会引发不对界访存异常，程序会直接退出。因此在从核向量编程时，编译程序一般需要添加 `-faddress_align=64` 选项，保证所有类型数组、结构体、联合体等的首地址是 64 字节对界的。

为满足从核向量对界要求，从核 C 库新增 `libc_aligned_malloc` 函数，该函数分配的空间首地址为 64 字节对界，需配合 `libc_aligned_free` 函数使用。

使用 `-faddress_align=n` 后，编译器会将所有的数组、结构体、联合体、向量的首地址进行对界设置，即使这些数组、结构体、联合体与向量的访存操作无关，这在一定程度上会增加程序的内存需求。对性能要求较高的用户，可以不使用这个选项，而是在程序中手工设置其对界属性，具体方法可参见本指南中对界部分相关内容。

B.3.5 SIMD 对 C 操作的扩展（主核）

假设定义了下面的一些变量：

```
intv8 x;  int y;  long la;  floatv4 fv;  
doublev4 *p, dv,  float f;  double d;
```

1、 `sizeof()`

`sizeof(x)`和 `sizeof(*p)`的返回值都是 32。

2、 赋值

扩展类型可以在定义变量的时候像数组一样的赋初值，例如：

va:

8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---

255 224 223 192 191 160 159 128 127 96 95 64 63 32 31 0

0	0	0	2.323
---	---	---	-------

$\begin{matrix} 255 & 192 & 191 & 128 & 127 & 64 & 63 & 0 \end{matrix}$

fv:

3.0	3.0	3.0	3.0
-----	-----	-----	-----

255 192 191 128 127 64 63 0

X:

2	2	2	2	2	2	2	2
---	---	---	---	---	---	---	---

255 224 223 192 191 160 159 128 127 96 95 64 63 32 31 0

va:

0	0	0	0	0	0	0	2
---	---	---	---	---	---	---	---

255 224 223 192 191 160 159 128 127 96 95 64 63 32 31 0

vb:

2	2	2	2	2	2	2	2
---	---	---	---	---	---	---	---

255 224 223 192 191 160 159 128 127 96 95 64 63 32 31 0

个扩展类型变量的指针。指针引用*p 隐含着从 p 的地址进行 256 位的装入操作。

5、 运算符的扩充

同时，本文对标准 C 中的运算符进行了扩充，例如“+、-、*、/”等运算符现在都可以用于扩展类型数据之间的运算。下面列出了所有的扩展情况，运算符后跟的数据类型表示用这些类型定义的变量可以进行相应的运算操作（“:”前是标准 C 中的运算符，“:”后面是该运算符扩展到的数据类型）：

```
+ : intv8, uintv8, int256, uint256, floatv4, doublev4
- : intv8, uintv8, int256, uint256, floatv4, doublev4
* : floatv4, doublev4
/ : floatv4, doublev4
& : intv8, uintv8, int256, uint256,
| : intv8, uintv8, int256, uint256,
^ : intv8, uintv8, int256, uint256,
>> : intv8, uintv8, int256, uint256,
<< : intv8, uintv8, int256, uint256,
== : intv8, uintv8, floatv4, doublev4
<= : intv8, uintv8, floatv4, doublev4
< : intv8, uintv8, floatv4, doublev4
>= : intv8, uintv8
```

在编写 SIMD 程序时可以直接使用这些运算符来进行扩展数据类型上的运算。在使用的时候需要注意以下几点：

- 1) 对 intv8 类型扩展的“>>”符号，与标准 C 中的约定一样，如果“>>”左边的操作数是有符号的，那么最终生成的指令为算术右移，如果是无符号的，最终生成的指令为逻辑右移。
- 2) int256 类型支持的加减法相当于 64*4 的四个 long 类型的向量加减法，因为 SW39000 众核处理器主核指令集里面针对 64*4 这种长整型向量的操作仅包含加法和减法，而 int256 本身也没有算术意义，从编译器具体实现的角度，我们认为没有必要再单独提供一种新类型，因而采用 int256 类型来实现 64*4 的长整型向量的加减。
- 3) “>>”运算符仅支持 uint256 类型，为逻辑右移，不支持 int256 类型。

B.3.6 SIMD 对 C 操作的扩展（从核）

假设定义了下面的一些变量：

```
intv16 x; int y; long la; floatv8 fv; doublev8 *p, dv.
float f; double d;
```

1、 sizeof()

sizeof(x)和 sizeof(*p)的返回值都是 64。

2、赋值

扩展类型可以在定义变量的时候像数组一样的赋初值，例如：

intv16 va = {1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8}; 赋值的结果为：

va:	8	7	6	5	4	3	2	1	8	7	6	5	4	3	2	1																
	511	480	479	448	447	416	415	384	383	352	351	320	319	288	287	256	255	224	223	192	191	160	159	128	127	96	95	64	63	32	31	0

或者

floatv8 vf = {2.323}; 赋值的结果为：

vf:	0	0	0	0	0	0	0	2.323								
	511	448	447	384	383	320	319	256	255	192	191	128	127	64	63	0

另外，除了向量浮点类型和向量整型之间不能互相赋值之外，其他的赋值都是允许的。比如：

fv = 3;

x = 2.345;

*p = dv;

其中，fv=3 的赋值结果为：

fv:

3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0								
511	448	447	384	383	320	319	256	255	192	191	128	127	64	63	0

x=2.345 的赋值结果为：

x:	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2																
	511	480	479	448	447	416	415	384	383	352	351	320	319	288	287	256	255	224	223	192	191	160	159	128	127	96	95	64	63	32	31	0

等号左/右侧的类型不一样隐含着强制类型转换，强制类型转换的具体含义参见 B.3.10 节《扩展类型的类型转换》。

intv16 va = {2};

表示给 va 的最低 32 位赋初值 2，va 的其他部分为缺省值 0。

va:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2		
	511	480	479	448	447	416	415	384	383	352	351	320	319	288	287	256	255	224	223	192	191	160	159	128	127	96	95	64	63	32	31	0

3、取地址符号

对向量类型的取地址符号是允许的，&x 表示取变量 x 的地址。

4、指针

指向扩展类型的指针运算与标准类型的相同。例如 p+1 表示指向下一个扩展类型变量的指针。指针引用*p 隐含着从 p 的地址进行 512 位的装入操作。

5、运算符的扩充

同时，本文本对标准 C 中的运算符进行了扩充，例如“+、-、*”等运算符现在都可以用于扩展类型数据之间的运算。下面列出了所有的扩展情况，运算符后跟的数据类型表示用这些类型定义的变量可以进行相应的运算操作（“：”前是标准 C 中的运算符号，“：”后面是该运算符扩展到的数据类型）：

+: intv16, uintv16, int512, uint512, floatv8, doublev8, float16v32

```

-: intv16, uintv16, int512, uint512, floatv8, doublev8, float16v32
*: floatv8, doublev8, float16v32
>>: intv16, uintv16
<<: intv16, uintv16
==: intv16, uintv16, floatv8, doublev8, float16v32
<=: intv16, uintv16, floatv8, doublev8, float16v32
< : intv16, uintv16, floatv8, doublev8, float16v32

```

在编写 SIMD 程序时可以直接使用这些运算符来进行扩展数据类型上的运算。在使用的时候需要注意以下几点：

- 1) 对 intv16 类型扩展的“>>”符号，与标准 C 中的约定一样，如果“>>”左边的操作数是有符号的，那么最终生成的指令为算术右移，如果是无符号的，最终生成的指令为逻辑右移。
- 2) int512 类型支持的加减法相当于 64*8 的四个 long 类型的向量加减法，因为 SW39000 众核处理器从核指令集里面针对 64*8 这种长整型向量的操作仅包含加法和减法，而 int512 本身也没有算术意义，从编译器具体实现的角度，我们认为没有必要再单独提供一种新类型，因而采用 int512 类型来实现 64*8 的长整型向量的加减。

B.3.7 SIMD 对 C 内部函数的扩展（主核）

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。编译器成功安装后，只需要将“simd.h” #include 到程序中就可以使用这些扩展数据类型和扩充的内部函数或宏定义了。

例如，要将一段普通的 C 程序向量化经常需要将标准类型的数据先映射到 SIMD 变量中，然后才进行运算。本文本扩充了 9 个 SIMD 宏定义接口，用于进行扩展数据类型与标准类型之间的映射操作。这 9 个宏定义是：

```

simd_load()   —— 对界装入，将标准类型的数据从向量要求的对界内存地址映射到扩展类型变量
simd_loadu()  —— 不对界装入，将标准类型的数据从不对界内存地址映射到扩展类型变量
simd_load_u() —— 强制对界装入，将不对界的内存地址低位清零形成强制对界地址，然后将标准类型的数据从对界内存映射到扩展类型变量
simd_loade()  —— 装入并扩展，将标准类型的数据从内存映射到扩展类型变量的低位并扩展到高位
simd_store()  —— 对界存储，将扩展类型变量中的数据映射到向量要求的对界内存中
simd_storeu() —— 不对界存储，将扩展类型变量中的数据映射到不对界内

```

存中

`simd_store_u()` —— 强制对界存储，将不对界的内存地址低位清零形成强制对界地址，然后将扩展类型变量中的数据映射到对界内存中

`simd_storeul()` —— 不对界存储，将扩展类型变量中的低位数据映射到不对界内存的低位中

`simd_storeuh()` —— 不对界存储，将扩展类型变量中的高位数据映射到不对界内存的高位中

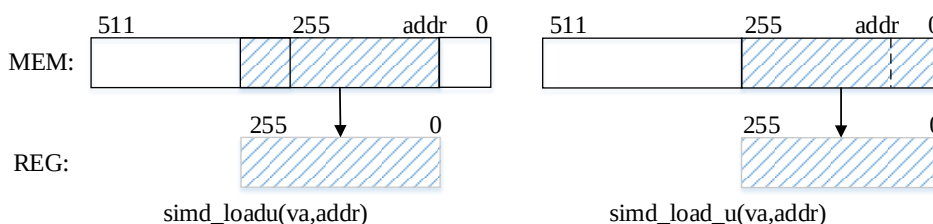


图 25 主核 `simd_loadu` 和 `simd_load_u` 示意图

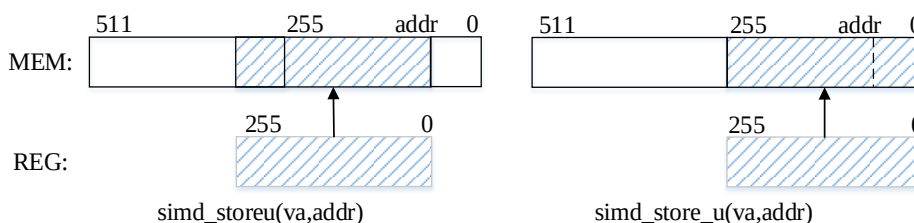


图 26 主核 `simd_storeu` 和 `simd_store_u` 示意图

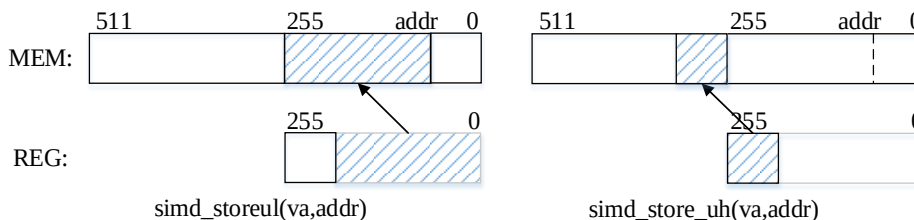


图 27 主核 `simd_storeul` 和 `simd_store_uh` 示意图

其中 `simd_load`、`simd_loadu`、`simd_store` 适用于所有扩展数据类型的装入/存储，`simd_loadu`、`simd_load_u`、`simd_storeu`、`simd_storeul`、`simd_storeuh` 支持 `intv8`、`uintv8`、`floatv4`、`doublev4` 类型的不对界装入/存储。这些宏定义有两个参数，第一个是扩展类型，第二个是标准数据类型的指针，参数具体允许的类型请参照 B.5 节《C 内部函数接口》。

在编写 SIMD 程序的过程中还经常需要给这些扩展类型的变量赋初值，本文提供了一些宏定义来做这件事情。例如定义一个 `intv8` 类型的变量 `va`，要给该变量赋初值，该初值应该包含八个整型常量，比如为 `{1,2,3,4,5,6,7,8}`，那么可以这样编写：

```
intv8 va = simd_set_intv8 (1, 2, 3, 4,5,6,7,8);
```

在变量 `va` 中，这 8 个值的存放位置是这样的：

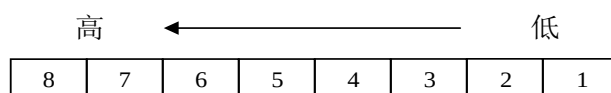


图 28 intv8 向量存储位置例图

相应的 uintv8、int256、uint256、doublev4 和 floatv4 类型也有类似的函数：`simd_set_uintv8`、`simd_set_int256`、`simd_set_uint256`、`simd_set_doublev4` 和 `simd_set_floatv4`，也可以用“=”符号直接为该类型的变量赋初值，例如：

```
doublev4 vlint;
vlint = 123.456;
```

对于扩展数据类型，许多操作是用标准 C 的运算符或库函数所无法表示的，这样的操作必须靠调用内部函数或宏定义来实现，例如：

```
simd_vinsfd () —— D 浮点向量插入
simd_vextfd () —— D 浮点向量提取
simd_vcopyfd () —— D 浮点向量拷贝
simd_vfseleqd () —— D 浮点向量等于选择
simd_vfselltd () —— D 浮点向量小于选择
simd_vfselled () —— D 浮点向量小于等于选择
```

扩展类型的打印，我们提供了 24 个打印函数，分别用于打印 6 种扩展类型，具体的打印函数见 B.5.2.6 节《打印函数接口》。

例如，

```
va = simd_set_intv8 (1,2,3,4,5,6,7,8);
simd_print_intv8(va);
```

打印的结果是：

```
$> [ 8, 7, 6, 5, 4, 3, 2, 1 ]
```

另外，还有一些操作，一般由编译器自动生成，这里也提供了一些内部函数接口供编程时直接调用，主要有扩展浮点类型的乘加、乘减、负乘加和负乘减操作等。

所有扩展运算符的运算操作也都提供了内部函数接口。

这里只是对扩充内部函数接口进行了简单的介绍，详细的描述请查阅 B.5 节《C 内部函数接口》。

B.3.8 SIMD 对 C 内部函数的扩展（从核）

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。编译器成功安装后，只需要将“`simd.h`”`#include` 到程序中就可以使用这些扩展数据类型和扩充的内部函数或宏定义了。

例如，要将一段普通的 C 程序向量化经常需要将标准类型的数据先映射到 SIMD 变量中，然后才进行运算。本文本扩充了 7 个 SIMD 宏定义接口，用于进行扩展数据类型与标准类型之间的映射操作。这 7 个宏定义是：

- `simd_load()` —— 对界装入，将标准类型的数据从向量要求的对界内存地址映射到扩展类型变量
- `simd_loade()` —— 装入并扩展，将标准类型的数据从内存映射到扩展类型变量的低位并扩展到高位
- `simd_loadu()` —— 不对界装入，将标准类型的数据从不对界内存地址映射到扩展类型变量
- `simd_load_u()` —— 强制对界装入，将不对界的内存地址低位清零形成强制对界地址，然后将标准类型的数据从对界内存映射到扩展类型变量
- `simd_store()` —— 对界存储，将扩展类型变量中的数据映射到向量要求的对界内存中
- `simd_storeu()/simd_store_u()` —— 不对界存储，将扩展类型变量中的数据映射到不对界内存中

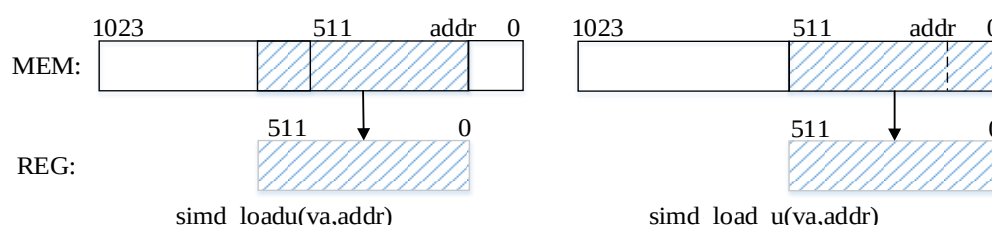


图 29 从核 `simd_loadu` 和 `simd_load_u` 示意图

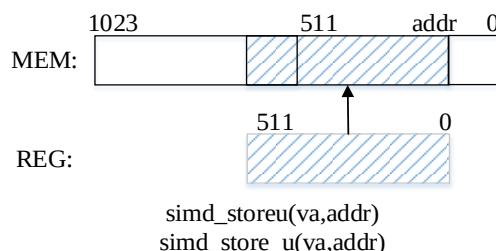


图 30 从核 `simd_storeu` 和 `simd_store_u` 示意图

其中 `simd_load`、`simd_store` 适用于所有扩展数据类型的装入/存储，`simd_loadu`、`simd_load_u`、`simd_storeu` 支持 `intv16`、`uintv16`、`floatv8`、`doublev8` 类型的不对界装入/存储，`simd_loade` 仅支持 `float16v32` 类型的装入并扩展。这些宏定义有两个参数，第一个是扩展类型，第二个是标准数据类型的指针，参数具体允许的类型请参照 B.5 节《C 内部函数接口》。

在编写 SIMD 程序的过程中还经常需要给这些扩展类型的变量赋初值，本文提供了一些宏定义来做这件事情。例如定义一个 `intv16` 类型的变量 `va`，要给该变量赋初值，该初值应该包含八个整型常量，比如为 {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16}，那么可以这样编写：

```
intv16 va = simd_set_intv16 (1, 2, 3, 4,5,6,7,8,9,10,11,12,13,14,15,16);
```

在变量 `va` 中，这 8 个值的存放位置是这样的：

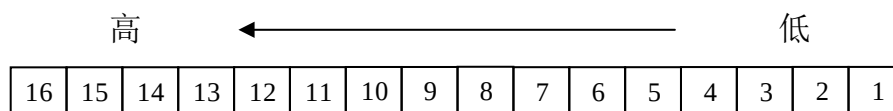


图 31 intv16 向量存储位置例图

相应的 uintv16、int512、uint512、doublev8、floatv8、float16v32 类型也有类似的函数：`simd_set_uintv16`、`simd_set_int512`、`simd_set_uint512`、`simd_set_doublev8`、`simd_set_floatv8` 和 `simd_set_float16v32`，也可以用“=”符号直接为该类型的变量赋初值，例如：

```
doublev8 vlint;
vlint = 123.456;
```

对于扩展数据类型，许多操作是用标准 C 的运算符或库函数所无法表示的，这样的操作必须靠调用内部函数或宏定义来实现，例如：

```
simd_vinsfd () —— D 浮点向量插入
simd_vextfd () —— D 浮点向量提取
simd_vcopyfd () —— D 浮点向量拷贝
simd_vfseleqd () —— D 浮点向量等于选择
simd_vfselltd () —— D 浮点向量小于选择
simd_vfselled () —— D 浮点向量小于等于选择
```

扩展类型的打印，我们提供了 28 个打印函数，分别用于打印 7 种扩展类型：具体的打印函数见 B.5.3.6 节《打印函数接口》。

例如，

```
va = simd_set_intv16 (1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16);
simd_print_intv16(va);
```

打印的结果是：

```
$> [16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

另外，还有一些操作，一般由编译器自动生成，这里也提供了一些内部函数接口供编程时直接调用，主要有扩展浮点类型的乘加、乘减、负乘加和负乘减操作等。

所有扩展运算符的运算操作也都提供了内部函数接口。

这里只是对扩充内部函数接口进行了简单的介绍，详细的描述请查阅 B.5 节《C 内部函数接口》。

B.3.9 扩展类型的类型转换（主核）

在扩展类型中，swgcc 支持如下几种转换：

1、扩展浮点类型单、双精度之间的转换

扩展浮点类型 floatv4 和 doublev4 之间的转换，意义上与标准类型的单、双精度的类型转换一样，例如：

```
floatv4   fv;      doublev4   dv;
```

fv = dv;

2、扩展整数类型 intv8 和 int256 之间的转换

这两种类型之间的转换，变量中的数据不会产生任何的变化。编译器之所以支持这两种类型之间的转换，只是为了有效利用系统提供的指令。例如下面的例子中，一个 int256 的数转换成 intv8 后，使用 intv8 的移位操作可以达到使用 int256 类型很多条操作才能达到的效果：

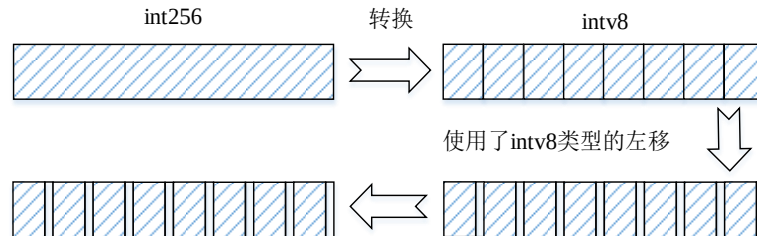


图 32 intv8 的移位操作示意图

同样的，intv8 类型的数据，转换成 int256 类型，并使用 int256 类型的操作后也可以收到事半功倍的效果。例如下图中先将 intv8 转换为 int256，使用 int256 的移位操作后可以很轻松的将最高的 32 位清空：



图 33 int256 的移位操作示意图

在编程过程中，用户灵活运用这两种类型的转换往往可以收到事半功倍的效果。

3、标准类型到扩展类型的转换

1) 值扩展

标准类型到 intv8，floatv4，doublev4 的转换是值的扩展。例如：

```
intv8 va;    int x;
```

```
va = x;
```

把 x 的值复制 8 份放到 va 中：

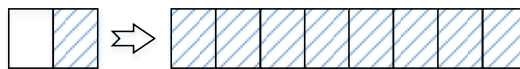


图 34 int 到 intv8 值扩展示意图

在 SW39000 众核处理器主核中，32 位的 int 数据是放在一个 64 位定点寄存器的低 32 位的。所以上面的图中，左边是一个 64 位的寄存器，其中低 32 位是有效数据，扩展到右边一个 256 位的向量寄存器中。再例如：

```
floatv4 fv;
```

```
fv = 3245.345;
```

结果是：

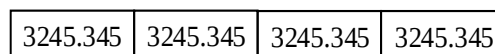


图 35 float 到 floatv4 值扩展示意图

long 到 intv8 的转换，例如：


```
intv8 vx;    long l;
vx = l;
```

相当于：

```
vx = (int)l;
```

float 到 intv8 的转换，例如：

```
float f;    intv8 va;
va = f;
```

相当于：

```
va = (int)f;
```

2) 符号扩展

标准类型到 int256 的转换是符号的扩展。例如：

```
int256 va;    long x=-2;
va = x;
```

把 x 的值复制到 va 的低 64 位中，同时把符号位扩展到高 192，结果是：

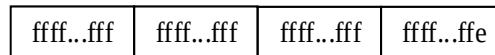


图 36 long 到 int256 值扩展示意图

int 到 int256 的转换，例如：

```
int256 vx; int l;
vx = l;
```

相当于：

```
vx = (long)l;
```

4、扩展类型到标准类型的转换

只是取扩展类型变量中最低的那部分。例如：

```
int a;    intv8 va = {1,2,3,4,5,6,7,8};
a = va;
printf ("%d\n",a);
```

打印结果为：

```
1
```

B.3.10 扩展类型的类型转换（从核）

在扩展类型中，swgcc 支持如下几种转换：

1、扩展浮点类型单、双精度之间的转换

扩展浮点类型 floatv8 和 doublev8 之间的转换，意义上与标准类型的单、双精度的类型转换一样，例如：

```
floatv8 fv;    doublev8 dv;
fv = dv;
```

2、扩展整数类型 intv16 和 int512 之间的转换

这两种类型之间的转换，变量中的数据不会产生任何的变化。编译器之所以支持这两种类型之间的转换，只是为了有效利用系统提供的指令。例如下面的例子中，一个 int512 的数转换成 intv16 后，使用 intv16 的移位操作可以达到使用 int512 类型很多条操作才能达到的效果：

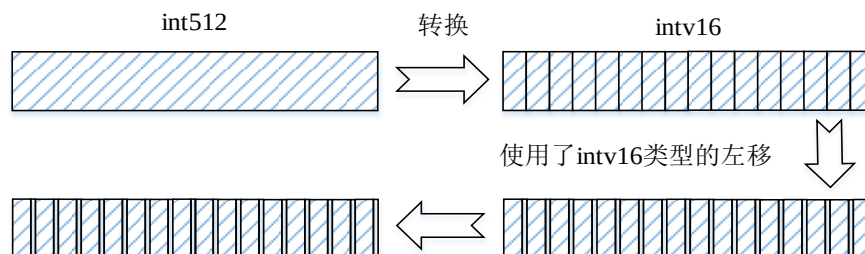


图 37 intv16 移位操作示意图

同样的，intv16 类型的数据，转换成 int512 类型，并使用 int512 类型的操作后也可以收到事半功倍的效果。例如下图中先将 intv16 转换为 int512，使用 int512 的移位操作后可以很轻松的将最高的 32 位清空：



图 38 int512 移位操作示意图

在编程过程中，用户灵活运用这两种类型的转换往往可以收到事半功倍的效果。

3、扩展类型到标准类型的转换

只是取扩展类型变量中最低的那部分。例如：

```
int a;   intv16 va = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
a = va;
printf ("%d\n",a);
```

打印结果为：

1

4、扩展浮点类型半精度、单精度之间的转换

floatv8 simd_vfcvths (float16v32 __A, const int __B): 扩展浮点类型 float16v32 到 floatv8 之间的转换，根据第二个 int 类型参数的值，从 float16v32 输入参数的每个 64 位中选择 1 个 16 位 HP 浮点数分量，转换成 64 位的 SP 浮点数扩展格式，将转换后的 8 个 64 位 SP 浮点数写入返回值。

float16v32 simd_vfcvtsh (floatv8 __A, const int __B): 扩展浮点类型 floatv8 到 float16v32 之间的转换，将 floatv8 输入参数中的 8 个 64 位 SP 扩展格式浮点数，转换成 8 个 16 位 HP 格式浮点数，然后根据第二个 int 类型参数的值写入 float16v32 返回值指定位置，并将其它位置清 0。

5、扩展浮点类型与扩展长字整形之间的转换

int512 可以作为扩展的长字整形向量存在，寄存器中的 512 位被划分为 8 个 64 位，每个 64 位存放一个长整形数据，此时支持它与扩展浮点类型

floatv8/doublev8 之间的转换，意义上与标准类型的长字整形/浮点类型的转换一样。例如：

```
int512 va; floatv8 vb; doublev8 vc;
```

编译器支持 int512 类型到 floatv8/doublev8 类型之间的转换：

```
vb = va; vc = va;
```

编译器支持 doublev8 类型到 int512 类型的转换：

```
va = vc;
```

因为从核没有直接的单精度浮点向量到长字向量的转换指令，编译器不支持 floatv8 类型到 int512 类型的转换，即下面的语句编译会报错：

```
va = vb;
```

B.3.11 扩展类型与标准类型之间的数据交换（主核）

swgcc 可以通过以下两种方法在标准数据类型与扩展类型的变量之间进行数据交换：（1）通过内存交换；（2）通过 SW39000 众核处理器主核提供的一些特殊操作如：vcpyf, vextf 等，上一节中的标准类型到扩展类型的强制类型转换编译器内部也是使用了这些操作完成的。后一种的性能一般要好于前一种，但是后一种一般由编译器优化使用。例如要将一个整型数组的 8 个连续整型数据传递到一个 intv8 类型的向量变量中，下面的写法是错误的：

```
intv8 vsum;
```

```
int vertex[8];
```

```
vsum=(intv8)vertex[0]; /* 错误写法 */
```

由于 vertex[0] 表示一个 int 类型的值，所以这种写法相当于上一节中的强制类型转换，是扩展方式的：

vertex[0]	vertex[0]	vertex[0]	vertex[0]	vertex[0]	vertex[0]	vertex[0]	vertex[0]
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------

而我们想要的结果实际上是：

vertex[7]	vertex[6]	vertex[5]	vertex[4]	vertex[3]	vertex[2]	vertex[1]	vertex[0]
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------

正确的方法主要有三种：

1、使用指针：

```
intv8 vsum, *p;
```

```
int vertex[8];
```

```
p = (intv8 *) (&(vertex[0]));
```

```
vsum = *p;
```

2、使用内部访存函数：

```
/*把以&(vertex[0])为起址，长度为 256 位的内容拷贝到 vsum 中*/
```

```
intv8 vsum; int vertex[8]
```

```
simd_load(vsum, &(vertex[0]));
```

3、使用内部赋值函数：

```
intv8 vsum;    int vertex[8];  
vsum = simd_set_intv8 (vertex[0], vertex[1], vertex[2], vertex[3], vertex[4],  
vertex[5], vertex[6], vertex[7]);
```

目前三种方法都使用了内存作为交换通道,在性能上没有差异。在本例子中,因为前两种方法都只需要一次访存,而第三种方法需要多次访存,前两种方法更好。但如果要传递的数据不在连续的内存区域中,比如,要将 8 个 int 类型的变量传递到一个 intv8 变量中,第三种方法就更好一些,例如:

```
intv8 va;  
int x,y,z,w,m,n,s,t;  
va = simd_set_intv8 (x, y, z, w, m, n, s, t);
```

B.3.12 扩展类型与标准类型之间的数据交换（从核）

swgcc 可以通过以下两种方法在标准数据类型与扩展类型的变量之间进行数据交换：（1）通过内存交换；（2）通过 SW39000 众核处理器从核提供的一些特殊操作如：vcpyf, vextf 等,上一节中的标准类型到扩展类型的强制类型转换编译器内部也是使用了这些操作完成的。后一种的性能一般要好于前一种,但是后一种一般由编译器优化使用。例如要将一个整型数组的 16 个连续整型数据传递到一个 intv16 类型的向量变量中,下面的写法是错误的:

```
intv16 vsum;  
int vertex[16];  
vsum=(intv16)vertex[0]; /* 错误写法 */
```

正确用法:

1、使用指针:

```
intv16 vsum, *p;  
int vertex[16];  
p = (intv16 *) (&(vertex[0]));  
vsum = *p;
```

2、使用内部访存函数:

/*把以&(vertex[0])为起址,长度为 512 位的内容拷贝到 vsum 中*/

```
intv16 vsum;    int vertex[16]  
simd_load(vsum, &(vertex[0]));
```

3、使用内部赋值函数:

```
intv16 vsum;    int vertex[16];  
vsum = simd_set_intv16 (vertex[0], vertex[1], vertex[2], vertex[3], vertex[4],  
vertex[5], vertex[6], vertex[7], vertex[8], vertex[9], vertex[10], vertex[11],  
vertex[12], vertex[13], vertex[14], vertex[15]);
```

目前三种方法都使用了内存作为交换通道,在性能上没有差异。在本例子中,因为前两种方法都只需要一次访存,而第三种方法需要多次访存,前两种方法更好。但如果要传递的数据不在连续的内存区域中,比如,要将 16 个 int 类型的变量传递到一个 intv16 变量中,第三种方法就更好一些,例如:

```
intv16 va;  
int x,y,z,w,m,n,s,t,a,s,d,f,g,h,j,k;  
va = simd_set_intv16 (x, y, z, w, m, n, s, t, a, s, d, f, g, h, j, k);
```

B.4 串行程序的向量化

在原有程序的基础上,要使用系统的 SIMD 功能,最直接的办法是先对程序进行向量化。

向量化时首先要考虑到一些限制:

主核硬件限制:

向量化受限于硬件的限制。在 SIMD 扩展中,向量化访存操作必须是地址连续的,且要求 32 字节(单精度浮点向量为 16 字节)对界的。意味着虽然有些循环是可向量化的,但具体针对 SW39000 众核处理器主核体系结构,需要进行程序变换后才能向量化。另外,由于扩展定点类型的指令集是不完备的,例如 8*32 的乘法、除法等,这样的循环在 SW39000 众核处理器主核上没有办法向量化。

从核硬件限制:

向量化受限于硬件的限制。在 SIMD 扩展中,向量化访存操作必须是地址连续的,且要求 64 字节(单精度浮点向量为 32 字节)对界的。意味着虽然有些循环是可向量化的,但具体针对 SW39000 众核处理器从核体系结构,需要进行程序变换后才能向量化。另外,由于扩展定点类型的指令集是不完备的,例如 16*32 的乘法、除法等,这样的循环在 SW39000 众核处理器从核上没有办法向量化。

程序形式:

循环体中有复杂的关键字、操作、数据访问和内存操作等,都会影响对程序的相关性分析和向量化分析,从而不利于用户或编译器对程序进行向量化。

了解了这些限制,你可以修改程序,避开这些限制,实现高效向量化。下面我们主要针对 C 语言介绍一些针对循环进行向量化的技术。

B.4.1 程序形式对向量化的限制

B.4.1.1 相关性的限制

向量化的基础是相关性分析。例如

例 3.

```
float data[N];
```

```

int i;
for (i=1; i<N-1; i++){
    data[i] = data[i-1]*0.25 + data[i]*0.5 + data[i+1]*0.25;
}

```

上述循环不能向量化，因为写当前元素 `data[i]` 时依赖于使用前面的元素 `data[i-1]`，而它在前一个迭代发生了改变。为了更清楚一点，下面列出头两个迭代的数组访问图：

```

i=1:  READ data[0]
      READ data[1]
      READ data[2]
      WRITE data[1]
i=2:  READ data[1]
      READ data[2]
      READ data[3]
      WRITE data[2]

```

按照循环的执行顺序，第二次迭代读 `data[1]` 的值就是第一次迭代所写的值。而向量化必须要求迭代间可以并行执行，且不改变原始循环的语义。

数据相关性分析包括寻找内存访问可能重叠的条件。给定程序的两个访问，相关的条件如下：

- 1、访问的变量在内存中是同一个（或者重叠）区域的别名
- 2、对数组访问，分析下标的关系是否可能相等

B.4.1.2 循环结构的限制

循环可以是常用的 `for`，`while-do` 或 `repeat-until` 结构，也可以是使用 `goto` 和标号组成的。然而，循环必须是只有一个入口和一个出口时才能被向量化。

例 4. 一个可向量化的循环：

```

double a[100], b[100], c[100];
while ( i<n ){
    a[i]= b[i] * c[i];
    i++;
}

```

例 5. 一个不可向量化的循环：

```

while ( i<n ){
    if ( condition ) break;
    /* 循环体的另一个出口 */
    i++;
}

```

B.4.1.3 循环退出条件的限制

循环退出条件决定循环执行的迭代次数。向量化需要迭代次数是可以计算的，也就是说迭代次数必须是下述表达式：

- 1、 常数
- 2、 循环不变量
- 3、 外层循环控制变量的线性函数

B.4.2 循环向量化步骤

通常，确定循环可以向量化后，向量化一个循环可以分以下步骤：循环展开和分裂、变量替换、操作替换。

B.4.2.1 循环展开和分裂

例 6. 对循环进行循环分裂（主核）

```
int vertex[n], sum;
for (i = 0; i < n; i++){
    sum=sum+vertex[i];
};
变为：
int sum_temp[8], vertex[n], sum;
for (i = 0; i < n/8; i++){
    for(j = 0; j < 8; j++){
        sum_temp[j] = sum_temp[j] + vertex[8*i+j];
    }
};
for(j = 0; j < 8; j++){
    sum = sum + sum_temp[j];
for (i = (n/8)*8; i < n; i++){
    sum=sum+vertex[i];
};
```

例如上面的例子中如果 n 不能被 8 整除，将最后不够一次向量操作的部分分裂出来，前面的部分就可以向量化了。

例 7. 对循环进行循环分裂（从核）

```
int vertex[n], sum;
for (i = 0; i < n; i++){
    sum=sum+vertex[i];
};
变为：
int sum_temp[16], vertex[n], sum;
```

```

for (i = 0; i < n/16; i++){
    for(j = 0; j < 16; j++){
        sum_temp[j] = sum_temp[j] + vertex[16*i+j];
    }
};
for(j = 0; j < 16; j++){
    sum = sum + sum_temp[j];
}
for (i = (n/16)*16; i < n; i++){
    sum=sum+vertex[i];
};

```

例如上面的例子中如果 n 不能被 16 整除，将最后不够一次向量操作的部分分裂出来，前面的部分就可以向量化了。

B.4.2.2 变量替换

声明扩展类型变量有两种策略，一种是在算法一级把关键变量都替换成扩展类型变量，这种方法改造程序彻底，但对程序改动较大；一种方法是在需要向量化的地方才引入扩展类型变量用于 SIMD 操作，该方法牵涉的地方较小，有利于快速向量化和调试。

主核：

```

int vsum[8]; → intv8 vsum_temp;
int vertex[n]; → intv8 tx;

```

从核：

```

int vsum[16]; → intv16 vsum_temp;
int vertex[n]; → intv16 tx;

```

B.4.2.3 操作替换

若不是全局替换，在用 SIMD 操作替换普通操作之前，需要把标准类型变量映射（或复制）到扩展类型变量上，操作完之后，再把扩展类型变量映射（或复制）回标准类型变量。如：

主核：

```

simd_load (tx, &(vertex[8*i])); //将标准类型映射到扩展类型
simd_store (tx, &(vertex[8*i])); //将扩展类型映射到标准类型

```

从核：

```

simd_load (tx, &(vertex[16*i])); //将标准类型映射到扩展类型
simd_store (tx, &(vertex[16*i])); //将扩展类型映射到标准类型

```


B.4.3 如何更好地向量化

B.4.3.1 合理使用数组

为了有效的进行向量化，程序中对数组的操作需要尽可能的连续访问，这意味着对于多维数组，尽量选择在最右边的维（C 语言）进行向量化。

另外，尽量让数组作为结构的一个域，而不是使用结构类型的数组。例如下面的例子中，右边的更好向量化。

表 14 数组的向量化比较

Array of Structures	Structure of Arrays
<pre>struct { float a, b, c, d, e, f } s[99]; int i, n; for (i=0; i < n; i++) x += s[i].a * s[i].e;</pre>	<pre>struct { float a[99], b[99], c[99], d[99], e[99], f[99] } s; int i, n; for (i=0; i < n; i++) x += s.a[i] * s.e[i];</pre>

B.4.3.2 对函数调用的处理

如果要向量化的循环中包含一个函数调用，占用很大开销，可以通过将该函数 `inline` 进循环体或者将该函数体向量化的方法来解决。例如下面的一段程序：

表 15 函数体的向量化比较

原来的程序	inline	函数体向量化
<pre>unsigned fc[FCL],q,k; for (k=0;k<FCL;k++) if (func(fc[k]&q)) s--; else s++; int func(unsigned s) { int i; for(i=16;i>=1;i>=>=1) s ^= s>>i; return s%2; }</pre>	<pre>int fc[FCL],q,tmp[8],k,i; intv8 v,vq,vi; vq=simd_set_intv8(q,q,q,q,q,q,q,q); for(k=0;k<FCL;k+=8){ simd_load(v,&fc[k]); v=v&vq; for(i=16;i>=1;i>=>=1) { vi= simd_set_intv8(i,i,i,i,i,i,i,i); v^=simd_vsraw(v,vi) simd_store(v,tmp); } for(i=0;i<8;i++) if (tmp[i]%2) s--; else s++; }</pre>	<pre>unsigned fc[FCL],q,t[8],k; intv8 vq,v; vq=simd_set_intv8(q,q,q,q,q,q,q,q); for(k=0;k<FCL;k+=8) { simd_load(v,&fc[k]); v=v&vq; vfunc(v,t); for(i=0;i<8;i++) if(t[i]%2) s--; else s++; } void vfunc(intv8 v,int*r){ int i; intv8 vi; for(i=16;i>=1;i>=>=1) { vi= simd_set_intv8(i,i,i,i,i,i,i,i); v^=simd_vsraw(v,vi) } simd_store(v,r); }</pre>

		}
--	--	---

在上面的例子中，扩展类型变量 `vs`（程序中是 `v`）作为函数的参数传递进 `vfunc`。在主核 SIMD 扩展中，扩展类型在函数调用过程中的使用约定与浮点类型的一致。最左边的六个参数通过寄存器 `$f16-$f21` 传递，第七个往后的参数通过栈传递。返回值放在 `$f0` 中。在从核 SIMD 扩展中，扩展类型在函数调用过程的中，整数和浮点数最左边的六个参数通过寄存器 `$16-$21` 传递，第七个往后的参数通过栈传递，返回值在 `$0` 中。而向量类型最左边的六个参数通过寄存器 `$48-$53` 传递，返回值放在 `$32` 中。

B.4.3.3 更有效的使用 cache

高效地使用各级 cache 对于性能的提高是极为重要的，到主存储器中访问数据要比在一级 cache 中访问数据慢数十倍。为了更好地使用 cache，程序需要尽量使用同一个 cache 行的所有数据而不是各不同 cache 行的部分数据，而且程序最好能在数据被替换出 cache 以前尽量多的重用这些数据。当然，为了从 SIMD 部件中获得性能的提升，也要求程序最好访问连续的内存区域，这一点来讲，cache 与 SIMD 部件对程序的要求是一样的。

例如在矩阵乘的例子中，第一个矩阵通常按照行优先的顺序访问，第二个矩阵则要按照列优先的顺序访问，显然对第一个矩阵的访问具有空间局部性，因为访问的是连续地址，而对第二个矩阵的访问每一次访问的经常是一个新的 cache 行。为了实施向量化，通常需要将第二个矩阵的某一列数据逐一装入，然后重新组织到向量寄存器中，这样会增加很大的开销。我们以程序的形式来说明：

```
for (i=0;i<n;i++)
    for(j=0;j<n;j++)
        for(k=0;k<n;k++)
            c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

如果直接进行向量化，则程序变换为：

```
for(k=0;k<n;k += 4) {
    simd_load(va, &a[i][k]);
    vb <- (b[k][j], b[k+1][j], b[k+2][j], b[k+3][j])
    vt += va*vb;
}
simd_store(vt,temp);
c[i][j] = temp[0]+temp[1]+temp[2]+temp[3];
```

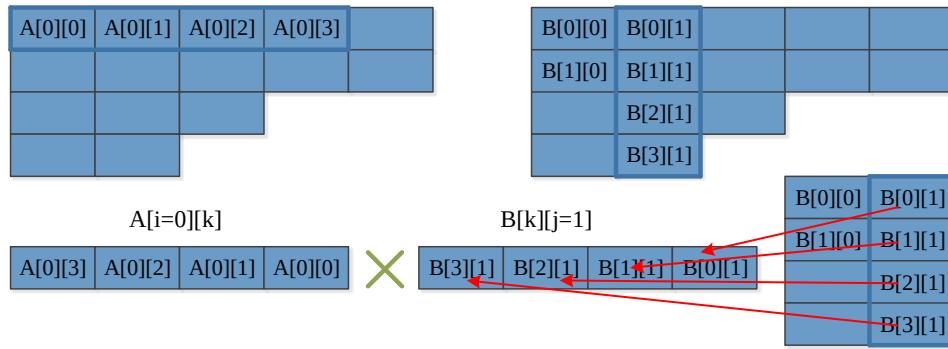


图 39 变换前向量化示意图

向量化的时候，数组 A 连续的 4 个元素可以直接装载到向量中，而数组 B 对应作乘法的元素在内存中并不连续，相当于是对多个 cache 行的访问，这样就需要花费较大的开销去拼接。但是，如果把程序做一下变换，就可以很好的避开访问不同 cache 行的开销，把 k 级循环和 j 级循环互换，然后再做向量化，如下：

```
for (i=0;i<n;i++)
    for(j=0;j<n;j++)
        for(k=0;k<n;k++)
            c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

向量化后程序变换为：

```
for(i=0;i<n;i++) {
    for (k=0;k<n;k++) {
        simd_load(va, &a[i][k]);
        for (j=0;j<n;j += 4) {
            simd_load(vb,&b[k][j]);
            vt = va*vb;
            simd_load(vs, &c[i][j]);
            vs += vt;
            simd_store ( vs,&c[i][j])
        }
    }
}
```

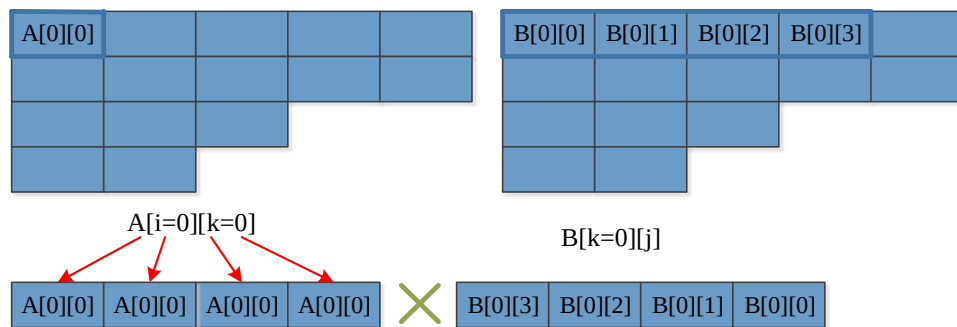


图 40 变换后向量化示意图

由于 j 是最内层循环，因此对 B[k][j] 的访问是连续的，而 A[i][k] 在 j 级循

环里面仅相当于一个常数，这样就可以用数组 A 的一个元素同时去乘数组 B 的连续四个元素，完全屏蔽了不连续的 cache 访问。

B.4.3.4 对界问题的处理

主核：在 B.4.2.3 节介绍的操作替换过程中，使用 `simd_load` 或 `simd_store` 操作进行变量映射的时候，需要保证标准类型变量为 32 字节对界（对于 `floatv4`，只需 16 字节对界）。如果出现了不对界的访存，执行的时候操作系统将最终处理该访存引起的异常，这将付出更昂贵的代价。

编译器可以通过选项 `-faddress_align=32` 来控制数组、结构体、`union` 联合体等的起始地址为 32 字节对界，其余要求用户保证。

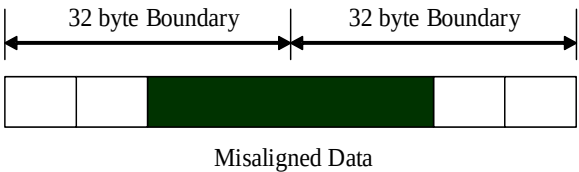


图 41 数据跨 32 字节的不对界访存

从核：在 B.4.2.3 节介绍的操作替换过程中，使用 `simd_load` 或 `simd_store` 操作进行变量映射的时候，需要保证标准类型变量为 64 字节对界（对于 `floatv8`，只需 32 字节对界）。如果出现了不对界的访存，执行的时候处理器硬件会直接报错退出。

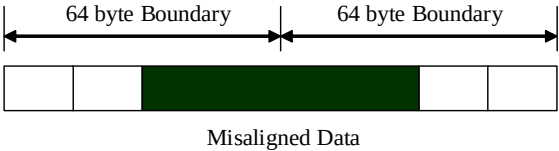


图 42 数据跨 64 字节的不对界访存

编译器可以通过选项 `-faddress_align=64` 来控制数组、结构体、`union` 联合体等的起始地址为 64 字节对界，其余要求用户保证。

对于有些不对界的情况，用户可以进行调整，例如如下主核例子：

例 8. 处理扩展类型的对界问题

```
double a[n],b[n+2];
for ( i=0; i<n; i++) {
    a[i]=a[i]+b[i+2];
}
```

在向量化该循环时，用户若保证数组 a 的对界要求，必然就不能保证数组 b 的对界要求，我们可以采用两种方法来解决该问题。

1、采用 SIMD 整理指令对数组 b 进行拼接

由于对数组 b 的访问不能保证 32 字节对界的要求，因此需要降低对数组 b 的访问粒度，将 `b[i+2]`、`b[i+3]`、`b[i+4]`、`b[i+5]` 分别取出，然后组合成向量与数组 a 进行向量运算。改写后的 SIMD 程序如下：

```

#include <simd.h>
double  a[n],b[n+2];
doublev4  aa, bb,bb1,vtmp;
for ( i=0; i<(n/4)*4; i +=4 ) {
    simd_load(aa, &(a[i]));
    simd_load(bb, &(b[i]));
    vtmp=simd_vextfd2(bb);
    bb1=simd_vinsfd0(vtmp,bb1);
    vtmp=simd_vextfd3(bb);
    bb1=simd_vinsfd1(vtmp,bb1);
    simd_load(bb, &(b[i+4]));
    vtmp=simd_vextfd0(bb);
    bb1=simd_vinsfd2(vtmp,bb1);
    vtmp=simd_vextfd1(bb);
    bb1=simd_vinsfd3(vtmp,bb1);
    aa=aa+bb1;
    simd_store(aa, &(a[i]));
}
for ( i=(n/4)*4; i<n; i++) {
    a[i]=a[i]+b[i+2];
}

```

调整后, 分别在**&b[i]**和**&b[i+4]**处使用两次对界的装入, 然后拼接成从**&b[i+2]**开始的连续的 4 个 64 位双精度浮点数进行运算。

2、使用不对界访存接口直接处理

根据向量不对界访存指令, 主核 SIMD 我们提供了五个不对界访存接口, 分别是向量不对界取 `simd_loadu`、`simd_load_u` 和向量不对界存 `simd_storeu`、`simd_storeul`、`simd_storeuh`。使用不对界访存指令可以不考虑向量类型 32 字节对界 (floatv4 为 16 字节对界) 的要求, 只需满足存取基本单元的对界要求即可, 如 `int` 和 `float` 类型需要 4 字节对界, `double` 类型需要 8 字节对界。从核 SIMD 我们提供了三个不对界访存接口, 分别是向量不对界取 `simd_loadu`、`simd_load_u` 和向量不对界存 `simd_storeu`。同样使用不对界访存指令可以不考虑向量类型 64 字节对界 (floatv4 为 32 字节对界) 的要求, 只需满足存取基本单元的对界要求即可, 如 `int` 和 `float` 类型需要 4 字节对界, `double` 类型需要 8 字节对界。上述主核程序转换如下:

```

#include <simd.h>
double  a[n],b[n+2];
doublev4  aa, bb;
for ( i=0; i<(n/4)*4; i += 4) {
    simd_load(aa, &(a[i]));
    simd_loadu(bb, &(b[i+2]));
    aa=aa+bb;
    simd_store(aa,&a[i]);
}

```

```

    }
    for ( i=(n/4)*4; i<n; i++) {
        a[i]=a[i]+b[i+2];
    }

```

调用不对界访存接口直接装入 b[i+2]，然后直接做向量加法。第二种方法在性能上要优于第一种。

3、使用不对界访存接口和拼接指令组合处理

simd_loadu(bb, &(b[i+2])) 实际上是由 simd_load_u(vtmp1, &(b[i+2]))、simd_load_u(vtmp2, &(b[i+6]))和 bb=simd_vcond (vtmp1, vtmp2, &(b[i+2])) 三条语句实现，连续使用 simd_loadu 取连续的数组数据时，相当于除了首尾数据外，都使用了 simd_load_u 取了两次，造成了访存浪费。用户可根据具体情况具体实现 simd_loadu 功能。上述主核程序可转化如下。

```

#include <simd.h>
double  a[n],b[n+2];
doublev4  aa, bb, vtmp1, vtmp2;
simd_load_u(vtmp1, &(b[2]));
for ( i=0; i<(n/4)*4; i += 4) {
    simd_load(aa, &(a[i]));
    simd_load_u(vtmp2, &(b[i+6]));
    bb=simd_vcond (vtmp1, vtmp2, &(b[i+2]));
    aa=aa+bb;
    simd_store(aa,&a[i]);
    vtmp1= vtmp2;
}
for ( i=(n/4)*4; i<n; i++) {
    a[i]=a[i]+b[i+2];
}

```

B.5 C 内部函数接口

B.5.1 使用的符号说明

VRav: 第一个向量参数

VRbv: 第二个向量参数

VRcv: 第三个向量参数

VRdv: 第四个向量参数

VRc: 向量返回值

Rbv: 无符号长整型变量，一般用于表示地址

Va: 一段内存地址中的数据 f

fp: FILE*类型的变量

B.5.2 主核内部函数接口

B.5.2.1 装入/存储函数接口

表 16 主核装入/存函数接口

宏定义	操作	参数说明	
		VRav	Rbv
simd_load	装入	扩展类型	int*,long*,float*,double*
simd_loade	装入并扩展	扩展类型	int*,float*,double*
simd_store	存储	扩展类型	int*,long*,float*,double*
simd_loadu	不对界装入	扩展类型	int*,float*,double*
simd_load_u	强制对界装入	扩展类型	int*,float*,double*
simd_storeu	不对界存储	扩展类型	int*,float*,double*
simd_store_u	强制对界存储	扩展类型	int*,float*,double*
simd_storeul	不对界存储	扩展类型	int*,long*,float*,double*
simd_storeuh	不对界存储	扩展类型	int*,long*,float*,double*

说明：当 Rbv 为 int * 时，VRav 只能是 intv8；当 Rbv 为 long * 类型时，VRav 只能是 int256；Rbv 为 float * 时，VRav 只能是 floatv4 类型；Rbv 为 double * 类型时，VRav 只能是 doublev4 类型。如果上述类型不匹配，编译器将报错。

函数名

simd_load —— 扩展类型装入

参数说明

- 1、intv8 类型的装入
[u]intv8 va;
[unsigned] int *addr; //intv8 类型的装入
- 2、int256 类型的装入
[u]int256 va;
[[unsigned] long *addr;]; //int256 类型的装入
- 3、floatv4 类型的装入
floatv4 va;
float *addr; //floatv4 类型的装入
- 4、doublev4 类型的装入
doublev4 va;
double *addr; //doublev4 类型的装入

装入结果：

- 1、intv8 类型的装入
[u]intv8 //intv8 类型的装入
- 2、int256 类型的装入

[u]int256 //int256 类型的装入

3、 floatv4 类型的装入

floatv4; //floatv4 类型的装入

4、 doublev4 类型的装入

doublev4; //doublev4 类型的装入

功能说明

扩展类型的装入，将 32 字节（floatv4 类型是 16 字节）长度的数据从连续内存区域中装入到一个向量变量中。

函数名

simd_loadc —— 扩展类型装入并扩展

参数说明

[u]intv8 va;

[unsigned] int *addr; //intv8 类型的装入

floatv4 va;

float *addr; //floatv4 类型的装入

doublev4 va;

double *addr; //doublev4 类型的装入

装入结果

[u]intv8 //intv8 类型的装入

floatv4; //floatv4 类型的装入

doublev4; //doublev4 类型的装入

功能说明

扩展类型的装入并扩展，将 8 字节（floatv4 类型是 4 字节）长度的数据从连续内存区域中装入到一个向量变量的低 64 位，并且将该数据扩展到高 192 位，使向量寄存器的四个寄存器具有相同的值。intv8 是取出 4 字节，然后扩展成 8 个分量。

函数名

simd_store —— 扩展类型存储

参数说明

[u]intv8 va;

[unsigned] int *addr;[[unsigned] long *addr;] //intv8 类型的存储

floatv4 va;

float *addr; //floatv4 类型的存储

doublev4 va;

double *addr; //doublev4 类型的存储

返回值

无

功能说明

扩展类型的存储，将一个向量变量中的数据存储到 32 字节（floatv4 类型是 16 字节）连续内存区域中。

函数名

simd_loadu —— 扩展类型不对界装入

参数说明

- 1、intv8 类型的装入

[u]intv8 va;

[unsigned] int *addr; //intv8 类型的装入

- 2、floatv4 类型的装入

floatv4 va;

float *addr; //floatv4 类型的装入

- 3、doublev4 类型的装入

doublev4 va;

double *addr; //doublev4 类型的装入

装入结果

- 1、intv8 类型的装入

[u]intv8 //intv8 类型的装入

- 2、floatv4 类型的装入

floatv4; //floatv4 类型的装入

- 3、doublev4 类型的装入

doublev4; //doublev4 类型的装入

功能说明

扩展类型的不对界装入，将 32 字节（floatv4 类型是 16 字节）长度的数据，从任意地址开始的内存区域中装入到一个向量变量中。

函数名

simd_load_u —— 扩展类型强制对界装入

参数说明

- 1、intv8 类型的装入

[u]intv8 va;

[unsigned] int *addr; //intv8 类型的装入

- 2、floatv4 类型的装入

floatv4 va;

float *addr; //floatv4 类型的装入

- 3、doublev4 类型的装入

```
doublev4 va;  
double *addr; //doublev4 类型的装入
```

装入结果

- 1、 intv8 类型的装入
[u]intv8 //intv8 类型的装入
- 2、 floatv4 类型的装入
floatv4; //floatv4 类型的装入
- 3、 doublev4 类型的装入
doublev4; //doublev4 类型的装入

功能说明

扩展类型的强制对界装入,将不对界地址 `addr` 低位清零形成强制对界地址,将 32 字节 (floatv4 类型是 16 字节) 长度的数据从强制对界地址开始的连续内存区域中装入到一个向量变量中。

函数名

simd_storeu —— 扩展类型的不对界存储

参数说明

```
[u]intv8 va;  
[unsigned] int *addr; //intv8 类型的存储  
floatv4 va;  
float *addr; //floatv4 类型的存储  
doublev4 va;  
double *addr; //doublev4 类型的存储
```

返回值

无

功能说明

扩展类型的不对界存储,将一个向量变量中的数据存储在 32 字节 (floatv4 类型是 16 字节) 连续内存区域中。

函数名

simd_store_u —— 扩展类型的强制对界存储

参数说明

```
[u]intv8 va;  
[unsigned] int *addr; //intv8 类型的存储  
floatv4 va;  
float *addr; //floatv4 类型的存储  
doublev4 va;  
double *addr; //doublev4 类型的存储
```

返回值

无

功能说明

扩展类型的强制对界存储，不对界地址低位清零形成强制对界地址，将一个向量变量中的数据存储到强制对界地址开始的 32 字节（floatv4 类型是 16 字节）连续内存区域中。

函数名

simd_storeul —— 扩展类型的不对界存储

参数说明

[u]intv8 va;
[unsigned] int *addr; //intv8 类型的存储
[u]int256 va;
[[unsigned] long *addr;]; //int256 类型的存储
floatv4 va;
float *addr; //floatv4 类型的存储
doublev4 va;
double *addr; //doublev4 类型的存储

返回值

无

功能说明

扩展类型的不对界存储，将一个向量变量中的低位数据存储到不对界地址的低位连续内存区域中。

函数名

simd_storeuh —— 扩展类型的不对界存储

参数说明

[u]intv8 va;
[unsigned] int *addr; //intv8 类型的存储
[u]int256 va;
[[unsigned] long *addr;]; //int256 类型的存储
floatv4 va;
float *addr; //floatv4 类型的存储
doublev4 va;
double *addr; //doublev4 类型的存储

返回值

无

功能说明

扩展类型的不对界存储，将一个向量变量中的高位数据存储到不对界地址的高位连续内存区域中。

B.5.2.2 定点向量运算函数接口

表 17 主核定点向量运算函数接口

内部函数	操作	参数说明			
		返回值	VRav	VRbv	VRcv
simd_vaddw	+	intv8	intv8	intv8/lit8	—
simd_vsubw	-	intv8	intv8	intv8/lit8	—
simd_vandw	&	intv8	intv8	intv8/lit8	—
simd_vbicw	与非	intv8	intv8	intv8/lit8	—
simd_vbisw		intv8	intv8	intv8/lit8	—
simd_vornotw	或非	intv8	intv8	intv8/lit8	—
simd_vxorw	^	intv8	intv8	intv8/lit8	—
simd_veqvw	等效	intv8	intv8	intv8/lit8	—
simd_vslw simd_vslwi	<<	intv8	intv8	int/lit5	—
simd_vsrw simd_vsrwi	>>	intv8	intv8	int/lit5	—
simd_vsrw simd_vsrwi	算术右移	intv8	intv8	int/lit5	—
simd_vcmpgew simd_vcmpgewi	大于等于比较	int	intv8	intv8/lit8	—
simd_vcmpeqw simd_vcmpeqwi	等于比较	intv8	intv8	intv8/lit8	—
simd_vcmplew simd_vcmplewi	小于等于比较	intv8	intv8	intv8/lit8	—
simd_vcmpltw simd_vcmpltwi	小于比较	intv8	intv8	intv8/lit8	—
simd_vcmpulew simd_vcmpulewi	无符号小于等于比较	intv8	intv8	intv8/lit8	—
simd_vcmpultw simd_vcmpultwi	无符号小于比较	intv8	intv8	intv8/lit8	—
simd_vrolw simd_vrolwi	循环左移	intv8	intv8	int/lit5	—
simd_vaddl simd_vaddli	长字向量加法	int256	int256	int256/lit8	—
simd_vsubl simd_vsubli	长字向量减法	int256	int256	int256/lit8	—
simd_sllw simd_sllwi	八倍字逻辑左移	int256	int256	int/lit8	—

simd_srlow simd_srlowi	八倍字逻辑右移	int256	int256	int/lit8	—
simd_ctpopow	八倍字数 1	int	int256	—	—
simd_ctlzow	八倍字数头 0	int	int256	—	—
simd_vucaddw simd_vucaddwi	字饱和加	intv8	intv8	intv8/lit8	—
simd_vucsubw simd_vucsubwi	字饱和减	intv8	intv8	intv8/lit8	—
simd_vucaddh simd_vucaddhi	半字饱和加	intv8	intv8	intv8/lit8	—
simd_vucsubh simd_vucsubhi	半字饱和减	intv8	intv8	intv8/lit8	—
simd_vucaddb simd_vucaddbi	字节饱和加	intv8	intv8	intv8/lit8	—
simd_vucsubb simd_vucsubbi	字节饱和减	intv8	intv8	intv8/lit8	—
simd_vseleqw simd_vseleqwi	等于比较选择	intv8	intv8	intv8	intv8/ lit5
simd_vselltw simd_vselltwi	小于比较选择	intv8	intv8	intv8	intv8/ lit5
simd_vsellew simd_vsellewi	小于等于比较选择	intv8	intv8	intv8	intv8/ lit5
simd_vsellbcw simd_vsellbcwi	低位为 0 比较选择	intv8	intv8	intv8	intv8/ lit5
simd_smaxw	大值比较选择	intv8	intv8	intv8	
simd_umaxw	无符号大值比较选择	uintv8	uintv8	uintv8	
simd_sminw	小值比较选择	intv8	intv8	intv8	
simd_uminw	无符号小值比较选择	uintv8	uintv8	uintv8	

内部函数	操作	参数说明				
		VRe (返回值)	VRav	VRbv	VRcv	VRdv
simd_vlog	可重构逻辑	intv8/ int256	0xzz	intv8/ int256	intv8/ int256	intv8/ int256

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；可重构逻辑接口 VRav、VRbv、VRcv、VRdv 分别为第一、第二、第三、第四个参数，VRe 为返回值。
- 2、“—”表示对应接口的该参数无效
- 3、比较指令、移位指令、字向量和长字向量加减法均支持运算符操作
- 4、simd_vlog 的 VRav 参数要求格式为 16 进制数，它指定参数里的 zz 域

函数名

simd_vaddw —— intv8 的加法

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 //intv8 类型的加法

功能说明

intv8 类型的加法, 将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 相加。可以使用“+”符号替换。

函数名

simd_vsubw —— intv8 的减法

参数说明

[u]intv8 va;

[u]intv8 vb (int8 b) ;

返回值

[u]intv8 //intv8 类型的减法

功能说明

intv8 类型的减法, 将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 相减。可以使用“-”符号替换。

函数名

simd_vandw —— intv8 的逻辑与

参数说明

[u]intv8 va;

[u]intv8 vb (int8 b) ;

返回值

[u]intv8

功能说明

intv8 类型的逻辑与, 将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 进行逻辑与运算。可以使用“&”符号替换。

函数名

simd_vbicw —— intv8 的逻辑与非

参数说明

[u]intv8 va;

[u]intv8 vb (int8 b) ;

返回值

[u]intv8

功能说明

intv8 类型的逻辑与非，将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 进行逻辑与非运算。可以使用“&”和“~”符号替换。例如：

```
vc = simd_vbicw(va,vb);等价于：
vc = va & (~vb);
```

函数名

simd_vbisw —— intv8 的逻辑或

参数说明

```
[u]intv8 va;
[u]intv8 vb (int8 b) ;
```

返回值

[u]intv8

功能说明

intv8 类型的逻辑或，将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 进行逻辑或运算。可以使用“|”符号替换。

函数名

simd_vornotw —— intv8 的逻辑或非

参数说明

```
[u]intv8 va;
[u]intv8 vb (int8 b) ;
```

返回值

[u]intv8

功能说明

intv8 类型的逻辑或非，将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 进行逻辑与非运算。可以使用“|”和“~”符号替换。例如：

```
vc = simd_vornotw(va,vb);等价于：
vc = va | (~vb);
```

函数名

simd_vxorw —— intv8 的逻辑异或

参数说明

```
[u]intv8 va;
```

[u]intv8 vb (int8 b) ;

返回值

[u]intv8

功能说明

intv8 类型的逻辑异或，将 va 中的 8 个 int 类型的整型数据分别与 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 进行逻辑异或运算。可以使用“^”符号替换。

函数名

simd_vllw —— intv8 的逻辑左移

simd_vllwi —— intv8 的逻辑左移（立即数格式）

参数说明

[u]intv8 va;

int/lit5 b;

返回值

[u]intv8 //intv8 类型的逻辑左移

功能说明

intv8 类型的逻辑左移，将 va 中的 8 个 int 类型的整型数据进行逻辑左移，移出的空位用“0”补充，八个数移位的位数相同，由 b 中的最低 5 位或者立即数决定。可以使用“<<”符号替换。

函数名

simd_vsrlw —— intv8 的逻辑右移

simd_vsrlwi —— intv8 的逻辑右移（立即数格式）

参数说明

[u]intv8 va;

int/lit5 b;

返回值

[u]intv8 //intv8 类型的逻辑右移

功能说明

intv8 类型的逻辑右移，将 va 中的 8 个 int 类型的整型数据进行逻辑右移，移出的空位用“0”补充，八个数移位的位数相同，由 b 中的最低 5 位或者 5 位立即数决定。可以使用“>>”符号替换。使用“>>”符号的时候，如果操作数是有符号的，生成算术右移指令，如果是无符号的，生成逻辑右移指令。

函数名

simd_vsraw —— intv8 的算术右移

simd_vsrawi —— intv8 的算术右移（立即数格式）

参数说明

[u]intv8 va;
int/lit5 b;

返回值

[u]intv8 //intv8 类型的算术右移

功能说明

intv8 类型的算术右移，将 va 中的 8 个 int 类型的整型数据进行算术右移，移出的空位分别用 8 个字的符号位填充，八个数移位的位数相同，由 b 中的最低 5 位或者 5 位立即数决定。可以使用“>>”符号替换。使用“>>”符号的时候，如果操作数是有符号的，生成算术右移指令，如果是无符号的，生成逻辑右移指令。

函数名

simd_veqvw —— intv8 的逻辑等效

参数说明

intv8 va;
intv8 vb (int8 b) ;

返回值

intv8

功能说明

intv8 类型的逻辑等效，将 va 中的 8 个 int 类型的整型数据和 vb 中的 8 个 int 类型的整型数据或 8 位立即数 b 分别对应按位逻辑等效（同或），结果保存到返回值中。

函数名

simd_vcmpgew —— intv8 的等于比较运算

simd_vcmpgewi —— intv8 的等于比较运算（立即数格式）

参数说明

[u]intv8 va;
[u]intv8 vb(int8 b);

返回值

int c

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行有符号的大于等于比较，只要有一个32位字整数元素满足比较条件，则c写入“1”，否则返回“0”。

函数名

simd_vcmpeqw —— intv8 的等于比较运算

simd_vcmpeqwi —— intv8 的等于比较运算（立即数格式）

参数说明

[u]intv8 va;
[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。

函数名

simd_vcmplew —— intv8 的小于等于比较运算

simd_vcmplewi —— intv8 的小于等于比较运算（立即数格式）

参数说明

[u]intv8 va;
[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行小于等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。

函数名

simd_vcmpltw —— intv8 的小于比较运算

simd_vcmpltwi —— intv8 的小于比较运算（立即数格式）

参数说明

[u]intv8 va;
[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行小于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。

函数名

simd_vcmpulew —— intv8 的无符号小于等于比较运算

simd_vcmpulewi —— intv8 的无符号小于等于比较运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行无符号小于等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。

函数名

simd_vcmpultw —— intv8 的无符号小于比较运算

simd_vcmpultwi —— intv8 的无符号小于比较运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

对va中8个32位的字整数向量元素和vb中对应8个32位的字整数向量元素（或8位立即数# b）分别进行无符号小于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。

函数名

simd_vrolw —— intv8 的循环左移

simd_vrolwi —— intv8 的循环左移（立即数格式）

参数说明

[u]intv8 va;

int/lit5 b;

返回值

[u]intv8

功能说明

intv8类型的循环左移，将va中的8个int类型的整型数据进行循环左移，移出的空位用移出位补充，八个数移位的位数相同，由b的低5位或5位立即数决定。

函数名

simd_sllow —— int256 的逻辑左移

simd_sllowi —— int256 的逻辑左移（立即数格式）

参数说明

[u]int256 va;

int b; （lit8 b）

返回值

[u]int256

功能说明

将va中256位整体左移，移出的空位用“0”填充，移位位数由b低8位或8位立即数确定，结果写入vc中。

函数名

simd_srlow —— int256 的逻辑右移

simd_srlowi —— int256 的逻辑右移（立即数格式）

参数说明

[u]int256 va;

int b; （lit8 b）

返回值

[u]int256

功能说明

将va中256位整体右移，移出的空位用“0”填充，移位位数由b低8位或8位立即数确定，结果写入vc中。

函数名

simd_vaddl —— 长字向量加法

simd_vaddli —— 长字向量加法（立即数格式）

参数说明

[u]int256 va;

[u]int256 vb(int8 b);

返回值

[u]int256

功能说明

将va和vb中4个64位长字（或8位立即数b）分别对应相加，结果保存到vc中对应位置。

函数名

simd_vsubl —— 长字向量减法

simd_vsubli —— 长字向量减法（立即数格式）

参数说明

[u]int256 va;
[u]int256 vb(int8 b);

返回值

[u]int256

功能说明

将va和vb中4个64位长字（或8位立即数b）分别对应相减，结果保存到vc中对应位置。

函数名

simd_ctpopow —— int256 的数 1

参数说明

[u]int256 va;

返回值

int vc

功能说明

返回va中的256位八倍字整数中“1”的个数。

函数名

simd_ctlzow —— int256 的数头 0

参数说明

[u]int256 va;

返回值

int vc

功能说明

返回va中的256位八倍字整数从最高位起连续“0”的个数。

函数名

simd_vucaddw —— intv8 的饱和加运算

simd_vucaddwi —— intv8 的饱和加运算（立即数格式）

参数说明

[u]intv8 va;
[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的8个32位字整数（或8位立即数#b零扩展的字整数）分别

进行32位的有符号加，取低32位结果保存到vc。若结果产生上溢，则将对应的32位结果置为0x7fff,ffff；若产生下溢，则将对应的32位结果置为0x8000,0000。

函数名

simd_vucsubw —— intv8 的饱和减运算

simd_vucsubwi —— intv8 的饱和减运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的8个32位字整数（或8位立即数#b零扩展的字整数）分别进行32位的有符号减，取低32位结果保存到vc。若结果产生上溢，则将对应的32位结果置为0x7fff,ffff；若产生下溢，则将对应的32位结果置为0x8000,0000。

函数名

simd_vucaddh —— 有符号半字向量的饱和加运算

simd_vucaddhi —— 有符号半字向量的饱和加运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的16个16位半字整数（或8位立即数#b零扩展的半字整数）分别进行16位的有符号加，取低16位结果保存到vc。若结果产生上溢，则将对应的16位结果置为0x7fff；若产生下溢，则将对应的16位结果置为0x8000。

函数名

simd_vucsubh —— 有符号半字向量的饱和减运算

simd_vucsubhi —— 有符号半字向量的饱和减运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的16个16位半字整数（或8位立即数#b零扩展的半字整数）分别进行16位的有符号减，取低16位结果保存到vc。若结果产生上溢，则将对应的16位结果置为0x7fff；若产生下溢，则将对应的16位结果置为0x8000。

函数名

simd_vucaddb —— 有符号字节向量的饱和加运算

simd_vucaddbi —— 有符号字节向量的饱和加运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的32个8位字节整数（或8位立即数#b）分别进行8位的有符号加，取低8位结果保存到vc。若结果产生上溢，则将对应的8位结果置为0x7f；若产生下溢，则将对应的8位结果置为0x80。

函数名

simd_vucsubb —— 有符号字节向量的饱和减运算

simd_vucsubbi —— 有符号字节向量的饱和减运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb(int8 b);

返回值

[u]intv8 vc

功能说明

将va和vb中对应的32个8位字节整数（或8位立即数#b）分别进行8位的有符号减，取低8位结果保存到vc。若结果产生上溢，则将对应的8位结果置为0x7f；若产生下溢，则将对应的8位结果置为0x80。

函数名

simd_vseleqw ——intv8 的等于选择运算

simd_vseleqwi ——intv8 的等于选择运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb;

[u]intv8 vc(int5 b);

返回值

[u]intv8 vd

功能说明

对va中每个字元素与0进行等于比较测试，若条件成立，则将向量vb对应的字元素写入vd的对应位置，否则将向量vc对应的字元素（或“0”扩展5位立即数#c形成的字整数）写入vd的对应位置。

函数名

simd_vsellew ——intv8 的小于等于选择运算

simd_vsellewi ——intv8 的小于等于选择运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb;

[u]intv8 vc(int5 b);

返回值

[u]intv8 vd

功能说明

对va中每个字元素与0进行小于等于比较测试，若条件成立，则将向量vb对应的字元素写入vd的对应位置，否则将向量vc对应的字元素（或“0”扩展5位立即数#c形成的字整数）写入vd的对应位置。

函数名

simd_vselltw ——intv8 的小于选择运算

simd_vselltwi ——intv8 的小于选择运算（立即数格式）

参数说明

[u]intv8 va;

[u]intv8 vb;

[u]intv8 vc(int5 b);

返回值

[u]intv8 vd

功能说明

对va中每个字元素与0进行小于比较测试，若条件成立，则将向量vb对应的字元素写入vd的对应位置，否则将向量vc对应的字元素（或“0”扩展5位立即数#c形成的字整数）写入vd的对应位置。

函数名

simd_vsellbcw ——intv8 的低位为 0 选择运算

simd_vsellbcwi ——intv8 的低位为 0 选择运算（立即数格式）

参数说明


```
[u]intv8 va;  
[u]intv8 vb;  
[u]intv8 vc(int5 b);
```

返回值

```
[u]intv8 vd
```

功能说明

对va中每个字元素的低位进行测试，若为0，则将向量vb对应的字元素写入vd的对应位置，否则将向量vc对应的字元素（或“0”扩展5位立即数#c形成的字整数）写入vd的对应位置。

函数名

simd_vlog —— intv8 的可重构逻辑运算

参数说明

```
zz;  
[u]intv8 va;  
[u]intv8 vb;  
[u]intv8 vc;
```

返回值

```
[u]intv8 vd
```

功能说明

将va、vb和vc中对应的每一位，组成3位的二进制数，从两位16进制参数zz的8位二进制值中选出一位，写入vd的对应位中。

函数名

simd_smaxw —— intv8 的大值比较选择

simd_umaxw —— uintv8 的无符号大值比较选择

参数说明

```
[u]intv8 va;  
[u]intv8 vb;
```

返回值

```
[u]intv8 vc
```

功能说明

对va与vb中每个字元素进行大于比较测试，若条件成立，则将向量va对应的字元素写入vc的对应位置，否则将向量vb对应的字元素写入vc的对应位置。

函数名

simd_sminw —— intv8 的小值比较选择

simd_uminw —— uintv8 的无符号小值比较选择

参数说明

[u]intv8 va;

[u]intv8 vb;

返回值

[u]intv8 vc

功能说明

对va与vb中每个字元素进行小于比较测试，若条件成立，则将向量va对应的字元素写入vc的对应位置，否则将向量vb对应的字元素写入vc的对应位置。

B.5.2.3 浮点向量运算函数接口

表 18 主核浮点向量运算函数接口

内部函数	操作	参数说明			
		VRc（返回值）	VRav	VRbv	VRcv
simd_vadds	+	floatv4	floatv4	floatv4	/
simd_vsubs	-	floatv4	floatv4	floatv4	/
simd_vmuls	*	floatv4	floatv4	floatv4	/
simd_vmas	乘加	floatv4	floatv4	floatv4	floatv4
simd_vmss	乘减	floatv4	floatv4	floatv4	floatv4
simd_vnmas	负乘加	floatv4	floatv4	floatv4	floatv4
simd_vnmss	负乘减	floatv4	floatv4	floatv4	floatv4
simd_vadddd	+	doublev4	doublev4	doublev4	/
simd_vsubdd	-	doublev4	doublev4	doublev4	/
simd_vmuldd	*	doublev4	doublev4	doublev4	/
simd_vmad	乘加	doublev4	doublev4	doublev4	doublev4
simd_vmsd	乘减	doublev4	doublev4	doublev4	doublev4
simd_vnmad	负乘加	doublev4	doublev4	doublev4	doublev4
simd_vnmsd	负乘减	doublev4	doublev4	doublev4	doublev4
simd_vfseleqs	条件选择	floatv4	floatv4	floatv4	floatv4
simd_vfseleqd		doublev4	doublev4	doublev4	doublev4
simd_vfsellts		floatv4	floatv4	floatv4	floatv4
simd_vfselltd		doublev4	doublev4	doublev4	doublev4
simd_vfselles	符号拷贝	floatv4	floatv4	floatv4	floatv4
simd_vfselled		doublev4	doublev4	doublev4	doublev4
simd_vcpyss		floatv4	floatv4	floatv4	/
simd_vcpysd		doublev4	doublev4	doublev4	/
simd_vcpysns	除法	floatv4	floatv4	floatv4	/
simd_vcpysnd		doublev4	doublev4	doublev4	/
simd_vdivs		floatv4	floatv4	floatv4	—
simd_vdivd		doublev4	doublev4	doublev4	—

simd_vsqrts	求平方根	floatv4	floatv4	—	—
simd_vsqrtd		doublev4	doublev4	—	—
simd_vfcmpeqs simd_vfcmpeqd	等于比较	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—
simd_vfcmplfs simd_vfcmplfd	小于等于比较	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—
simd_vfcmplts simd_vfcmpltd	小于比较	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—
simd_vfcmpuns simd_vfcmpund	无序比较	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—
simd_vslsln simd_vslldn (n=1,2,3)	浮点整体 逻辑左移	floatv4 doublev4	floatv4 doublev4	—	—
simd_vsrsln simd_vsrldn (n=1,2,3)	浮点整体 逻辑右移	floatv4 doublev4	floatv4 doublev4	—	—
simd_smaxs simd_smaxd	大值比较选择	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—
simd_smins simd_smind	小值比较选择	floatv4 doublev4	floatv4 doublev4	floatv4 doublev4	—

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效
- 3、加法、减法、乘法、除法、乘加类运算支持运算符操作，浮点逻辑移位不支持运算符操作。

函数名

simd_vaddd —— doublev4 的加法运算

simd_vadds —— floatv4 的加法运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4

功能说明

加法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行加法运算。
可以用“+”符号替换。

函数名

simd_vsubd —— doublev4 的减法运算

simd_vsubs —— floatv4 的减法运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4

功能说明

减法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行减法运算。
可以用“-”符号替换。

函数名

simd_vmuld —— doublev4 的乘法运算

simd_vmuls —— floatv4 的乘法运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4

功能说明

乘法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行乘法运算。
可以用“*”符号替换。

函数名

simd_vmad —— doublev4 的乘加运算

simd_vmas —— floatv4 的乘加运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

乘加运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行乘加运算。可以用“*”和“+”符号替换。

ret = va*vb + vc;

函数名

simd_vmsd —— doublev4 的乘减运算

simd_vmss —— floatv4 的乘减运算

参数说明

doublev4/floatv4 va;
doublev4/floatv4 vb;
doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

乘减运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行乘减运算。可以用“*”和“-”符号替换。

ret=va*vb-vc;

函数名

simd_vnmad —— doublev4 的负乘加运算

simd_vnmas —— floatv4 的负乘加运算

参数说明

doublev4/floatv4 va;
doublev4/floatv4 vb;
doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

负乘加运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。

ret = -va*vb + vc;

函数名

simd_vnmsd —— doublev4 的负乘减运算

simd_vnmss —— floatv4 的负乘减运算

参数说明

doublev4/floatv4 va;
doublev4/floatv4 vb;
doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

负乘减运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行负乘减运算。可以用“*”和“+”和“-”符号替换。

```
ret = -va*vb - vc;
```

函数名

simd_vfseleqd —— doublev4 的等于选择运算

simd_vfseleqs —— floatv4 的等于选择运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

对 doublev4/floatv4 类型的参数 va 中的对应浮点向量元素进行比较，如果等于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。

函数名

simd_vfselltd —— doublev4 的小于选择运算

simd_vfsellts —— floatv4 的小于选择运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

对 doublev4/floatv4 类型的参数 va 中的对应浮点向量元素进行比较，如果小于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。

函数名

simd_vfselled —— doublev4 的小于等于选择运算

simd_vfselles —— floatv4 的小于等于选择运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

doublev4/floatv4 vc;

返回值

doublev4/floatv4

功能说明

对 doublev4/floatv4 类型的参数 **va** 中的对应浮点向量元素进行比较，如果小于等于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。

函数名

simd_vcpysd —— doublev4 的拷贝符号

simd_vcpyss —— floatv4 的拷贝符号

参数说明

doublev4/floatv4 **va**;

doublev4/floatv4 **vb**;

返回值

doublev4/floatv4

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。

函数名

simd_vcpysed —— doublev4 的拷贝符号和指数

simd_vcpyses —— floatv4 的拷贝符号和指数

参数说明

doublev4/floatv4 **va**;

doublev4/floatv4 **vb**;

返回值

doublev4/floatv4

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位和指数位是 **va** 中对应浮点向量的符号位和指数位，返回值中每个浮点向量的尾数位是 **vb** 中对应浮点向量的尾数位。

函数名

simd_vcpysnd —— doublev4 的拷贝符号反码

simd_vcpysns —— floatv4 的拷贝符号反码

参数说明

doublev4/floatv4 **va**;

doublev4/floatv4 **vb**;

返回值

doublev4/floatv4

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位的反码，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。

函数名

simd_vdivd —— doublev4 的除法运算

simd_vdivs —— floatv4 的除法运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4

功能说明

除法运算，将 **va** 中的 4 个浮点数分别与 **vb** 中的 4 个浮点数进行除法运算。

函数名

simd_vsqrtd —— doublev4 的求平方根运算

simd_vsqrts —— floatv4 的求平方根运算

参数说明

doublev4/floatv4 va;

返回值

doublev4/floatv4

功能说明

求平方根运算，将 **va** 中的 4 个浮点数分别求平方根。

函数名

simd_vfcmpeqd —— doublev4 的等于比较运算

simd_vfcmpeqs —— floatv4 的等于比较运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4 vc

功能说明

将**va**和**vb**中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”浮点值（1.0）写入**vc**的对应位置，否则将真“0”写入**vc**的对应位置。

函数名

simd_vfcmple —— doublev4 的小于等于比较运算

simd_vfcmple —— floatv4 的小于等于比较运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4 vc

功能说明

将va和vb中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（1.0）写入vc的对应位置，否则将真“0”写入vc的对应位置。

函数名

simd_vfcmpltd —— doublev4 的小于比较运算

simd_vfcmplts —— floatv4 的小于比较运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4 vc

功能说明

将va和vb中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（1.0）写入vc的对应位置，否则将真“0”写入vc的对应位置。

函数名

simd_vfcmpund —— doublev4 的无序比较运算

simd_vfcmpuns —— floatv4 的无序比较运算

参数说明

doublev4/floatv4 va;

doublev4/floatv4 vb;

返回值

doublev4/floatv4 vc

功能说明

将va和vb中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（1.0）写入vc的对应位置，否则将真“0”写入vc的对应位置。

函数名

simd_vslldn —— D 浮点整体逻辑左移运算

simd_vslldn —— S 浮点整体逻辑左移运算

参数说明

doublev4/floatv4 va;

返回值

doublev4/floatv4

功能说明

将浮点向量参数 va 整体左移 $n*64$ 位 (n 的有效值为 1~3), 同时空位补 0, 组成并返回新的浮点向量。

该接口实际上对应3个接口, 分别是simd_vslld1、simd_vslld2、simd_vslld3, 比如:

vb=simd_vslld1(va)

表示 va 整体左移 64 位, 同时低 64 位数据补 0, 组成新的向量返回给 vb。

函数名

simd_vsrlldn——D 浮点整体逻辑右移运算

simd_vsrlsn——S 浮点整体逻辑右移运算

参数说明

doublev4/floatv4 va;

返回值

doublev4/floatv4

功能说明

将浮点向量参数 va 整体右移 $n*64$ 位 (n 的有效值为 1~3), 同时空位补 0, 组成并返回新的浮点向量。

该接口实际上对应3个接口, 分别是simd_vsrlld1、simd_vsrlld2、simd_vsrlld3, 比如:

vb=simd_vsrlld1(va)

表示 va 整体右移 64 位, 同时高 64 位数据补 0, 组成新的向量返回给 vb。

函数名

simd_smaxs ——floatv4 的大值比较选择

simd_smaxd ——doublev4 的大值比较选择

参数说明

floatv4/doublev4 va;

floatv4/doublev4 vb;

返回值

floatv4/doublev4 vc

功能说明

对va与vb中每个浮点元素进行大于比较测试, 若条件成立, 则将向量va对应的浮点元素写入vc的对应位置, 否则将向量vb对应的浮点元素写入vc的对应

位置。

函数名

simd_smins ——floatv4 的小值比较选择

simd_smind ——doublev4 的小值比较选择

参数说明

floatv4/doublev4 va;

floatv4/doublev4 vb;

返回值

floatv4/doublev4 vc

功能说明

对va与vb中每个浮点元素进行小于比较测试，若条件成立，则将向量va对应的浮点元素写入vc的对应位置，否则将向量vb对应的浮点元素写入vc的对应位置。

B.5.2.4 数据整理函数接口

表 19 主核数据整理函数接口

内部函数	操作	参数说明			
		VRc (返回值)	VRav	VRbv	VRcv
simd_vinswn n=(0,1,...,7)	字向量插入	intv8	int	intv8	/
simd_vinsfdn simd_vinsfsn n=(0,1,2,3)	浮点向量插入	floatv4 doublev4	float double	floatv4 doublev4	/
simd_vextwn n=(0,1,...,7)	字向量提取	int	intv8	/	/
simd_vextfsn simd_vextfdn n=(0,1,2,3)	浮点向量提取	float double	floatv4 doublev4	/	/
simd_vcpyw	字向量拷贝	intv8	int	/	/
simd_vcpyfs simd_vcpyfd	浮点向量拷贝	floatv4 doublev4	float double	/	/
simd_vconw	字向量元素拼接	intv8	intv8	intv8	void*
simd_vcons	浮点向量元素拼接	floatv4	floatv4	floatv4	void*
simd_vcond		doublev4	doublev4	doublev4	void*
simd_vshfw	字向量全混洗	intv8	intv8	intv8	double

说明:

simd_vinswn 和 simd_vextwn 中 n 的取值只能是 0、1、2、3、4、5、6、7;

simd_vinsfdn 和 simd_vextfdn 中 n 的取值只能是 0、1、2、3。

函数名

simd_vinswn—— 字向量元素插入运算

参数说明

int a;
intv8 vb;

返回值

intv8

功能说明

将 32 位字整数 a 替代 vb 中由 n（有效值为 0~7）指定 32 位字元素，组成并返回新的字整数向量。

该接口实际上对应 8 个接口，分别是 simd_vinsw0、simd_vinsw1、simd_vinsw2、simd_vinsw3、simd_vinsw4、simd_vinsw5、simd_vinsw6、simd_vinsw7，比如：

vc=simd_vinsw2(a,vb)

表示将 32 位字整数 a 替代 vb 的[95:64]这 32 位的字元素，组成新的 vb 返回给 vc。

函数名

simd_vinsfdn—— D 浮点向量元素插入运算

simd_vinsfsn—— S 浮点向量元素插入运算

参数说明

double/float a;
doublev4/floatv4 vb;

返回值

doublev4/floatv4

功能说明

将浮点寄存器 a 的 64 位浮点数据替代 vb 中由 n（有效值为 0~3）指定的 64 位浮点元素，组成并返回新的浮点向量。

该接口实际上对应 4 个接口，分别是 simd_vinsfd0、simd_vinsfd1、simd_vinsfd2、simd_vinsfd3，比如：

vc=simd_vinsfd1(a,vb)

表示 a 的 64 位浮点数据替代 vb 的[127:64]这 64 位的浮点元素，组成新的 vb 返回给 vc。

函数名

simd_vextwn—— 字向量元素提取运算

参数说明

intv8 va;

返回值

int a

功能说明

将 va 中由 n（有效值为 0~7）指定的 32 位字元素，变为 64 位寄存器格式的字整数写入目的寄存器 a。

该接口实际上对应 8 个接口，分别是 simd_vextw0、simd_vextw1、simd_vextw2、simd_vextw3、simd_vextw4、simd_vextw5、simd_vextw6、simd_vextw7，比如：

a=simd_vextw2(va)

表示 va 的 [95:64]这 32 位的字元素，变为 64 位寄存器格式的字整数写入 a。

函数名

simd_vextfdn—— D 浮点向量元素提取运算

simd_vextfsn—— S 浮点向量元素提取运算

参数说明

doublev4/floatv4 va;

返回值

double/float a

功能说明

将 va 中由 n（有效值为 0~3）指定的 S 浮点或 D 浮点元素存入目的浮点寄存器 a 中。此指令可实现 S 浮点和 D 浮点的提取。

该接口实际上对应 4 个接口，分别是 simd_vextfd0、simd_vextfd1、simd_vextfd2、simd_vextfd3，比如：

a=simd_vextfd1(va)

表示 va 的 [127:64]这 64 位的浮点元素存入 a 的<63:0>中。

函数名

simd_vcopyw—— 字向量元素拷贝运算

参数说明

int a;

返回值

intv8

功能说明

将 32 位字整数 a 复制成 8 个相同元素组成新的字整数向量写入目的寄存器。

函数名

simd_vcopyfd—— D 浮点向量元素拷贝运算

simd_vcopyfs—— S 浮点向量元素拷贝运算

参数说明

double/float a;

返回值

doublev4/floatv4

功能说明

将浮点数 a 复制成 4 个相同元素，组成新的浮点向量写入目的寄存器。

函数名

simd_vconw—— 字向量元素拼接

参数说明

intv8 va;

intv8 vb;

void*;

返回值

intv8 vc

功能说明

字向量拼接，以字分量为单位，va 存放需拼接的字向量低段，vb 存放需拼接的字向量高段，addr 存放不对界访存地址低 5 位（忽略其低 2 位）。根据不对界访存地址低 5 位，将 vb 的低段数据放入目标向量的高段，将 va 的高段数据放入目标向量的低段，拼接后形成新的字向量，存放到 vc 中。

函数名

simd_vcons—— S 浮点向量元素拼接

参数说明

floatv4 va;

floatv4 vb;

void*;

返回值

floatv4 vc

功能说明

S 浮点向量拼接，以 64 位分量为单位，va 存放需拼接的 S 浮点向量低段，vb 存放需拼接的 S 浮点向量高段，addr 存放不对界访存地址低 4 位（忽略其低 2 位）。根据不对界访存地址低 4 位，将 vb 的低段数据放入目标向量的高段，将 va 的高段数据放入目标向量的低段，拼接后形成新的 S 浮点向量，存放到 vc 中。

函数名

simd_vcond—— D 浮点向量元素拼接

参数说明

```
doublev4 va;  
doublev4 vb;  
void*;
```

返回值

```
doublev4 vc
```

功能说明

D 浮点向量拼接，以 64 位分量为单位，va 存放需拼接的 D 浮点向量低段，vb 存放需拼接的 D 浮点向量高段，addr 存放不对界访存地址低 5 位（忽略其低 3 位）。根据不对界访存地址低 5 位，将 vb 的低段数据放入目标向量的高段，将 va 的高段数据放入目标向量的低段，拼接后形成新的 D 浮点向量，存放到 vc 中。

函数名

simd_vshfw—— 字向量全混洗

参数说明

```
intv8 va;  
intv8 vb;  
double Fc;
```

返回值

```
intv8 vd
```

功能说明

操作数寄存器为va、vb、Fc和vd。vd的八个32bit的数由va和vb装填，具体如何装填由Fc说明：

Fc的<31:28>指出vd的<255:224>来自va或vb的位置
Fc的<27:24>指出vd的<223:192>来自va或vb的位置
Fc的<23:20>指出vd的<191:160>来自va或vb的位置
Fc的<19:16>指出vd的<159:128>来自va或vb的位置
Fc的<15:12>指出vd的<127:96>来自va或vb的位置
Fc的<11:8>指出vd的<95:64>来自va或vb的位置
Fc的<7:4>指出vd的<63:32>来自va或vb的位置
Fc的<3:0>指出vd的<31:0>来自va或vb的位置

Fc 相应的 4bit 位置的最高位为“0”时，vd 相应的数来自 va；为“1”时，vd 相应的数来自 vb。va/vb 分成自然对界的 8 个 32bit，Fc 相应的 4bit 位置的低 3bit（0~7）用于指示是第几个 32bit 数据，为“0”表示第 0 个 32bit（[31:0]），依次类推。

例子：

因为存储管理按小端地址格式（little-endian）来实现字节寻址，所以建议 Fc 参数从右往左排列。如果需要间隔装填 va 和 vb 的数，那么可以设置 Fc 为 16 进制数：

```
unsigned long c=0xF6D4B290;
```

注意需要做类型转换： `*(double *)&(c)`

相当于二进制数：

```
unsigned long c= 0B11110110110101001011001010010000
```

以 4bit 为一组，从标红的最高位置可以看出是间隔装填 va 和 vb 的数。

```
#include <stdlib.h>
#include <simd.h>
intv8 va,vb,vd;
int main()
{
    int a[8] = {0,1,2,3,4,5,6,7};
    int b[8] = {999,999,999,999,999,999,999,999};
    int d[8] = {0,999,2,999,4,999,6,999};
    unsigned long c=0xF6D4B290;
    simd_load(va,&a[0]);
    simd_load(vb,&b[0]);
    vd = simd_vshfw(va,vb,*(double *)&(c));
    simd_print_intv8(vd);
    if(memcmp(&vd,d,32)!=0 ){
        printf("Result Error!\n");
        exit(-1);
    }
    printf("Result OK!\n");
}
```

输出为： [999, 6, 999, 4, 999, 2, 999, 0]

Result OK!

B.5.2.5 赋值函数接口

表 20 主核赋值函数接口

宏定义	操作	参数说明	
		返回值	参数
simd_set_intv8	赋值	32*8 向量类型	8 个 int 类型数
simd_set_uintv8	赋值	32*8 向量类型	8 个 unsigned int 类型数
simd_set_int256	赋值	256 位八倍字	4 个 long 类型数
simd_set_uint256	赋值	256 位八倍字	4 个 unsigned long 类型数
simd_set_floatv4	赋值	floatv4 类型	4 个单精浮点
simd_set_doublev4	赋值	doublev4 类型	4 个双精浮点

函数名

simd_set_doublev4—— doublev4 赋值函数

simd_set_floatv4 —— floatv4 赋值函数

参数说明

double/float a;
double/float b;
double/float c;
double/float d;

返回值

doublev4/floatv4

功能说明

doublev4/floatv4 类型的赋值函数，将 4 个 double/float 类型的数据传递到向量变量中。

例如：doublev4 va=simd_set_doublev4(1.0, 2.0, 3.0, 4.0);

va 中的值是这样的：

4.0	3.0	2.0	1.0
255 192 191	128 127	64 63	0

函数名

simd_set_intv8 —— intv8 赋值函数

simd_set_uintv8 —— uintv8 赋值函数

参数说明

int/ unsigned int a;
int/ unsigned int b;
int/ unsigned int c;
int/ unsigned int d;
int/ unsigned int e;
int/ unsigned int f;
int/ unsigned int g;
int/ unsigned int h;

返回值

intv8/uintv8

功能说明

intv8/uintv8 类型的赋值函数，将 8 个 int/ unsigned int 类型的数据传递到向量变量中。

例如：va=simd_set_intv8(1,2,3,4,5,6,7,8);

va 中的值是这样的：

8	7	6	5	4	3	2	1
255 224 223	192 191	160 159	128 127	96 95	64 63	32 31	0

函数名

simd_set_int256 —— int256 赋值函数

simd_set_uint256 —— uint256 赋值函数

参数说明

long/ unsigned long a;

long/ unsigned long b;

long/ unsigned long c;

long/ unsigned long d;

返回值

int256/uint256

功能说明

int256/uint256 类型的赋值函数，将 4 个 long/unsigned long 类型的数据传递到向量变量中。

例如：va=simd_set_int256(1,2,3,4);

va 中的值是这样的：

4	3	2	1
255 192 191	128 127	64 63	0

B.5.2.6 打印函数接口

表 21 主核扩展类型打印函数接口

函数	操作	参数说明	
		返回值	参数
simd_fprint_intv8	打印	/	FILE*, intv8
simd_fprint_uintv8	打印	/	FILE *, uint8
simd_fprint_int256	打印	/	FILE*, int256
simd_fprint_uint256	打印	/	FILE *, uint256
simd_fprint_floatv4	打印	/	FILE*, floatv4
simd_fprint_doublev4	打印	/	FILE*, doublev4
simd_print_intv8	打印	/	intv8
simd_print_uintv8	打印	/	uintv8
simd_print_int256	打印	/	int256
simd_print_uint256	打印	/	uint256
simd_print_floatv4	打印	/	floatv4
simd_print_doublev4	打印	/	doublev4
simd_fprint_intv8_X	打印	/	FILE*, intv8
simd_fprint_uintv8_X	打印	/	FILE *, uint8
simd_fprint_int256_X	打印	/	FILE*, int256
simd_fprint_uint256_X	打印	/	FILE *, uint256

simd_fprint_floatv4_X	打印	/	FILE*, floatv4
simd_fprint_doublev4_X	打印	/	FILE*, doublev4
simd_print_intv8_X	打印	/	intv8
simd_print_uintv8_X	打印	/	uintv8
simd_print_int256_X	打印	/	int256
simd_print_uint256_X	打印	/	uint256
simd_print_floatv4_X	打印	/	floatv4
simd_print_doublev4_X	打印	/	doublev4

函数名

simd_fprint_intv8 —— intv8 打印到文件

simd_fprint_uintv8 —— uintv8 打印到文件

simd_fprint_intv8_X —— intv8 以 16 进制打印到文件

simd_fprint_uintv8_X —— uintv8 以 16 进制打印到文件

参数说明

FILE *file;

intv8/uintv8 va;

返回值

无

功能说明

intv8/uintv8 类型的数据以 8 个 int/unsigned int 的形式打印到文件 file 中，其中 simd_fprint_intv8_X、simd_fprint_uintv8_X 以 16 进制格式打印。

例如：va=simd_set_intv8(1,2,3,4,5,6,7,8);

simd_fprint_intv8(stderr, va);

simd_fprint_intv8_X (stderr, va);

打印的结果是这样的：

[8, 7, 6, 5, 4, 3, 2, 1]

[0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1]

函数名

simd_fprint_int256 —— int256 打印到文件

simd_fprint_uint256 —— uint256 打印到文件

simd_fprint_int256_X —— int256 以 16 进制打印到文件

simd_fprint_uint256_X —— uint256 以 16 进制打印到文件

参数说明

FILE *file;

int256/uint256 va;

返回值

无

功能说明

int256/uint256 类型的数据以 4 个 long/unsigned long 的形式打印到文件 file 中，其中 simd_fprint_int256_X、simd_fprint_uint256_X 以 16 进制格式打印。

例如：va=simd_set_uint256(1,2,3,4);
 simd_fprint_uint256(stderr, va);
 simd_fprint_uint256_X (stderr, va);

打印的结果是这样的：

[4, 3, 2, 1]
[0x4, 0x3, 0x2, 0x1]

函数名

simd_fprint_floatv4 —— floatv4 打印到文件
simd_fprint_doublev4 —— doublev4 打印到文件
simd_fprint_floatv4_X —— floatv4 以 16 进制打印到文件
simd_fprint_doublev4_X —— doublev4 以 16 进制打印到文件

参数说明

FILE *file;
floatv4/doublev4 va;

返回值

无

功能说明

floatv4/doublev4 类型的数据以 4 个 float/double 的形式打印到文件 file 中，其中 simd_fprint_floatv4_X、simd_fprint_doublev4_X 以十六进制格式打印。

例如：va=simd_set_floatv4(1.0,2.0,3.0,4.0);
 simd_fprint_floatv4(stderr, va);
 vb=simd_set_doublev4(1.0,2.0,3.0,4.0);
 simd_fprint_floatv4_X (stderr, va);
 simd_fprint_doublev4_X (stderr, vb);

打印的结果是这样的：

[3.0, 4.0, 2.0, 1.0]
[0x40800000, 0x40400000, 0x40000000, 0x3f800000]
[0x4010000000000000, 0x4008000000000000, 0x4000000000000000,
0x3ff0000000000000]

函数名

simd_print_intv8 —— intv8 打印到屏幕
simd_print_uintv8 —— uintv8 打印到屏幕
simd_print_intv8_X —— intv8 以 16 进制打印到屏幕

simd_print_uintv8_X —— uintv8 以 16 进制打印到屏幕

参数说明

intv8/uintv8 va;

返回值

无

功能说明

intv8/uintv8 类型的数据以 8 个 int/unsigned int 的形式打印到屏幕，其中 simd_print_intv8_X、simd_print_uintv8_X 以十六进制格式打印。

例如： va=simd_set_intv8(1,2,3,4,5,6,7,8);

simd_print_intv8(va);

simd_print_intv8_X (va);

打印的结果是这样的：

[8, 7, 6, 5, 4, 3, 2, 1]

[0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1]

函数名

simd_print_int256 —— int256 打印到屏幕

simd_print_uint256 —— uint256 打印到屏幕

simd_print_int256_X —— int256 以 16 进制打印到屏幕

simd_print_uint256_X —— uint256 以 16 进制打印到屏幕

参数说明

int256/uint256 va;

返回值

无

功能说明

int256/uint256 类型的数据以 4 个 long/unsigned long 的形式打印到屏幕，其中 simd_print_int256_X、simd_print_uint256_X 以十六进制格式打印。

例如： va=simd_set_int256(1,2,3,4);

simd_print_int256(va);

simd_print_int256_X (va);

打印的结果是这样的：

[4, 3, 2, 1]

[0x4, 0x3, 0x2, 0x1]

函数名

simd_print_floatv4 —— floatv4 打印到屏幕

simd_print_doublev4 —— doublev4 打印到屏幕

simd_print_floatv4_X —— floatv4 以 16 进制打印到屏幕

simd_print_doublev4_X —— doublev4 以 16 进制打印到屏幕

参数说明

floatv4/doublev4 va;

返回值

无

功能说明

floatv4/doublev4 类型的数据以 4 个 float/double 的形式打印到屏幕，其中 simd_print_floatv4_X、simd_print_doublev4_X 以十六进制格式打印。

例如：va=simd_set_floatv4(1.0,2.0,3.0,4.0);

simd_print_floatv4(va);

simd_print_floatv4_X (va);

vb=simd_set_doublev4(1.0,2.0,3.0,4.0);

simd_print_doublev4_X (vb);

打印的结果是这样的：

[3.0, 4.0, 2.0, 1.0]

[0x40800000, 0x40400000, 0x40000000, 0x3f800000]

[0x4010000000000000, 0x4008000000000000, 0x4000000000000000, 0x3ff0000000000000]

B.5.2.7 向量归约接口

表 22 主核向量归约接口

函数	操作	参数说明	
		返回值	参数
simd_reduc_plusw	加法归约	int	intv8
simd_reduc_pluss		float	floatv4
simd_reduc_plusd		double	doublev4
simd_reduc_umaxw	无符号最大值归约	unsigned int	uintv8
simd_reduc_smaxw	最大值归约	int	intv8
simd_reduc_smaxs		float	floatv4
simd_reduc_smaxd		double	doublev4
simd_reduc_uminw	无符号最小值归约	unsigned int	uintv8
simd_reduc_sminw	最小值归约	int	intv8
simd_reduc_smins		float	floatv4
simd_reduc_smind		double	doublev4

函数名

simd_reduc_plusw —— intv8 的加法归约

参数说明

intv8 va;

返回值

int

功能说明

intv8 类型的加法归约，将 va 中的 8 个 int 类型的整型数据归约求和。

函数名

[simd_reduc_pluss](#) —— floatv4 的加法归约

[simd_reduc_plusd](#) —— doublev4 的加法归约

参数说明

floatv4/doublev4 va;

返回值

float/double

功能说明

加法归约，将 va 中的 4 个浮点数据归约求和。

函数名

[simd_reduc_smaxw](#) —— intv8 的最大值归约

[simd_reduc_umaxw](#) —— uintv8 的无符号最大值归约

参数说明

intv8/uintv8 va;

返回值

int/unsigned int

功能说明

最大值归约，求 va 中的 8 个整型数据中的最大值。

函数名

[simd_reduc_smaxs](#) —— floatv4 的最大值归约

[simd_reduc_smaxd](#) —— doublev4 的最大值归约

参数说明

floatv4/doublev4 va;

返回值

float/double

功能说明

最大值归约，求 va 中的 4 个浮点数据中的最大值。

函数名

[simd_reduc_sminw](#) —— intv8 的最小值归约

[simd_reduc_uminw](#) —— uintv8 的无符号最小值归约

参数说明

intv8/uintv8 va;
 返回值
 int/unsigned int
 功能说明
 最小值归约，求 va 中的 8 个整型数据中的最小值。

函数名
simd_reduc_smins —— floatv4 的最小值归约
simd_reduc_smind —— doublev4 的最小值归约

参数说明
 floatv4/doublev4 va;
 返回值
 float/double
 功能说明
 最小值归约，求 va 中的 4 个浮点数据中的最小值。

B.5.3 从核内部函数接口

B.5.3.1 装入/存储函数接口

表 23 从核装入/存储函数接口

宏定义	操作	参数说明	
		VRav	Rbv
simd_load	装入	扩展类型	int*,long*,float*,double*,_Float16*
simd_loade	装入并扩展	float16v32	_Float16*
simd_store	存储	扩展类型	int*,float*,double*,_Float16*
simd_loadu	不对界装入	扩展类型	int*,float*,double*,_Float16*
simd_load_u	强制对界装入	扩展类型	int*,float*,double*,_Float16*
simd_storeu simd_store_u	不对界存储	扩展类型	int*,float*,double*,_Float16*

说明：当 Rbv 为 int * 类型时，VRav 只能是 intv16，当 Rbv 为 long *类型时，VRav 只能是 int512；Rbv 为 float *时，VRav 只能是 floatv8 类型；Rbv 为 double *类型时，VRav 只能是 doublev8 类型；Rbv 为 _Float16*类型时，VRav 只能是 float16v32 类型。如果上述类型不匹配，编译器将报错。

函数名
simd_load —— 扩展类型装入

参数说明

- 1、intv16 类型的装入
[u]intv16 va;
[unsigned] int *addr;
- 2、int512 类型的装入
[u]int512 va;
[[unsigned] long *addr;] //int512 类型的装入
- 3、floatv8 类型的装入
floatv8 va;
float *addr; //floatv8 类型的装入
- 4、doublev8 类型的装入
doublev8 va;
double *addr; //doublev8 类型的装入
- 5、float16v32 类型的装入
float16v32 va;
_Float 16 *addr; //float16v32 类型的装入

装入结果

- 1、intv16 类型的装入
[u]intv16 //intv16 类型的装入
- 2、int512 类型的装入
[u]int512; //int512 类型的装入
- 3、floatv8 类型的装入
floatv8; //floatv8 类型的装入
- 4、doublev8 类型的装入
doublev8; //doublev8 类型的装入
- 5、float16v32 类型的装入
float16v32; //float16v32 类型的装入

功能说明

扩展类型的装入，将 64 字节（floatv8 类型是 32 字节）长度的数据从连续内存区域中装入到一个向量变量中。

函数名

simd_loadv —— 半精度浮点类型装入并扩展

参数说明

float16v32 va;
_Float 16 *addr; //float16v32 类型的装入

装入结果

float16v32; //float16v32 类型的装入

功能说明

半精度浮点类型的装入并扩展，将 2 字节长度的数据从连续内存区域中装入到一个向量变量的低 16 位，并且将该数据扩展到高 496 位，使向量寄存器的 32 个半精度分量具有相同的值。从核仅支持半精度浮点向量的装入并扩展。

函数名

simd_store —— 扩展类型存储

参数说明

[u]intv16 va;
[unsigned] int *addr; //intv16 类型的存储
floatv8 va;
float *addr; //floatv8 类型的存储
doublev8 va;
double *addr; //doublev8 类型的存储
float16v32 va;
_Float *addr; //float16v32 类型的存储

返回值

无

功能说明

扩展类型的存储，将一个向量变量中的数据存储到 64 字节（floatv8 类型是 32 字节）连续内存区域中。

函数名

simd_loadu —— 扩展类型不对界装入

参数说明

- 1、intv16 类型的装入
[u]intv16 va;
[unsigned] int *addr; //intv16 类型的装入
- 2、floatv8 类型的装入
floatv8 va;
float *addr; //floatv8 类型的装入
- 3、doublev8 类型的装入
doublev8 va;
double *addr; //doublev8 类型的装入
- 4、float16v32 类型的装入
float16v32 va;
_Float16 *addr; //float16v32 类型的装入

装入结果

- 1、intv16 类型的装入

- [u]intv16 //intv16 类型的装入
- 2、 floatv8 类型的装入
floatv8; //floatv8 类型的装入
- 3、 doublev8 类型的装入
doublev8; //doublev8 类型的装入
- 4、 float16v32 类型的装入
float16v32; //float16v32 类型的装入

功能说明

扩展类型的不对界装入，将 64 字节（floatv8 类型是 32 字节）长度的数据，从任意地址（须满足 4 字节对齐）开始的内存区域中装入到一个向量变量中。

函数名

simd_load_u —— 扩展类型强制对界装入

参数说明

- 1、 intv16 类型的装入
[u]intv16 va;
[unsigned] int *addr; //intv16 类型的装入
- 2、 floatv8 类型的装入
floatv8 va;
float *addr; //floatv8 类型的装入
- 3、 doublev8 类型的装入
doublev8 va;
double *addr; //doublev8 类型的装入
- 4、 float16v32 类型的装入
float16v32 va;
_Float16 *addr; //float16v32 类型的装入

装入结果

- 1、 intv16 类型的装入
[u]intv16 //intv16 类型的装入
- 2、 floatv8 类型的装入
floatv8; //floatv8 类型的装入
- 3、 doublev8 类型的装入
doublev8; //doublev8 类型的装入
- 4、 float16v32 类型的装入
float16v32; //float16v32 类型的装入

功能说明

扩展类型的强制对界装入，将不对界地址 addr 低位清零形成强制对界地址，将 64 字节（floatv8 类型是 32 字节）长度的数据从强制对界地址开始的连续内存

区域中装入到一个向量变量中。

函数名

simd_storeu/ simd_store_u —— 扩展类型的不对界存储

参数说明

[u]intv16 va;
[unsigned] int *addr; //intv16 类型的存储
floatv8 va;
float *addr; //floatv8 类型的存储
doublev8 va;
double *addr; //doublev4 类型的存储
float16v32 va;
_Float16*addr; //float16v32 类型的存储

返回值

无

功能说明

扩展类型的不对界存储，将一个向量变量中的数据存储到 64 字节（floatv8 类型是 32 字节）连续内存区域中。

B.5.3.2 定点向量运算函数接口

表 24 从核定点向量运算函数接口

内部函数	操作	参数说明			
		返回值	VRav	VRbv	VRcv
simd_vaddw	+	intv16	intv16	intv16/lit8	—
simd_vsubw	-	intv16	intv16	intv16/lit8	—
simd_vmulw	*	intv16	intv16	intv16/lit8	—
simd_vdivw	/	intv16	intv16	intv16/lit8	—
simd_vslw simd_vslwi	<<	intv16	intv16	intv16/lit5	—
simd_vsrw simd_vsrwi	>>	intv16	intv16	intv16/lit5	—
simd_vsraw simd_vsrawi	算术右移	intv16	intv16	intv16/lit5	—
simd_vrolw simd_vrolwi	循环左移	intv16	intv16	intv16/lit5	—
simd_vaddl simd_vaddli	长字向量加法	int512	int512	int512/lit8	—
simd_vsubl simd_vsubli	长字向量减法	int512	int512	int512/lit8	—

simd_vornotw	或非	intv16	intv16	intv16/lit8	—
simd_vxorw	^	intv16	intv16	intv16/lit8	—
simd_vandw	&	intv16	intv16	intv16/lit8	—
simd_vbicw	与非	intv16	intv16	intv16/lit8	—
simd_vbisw		intv16	intv16	intv16/lit8	—
simd_vcmpeqw simd_vcmpeqwi	等于比较	intv16	intv16	intv16/lit8	—
simd_vcmplew simd_vcmplewi	小于等于比较	intv16	intv16	intv16/lit8	—
simd_vcmpltw simd_vcmpltwi	小于比较	intv16	intv16	intv16/lit8	—
simd_vcmpulew simd_vcmpulewi	无符号小于等于比较	uintv16	uintv16	uintv16/lit8	—
simd_vcmpultw simd_vcmpultwi	无符号小于比较	uintv16	uintv16	uintv16/lit8	—
simd_sllx	八倍字逻辑左移	int512	int512	int/lit9	
simd_srlx	八倍字逻辑右移	int512	int512	int/lit9	

内部函数	操作	参数说明				
		VRe (返回值)	VRav	VRbv	VRcv	VRdv
simd_vlog2xi	可重构逻辑	intv16	intv16	intv16	0xzz	—
simd_vlog2xi_i	可重构逻辑	intv16	intv16	lit8	0xzz	—
simd_vlog2xx	可重构逻辑	int512	int512	int512	0xzz	—
simd_vlog2xx_i	可重构逻辑	int512	int512	lit8	0xzz	—
simd_vlog3ri	可重构逻辑	intv16	intv16	intv16	intv16	0xzz
simd_vlog3rx	可重构逻辑	int512	int512	int512	int512	0xzz

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；可重构逻辑接口 VRav、VRbv、VRcv、VRdv 分别为第一、第二、第三、第四个参数，VRe 为返回值。
- 2、“—”表示对应接口的该参数无效
- 3、比较指令、移位指令、字向量和长字向量加减法均支持运算符操作
- 4、simd_vlog 的 VRav 参数要求格式为 16 进制数，它指定参数里的 zz 域

函数名

simd_vaddw —— intv16 的加法

参数说明

[u]intv16 va;

[u]intv16 vb(int8 b);

返回值

[u]intv16 //intv16 类型的加法

功能说明

intv16 类型的加法，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 相加。可以使用“+”符号替换。约 1 拍。

函数名

simd_vsubw —— intv16 的减法

参数说明

[u]intv16 va;

[u]intv16 vb (int8 b) ;

返回值

[u]intv16 //intv16 类型的减法

功能说明

intv16 类型的减法，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 相减。可以使用“-”符号替换。约 1 拍。

函数名

simd_vmulw —— intv16 的乘法

参数说明

[u]intv16 va;

[u]intv16 vb(int8 b);

返回值

[u]intv16 //intv16 类型的乘法

功能说明

intv16 类型的乘法，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 相乘。约 60 拍。

函数名

simd_vdivw —— intv16 的除法

参数说明

[u]intv16 va;

[u]intv16 vb (int8 b) ;

返回值

[u]intv16 //intv16 类型的除法

功能说明

intv16 类型的除法，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 相除。约 600 拍。

函数名

simd_vslw —— intv16 的逻辑左移

simd_vslwi —— intv16 的逻辑左移（立即数格式）

参数说明

[u]intv16 va;

intv16/lit5 vb/b;

返回值

[u]intv16 //intv16 类型的逻辑左移

功能说明

intv16 类型的逻辑左移, 将 va 中的 16 个 int 类型的整型数据进行逻辑左移, 移出的空位用“0”补充, 移位位数由 vb 中对应位置字分量的低 5 位或立即数 b 的低 5 位确定。可以使用“<<”符号替换。约 1 拍。

函数名

simd_vsrlw —— intv16 的逻辑右移

simd_vsrlwi —— intv16 的逻辑右移（立即数格式）

参数说明

[u]intv16 va;

intv16/lit5 vb/b;

返回值

[u]intv16 //intv16 类型的逻辑右移

功能说明

intv16 类型的逻辑右移, 将 va 中的 16 个 int 类型的整型数据进行逻辑右移, 移出的空位用“0”补充, 移位位数由 vb 中对应位置字分量的低 5 位或立即数 b 的低 5 位确定。可以使用“>>”符号替换。使用“>>”符号的时候, 如果操作数是有符号的, 生成算术右移指令, 如果是无符号的, 生成逻辑右移指令。约 1 拍。

函数名

simd_vsraw —— intv16 的算术右移

simd_vsrawi —— intv16 的算术右移（立即数格式）

参数说明

[u]intv16 va;

intv16/lit5 vb/b;

返回值

[u]intv16 //intv16 类型的算术右移

功能说明

intv16 类型的算术右移, 将 va 中的 16 个 int 类型的整型数据进行算术右移, 移出的空位分别用 16 个 int 类型的整型数据的符号位填充, 移位位数由 vb 中对

应位置字分量的低 5 位或立即数 b 的低 5 位确定。可以使用“>>”符号替换。使用“>>”符号的时候,如果操作数是有符号的,生成算术右移指令,如果是无符号的,生成逻辑右移指令。约 1 拍。

函数名

simd_vrolw —— intv16 的循环左移

simd_vrolwi —— intv16 的循环左移（立即数格式）

参数说明

[u]intv16 va;

intv16/lit5 vb/b;

返回值

[u]intv16

功能说明

intv16 类型的循环左移,将 va 中的 16 个 int 类型的整型数据进行循环左移,移出的空位用移出位补充,移位位数由 vb 中对应位置字分量的低 5 位或立即数 b 的低 5 位确定。

函数名

simd_vaddl —— 长字向量加法

simd_vaddli —— 长字向量加法（立即数格式）

参数说明

[u]int512 va;

[u]int512 vb(int8 b);

返回值

[u]int512

功能说明

将va和vb中8个64位长字（或8位立即数b）分别对应相加，结果保存到vc中对应位置。约1拍。

函数名

simd_vsubl —— 长字向量减法

simd_vsubli —— 长字向量减法（立即数格式）

参数说明

[u]int512 va;

[u]int512 vb(int8 b);

返回值

[u]int512

功能说明

将va和vb中8个64位长字（或8位立即数b）分别对应相减，结果保存到vc中对应位置。约1拍。

函数名

simd_vornotw —— intv16 的逻辑或非

参数说明

[u]intv16 va;
[u]intv16 vb (int16 b) ;

返回值

[u]intv16

功能说明

intv16 类型的逻辑或非，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 进行逻辑与非运算。可以使用“|”和“~”符号替换。约 1 拍。

例如：

vc = simd_vornotw(va,vb);等价于：
vc = va | (~vb);

函数名

simd_vxorw —— intv16 的逻辑异或

参数说明

[u]intv16 va;
[u]intv16 vb (int16 b) ;

返回值

[u]intv16

功能说明

intv16 类型的逻辑异或，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 进行逻辑异或运算。可以使用“^”符号替换。约 1 拍。

函数名

simd_vbicw —— intv16 的逻辑与非

参数说明

[u]intv16 va;
[u]intv16 vb (int16 b) ;

返回值

[u]intv16

功能说明

intv16 类型的逻辑与非，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 进行逻辑与非运算。可以使用“&”和“~”符号替换。约 1 拍。

例如：

```
vc = simd_vbicw(va,vb);等价于：
vc = va & (~vb);
```

函数名

simd_vbisw —— intv16 的逻辑或

参数说明

```
[u]intv16 va;
[u]intv16 vb (int8 b) ;
```

返回值

```
[u]intv16
```

功能说明

intv16 类型的逻辑或，将 va 中的 16 个 int 类型的整型数据分别与 vb 中的 16 个 int 类型的整型数据或 8 位立即数 b 进行逻辑或运算。可以使用“|”符号替换。约 1 拍。

函数名

simd_vlog2xi —— 2 操作数 intv16 的可重构逻辑运算

参数说明

```
[u]intv16 va;
[u]intv16 vb;
const int zz;
```

返回值

```
[u]intv16 vc
```

功能说明

将 va 和 vb（或 8 位立即数）中对应的每一位，组成 2 位的二进制数，zz 表示的 4 位二进制值中选出一位，写入 vc 的对应位中。

对立即数格式的 vlog2x 指令，先将 8 位立即数零扩展为字整数，再分别与 va 寄存器中字整数向量的每个元素进行运算。约 1 拍。

函数名

simd_vlog2xi_i —— 2 操作数 intv16 的可重构逻辑运算

参数说明

```
[u]intv16 va;
int8 b;
```

const int zz;

返回值

[u]intv16 vc

功能说明

将 va 和 8 位立即数 b 中对应的每一位，组成 2 位的二进制数，zz 表示的 4 位二进制值中选出一位，写入 vc 的对应位中。

对立即数格式的vlog2x指令，先将8位立即数零扩展为字整数，再分别与va寄存器中字整数向量的每个元素进行运算。约1拍

函数名

simd_vlog3ri —— 3 操作数 intv16 的可重构逻辑运算

参数说明

[u]intv16 va;

[u]intv16 vb;

[u]intv16 vc;

const int zz;

返回值

[u]intv16 vd

功能说明

根据zz值，从相应的配置寄存器取出真值表<TRUTH>，然后将va、vb、vc中对应的每一位，组成3位的二进制数，从真值表<TRUTH>中的8位二进制值中选出一位，写入vd的对应位中。SW39000众核处理器从核提供16个真值表配置寄存器（与LOG3R指令共用），zz有效值为0~15。zz超出有效范围，将引起非法指令异常。约1拍

函数名

simd_vlog2xx —— 2 操作数 int512 的可重构逻辑运算

参数说明

[u]int512 va;

[u]int512 vb;

const int zz;

返回值

[u]int512 vc

功能说明

将 va 和 vb（或 8 位立即数）中对应的每一位，组成 2 位的二进制数，从 zz 表示的 4 位二进制值中选出一位，写入 vc 的对应位中。

对立即数格式的vlog2x指令，先将8位立即数零扩展为字整数，再分别与va寄存器中字整数向量的每个元素进行运算。约1拍

函数名

simd_vlog2xx_i —— 2 操作数 int512 的可重构逻辑运算

参数说明

[u]int512 va;
int8 b;
const int zz;

返回值

[u]int512 vc

功能说明

将 va 和 8 位立即数 b 中对应的每一位，组成 2 位的二进制数，从 zz 表示的 4 位二进制值中选出一位，写入 vc 的对应位中。

对立即数格式的vlog2x指令，先将8位立即数零扩展为字整数，再分别与va寄存器中字整数向量的每个元素进行运算。约1拍

函数名

simd_vlog3rx —— 3 操作数 int512 的可重构逻辑运算

参数说明

[u]int512 va;
[u]int512 vb;
[u]int512 vc;
const int zz;

返回值

[u]int512 vd

功能说明

根据zz值，从相应的配置寄存器取出真值表<TRUTH>，然后将va、vb、vc中对应的每一位，组成3位的二进制数，从真值表<TRUTH>中的8位二进制值中选出一位，写入vd的对应位中。SW39000众核处理器从核提供16个真值表配置寄存器（与LOG3R指令共用），zz有效值为0~15。zz超出有效范围，将引起非法指令异常。约1拍

函数名

simd_vcmpeqw —— intv16的等于比较运算

simd_vcmpeqwi —— intv16的等于比较运算（立即数格式）

参数说明

[u]intv16 va;
[u]intv16 vb(int8 b);

返回值

[u]intv16 vc

功能说明

对va中16个32位的字整数向量元素和vb中对应16个32位的字整数向量元素（或8位立即数#b）分别进行等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。约1拍

函数名

simd_vcamplew —— intv16的小于等于比较运算

simd_vcamplewi —— intv16的小于等于比较运算（立即数格式）

参数说明

[u]intv16 va;

[u]intv16 vb(int8 b);

返回值

[u]intv16 vc

功能说明

对va中16个32位的字整数向量元素和vb中对应16个32位的字整数向量元素（或8位立即数#b）分别进行小于等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。约1拍

函数名

simd_vcmltw —— intv16的小于比较运算

simd_vcmltwi —— intv16的小于比较运算（立即数格式）

参数说明

[u]intv16 va;

[u]intv16 vb(int8 b);

返回值

[u]intv16 vc

功能说明

对va中16个32位的字整数向量元素和vb中对应16个32位的字整数向量元素（或8位立即数#b）分别进行小于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。约1拍

函数名

simd_vcmpulew —— intv16的无符号小于等于比较运算

simd_vcmpulewi —— intv16的无符号小于等于比较运算（立即数格式）

式)

参数说明

[u]intv16 va;

[u]intv16 vb(int8 b);

返回值

[u]intv16 vc

功能说明

对va中16个32位的字整数向量元素和vb中对应16个32位的字整数向量元素（或8位立即数#b）分别进行无符号小于等于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。约1拍

函数名

simd_vcmpultw —— intv16的无符号小于比较运算
simd_vcmpultwi —— intv16的无符号小于比较运算（立即数格式）

参数说明

[u]intv16 va;
[u]intv16 vb(int8 b);

返回值

[u]intv16 vc

功能说明

对va中16个32位的字整数向量元素和vb中对应16个32位的字整数向量元素（或8位立即数#b）分别进行无符号小于比较运算，若满足指定关系，则vc中对应的字元素为“1”，否则为“0”。约1拍

函数名

simd_sllx —— int512 的逻辑左移
simd_srlx —— int512 的逻辑右移

参数说明

[u]int512 va;
int b; （lit9 b）

返回值

[u]int512 vc

功能说明

将va中512位整体左移/右移，移出的空位用“0”填充，移位位数由b低9位或9位立即数确定，结果写入vc中。约1拍

B.5.3.3 浮点向量运算函数接口

表 25 从核浮点向量运算函数接口

内部函数	操作	参数说明			
		VRc（返回值）	VRav	VRbv	VRcv
simd_vadds	+	floatv8	floatv8	floatv8	—
simd_vsubs	-	floatv8	floatv8	floatv8	—

simd_vmuls	*	floatv8	floatv8	floatv8	—
simd_vmas	乘加	floatv8	floatv8	floatv8	floatv8
simd_vmss	乘减	floatv8	floatv8	floatv8	floatv8
simd_vnmas	负乘加	floatv8	floatv8	floatv8	floatv8
simd_vnmss	负乘减	floatv8	floatv8	floatv8	floatv8
simd_vaddd	+	doublev8	doublev8	doublev8	—
simd_vsubd	-	doublev8	doublev8	doublev8	—
simd_vmuld	*	doublev8	doublev8	doublev8	—
simd_vmad	乘加	doublev8	doublev8	doublev8	doublev8
simd_vmsd	乘减	doublev8	doublev8	doublev8	doublev8
simd_vnmad	负乘加	doublev8	doublev8	doublev8	doublev8
simd_vnmsd	负乘减	doublev8	doublev8	doublev8	doublev8
simd_vfseleqs simd_vfseleqd	条件选择	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8
simd_vfsellts simd_vfselltd		floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8
simd_vfselles simd_vfselled		floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8
simd_vcpyss simd_vcpysd	符号拷贝	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vcpyses simd_vcpysed		floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vcpysns simd_vcpysnd		floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vfcmpeqs simd_vfcmpeqd	等于比较	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vfcmplts simd_vfcmpltd	小于等于 比较	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vfcmplts simd_vfcmpltd	小于比较	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vfcmpuns simd_vfcmpund	无序比较	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_smaxs simd_smaxd	大值比较 选择	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_smins simd_smind	小值比较 选择	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vdivs simd_vdivd	除法	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	—
simd_vfrecs simd_vfrecd	近似倒数	floatv8 doublev8	floatv8 doublev8	—	—

simd_vfrecps simd_vfrecpd	精确倒数	floatv8 doublev8	floatv8 doublev8	—	—
simd_vsqrts simd_vsqrtd	平方根	floatv8 doublev8	floatv8 doublev8	—	—
simd_vrsqrts simd_vrsqrtd	平方根 倒数	floatv8 doublev8	floatv8 doublev8	—	—
simd_vaddh	+	float16v32	float16v32	float16v32	—
simd_vsubh	-	float16v32	float16v32	float16v32	—
simd_vmulh	*	float16v32	float16v32	float16v32	—
simd_vdivh	除法	float16v32	float16v32	float16v32	—
simd_vsqrth	平方根	float16v32	float16v32	—	—
simd_vmah	乘加	float16v32	float16v32	float16v32	float16v32
simd_vmsh	乘减	float16v32	float16v32	float16v32	float16v32
simd_vnmah	负乘加	float16v32	float16v32	float16v32	float16v32
simd_vnmsh	负乘减	float16v32	float16v32	float16v32	float16v32
simd_vseleqh	条件选择	float16v32	float16v32	float16v32	float16v32
simd_vsellth		float16v32	float16v32	float16v32	float16v32
simd_vselleh		float16v32	float16v32	float16v32	float16v32
simd_vcpysh	符号拷贝	float16v32	float16v32	float16v32	—
simd_vcpyseh		float16v32	float16v32	float16v32	—
simd_vcpysnh		float16v32	float16v32	float16v32	—
simd_vfcmpeqh	等于比较	float16v32	float16v32	float16v32	—
simd_vfcmpleh	小于等于 比较	float16v32	float16v32	float16v32	—
simd_vfcmplth	小于比较	float16v32	float16v32	float16v32	—
simd_vfcmpunh	无序比较	float16v32	float16v32	float16v32	—
simd_smaxh	大值比较 选择	float16v32	float16v32	float16v32	—
simd_sminh	小值比较 选择	float16v32	float16v32	float16v32	—
simd_vfcvtsh	向量转换	float16v32	floatv8	lit2	—
simd_vfcvths	向量转换	floatv8	float16v32	lit2	—

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效
- 3、加法、减法、乘法、除法、乘加类运算支持运算符操作。

函数名

simd_vaddd —— doublev8 的加法运算

simd_vadds —— floatv8 的加法运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

加法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行加法运算。
可以用“+”符号替换。约 7 拍。

函数名

simd_vsubd —— doublev8 的减法运算

simd_vsubs —— floatv8 的减法运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

减法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行减法运算。
可以用“-”符号替换。约 7 拍。

函数名

simd_vmuld —— doublev8 的乘法运算

simd_vmuls —— floatv8 的乘法运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

乘法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行乘法运算。
可以用“*”符号替换。约 7 拍。

函数名

simd_vmad —— doublev8 的乘加运算

simd_vmas —— floatv8 的乘加运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

乘加运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行乘加运算。可以用“*”和“+”符号替换。约 7 拍。

ret = va*vb + vc;

函数名

simd_vmsd —— doublev8 的乘减运算

simd_vmss —— floatv8 的乘减运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

乘减运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行乘减运算。可以用“*”和“-”符号替换。约 7 拍。

ret=va*vb-vc;

函数名

simd_vnmad —— doublev8 的负乘加运算

simd_vnmas —— floatv8 的负乘加运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

负乘加运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个

浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。约 7 拍。

```
ret = -va*vb + vc;
```

函数名

simd_vnmsd —— doublev8 的负乘减运算

simd_vnmss —— floatv8 的负乘减运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

负乘减运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行负乘减运算。可以用“*”和“+”和“-”符号替换。约 7 拍。

```
ret = -va*vb - vc;
```

函数名

simd_vfseleqd —— doublev8 的等于选择运算

simd_vfseleqs —— floatv8 的等于选择运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

对 doublev8/floatv8 类型的参数 va 中的对应浮点向量元素进行比较，如果等于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。约 1 拍。

函数名

simd_vfselltd —— doublev8 的小于选择运算

simd_vfsellts —— floatv8 的小于选择运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

对 doublev8/floatv8 类型的参数 va 中的对应浮点向量元素进行比较，如果小于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。约 1 拍。

函数名

simd_vfselled —— doublev8 的小于等于选择运算

simd_vfselles —— floatv8 的小于等于选择运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

doublev8/floatv8 vc;

返回值

doublev8/floatv8

功能说明

对 doublev8/floatv8 类型的参数 va 中的对应浮点向量元素进行比较，如果小于等于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。约 1 拍。

函数名

simd_vcpysd —— doublev8 的拷贝符号

simd_vcpyss —— floatv8 的拷贝符号

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 va 中对应浮点向量的符号位，返回值中每个浮点向量的指数位和尾数位是 vb 中对应浮点向量的指数位和尾数位。约 1 拍。

函数名

simd_vcpysed —— doublev8 的拷贝符号和指数

simd_vcpyses —— floatv8 的拷贝符号和指数

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位和指数位是 **va** 中对应浮点向量的符号位和指数位，返回值中每个浮点向量的尾数位是 **vb** 中对应浮点向量的尾数位。约 1 拍。

函数名

simd_vcpysnd —— doublev8 的拷贝符号反码

simd_vcpysns —— floatv8 的拷贝符号反码

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位的反码，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。约 1 拍。

函数名

simd_vfcmpeqd —— doublev8 的等于比较运算

simd_vfcmpeqs —— floatv8 的等于比较运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8 vc

功能说明

将 **va** 和 **vb** 中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”浮点值（1.0）写入 **vc** 的对应位置，否则将真“0”写入 **vc** 的对应位置。

函数名

simd_vfcmpled —— doublev8 的小于等于比较运算

simd_vfcmple —— floatv8 的小于等于比较运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_vfcmpltd —— doublev8 的小于比较运算

simd_vfcmplts —— floatv8 的小于比较运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_vfcmpund —— doublev8 的无序比较运算

simd_vfcmpuns —— floatv8 的无序比较运算

参数说明

doublev8/floatv8 va;

doublev8/floatv8 vb;

返回值

doublev8/floatv8 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_smaxs —— floatv8 的大值比较选择

simd_smaxd —— doublev8 的大值比较选择

参数说明

floatv8/doublev8 va;

floatv8/doublev8 vb;

返回值

floatv8/doublev8 vc

功能说明

对 va 与 vb 中每个浮点元素进行大于比较测试，若条件成立，则将向量 va 对应的浮点元素写入 vc 的对应位置，否则将向量 vb 对应的浮点元素写入 vc 的对应位置。约 20 拍。

函数名

simd_smins ——floatv8 的小值比较选择

simd_smind ——doublev8 的小值比较选择

参数说明

floatv8/doublev8 va;

floatv8/doublev8 vb;

返回值

floatv8/doublev8 vc

功能说明

对 va 与 vb 中每个浮点元素进行小于比较测试，若条件成立，则将向量 va 对应的浮点元素写入 vc 的对应位置，否则将向量 vb 对应的浮点元素写入 vc 的对应位置。约 1 拍。约 20 拍。

函数名

simd_vdivs ——floatv8 的除法

simd_vdivd ——doublev8 的除法

参数说明

floatv8/doublev8 va;

floatv8/doublev8 vb;

返回值

floatv8/doublev8 vc

功能说明

除法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行除法运算。floatv8 约 55 拍，doublev8 约 75 拍。

函数名

simd_vfrecs ——floatv8 的近似倒数

simd_vfrecd ——doublev8 的近似倒数

参数说明

floatv8/doublev8 va;

返回值

floatv8/doublev8 vc

功能说明

求近似倒数运算，将 va 中的 8 个浮点数分别求近似倒数。约 8 拍。

函数名

simd_vfrecps ——floatv8 的精确倒数

simd_vfrecpd ——doublev8 的精确倒数

参数说明

floatv8/doublev8 va;

返回值

floatv8/doublev8 vc

功能说明

求精确倒数运算，将 va 中的 8 个浮点数分别求精确倒数。约 8 拍。

函数名

simd_vsqrts ——floatv8 的平方根

simd_vsqrtd ——doublev8 的平方根

参数说明

floatv8/doublev8 va;

返回值

floatv8/doublev8 vc

功能说明

求平方根运算，将 va 中的 8 个浮点数分别求平方根。floatv8 约 57 拍，doublev8 约 78 拍。

函数名

simd_vrsqrts ——floatv8 的平方根倒数

simd_vrsqrtd ——doublev8 的平方根倒数

参数说明

floatv8/doublev8 va;

返回值

floatv8/doublev8 vc

功能说明

求平方根倒数运算，将 va 中的 8 个浮点数分别求平方根倒数。floatv8 约 119 拍，doublev8 约 539 拍。

函数名

simd_vaddh —— float16v32 的加法运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

float16v32 类型的加法运算，将 va 中的 32 个_Float16 浮点数分别与 vb 中的 32 个_Float16 浮点数进行加法运算。可以用“+”符号替换。约 4 拍。

函数名

simd_vsubh —— float16v32 的减法运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

float16v32 类型的减法运算，将 va 中的 32 个_Float16 浮点数分别与 vb 中的 32 个_Float16 浮点数进行减法运算。可以用“-”符号替换。约 4 拍。

函数名

simd_ymulh —— float16v32 的乘法运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

float16v32 类型的乘法运算，将 va 中的 32 个_Float16 浮点数分别与 vb 中的 32 个_Float16 浮点数进行乘法运算。可以用“*”符号替换。约 4 拍。

函数名

simd_vdivh —— float16v32 的除法

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

除法运算，将 va 中的 32 个_Float16 浮点数分别与 vb 中的 32 个_Float16 浮点数进行除法运算。约 40 拍。

函数名

simd_vsqrth ——float16v32 的平方根

参数说明

float16v32 va;

返回值

float16v32 vc

功能说明

求平方根运算，将 va 中的 32 个_Float16 浮点数分别求平方根。约 53 拍。

函数名

simd_vmah —— float16v32 的乘加运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

float16v32 类型的乘加运算，将 va 中的 32 个_Float16 浮点数和 vb 中的 32 个_Float16 浮点数以及 vc 中的 32 个_Float16 浮点数分别进行乘加运算。可以用“*”和“+”符号替换。约 4 拍。

ret = va*vb + vc;

函数名

simd_vmsh —— float16v32 的乘减运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

float16v32 类型的乘减运算，将 va 中的 32 个_Float16 浮点数和 vb 中的 32

个_Float16 浮点数以及 vc 中的 32 个_Float16 浮点数分别进行乘减运算。可以用“*”和“-”符号替换。约 4 拍。

```
ret=va*vb-vc;
```

函数名

simd_vnmah —— float16v32 的负乘加运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

float16v32 类型的负乘加运算, 将 va 中的 32 个_Float16 浮点数和 vb 中的 32 个_Float16 浮点数以及 vc 中的 32 个_Float16 浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。约 4 拍。

```
ret = -va*vb + vc;
```

函数名

simd_vnmsh —— float16v32 的负乘减运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

float16v32 类型的负乘减运算, 将 va 中的 32 个_Float16 浮点数和 vb 中的 32 个_Float16 浮点数以及 vc 中的 32 个_Float16 浮点数分别进行负乘减运算。可以用“*”和“+”和“-”符号替换。约 4 拍。

```
ret = -va*vb - vc;
```

函数名

simd_vseleqh —— float16v32 的等于选择运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

对 float16v32 类型的参数 **va** 中的对应浮点向量元素进行比较，如果等于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。约 1 拍。

函数名

simd_vsellth —— float16v32 的小于选择运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

对 float16v32 类型的参数 **va** 中的对应浮点向量元素进行比较，如果小于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。约 1 拍。

函数名

simd_vselleh —— float16v32 的小于等于选择运算

参数说明

float16v32 va;

float16v32 vb;

float16v32 vc;

返回值

float16v32

功能说明

对 float16v32 类型的参数 **va** 中的对应浮点向量元素进行比较，如果小于等于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。约 1 拍。

函数名

simd_vcpysh —— float16v32 的拷贝符号

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。约 1 拍。

函数名

simd_vcpyseh —— float16v32 的拷贝符号和指数

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位和指数位是 **va** 中对应浮点向量的符号位和指数位，返回值中每个浮点向量的尾数位是 **vb** 中对应浮点向量的尾数位。约 1 拍。

函数名

simd_vcpysnh —— float16v32 的拷贝符号反码

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位的反码，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。约 1 拍。

函数名

simd_vfcmpeqh —— float16v32 的等于比较运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

将 **va** 和 **vb** 中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”

浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_vfcmpleh —— float16v32 的小于等于比较运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_vfcmplth —— float16v32 的小于比较运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_vfcmpunh —— float16v32 的无序比较运算

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（1.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。约 1 拍。

函数名

simd_smaxh —— float16v32 的大值比较选择

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

对 va 与 vb 中每个浮点元素进行大于比较测试，若条件成立，则将向量 va 对应的浮点元素写入 vc 的对应位置，否则将向量 vb 对应的浮点元素写入 vc 的对应位置。约 17 拍。

函数名

simd_sminh ——float16v32 的小值比较选择

参数说明

float16v32 va;

float16v32 vb;

返回值

float16v32 vc

功能说明

对 va 与 vb 中每个浮点元素进行小于比较测试，若条件成立，则将向量 va 对应的浮点元素写入 vc 的对应位置，否则将向量 vb 对应的浮点元素写入 vc 的对应位置。约 17 拍。

函数名

simd_vfcvtsh ——floatv8 向量到 float16v32 向量的转换

参数说明

floatv8 va;

lit2 b;

返回值

float16v32 vc

功能说明

va 中的 8 个 64 位单精度扩展格式浮点数，转换成 8 个 16 位半精度格式浮点数，然后根据立即数 b 的值写入 vc 指定位置，并将 vc 其它位置清 0。约 2 拍。

例如：floatv8 va=simd_set_floatv8(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0);

float16v32 vc = simd_vfcvtsh(va, 1);

simd_print_float16v32(vc);

打印的结果为： [0.0, 0.0, 8.0, 0.0, 0.0, 0.0, 7.0, 0.0, 0.0, 0.0, 6.0, 0.0, 0.0, 0.0, 5.0, 0.0, 0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 3.0, 0.0, 0.0, 0.0, 2.0, 0.0, 0.0, 0.0, 1.0, 0.0]

函数名

simd_vfcvths ——float16v32 向量到 floatv8 向量的转换

参数说明

float16v32 va;

lit2 b;

返回值

floatv8 vc

功能说明

根据立即数#b 的值，从 va 的每个 64 位中选择 1 个 16 位半精度浮点数分量，转换成 64 位的单精度浮点数扩展格式，将转换后的 8 个 64 位 SP 浮点数写入 vc。约 2 拍。

例如： float16v32 va=simd_set_float16v32 (0.0,1.0,2.0,3.0,4.0,5.0,6.0,7.0,
8.0,9.0,10.0,11.0,12.0,13.0,14.0,15.0,
16.0,17.0,18.0,19.0,20.0,21.0,22.0,23.0,
24.0,25.0,26.0,27.0,28.0,29.0,30.0,31.0);

floatv8 vc = simd_vfcvths(va, 1);

simd_print_float16v32(vc);

打印的结果为： [29.0, 25.0, 21.0, 17.0, 13.0, 9.0, 5.0, 1.0]

B.5.3.4 数据整理函数接口

表 26 从核数据整理函数接口

内部函数	操作	参数说明			
		VRc（返回值）	VRav	VRbv	VRcv
simd_vinswn n=(0,1,...,15)	字向量插入	intv16	int	intv16	/
simd_vinsfdn simd_vinsfsn n=(0,1...,7)	浮点向量插入	floatv8 doublev8	float double	floatv8 doublev8	/
simd_vextwn n=(0,1,...,15)	字向量提取	int	intv16	/	/
simd_vshfw	字向量混洗	intv16	intv16	intv16	intv16
simd_vshfs simd_vshfd	浮点向量混洗	floatv8 doublev8	floatv8 doublev8	floatv8 doublev8	Intv16
simd_vextfsn simd_vextfdn n=(0,1...,7)	浮点向量提取	float double	floatv8 doublev8	/	/
simd_vcopyw	字向量拷贝	intv16	int	/	/
simd_vcopyfs simd_vcopyfd	浮点向量拷贝	floatv8 doublev8	float double	/	/
simd_vconw	字向量元素拼接	intv16	intv16	intv16	void*

simd_vcons	浮点向量元素拼接	floatv8	floatv8	floatv8	void*
simd_vcond		doublev8	doublev8	doublev8	void*
simd_vinsh	半精度向量插入	float16v32	_Float16	float16v32	int
simd_vexth	半精度向量提取	_Float16	float16v32	int	/
simd_vcpyh	半精度向量拷贝	float16v32	_Float16	/	/
simd_vshfh	半精度向量混洗	float16v32	float16v32	float16v32	intv16

说明：

simd_vinswn 和 simd_vextwn 中 n 的取值只能是 0、1、2、3、4、5、6、7、8、9、10、11、12、13、14、15；simd_vinsfdn 和 simd_vextfdn 中 n 的取值只能是 0、1、2、3、4、5、6、7。

函数名

simd_vinswn—— 字向量元素插入运算

参数说明

int a;
intv16 vb;

返回值

intv16

功能说明

将32位字整数a替代vb中由n（有效值为0~15）指定32位字元素，组成并返回新的字整数向量。约3拍。

该接口实际上对应16个接口，分别是simd_vinsw0、simd_vinsw1、simd_vinsw2、simd_vinsw3、simd_vinsw4、simd_vinsw5、simd_vinsw6、simd_vinsw7、simd_vinsw8、simd_vinsw9、simd_vinsw10、simd_vinsw11、simd_vinsw12、simd_vinsw13、simd_vinsw14、simd_vinsw15，比如：

vc=simd_vinsw2(a,vb)

表示将32位字整数a替代vb的[95:64]这32位的字元素，组成新的vb返回给vc。

函数名

simd_vinsfdn—— D 浮点向量元素插入运算

simd_vinsfsn—— S 浮点向量元素插入运算

参数说明

double/float a;
doublev8/floatv8 vb;

返回值

doublev8/floatv8

功能说明

将浮点寄存器a的64位浮点数据替代vb中由n（有效值为0~7）指定的64位浮点元素，组成并返回新的浮点向量。

该接口实际上对应8个接口，分别是simd_vinsfd0、simd_vinsfd1、simd_vinsfd2、simd_vinsfd3、simd_vinsfd4、simd_vinsfd5、simd_vinsfd6、simd_vinsfd7，比如：

vc=simd_vinsfd1(a,vb)

表示a的64位浮点数据替代vb的[127:64]这64位的浮点元素，组成新的vb返回给vc。

函数名

simd_vextwn—— 字向量元素提取运算

参数说明

intv16 va;

返回值

int a

功能说明

将va中由n（有效值为0～15）指定的32位字元素，变为64位寄存器格式的字整数写入目的寄存器a。约3拍。

该接口实际上对应16个接口，分别是simd_vextw0、simd_vextw1、simd_vextw2、simd_vextw3、simd_vextw4、simd_vextw5、simd_vextw6、simd_vextw7、simd_vextw8、simd_vextw9、simd_vextw10、simd_vextw11、simd_vextw12、simd_vextw13、simd_vextw14、simd_vextw15，比如：

a=simd_vextw2(va)

表示va的 [95:64]这32位的字元素，变为64位寄存器格式的字整数写入a。

函数名

simd_vextfdn—— D 浮点向量元素提取运算

simd_vextfsn—— S 浮点向量元素提取运算

参数说明

doublev8/floatv8 va;

返回值

double/float a

功能说明

将va中由n（有效值为0～7）指定的S浮点或D浮点元素存入目的浮点寄存器a中。此指令可实现S浮点和D浮点的提取。约3拍。

该接口实际上对应8个接口，分别是simd_vextfd0、simd_vextfd1、simd_vextfd2、simd_vextfd3、simd_vextfd4、simd_vextfd5、simd_vextfd6、simd_vextfd7，比如：

a=simd_vextfd1(va)

表示va的 [127:64]这64位的浮点元素存入a的<63:0>中。

函数名

simd_vcopyw—— 字向量元素拷贝运算

参数说明

int a;

返回值

intv16

功能说明

将32位字整数a复制成16个相同元素组成新的字整数向量写入目的寄存器。约3拍。

函数名

simd_vcopyfd—— D 浮点向量元素拷贝运算

simd_vcopyfs—— S 浮点向量元素拷贝运算

参数说明

double/float a;

返回值

doublev8/floatv8

功能说明

将浮点数a复制成8个相同元素，组成新的浮点向量写入目的寄存器。约3拍。

函数名

simd_vconw—— 字向量元素拼接

参数说明

intv16 va;

intv16 vb;

void* addr;

返回值

intv16 vc

功能说明

字向量拼接，以字分量为单位，va存放不对界高段，vb存放不对界低段，addr存放不对界地址。根据不对界地址低6位（忽略其低2位），将va的高段数据和vb的低段数据拼接后形成新的向量，存放到vc中。约3拍。

函数名

simd_vcons—— S 浮点向量元素拼接

参数说明

floatv8 va;

floatv8 vb;

void* addr;

返回值

floatv8 vc

功能说明

S浮点向量拼接，以64位分量为单位，va存放不对界高段，vb存放不对界低段，addr存放不对界地址。根据不对界地址低5位（忽略其低2位），将va的高段数据和vb的低段数据拼接后形成新的向量，存放到vc中。约3拍。

函数名

simd_vcond——D 浮点向量元素拼接

参数说明

doublev8 va;

doublev8 vb;

void* addr;

返回值

doublev8 vc

功能说明

D浮点向量拼接，以64位分量为单位，va存放不对界高段，vb存放不对界低段，addr存放不对界地址。根据不对界地址低6位（忽略其低2位），将va的高段数据和vb的低段数据拼接后形成新的向量，存放到vc中。约3拍。

函数名

simd_vinsh——半精度浮点向量元素插入运算

参数说明

_Float16 a;

float16v32 vb;

const int c;

返回值

float16v32

功能说明

将16位半精度浮点数a替代vb中由c（有效值为0~31）指定的16位半精度浮点元素，组成并返回新的半精度浮点数向量。约3拍。

函数名

simd_vexth——半精度浮点向量元素提取运算

参数说明

float16v32 va;

const int b;

返回值

`_Float16 a`

功能说明

将`va`中由`b`（有效值为0~31）指定的16位半精度浮点元素，变为16位寄存器格式的半精度浮点数写入目的寄存器`a`。约3拍。

函数名

`simd_vcpyh`——半精度浮点向量元素拷贝运算

参数说明

`_Float16 a;`

返回值

`float16v32`

功能说明

将16位半精度浮点数`a`复制成32个相同元素组成新的半精度浮点数向量写入目的寄存器。约3拍。

函数名

`simd_vshfw`——字向量全混洗

参数说明

`intv16 va;`

`intv16 vb;`

`intv16 vc;`

返回值

`intv16 vd`

功能说明

操作数寄存器为`va`、`vb`、`vc`和`vd`。`vd`的十六个32bit的数由`va`和`vb`装填，约3拍。具体如何装填由`vc`说明：

`vc`的<79:75>指出`vd`的<511:480>来自`va`或`vb`的位置

`vc`的<74:70>指出`vd`的<479:448>来自`va`或`vb`的位置

.....

`vc`的<14:10>指出`vd`的<95:64>来自`va`或`vb`的位置

`vc`的<9:5>指出`vd`的<63:32>来自`va`或`vb`的位置

`vc`的<4:0>指出`vd`的<31:0>来自`va`或`vb`的位置

`vc`的5bit分量表示`vd`对应分量的来源，5bit分量的最高位为0时，`vd`相应的数来自`va`；为1时，`vd`相应的数来自`vb`。5bit分量位置的低位4bit（0~15）含义为：`va/vb`分成自然对界的16个32bit，位置为0表示第0个32bit（[31:0]），依次类推。

`vc`的[511:80]位无意义。

例子：

因为存储管理按小端地址格式（little-endian）来实现字节寻址，所以 vc 参数需要拼接前后参数来完整查看。如果只需要装填 va 的数，那么可以设置 vc 为 16 进制数：

```
int c[16] = {0x8a418820,0xc5a92839,0x7b9a,0,0,0,0,0,0,0,0,0,0,0,0};
```

```
simd_load(vc, c);
```

相当于二进制数：

```
int c[16] =
{0B10001010010000011000100000100000,0B110001011010100100101000001
11001,0B0111101110011010,0,0,0,0,0,0,0,0,0,0,0,0};
```

为了直观展现，以 5bit 为一组，按小端地址格式排列 c[16]是这样的效果：
0...0B01111 01110 01101 0,0B1100 01011 01010 01001 01000 00111 001,0B10 00101
00100 00011 00010 00001 00000

所以 vc 参数只装填了 va 的数

```
#include <simd.h>
#include <stdlib.h>
void main()
{
    int a[16] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
    int b[16] = {321,321,321,321,321,321,321,321,321,321,321,321,321,321,321};
    int c[16] = {0x8a418820,0xc5a92839,0x7b9a,0,0,0,0,0,0,0,0,0,0,0,0};
    int d[16] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};

    intv16 va, vb, vc, vd;
    simd_load(va, a);
    simd_load(vb, b);
    simd_load(vc, c);

    vd = simd_vshfw(va, vb, vc);
    simd_print_intv16(vd);
}
```

输出为： [16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

函数名

simd_vshfs—— 单精度向量全混洗

simd_vshfd—— 双精度向量全混洗

参数说明

floatv8 va/doublev8 va;

floatv8 vb/doublev8 vb;

intv16 vc;

返回值

floatv8 vd/doublev8 vd

功能说明

操作数寄存器为 va、vb、vc 和 vd。硬件没有浮点向量混洗指令，所以此时我们不能简单的将向量 va、vb 和 vd 看成 8 个 64 位浮点标量组成的向量，而需要将其看成由 16 个 32 位标量组成的向量。vd 的 16 个 32bit 的数由 va 和 vb 装填，约 3 拍。具体如何装填由 vc 说明。

vc 的<79:75>指出 vd 的<511:480>来自 va 或 vb 的位置

vc 的<74:70>指出 vd 的<479:448>来自 va 或 vb 的位置

.....

vc 的<9:5>指出 vd 的<63:32>来自 va 或 vb 的位置

vc 的<4:0>指出 vd 的<31:0>来自 va 或 vb 的位置

vc 的[79:0]位，拆分成 16 个 5bit 位置分量。每个 5bit 位置分量的最高位为 0 时，vd 相应的数来自 va；为 1 时，vd 相应的数来自 vb。5bit 位置的低 4bit（0~15）含义为：va/vb 分成自然对界的 16 个 32bit，位置为 0 表示第 0 个 32bit（[31:0]），依次类推。

vc 的[511:80]位无意义。

附加说明

由于单双精度浮点向量混洗接口借助字向量混洗指令实现，混洗的时候是以字（32 位）为单位进行，而非浮点数据在寄存器中占用的位数（64 位），故使用时若以浮点向量（8 个分量元素）为视角，则需要 vc 以每 10bit 为一组：

vc 的<4:0>决定浮点向量第一个元素来自 va 或 vb 的位置（此时需将 va 或 vb 当成 16 个 32 位数来看待，下同），vc 的<9:5>的值为 vc 的<4:0>的值加 1；

vc 的<14:10>决定向量第二个元素来自 va 或 vb 的位置，vc 的<19:15>的值为 vc 的<14:10>的值加 1；

.....

vc 的<74:70>决定向量第八个元素来自 va 或 vb 的位置，vc 的<79:75>的值为 vc 的<74:70>的值加 1。

数组 vc 的首地址需要满足 64 字节对界。用户使用时，需要根据希望实现的混洗方式，对 vc 的前 80 位赋以适当的值。

例子：

因为存储管理按小端地址格式（little-endian）来实现字节寻址，所以 vc 参数需要拼接前后参数来完整地查看。如果需要间隔装填 va 和 vb 的数，可以使用如下 16 进制参数：

```
int c[16]={0x8A49C820,0xCDE928BD,0xFF9A,0,0,0,0,0,0,0,0,0,0,0,0,0};
```

换算成二进制为：

```
int c[16]=
```

```
{0B10001010010010011100100000100000,0B11001101111010010010100010111101,0B1111111110011010,0,0,0,0,0,0,0,0,0,0,0,0};
```

为了直观展现，按小端地址格式排列 c[16]是这样的效果：

```
0...0B11111 11110 01101 0,0B1100 11011 11010 01001 01000 10111 101,0B1000101 00100 10011 10010 00001 00000
```

可以看到，每 10bit 为一组，两个 5bit 首位相同。

```
#include <stdlib.h>
#include <simd.h>
#include <slave.h>
doublev8 va,vb,vd;
intv16 vc;
int main()
{
    double a[8] = {0,1,2,3,4,5,6,7};
    double b[8] = {999,999,999,999,999,999,999,999};
    double d[16] = {0,999,2,999,4,999,6,999};
    int c[16]={0x8A49C820,0xCDE928BD,0xFF9A,0,0,0,0,0,0,0,0,0,0,0,0};

    simd_load(va,a);
    simd_load(vb,b);
    simd_load(vc,c);

    vd = simd_vshfd(va,vb,vc);
    simd_print_doublev8(vd);
}
```

输出为： [9.990000e+02, 6.000000e+00, 9.990000e+02, 4.000000e+00, 9.990000e+02, 2.000000e+00, 9.990000e+02, 0.000000e+00]

函数名

simd_vshfh—— 半精度向量全混洗

参数说明

```
float16v32 va;
float16v32 vb;
intv16 vc;
```

返回值

```
float16v32 vd
```

功能说明

操作数寄存器为 va、vb、vc 和 vd。vd 的 32 个 16bit 的数由 va 和 vb 装填，

约 3 拍。具体如何装填由 vc 说明：

vc 的<191:186>指出 vd 的<511:496>来自 va 或 vb 的位置

vc 的<185:180>指出 vd 的<495:480>来自 va 或 vb 的位置

.....

vc 的<11:6>指出 vd 的<31:16>来自 va 或 vb 的位置

vc 的<5:0>指出 vd 的<15:0>来自 va 或 vb 的位置

vc 的[191:0]位，拆分成 32 个 6bit 位置分量。每个 6bit 位置分量的最高位为 0 时，vd 相应的数来自 va；为 1 时，vd 相应的数来自 vb。6bit 位置的低 5bit（0~31）含义为：位置为 0 表示第 0 个 16bit（[15:0]），依次类推。

vc 的[511:192]位无意义。

例子：

因为存储管理按小端地址格式（little-endian）来实现字节寻址，所以 vc 需要拼接前后参数来完整查看。参数 vc 以 6bit 为一组，本例中如果 b[i]是个位数，则装填到 vd，如果 b[i]是十位数，则装填 a[i]到 vd，vc 参数 16 进制为：

```
int c[16] =
{0x648E2860,0xA2689E69,0x3CE34C2C,0x74CF2C70,0xA678DF6D,0x7DE7
5C6D,0,0,0,0,0,0,0,0,0,0};
```

二进制数为：

```
int c[16] =
{0B01100100100011100010100001100000,0B101000100110100010011110011
01001,0B00111100111000110100110000101100,0B011101001100111100101100011
10000,0B10100110011110001101111101101101,0B01111101111100111010111000110
1101,0,0,0,0,0,0,0,0,0,0};
```

为了直观展现，以 6bit 为一组，按小端地址格式排列 c[16]是这样的效果：

```
0...00B011111 011110 011101 011100 011011 01,0B1010 011001 111000 110111
110110 1101,0B01 110100 110011 110010 110001 110000,0B001111 001110 001101
001100 001011 00,0B1010 001001 101000 100111 100110 1001,0B01 100100 100011
100010 100001 100000
```

```
#include <simd.h>
```

```
#include <stdlib.h>
```

```
void main()
```

```
{
```

```
    _Float16 a[32] = {0};
```

```
    _Float16                                     b[32]                                     =
```

```
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
```

```
    //选择 vb 的个位数字装填到 vd，其他部分选择 va 装填到 vd
```

```
    int                                     c[16]                                     =
```

```
{0x648E2860,0xA2689E69,0x3CE34C2C,0x74CF2C70,0xA678DF6D,0x7DE75C6D,0,0,0,0,0,0,
```

```

0,0,0,0};

_Float16 d[32] = {1,2,3,4,5,6,7,8,9,0,0,0,0,0,0,1,2,3,4,5,6,7,8,9,0,0,0,0,0,0};
float16v32 va, vb, vd;
intv16 vc;
simd_load(va, a);
simd_load(vb, b);
simd_load(vc, c);

vd = simd_vshfh(va, vb, vc);
simd_print_float16v32(vd);
if(memcmp(d,&vd,64)!=0 ){
    printf("Result Error!\n");
    exit(-1);
}
printf("Result OK!\n");
}

```

输出为:

```

[ 0.000000e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00,
0.000000e+00, 9.000000e+00, 8.000000e+00, 7.000000e+00, 6.000000e+00, 5.000000e+00,
4.000000e+00, 3.000000e+00, 2.000000e+00, 1.000000e+00, 0.000000e+00, 0.000000e+00,
0.000000e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00, 9.000000e+00,
8.000000e+00, 7.000000e+00, 6.000000e+00, 5.000000e+00, 4.000000e+00, 3.000000e+00,
2.000000e+00, 1.000000e+00 ]
Result OK!

```

B.5.3.5 赋值函数接口

表 27 从核赋值函数接口

宏定义	操作	参数说明	
		返回值	参数
simd_set_intv16	赋值	intv16 类型	16 个 int 类型数
simd_set_uintv16	赋值	uintv16 类型	16 个 unsigned int 类型数
simd_set_int512	赋值	512 位八倍字	8 个 long
simd_set_uint512	赋值	512 位八倍字	8 个 unsigned long
simd_set_floatv8	赋值	floatv8 类型	8 个单精度浮点
simd_set_doublev8	赋值	doublev8 类型	8 个双精度浮点
simd_set_float16v32	赋值	float16v32 类型	32 个半精度浮点

函数名

simd_set_doublev8—— doublev8 赋值函数

simd_set_floatv8—— floatv8 赋值函数

参数说明

double/float K;
double/float L;
double/float M;
double/float N;
double/float O;
double/float P;
double/float Q;
double/float R;

返回值

doublev8/floatv8

功能说明

doublev8/floatv8 类型的赋值函数，将 8 个 double/float 类型的数据传递到向量变量中。

例如：doublev8 va=simd_set_doublev8(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0);
va 中的值是这样的：

8.0	7.0	6.0	5.0	4.0	3.0	2.0	1.0
512	448 447	384 383	320 319	256 255	192 191	128 127	64 63 0

函数名

simd_set_intv16—— intv16 赋值函数

simd_set_uintv16—— uintv16 赋值函数

参数说明

int/ unsigned int a0;
int/ unsigned int a1;
int/ unsigned int a2;
.....
int/ unsigned int a14;
int/ unsigned int a15;

返回值

intv16/uintv16

功能说明

intv16/uintv16 类型的赋值函数，将 16 个 int/unsigned int 类型的数据传递到向量变量中。

例如：va=simd_set_intv16(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16);
va 中的值是这样的：

16	15		3	2	1
511 480 479 448			95 64 63	32 31	0

函数名

[simd_set_int512](#) —— int512 赋值函数

[simd_set_uint512](#) —— uint512 赋值函数

参数说明

long/ unsigned long a;

long/ unsigned long b;

long/ unsigned long c;

long/ unsigned long d;

long/ unsigned long e;

long/ unsigned long f;

long/ unsigned long d;

long/ unsigned long h;

返回值

int512/uint512

功能说明

int512/uint512 类型的赋值函数，将 8 个 long/ unsigned long 类型的数据传递到向量变量中。

例如：va=simd_set_int512(1,2,3,4,5,6,7,8);

va 中的值是这样的：

8	7	6	5	4	3	2	1
512 448 447 384 383 320 319 256 255 192 191 128 127 64 63 0							

函数名

[simd_set_float16v32](#) ——float16v32 赋值函数

参数说明

_Float16 a0;

_Float16 a1;

_Float16 a2;

.....;

_Float16 a30;

_Float16 a31;

返回值

float16v32

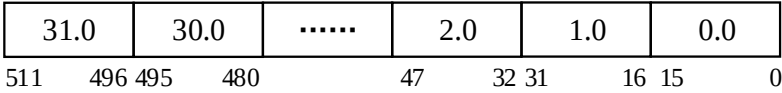
功能说明

float16v32 类型的赋值函数，将 32 个 _Float16 类型的数据传递到向量变量

中。

```
例如：va=simd_set_float16v32 (0.0,1.0,2.0,3.0,4.0,5.0,6.0,7.0,  
                                8.0,9.0,10.0,11.0,12.0,13.0,14.0,15.0,  
                                16.0,17.0,18.0,19.0,20.0,21.0,22.0,23.0,  
                                24.0,25.0,26.0,27.0,28.0,29.0,30.0,31.0);
```

va 中的值是这样的：



B.5.3.6 打印函数接口

表 28 从核扩展类型的打印函数接口

函数	操作	参数说明	
		返回值	参数
simd_fprint_intv16	打印	/	FILE*, intv16
simd_fprint_uintv16	打印	/	FILE *, uint16
simd_fprint_int512	打印	/	FILE*, int512
simd_fprint_uint512	打印	/	FILE *, uint512
simd_fprint_floatv8	打印	/	FILE*, floatv8
simd_fprint_doublev8	打印	/	FILE*, doublev8
simd_fprint_float16v32	打印	/	FILE*, float16v32
simd_print_intv16	打印	/	intv16
simd_print_uintv16	打印	/	uintv16
simd_print_int512	打印	/	int512
simd_print_uint512	打印	/	uint512
simd_print_floatv8	打印	/	floatv8
simd_print_doublev8	打印	/	doublev8
simd_print_float16v32	打印	/	float16v32
simd_fprint_intv16_X	打印	/	FILE*, intv16
simd_fprint_uintv16_X	打印	/	FILE *, uint16
simd_fprint_int512_X	打印	/	FILE*, int512
simd_fprint_uint512_X	打印	/	FILE *, uint512
simd_fprint_floatv8_X	打印	/	FILE*, floatv8
simd_fprint_doublev8_X	打印	/	FILE*, doublev8
simd_fprint_float16v32_X	打印	/	FILE*, float16v32
simd_print_intv16_X	打印	/	intv16
simd_print_uintv16_X	打印	/	uintv16

simd_print_int512_X	打印	/	int512
simd_print_uint512_X	打印	/	uint512
simd_print_floatv8_X	打印	/	floatv8
simd_print_doublev8_X	打印	/	doublev8
simd_print_float16v32_X	打印	/	float16v32

函数名

simd_fprint_intv16 —— intv16 打印到文件

simd_fprint_uintv16 —— uintv16 打印到文件

simd_fprint_intv16_X —— intv16 以 16 进制打印到文件

simd_fprint_uintv16_X —— uintv16 以 16 进制打印到文件

参数说明

FILE *file;

intv16/uintv16 va;

返回值

无

功能说明

intv16/uintv16 类型的数据以 16 个 int/ unsigned int 的形式打印到文件 file 中，其中 simd_fprint_intv16_X、simd_fprint_uintv16_X 以 16 进制格式打印。

例如： va=simd_set_intv16(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16);
 simd_fprint_intv16(stderr, va);
 simd_fprint_intv16_X(stderr, va);

打印的结果是这样的：

[16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

[0x10, 0xf, 0xe, 0xd, 0xc, 0xb, 0xa, 0x9, 0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1]

函数名

simd_fprint_int512 —— int512 打印到文件

simd_fprint_uint512 —— uint512 打印到文件

simd_fprint_int512_X —— int512 以 16 进制打印到文件

simd_fprint_uint512_X —— uint512 以 16 进制打印到文件

参数说明

FILE *file;

int512/uint512 va;

返回值

无

功能说明

int512/uint512 类型的数据以 8 个 long/ unsigned long 的形式打印到文件 file 中，其中 simd_fprint_int512_X、simd_fprint_uint512_X 以 16 进制格式打印。

例如: va=simd_set_int512(1,2,3,4,5,6,7,8);
 simd_fprint_int512(stderr, va);
 simd_fprint_int512_X(stderr, va);

打印的结果是这样的:

```
[ 8, 7, 6, 5, 4, 3, 2, 1 ]  
[ 0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1 ]
```

函数名

simd_fprint_floatv8 —— floatv8 打印到文件

simd_fprint_doublev8 —— doublev8 打印到文件

simd_fprint_floatv8_X —— floatv8 以 16 进制打印到文件

simd_fprint_doublev8_X —— doublev8 以 16 进制打印到文件

参数说明

FILE *file;
floatv8/doublev8 va;

返回值

无

功能说明

floatv8/doublev8 类型的数据以 8 个 float/double 的形式打印到文件 file 中, 其中 simd_fprint_floatv8_X、simd_fprint_doublev8_X 以 16 进制格式打印。

例如: va=simd_set_floatv8(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0);
 simd_fprint_floatv8(stderr, va);
 simd_fprint_floatv8_X(stderr, va);
 vb=simd_set_doublev8(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0);
 simd_fprint_doublev8_X(stderr, vb);

打印的结果是这样的:

```
[ 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0 ]  
[ 0x41000000, 0x40e00000, 0x40c00000, 0x40a00000, 0x40800000, 0x40400000,  
0x40000000, 0x3f800000 ]  
[   0x4020000000000000,   0x401c000000000000,   0x4018000000000000,  
0x4014000000000000, 0x4010000000000000, 0x4008000000000000, 0x4000000000000000,  
0x3ff0000000000000 ]
```

函数名

simd_fprint_float16v32 —— float16v32 打印到文件

simd_fprint_float16v32_X —— float16v32 以 16 进制打印到文件

参数说明

FILE *file;
float16v32 va;

返回值

无

功能说明

float16v32 类型的数据以 32 个_float16 的形式打印到文件 file 中，其中 simd_fprint_float16v32_X 以 16 进制格式打印。

```
例如： va=simd_set_float16v32 (0.0,1.0,2.0,3.0,4.0,5.0,6.0,7.0,
                                8.0,9.0,10.0,11.0,12.0,13.0,14.0,15.0,
                                16.0,17.0,18.0,19.0,20.0,21.0,22.0,23.0,
                                24.0,25.0,26.0,27.0,28.0,29.0,30.0,31.0);

simd_fprint_float16v32(stderr, va);
simd_fprint_float16v32_X(stderr, va);
```

打印的结果是这样的：

```
[ 31.0, 30.0, 29.0, 28.0, 27.0, 26.0, 25.0, 24.0, 23.0, 22.0, 21.0, 20.0, 19.0, 18.0, 17.0,
16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0, 0.0 ]
[ 0x4fc0, 0x4f80, 0x4f40, 0x4f00, 0x4ec0, 0x4e80, 0x4e40, 0x4e00, 0x4dc0, 0x4d80,
0x4d40, 0x4d00, 0x4cc0, 0x4c80, 0x4c40, 0x4c00, 0x4b80, 0x4b00, 0x4a80, 0x4a00, 0x4980,
0x4900, 0x4880, 0x4800, 0x4700, 0x4600, 0x4500, 0x4400, 0x4200, 0x4000, 0x3c00, 0x0 ]
```

函数名

simd_print_intv16 —— intv16 打印到屏幕

simd_print_uintv16 —— uintv16 打印到屏幕

simd_print_intv16_X —— intv16 以 16 进制打印到屏幕

simd_print_uintv16_X —— uintv16 以 16 进制打印到屏幕

参数说明

intv16/uintv16 va;

返回值

无

功能说明

intv16/uintv16 类型的数据以 16 个 int/ unsigned int 的形式打印到屏幕，其中 simd_print_intv16_X、simd_print_uintv16_X 以 16 进制格式打印。

```
例如： va=simd_set_intv16(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16);
simd_print_intv16(va);
simd_print_intv16_X(va);
```

打印的结果是这样的：

```
[16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 ]
[ 0x10, 0xf, 0xe, 0xd, 0xc, 0xb, 0xa, 0x9, 0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1 ]
```

函数名

simd_print_int512 —— int512 打印到屏幕

simd_print_uint512 —— uint512 打印到屏幕

simd_print_int512_X —— int512 以 16 进制打印到屏幕

simd_print_uint512_X —— uint512 以 16 进制打印到屏幕

参数说明

int512/uint512 va;

返回值

无

功能说明

int512/uint512 类型的数据以 8 个 long/ unsigned long 的形式打印到屏幕，其中 simd_print_int512_X、simd_print_uint512_X 以 16 进制格式打印。

例如： va=simd_set_int512(1,2,3,4,5,6,7,8);
simd_print_int512(va);
simd_print_int512_X(va);

打印的结果是这样的：

[8, 7, 6, 5, 4, 3, 2, 1]
[0x8, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1]

函数名

simd_print_floatv8 —— floatv8 打印到屏幕

simd_print_doublev8 —— doublev8 打印到屏幕

simd_print_floatv8_X —— floatv8 以 16 进制打印到屏幕

simd_print_doublev8_X —— doublev8 以 16 进制打印到屏幕

参数说明

floatv8/doublev8 va;

返回值

无

功能说明

floatv8/doublev8 类型的数据以 8 个 float/double 的形式打印到屏幕，其中 simd_print_floatv8_X、simd_print_doublev8_X 以 16 进制格式打印。

例如： va=simd_set_floatv8(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0);
simd_print_floatv8(va);
simd_print_floatv8_X(va);
vb=simd_set_doublev8(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0);
simd_print_doublev8_X(vb);

打印的结果是这样的：

[8.0,7.0, 6.0,5.0, 4.0, 3.0, 2.0, 1.0]
[0x41000000, 0x40e00000, 0x40c00000, 0x40a00000, 0x40800000, 0x40400000, 0x40000000, 0x3f800000]
[0x4020000000000000, 0x401c000000000000, 0x4018000000000000, 0x4014000000000000, 0x4010000000000000, 0x4008000000000000, 0x4000000000000000, 0x3ff0000000000000]

函数名

simd_print_float16v32 —— float16v32 打印到屏幕

simd_print_float16v32_X —— float16v32 以 16 进制打印到屏幕

参数说明

float16v32 va;

返回值

无

功能说明

float16v32 类型的数据以 32 个_Float16 的形式打印到屏幕中，其中 simd_print_float16v32_X 以 16 进制格式打印。

例如： va=simd_set_float16v32 (0.0,1.0,2.0,3.0,4.0,5.0,6.0,7.0,
8.0,9.0,10.0,11.0,12.0,13.0,14.0,15.0,
16.0,17.0,18.0,19.0,20.0,21.0,22.0,23.0,
24.0,25.0,26.0,27.0,28.0,29.0,30.0,31.0);

```
simd_print_float16v32(va);  
simd_print_float16v32_X(va);
```

打印的结果是这样的：

```
[ 31.0, 30.0, 29.0, 28.0, 27.0, 26.0, 25.0, 24.0, 23.0, 22.0, 21.0, 20.0, 19.0, 18.0, 17.0,  
16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0, 0.0 ]  
[ 0x4fc0, 0x4f80, 0x4f40, 0x4f00, 0x4ec0, 0x4e80, 0x4e40, 0x4e00, 0x4dc0, 0x4d80,  
0x4d40, 0x4d00, 0x4cc0, 0x4c80, 0x4c40, 0x4c00, 0x4b80, 0x4b00, 0x4a80, 0x4a00, 0x4980,  
0x4900, 0x4880, 0x4800, 0x4700, 0x4600, 0x4500, 0x4400, 0x4200, 0x4000, 0x3c00, 0x0 ]
```

B.5.3.7 向量归约接口

表 29 从核向量归约接口

函数	操作	参数说明	
		返回值	参数
simd_reduc_plusw	加法归约	int	intv16
simd_reduc_pluss		float	floatv8
simd_reduc_plusd		double	doublev8
simd_reduc_plush		_Float16	float16v32
simd_reduc_smaxs	最大值归约	float	floatv8
simd_reduc_smaxd		double	doublev8
simd_reduc_smaxh		_Float16	float16v32
simd_reduc_smins	最小值归约	float	floatv8
simd_reduc_smind		double	doublev8
simd_reduc_sminh		_Float16	float16v32

函数名

simd_reduc_plusw —— intv16 的加法归约

参数说明

intv16 va;

返回值

int

功能说明

intv16 类型的加法归约，将 va 中的 16 个 int 类型的整型数据归约求和。约 29 拍。

函数名

simd_reduc_pluss —— floatv8 的加法归约

simd_reduc_plusd —— doublev8 的加法归约

参数说明

floatv8/doublev8 va;

返回值

float/double

功能说明

加法归约，将 va 中的 8 个 float/doublet 类型的浮点数据归约求和。约 42 拍。

函数名

simd_reduc_smaxs —— floatv8 的最大值归约

simd_reduc_smaxd —— doublev8 的最大值归约

参数说明

floatv8/doublev8 va;

返回值

float/double

功能说明

最大值归约，求 va 中的 8 个 float/double 类型浮点数据中的最大值。约 45 拍。

函数名

simd_reduc_smins —— floatv8 的最小值归约

simd_reduc_smind —— doublev8 的最小值归约

参数说明

floatv8/doublev8 va;

返回值

float/double

功能说明

最小值归约，求 va 中的 8 个 float/double 类型浮点数据中的最小值。约 45 拍。

函数名

simd_reduc_plush —— float16v32 的加法归约

参数说明

Float16v32 va;

返回值

_Float16

功能说明

float16v32 类型的加法归约，将 va 中的 32 个 _Float16 类型的浮点数据归约求和。约 49 拍。

函数名

simd_reduc_smaxh —— float16v32 的最大值归约

参数说明

float16v32 va;

返回值

_Float16

功能说明

float16v32 类型的最大值归约，求 va 中的 32 个 _Float16 类型浮点数据中的最大值。约 54 拍。

函数名

simd_reduc_sminh —— float16v32 的最小值归约

参数说明

float16v32 va;

返回值

_Float16

功能说明

float16v32 类型的最小值归约，求 va 中的 32 个 _Float16 类型浮点数据中的最小值。约 54 拍。

附录 C：SW39000 众核处理器 SIMD 编程接口— Fortran 语言

C.1 简要说明

附录 B 以 C 语言为蓝本详细描述了基础编译器提供的向量功能、用法、注意事项等。本章主要介绍基础编译器针对 Fortran 语言提供的向量化功能、接口等。与程序设计语言前端无关的向量化方法、硬件结构特点等在这里不再赘述，请借鉴附录 B 中说明。鉴于使用 Fortran 语言的用户主要以科学与工程计算为主，这里面使用的数据类型主要以浮点数据为主，我们只提供了浮点的向量数据类型以及与之相关的运算、函数接口等。

C.2 快速入门

本节给出一个简单的 Fortran SIMD 程序，用来说明 Fortran 向量语言的某些基本特征。

C.2.1 一个简单的例子

```
program main
  v4real(kind=4) :: vf=(1.0,2.0,3.0,4.0)
  v4real(kind=4) :: vn=(2.0,3.0,4.0,5.0)
  real(kind=4),dimension(8) :: aaa
  aaa = (/2.,3.,4.,5.,1.,2.,3.,4./)
  call simd_load(vf,aaa(1))
  vf=simd_vadds(vf, vn)
  write(*,*)vf
end
```

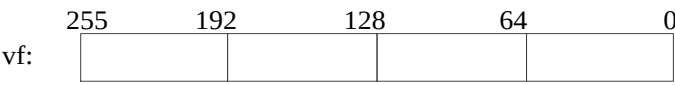
C.2.2 变量声明

在 SIMD 程序中，变量的声明方式与 Fortran 中是一致的，只是在标准 Fortran 的基础上扩展了一些基本类型。

主核：上面的例子中使用了：

```
v4real(kind=4) :: vf
```

声明了一个 `v4real(kind=4)` 类型的 SIMD 变量，表示 `vf` 是 4 个 `real(kind=4)`（64 位单精度浮点）组成的向量类型变量，对应于 C 语言中的 `floatv4` 数据类型。

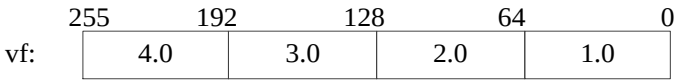


C.2.3 将标准类型的数据赋值给扩展类型

编译器扩展了一些方式用于将标准类型的数据赋给扩展类型。
例如，上面的例子中，

```
v4real(kind=4) :: vf=(1.0,2.0,3.0,4.0)
```

表示将 4 个 `real(kind=4)` 类型的数据赋值给 `vf`，执行完该语句，`vf` 的值为：



C.2.4 扩展类型变量的运算

编译器还对运算符进行了扩展，可以使用 Fortran 语言中的一些运算符对扩展类型的变量进行运算。

例如上面的例子中，

```
vf=simd_vadds(vf, vn)
```

其功能为将两个向量类型变量进行相加，实际上他也可以使用“+”符号实现：

```
vf = vf + vn
```

C.2.5 扩展类型的类型转换

编译器支持几种扩展类型的转换，所支持的几种类型转换和它们各自的用法在后面的章节中有详细的介绍。

C.2.6 扩展类型的打印

编译器提供了扩展类型的打印（半精度浮点向量不支持）。如例子中的：

```
write(*,*) vf
```

256 位数据被分成 4 个部分进行打印，其结果如下：

```
[ 10.0000000, 8.00000000, 6.00000000, 4.00000000]
```

C.3 SIMD 对 Fortran 语言的扩展

本节主要介绍 swgfotran 编译器针对 Fortran 语言进行的 SIMD 扩展。

C.3.1 主核数据类型的扩展

在标准 Fortran 的基础上扩展了 2 种数据类型：

- ◆ `v4real(kind=4)`: 64*4 单精度浮点
- ◆ `v4real(kind=8)`: 64*4 双精度浮点

表 30 Fortran 主核扩展数据类型

扩展类型	说明	值的范围
<code>v4real(kind=4)</code>	4 个 <code>real(kind=4)</code>	1.17549435e-38...3.40232847e38
<code>v4real(kind=8)</code>	4 个 <code>real(kind=8)</code>	2. 2250738585072013e-308 ...1.7976931328623158e308

C.3.2 从核数据类型的扩展

在标准 Fortran 的基础上扩展了 3 种数据类型：

- ◆ `v8real(kind=4)`: 64*8 单精度浮点
- ◆ `v8real(kind=8)`: 64*8 双精度浮点
- ◆ `v32real(kind=2)` : 16*32 半精度浮点

表 31 Fortran 从核扩展数据类型

扩展类型	说明	值的范围
<code>v8real(kind=4)</code>	8 个 <code>real(kind=4)</code>	1.17549435e-38...3.40232847e38
<code>v8real(kind=8)</code>	8 个 <code>real(kind=8)</code>	2. 2250738585072013e-308 ...1.7976931328623158e308
<code>v32real(kind=2)</code>	32 个 <code>real(kind=2)</code>	6.103515625e-5...65504

C.3.3 主核对界 (Host Alignment)

本节描述 SIMD 的主核对界要求，当使用 SIMD 数据进行编程的时候，用户必须清楚这些对界问题，因为编译器不会在对界方面产生报警或错误信息，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

向量编程时，编译时一般需要添加 `-faddress_align=n` 选项，一般情况下，主核向量程序设置 `n=32`。向量类型的数据项在内存中除了 `v4real(kind=4)` 类型是 16 字节对界，其余都是 32 字节对界。编译器保证用向量类型定义的变量在内存中是 32 字节（16 字节）对界的，编译器还保证所有类型数组的首地址是 32 字节对界的。在主核构架中，不对界的 Load/Store 会引发异常，操作系统收到异常信号后就会将这些 Load/Store 拆分成标准类型的 Load/Store，从而大大降低性能。

C.3.4 从核对界 (Slave Alignment)

本节描述 SIMD 的从核对界要求，当使用 SIMD 数据进行编程的时候，用户必须清楚这些对界问题，因为编译器不会在对界方面产生报警或错误信息，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

向量编程时，编译时一般需要添加 `-faddress_align=n` 选项，一般情况下，从核向量程序设置 `n=64`。向量类型的数据项在内存中除了 `v8real(kind=4)` 类型是 32 字节对界，其余都是 64 字节对界。编译器保证用向量类型定义的变量在内存中是 64 字节（32 字节）对界的，编译器还保证所有类型数组的首地址是 64 字节对界的。在从核架构中，不对界的 Load/Store 会引发异不对界访存异常，程序会直接退出。

使用 `-faddress_align=n` 后，编译器会将所有的数组首地址进行对界设置，即使这些数组与向量的访存操作无关，这在一定程度上会增加程序的内存需求。对性能要求较高的用户，可以不使用这个选项，而是在程序中手工设置其对界属性，具体方法可参见本指南中对界部分相关内容。

C.3.5 SIMD 对 Fortran 操作的扩展（主核）

假设定义了下面的一些变量：

```
v4real(kind=4) vf;
```

1、赋值

扩展类型可以在定义变量的时候像数组一样的赋初值，例如：

```
v4real(kind=4) :: vf=(1.0,2.0,3.0,4.0); 赋值的结果为：
```

	255	192	128	64	0
vf:	4.0	3.0	2.0	1.0	

或者

```
v4real(kind=4) :: vf=1.0; 赋值的结果为：
```

	255	192	128	64	0
vf:	1.0	1.0	1.0	1.0	

2、运算符的扩充

同时，本文本对标准 Fortran 中的运算符进行了扩充，例如“+、-、*、/”等运算符现在都可以用于扩展类型数据之间的运算。下面列出了所有的扩展情况，运算符后跟的数据类型表示用这些类型定义的变量可以进行相应的运算操作（“：”前是标准 Fortran 中的运算符号，“：”后面是该运算符扩展到的数据类型）：

+、-、*、/、== (.EQ.)、/= (.NE.)、<= (.LE.)、< (.LT.)、>= (.GE.)、> (.GT.):

```
v4real(kind=4/8)
```

在编写 SIMD 程序时可以直接使用这些运算符来进行扩展数据类型上的运

算。在使用的时候需要注意：在标准的 Fortran 语法中，比较类操作的结果为 logical 类型，对于扩展类型的比较类操作而言，为了与硬件指令的实际意义一致，其结果与源操作数的类型一致。

C.3.6 SIMD 对 Fortran 操作的扩展（从核）

假设定义了下面的一些变量：

`v8real(kind=4) vf;`

1、赋值

扩展类型可以在定义变量的时候像数组一样的赋初值，例如：

`vf=(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0);` 赋值的结果为：

	511	448	384	320	256	192	128	64	0
vf:	8.0	7.0	6.0	5.0	4.0	3.0	2.0	1.0	

或者

`vf=1.0;` 赋值的结果为：

	511	448	384	320	256	192	128	64	0
vf:	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	

对于 `v32real(kind=2)` 半精度浮点向量而言，综合考虑 Fortran 语句的换行问题、词法分析、语法分析的严谨性，以及实用性，暂不支持 `v32real(kind=2) :: vf = (1.0, 2.0, 3.0, 4.0, ……………, 31.0, 32.0)` 这样的初始化赋值。支持如下的赋值：

`v32real(kind=2) :: vf=1.0;`

2、运算符的扩充

同时，本文本对标准 Fortran 中的运算符进行了扩充，例如“+、-、*、/”等运算符现在都可以用于扩展类型数据之间的运算。下面列出了所有的扩展情况，运算符后跟的数据类型表示用这些类型定义的变量可以进行相应的运算操作（“：”前是标准 Fortran 中的运算符，“：”后面是该运算符扩展到的数据类型）：

+、-、*、/、== (.EQ.)、/= (.NE.)、<= (.LE.)、< (.LT.)、>= (.GE.)、> (.GT.)：
v8real(kind=4/8)、v32real(kind=2)

在编写 SIMD 程序时可以直接使用这些运算符来进行扩展数据类型上的运算。在使用的时候需要注意：在标准的 Fortran 语法中，比较类操作的结果为 logical 类型，对于扩展类型的比较类操作而言，为了与硬件指令的实际意义一致，其结果与源操作数的类型一致。

C.3.7 SIMD 对 Fortran 内部函数的扩展（主核）

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。

例如，要将一段普通的 Fortran 程序向量化经常需要将标准类型的数据先映射到 SIMD 变量中，然后才进行运算。本文扩充了 8 个 SIMD 宏定义接口，用于进行扩展数据类型与标准类型之间的映射操作。这 8 个宏定义是：

`simd_load()` —— 对界装入，将标准类型的数据从向量要求的对界内存地址映射到扩展类型变量

`simd_loadu()` —— 不对界装入，取不对界地址低位清零形成强制对界地址的连续两个向量，根据不对界地址低位进行拼接，形成不对界地址向量装入

`simd_load_u()` —— 不对界装入，将不对界的内存地址低位清零形成强制对界地址，然后将标准类型的数据从对界内存映射到扩展类型变量

`simd_loade()` —— 装入并扩展，将标准类型的数据从内存映射到扩展类型变量的低位并扩展到高位

`simd_store()` —— 对界存储，将扩展类型变量中的数据映射到向量要求的对界内存中

`simd_storeu()` —— 不对界存储，将不对界的内存地址低位清零形成强制对界地址，然后将扩展类型变量中的数据映射到对界内存中

`simd_storeul()` —— 不对界存储，将扩展类型变量中的低位数据映射到不对界内存的低位中

`simd_storeuh()` —— 不对界存储，将扩展类型变量中的高位数据映射到不对界内存的高位中

这些宏定义有两个参数，第一个是扩展类型，第二个是标准数据类型的指针，参数具体允许的类型请参照 C.4 节《Fortran 内部函数接口》。

对于扩展数据类型，许多操作是用标准 Fortran 的运算符号或库函数所无法表示的，这样的操作必须靠调用内部函数或宏定义来实现，例如：

`simd_vinsfd()` —— 浮点向量插入

`simd_vextfd()` —— 浮点向量提取

`simd_vcopyfd()` —— 浮点向量拷贝

`simd_vfseleqd()` —— 浮点向量等于选择

`simd_vfselltd()` —— 浮点向量小于选择

`simd_vfselled()` —— 浮点向量小于等于选择

另外，还有一些操作，一般由编译器自动生成，这里也提供了一些内部函数接口供编程时直接调用，主要有扩展浮点类型的乘加、乘减、负乘加和负乘减操作等。

所有扩展运算符的运算操作也都提供了内部函数接口。

这里只是对扩充内部函数接口进行了简单的介绍，详细的描述请查阅 C.4 节《Fortran 内部函数接口》。

C.3.8 SIMD 对 Fortran 内部函数的扩展（从核）

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。

例如，要将一段普通的 Fortran 程序向量化经常需要将标准类型的数据先映射到 SIMD 变量中，然后才进行运算。本文本扩充了 5 个 SIMD 宏定义接口，用于进行扩展数据类型与标准类型之间的映射操作。这 5 个宏定义是：

`simd_load()` —— 对界装入，将标准类型的数据从向量要求的对界内存地址映射到扩展类型变量

`simd_loadu()` —— 不对界装入，取不对界地址低位清零形成强制对界地址的连续两个向量，根据不对界地址低位进行拼接，形成不对界地址向量装入

`simd_load_u()` —— 不对界装入，将不对界的内存地址低位清零形成强制对界地址，然后将标准类型的数据从对界内存映射到扩展类型变量

`simd_store()` —— 对界存储，将扩展类型变量中的数据映射到向量要求的对界内存中

`simd_storeu()` —— 不对界存储，将不对界的内存地址低位清零形成强制对界地址，然后将扩展类型变量中的数据映射到对界内存中

这些宏定义有两个参数，第一个是扩展类型，第二个是标准数据类型的指针，参数具体允许的类型请参照 C.4 节《Fortran 内部函数接口》。

对于扩展数据类型，许多操作是用标准 Fortran 的运算符或库函数所无法表示的，这样的操作必须靠调用内部函数或宏定义来实现，例如：

`simd_vinsfd()` —— 浮点向量插入

`simd_vextfd()` —— 浮点向量提取

`simd_vcopyfd()` —— 浮点向量拷贝

`simd_vfseleqd()` —— 浮点向量等于选择

`simd_vfselltd()` —— 浮点向量小于选择

`simd_vfselled()` —— 浮点向量小于等于选择

另外，还有一些操作，一般由编译器自动生成，这里也提供了一些内部函数接口供编程时直接调用，主要有扩展浮点类型的乘加、乘减、负乘加和负乘减操作等。

所有扩展运算符的运算操作也都提供了内部函数接口。

这里只是对扩充内部函数接口进行了简单的介绍，详细的描述请查阅 C.4 《Fortran 内部函数接口》。

C.3.9 扩展类型的类型转换（主核）

在扩展类型中，编译器支持如下几种转换：

1、扩展浮点类型单、双精度之间的转换

扩展浮点类型 `v4real(kind=4)`和 `v4real(kind=8)`之间的转换，意义上与标准类型的单、双精度的类型转换一样，例如：

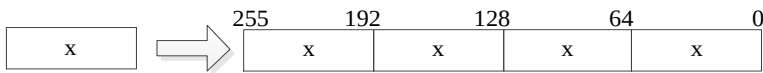
```
v4real(kind=4)  fv;      v4real(kind=8)  dv;
fv = dv;
```

2、标准类型到扩展类型的转换

标准类型到向量类型的转换是值的扩展。例如：

```
v4real(kind=4) va;      real(kind=4) x;
va = x;
```

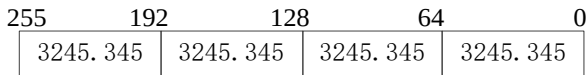
把 `x` 的值复制 4 份放到 `va` 中：



再例如：

```
v4real(kind=4)  fv;
fv = 3245.345;
```

结果是：



3、扩展类型到标准类型的转换

只是取扩展类型变量中最低的那部分。例如：

```
real(kind=4) a;
v4real(kind=4):: va = (1.0,2.0,3.0,4.0)
a = va
write(*,*) a
```

打印结果为：

1.0

C.3.10 扩展类型的类型转换（从核）

在扩展类型中，编译器支持如下几种转换：

1、扩展浮点类型单、双精度之间的转换

扩展浮点类型 `v8real(kind=4)`和 `v8real(kind=8)`之间的转换，意义上与标准类型的单、双精度的类型转换一样，例如：

```
v8real(kind=4)  fv;      v8real(kind=8)  dv;
fv = dv;
```

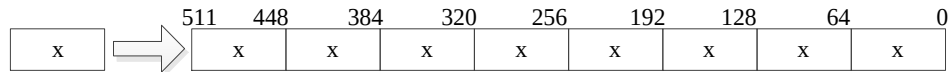
需要注意的是，因为半精度浮点向量与单/双精度浮点向量的元素个数不一致，因此不支持半精度浮点向量与单/双精度浮点向量之间的直接转换，如果需要进行他们之间的转换，请调用 C.4.3.2 节中的向量转换内部函数接口实现。

2、标准类型到扩展类型的转换

标准类型到向量类型的转换是值的扩展。例如：

```
v8real(kind=4) va;          real(kind=4) x;
va = x;
```

把 x 的值复制 8 份放到 va 中：



再例如：

```
v8real(kind=4) fv;
fv = 3245.345;
```

结果是：

511	448	384	320	256	192	128	64	0
3245.345	3245.345	3245.345	3245.345	3245.345	3245.345	3245.345	3245.345	3245.345

对于 v32real(kind=2)类型的半精度浮点向量，他的转换是将半精度浮点标量复制 32 份存放到向量中。

3、扩展类型到标准类型的转换

只是取扩展类型变量中最低的那部分。例如：

```
real(kind=4) a;
v8real(kind=4):: va = (1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0)
a = va
write(*,*) a
```

打印结果为：

1.0

C.4 Fortran 内部函数接口

C.4.1 使用的符号说明

VRav: 第一个向量参数

VRbv: 第二个向量参数

VRcv: 第三个向量参数

VRdv: 第四个向量参数

VRc: 向量返回值

Rbv: 无符号长整型变量，一般用于表示地址

Va: 一段内存地址中的数据 f

C.4.2 主核内部函数接口

C.4.2.1 装入/存储函数接口

表 32 Fortran 主核装入/存储函数接口

宏定义	操作	参数说明	
		VRav	Rbv
call simd_load()	装入	扩展类型	real(kind=4/8)*
call simd_loade()	装入并扩展		
call simd_loadu()	不对界装入		
call simd_load_u()			
call simd_store()	存储		
call simd_storeu()	不对界存储		
call simd_storeul()			
call simd_storeuh()			

说明：当 Rbv 为 real(kind=4)* 时，VRav 只能是 v4real(kind=4) 类型；Rbv 为 real(kind=8)* 类型时，VRav 只能是 v4real(kind=8) 类型。如果上述类型不匹配，编译器将报错。

函数名

call **simd_load** —— 扩展类型装入

参数说明

- 1、v4real(kind=4) 类型的装入
v4real(kind=4) va;
real(kind=4) *addr;
- 2、v4real(kind=8) 类型的装入
v4real(kind=8) va;
real(kind=8) *addr;

返回值：

无

功能说明

扩展类型的装入，将 32 字节（v4real(kind=4) 类型的装入是 16 字节）长度的数据从连续内存区域中装入到一个向量变量中。一般要求内存地址满足 32 字节（v4real(kind=4) 类型的装入是 16 字节）对界要求。

函数名

call **simd_loade** —— 扩展类型装入并扩展

参数说明

- 1、v4real(kind=4) 类型的装入并扩展

```
v4real(kind=4) va;  
real(kind=4) *addr;
```

2、v4real(kind=8)类型的装入并扩展

```
v4real(kind=8) va;  
real(kind=8) *addr;
```

返回值:

无

功能说明

扩展类型的装入并扩展，将 8 字节（v4real(kind=4)类型的装入是 4 字节）长度的数据从连续内存区域中装入到一个向量变量的低 64 位，并且将该数据扩展到高 192 位，使向量寄存器的四个寄存器具有相同的值。一般要求内存地址满足 8 字节（v4real(kind=4)类型的装入是 4 字节）对界要求。

函数名

call **simd_loadu** ——扩展类型不对界装入

参数说明

1、v4real(kind=4)类型的不对界装入

```
v4real(kind=4) va;  
real(kind=4) *addr;
```

2、v4real(kind=8)类型的不对界装入

```
v4real(kind=8) va;  
real(kind=8) *addr;
```

返回值:

无

功能说明

扩展类型的不对界装入，将 32 字节（v4real(kind=4)类型的装入是 16 字节）长度的数据，从任意地址开始的内存区域中装入到一个向量变量中。一般要求内存地址满足 8 字节（v4real(kind=4)类型的装入是 4 字节）对界要求。

函数名

call **simd_load_u** ——扩展类型不对界装入

参数说明

1、v4real(kind=4)类型的不对界装入

```
v4real(kind=4) va;  
real(kind=4) *addr;
```

2、v4real(kind=8)类型的不对界装入

```
v4real(kind=8) va;  
real(kind=8) *addr;
```

返回值:

无

功能说明

扩展类型的不对界装入，将不对界地址 `addr` 低位清零形成强制对界地址，将 32 字节（`v4real(kind=4)`类型的装入是 16 字节）长度的数据，从强制对界地址开始的连续内存区域中装入到一个向量变量中。

函数名

call **`simd_store`** —— 扩展类型存储

参数说明

- 1、`v4real(kind=4)`类型的存储

```
v4real(kind=4) va;  
real(kind=4) *addr;
```

- 2、`v4real(kind=8)`类型的存储

```
v4real(kind=8) va;  
real(kind=8) *addr;
```

返回值:

无

功能说明

扩展类型的存储，将一个向量变量中的数据存储到 32 字节（`v4real(kind=4)`类型的存储是 16 字节）连续内存区域中。一般要求内存地址满足 32 字节（`v4real(kind=4)`类型的装入是 16 字节）对界要求。

函数名

call **`simd_storeu`** —— 扩展类型不对界存储

参数说明

- 1、`v4real(kind=4)`类型的不对界存储

```
v4real(kind=4) va;  
real(kind=4) *addr;
```

- 2、`v4real(kind=8)`类型的不对界存储

```
v4real(kind=8) va;  
real(kind=8) *addr;
```

返回值:

无

功能说明

扩展类型的不对界存储，不对界地址低位清零形成强制对界地址，将一个向量变量中的数据存储到强制对界地址开始的 32 字节（`v4real(kind=4)`类型的装入是 16 字节）连续内存区域中。

函数名

call **simd_storeul** —— 扩展类型不对界存储

参数说明

- 1、v4real(kind=4)类型的不对界存储
v4real(kind=4) va;
real(kind=4) *addr;
- 2、v4real(kind=8)类型的不对界存储
v4real(kind=8) va;
real(kind=8) *addr;

返回值:

无

功能说明

扩展类型的不对界存储, 将一个向量变量中的低位数据存储到不对界地址的低位连续内存区域中。一般要求内存地址满足 8 字节 (v4real(kind=4)类型的装入是 4 字节) 对界要求。

函数名

call **simd_storeuh** —— 扩展类型不对界存储

参数说明

- 1、v4real(kind=4)类型的不对界存储
v4real(kind=4) va;
real(kind=4) *addr;
- 2、v4real(kind=8)类型的不对界存储
v4real(kind=8) va;
real(kind=8) *addr;

返回值:

无

功能说明

扩展类型的不对界存储, 将一个向量变量中的高位数据存储到不对界地址的高位连续内存区域中。一般要求内存地址满足 8 字节 (v4real(kind=4)类型的装入是 4 字节) 对界要求。

C.4.2.2 浮点向量运算函数接口

表 33 Fortran 主核浮点向量运算函数接口

内部函数	操作	参数说明			
		VRc (返回值)	VRav	VRbv	VRcv

simd_vadds	+	v4real(kin d=4/8)	v4real(kin d=4/8)	v4real(kin d=4/8)	—	
simd_vaddd						
simd_vsubs						-
simd_vsubd						
simd_vmuls	*					
simd_vmuld						
simd_vdivs						/
simd_vdivd						
simd_vsqrts	求平方根					
simd_vsqrtd						
simd_vmas	乘加			v4real(kin d=4/8)	v4real(kin d=4/8)	
simd_vmad						
simd_vmss	乘减					
simd_vmsd						
simd_vnmas	负乘加					
simd_vnmad						
simd_vnmss	负乘减					
simd_vnmsd						
simd_vfcmpeqs	==					
simd_vfcmpeqd	.EQ.					
simd_vfcmpnes	/=					
simd_vfcmpned	.NE.					
simd_vfcmplns	<=					
simd_vfcmplnd	.LE.					
simd_vfcmplts	<					
simd_vfcmpltd	.LT.					
simd_vfcmpges	>=					
simd_vfcmpged	.GE.					
simd_vfcmpgts	>					
simd_vfcmpgtd	.GT.					
simd_vfcmpuns	无序比较					
simd_vfcmpund						
simd_vfseleqs	等于 0 条件 选择	v4real(kin d=4/8)				
simd_vfseleqd						
simd_vfsellts	小于 0 条件 选择					
simd_vfselltd						
simd_vfselles	小于等于 0 条件选择					
simd_vfselled						
simd_vcpyss	拷贝符号		—			
simd_vcpysd						

simd_vcpyses	拷贝符号和 指数				
simd_vcpysed					
simd_vcpysns	拷贝符号的 反码				
simd_vcpysnd					

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效
- 3、加法、减法、乘法、除法、乘加类运算、比较类操作（除无序比较外）支持运算符操作。

函数名

simd_vadddd —— v4real(kind=8)类型的加法运算

simd_vadds —— v4real(kind=4)类型的加法运算

参数说明

- 1、v4real(kind=8)类型的加法运算
v4real(kind=8) va;
- 2、v4real(kind=8) vb;
- 3、v4real(kind=4)类型的加法运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

- 1、v4real(kind=8)
- 2、v4real(kind=4)

功能说明

浮点向量加法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行加法运算。可以用“+”符号替换。

函数名

simd_vsubdd —— v4real(kind=8)类型的减法运算

simd_vsubs —— v4real(kind=4)类型的减法运算

参数说明

- 1、v4real(kind=8)类型的减法运算
v4real(kind=8) va;
v4real(kind=8) vb;
- 2、v4real(kind=4)类型的减法运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

1、 v4real(kind=8)

2、 v4real(kind=4)

功能说明

浮点向量减法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行减法运算。可以用“-”符号替换。

函数名

simd_vmuld —— v4real(kind=8)类型的乘法运算

simd_vmuls ——v4real(kind=4)类型的乘法运算

参数说明

1、 v4real(kind=8)类型的乘法运算

v4real(kind=8) va;

v4real(kind=8) vb;

2、 v4real(kind=4)类型的乘法运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

1、 v4real(kind=8)

2、 v4real(kind=4)

功能说明

浮点向量乘法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行乘法运算。可以用“*”符号替换。

函数名

simd_vdivd —— v4real(kind=8)类型的除法运算

simd_vdivs ——v4real(kind=4)类型的除法运算

参数说明

1、 v4real(kind=8)类型的除法运算

v4real(kind=8) va;

v4real(kind=8) vb;

2、 v4real(kind=4)类型的除法运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

1、 v4real(kind=8)

2、 v4real(kind=4)

功能说明

浮点向量除法运算，将 va 中的 4 个浮点数分别与 vb 中的 4 个浮点数进行除

法运算。可以用“/”符号替换。

函数名

simd_vsqrtd —— v4real(kind=8)类型的求平方根运算

simd_vsqrts —— v4real(kind=4)类型的求平方根运算

参数说明

- 1、 v4real(kind=8)类型的求平方根运算

v4real(kind=8) va;

- 2、 v4real(kind=4)类型的求平方根运算

v4real(kind=4) va;

返回值

- 1、 v4real(kind=8)

- 2、 v4real(kind=4)

功能说明

浮点向量求平方根运算，将 va 中的 4 个浮点数分别进行求平方根运算。可以用“sqrt”函数替换。

vb = sqrt(va);

函数名

simd_ymad —— v4real(kind=8)类型的乘加运算

simd_ymas —— v4real(kind=4)类型的乘加运算

参数说明

- 1、 v4real(kind=8)类型的乘加运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、 v4real(kind=4)类型的乘加运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、 v4real(kind=8)

- 2、 v4real(kind=4)

功能说明

浮点向量乘加运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行乘加运算。可以用“*”和“+”符号替换。

vd = va*vb + vc;

函数名

simd_vmsd —— v4real(kind=8)类型的乘减运算

simd_vmss ——v4real(kind=4)类型的乘减运算

参数说明

- 1、v4real(kind=8)类型的乘减运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的乘减运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量乘减运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行乘减运算。可以用“*”和“-”符号替换。

vd = va*vb - vc;

函数名

simd_ynmad —— v4real(kind=8)类型的负乘加运算

simd_ynmas ——v4real(kind=4)类型的负乘加运算

参数说明

- 1、v4real(kind=8)类型的负乘加运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的负乘加运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量负乘加运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。

$vd = -va * vb + vc;$

函数名

simd_vnmsd —— v4real(kind=8)类型的负乘减运算

simd_vnmss —— v4real(kind=4)类型的负乘减运算

参数说明

- 1、v4real(kind=8)类型的负乘减运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的负乘减运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量负乘减运算，将 va 中的 4 个浮点数和 vb 中的 4 个浮点数以及 vc 中的 4 个浮点数分别进行负乘减运算。可以用“*”和“-”符号替换。

$vd = -va * vb - vc;$

函数名

simd_vfcmpeqd —— v4real(kind=8)类型的等于比较运算

simd_vfcmpeqs —— v4real(kind=4)类型的等于比较运算

参数说明

- 1、v4real(kind=8)类型的等于比较运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、v4real(kind=4)类型的等于比较运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、v4real(kind=8) vc

- 2、v4real(kind=4) vc

功能说明

浮点向量等于比较运算，将 va 和 vb 中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入

vc 的对应位置。可以用“==”或者“.EQ.”符号替换。

函数名

simd_vfcmpned —— v4real(kind=8)类型的不等于比较运算

simd_vfcmpnes ——v4real(kind=4)类型的不等于比较运算

参数说明

- 1、 v4real(kind=8)类型的不等于比较运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、 v4real(kind=4)类型的不等于比较运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc

- 2、 v4real(kind=4) vc

功能说明

浮点向量不等于比较运算，将 va 和 vb 中的对应浮点向量元素进行不等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“/=”或者“.NE.”符号替换。

函数名

simd_vfcmpged —— v4real(kind=8)类型的大于等于比较运算

simd_vfcmpges ——v4real(kind=4)类型的大于等于比较运算

参数说明

- 1、 v4real(kind=8)类型的大于等于比较运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、 v4real(kind=4)类型的大于等于比较运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc

- 2、 v4real(kind=4) vc

功能说明

浮点向量大于等于比较运算，将 va 和 vb 中的对应浮点向量元素进行大于等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“>=”或者“.GE.”符号替换。

函数名

simd_vfcmpgtd —— v4real(kind=8)类型的大于比较运算

simd_vfcmpgts ——v4real(kind=4)类型的大于比较运算

参数说明

- 1、 v4real(kind=8)类型的大于比较运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、 v4real(kind=4)类型的大于比较运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc

- 2、 v4real(kind=4) vc

功能说明

浮点向量大于比较运算，将 va 和 vb 中的对应浮点向量元素进行大于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“>”或者“.GT.”符号替换。

函数名

simd_vfcmpltd —— v4real(kind=8)类型的小于比较运算

simd_vfcmplts ——v4real(kind=4)类型的小于比较运算

参数说明

- 1、 v4real(kind=8)类型的小于比较运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、 v4real(kind=4)类型的小于比较运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc

- 2、 v4real(kind=4) vc

功能说明

浮点向量小于比较运算，将 va 和 vb 中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“<”或者“.LT.”符号替换。

函数名

simd_vfcmpled —— v4real(kind=8)类型的小于等于比较运算

simd_vfcmples ——v4real(kind=4)类型的小于等于比较运算

参数说明

- 1、 v4real(kind=8)类型的小于等于比较运算
v4real(kind=8) va;
v4real(kind=8) vb;
- 2、 v4real(kind=4)类型的小于等于比较运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc
- 2、 v4real(kind=4) vc

功能说明

浮点向量小于等于比较运算，将 va 和 vb 中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“<=”或者“.LE.”符号替换。

函数名

simd_vfcmpund —— v4real(kind=8)类型的无序比较运算

simd_vfcmpuns ——v4real(kind=4)类型的无序比较运算

参数说明

- 1、 v4real(kind=8)类型的无序比较运算
v4real(kind=8) va;
v4real(kind=8) vb;
- 2、 v4real(kind=4)类型的无序比较运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc
- 2、 v4real(kind=4) vc

功能说明

浮点向量无序比较运算，将 va 和 vb 中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_vfseleqd —— v4real(kind=8)类型的等于 0 条件选择运算

simd_vfseleqs ——v4real(kind=4)类型的等于 0 条件选择运算

参数说明

- 1、v4real(kind=8)类型的等于 0 条件选择运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的等于 0 条件选择运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量等于 0 条件选择运算，对 va 的每个浮点向量元素分别进行等于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vfselltd —— v4real(kind=8)类型的小于 0 条件选择运算

simd_vfsellts —— v4real(kind=4)类型的小于 0 条件选择运算

参数说明

- 1、v4real(kind=8)类型的小于 0 条件选择运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的小于 0 条件选择运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量小于 0 条件选择运算，对 va 的每个浮点向量元素分别进行小于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vfselled —— v4real(kind=8)类型的小于等于 0 条件选择运算

simd_vfselles ——v4real(kind=4)类型的小于等于 0 条件选择运算

参数说明

- 1、v4real(kind=8)类型的小于等于 0 条件选择运算

v4real(kind=8) va;

v4real(kind=8) vb;

v4real(kind=8) vc;

- 2、v4real(kind=4)类型的小于等于 0 条件选择运算

v4real(kind=4) va;

v4real(kind=4) vb;

v4real(kind=4) vc;

返回值

- 1、v4real(kind=8)

- 2、v4real(kind=4)

功能说明

浮点向量小于等于 0 条件选择运算，对 va 的每个浮点向量元素分别进行小于等于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vcpysd —— v4real(kind=8)类型的拷贝符号运算

simd_vcpyss ——v4real(kind=4)类型的拷贝符号运算

参数说明

- 1、v4real(kind=8)类型的拷贝符号运算

v4real(kind=8) va;

v4real(kind=8) vb;

- 2、v4real(kind=4)类型的拷贝符号运算

v4real(kind=4) va;

v4real(kind=4) vb;

返回值

- 1、v4real(kind=8) vc

- 2、v4real(kind=4) vc

功能说明

浮点向量拷贝符号运算，对va的每个浮点向量元素分别取其符号位，与向量vb对应浮点向量元素的指数和尾数拼接起来组成新的浮点向量元素写入vc。

函数名

simd_vcpysed —— v4real(kind=8)类型的拷贝符号和指数运算

simd_vcpyses ——v4real(kind=4)类型的拷贝符号和指数运算

参数说明

- 1、 v4real(kind=8)类型的拷贝符号和指数运算
v4real(kind=8) va;
v4real(kind=8) vb;
- 2、 v4real(kind=4)类型的拷贝符号和指数运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc
- 2、 v4real(kind=4) vc

功能说明

浮点向量拷贝符号和指数运算，对va的每个浮点向量元素分别取其符号位和指数，与向量vb对应浮点向量元素的尾数拼接起来组成新的浮点向量元素写入vc。

函数名

simd_vcpysnd —— v4real(kind=8)类型的拷贝符号反码运算
simd_vcpysns ——v4real(kind=4)类型的拷贝符号反码运算

参数说明

- 1、 v4real(kind=8)类型的拷贝符号反码运算
v4real(kind=8) va;
v4real(kind=8) vb;
- 2、 v4real(kind=4)类型的拷贝符号反码运算
v4real(kind=4) va;
v4real(kind=4) vb;

返回值

- 1、 v4real(kind=8) vc
- 2、 v4real(kind=4) vc

功能说明

浮点向量拷贝符号反码运算，对va的每个浮点向量元素分别取其符号位的反码作为符号位，与向量vb对应浮点向量元素的指数和尾数拼接起来组成新的浮点向量元素写入vc。

C.4.2.3 数据整理函数接口

表 34 Fortran 主核数据整理函数接口

内部函数	操作	参数说明			
		VRc (返回值)	VRav	VRbv	VRcv

simd_vinsfs	向量元素插入	v4real(kind=4/8)	real(kind=4/8)	v4real(kind=4/8)	lit5
simd_vinsfd					
simd_vextfs	向量元素提取	real(kind=4/8)	v4real(kind=4/8)	lit5	—
simd_vextfd					
simd_vcpyfs	向量元素拷贝	v4real(kind=4/8)	real(kind=4/8)	—	
simd_vcpyfd					
simd_vcons	向量元素拼接		v4real(kind=4/8)	v4real(kind=4/8)	integer(kind=8)
simd_vcond					

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效

函数名

simd_vinsfd —— v4real(kind=8)类型的向量元素插入操作

simd_vinsfs —— v4real(kind=4)类型的向量元素插入操作

参数说明

- 1、v4real(kind=8)类型的向量元素插入操作

```
real(kind=8) a;
v4real(kind=8) vb;
lit5 c;
```

- 2、v4real(kind=4)类型的向量元素插入操作

```
real(kind=4) a;
v4real(kind=4) vb;
lit5 c;
```

返回值

- 1、v4real(kind=8) vd
- 2、v4real(kind=4) vd

功能说明

浮点向量元素插入操作，将浮点变量 a 的 64 位浮点数据替代 vb 中由 c（有效值为 0~3，只有最低两位有效）指定的 64 位浮点元素，组成并返回新的浮点向量。比如：

```
vd=simd_vinsfd(a, vb, 2)
```

表示 a 的 64 位浮点数据替代 vb 的[191:128]这 64 位的浮点元素，组成新的 vb 返回给 vd。

函数名

simd_vextfd —— v4real(kind=8)类型的向量元素提取操作

simd_vextfs —— v4real(kind=4)类型的向量元素提取操作

参数说明

- 1、v4real(kind=8)类型的向量元素提取操作

v4real(kind=8) va;

lit5 b

- 2、v4real(kind=4)类型的向量元素提取操作

v4real(kind=4) va;

lit5 b

返回值

- 1、real(kind=8) c

- 2、real(kind=4) c

功能说明

浮点向量元素提取操作，将向量 va 中由 b（有效值为 0~3，只有最低两位有效）指定的 64 位浮点元素提取出来，返回给 c。比如：

c = simd_vextfd(va, 1)

表示将 va 中[127:64]这 64 位浮点元素赋值给 c 作为返回结果。

函数名

simd_vcopyfd —— v4real(kind=8)类型的向量元素拷贝操作

simd_vcopyfs ——v4real(kind=4)类型的向量元素拷贝操作

参数说明

- 1、v4real(kind=8)类型的向量元素拷贝操作

real(kind=8) a;

- 2、v4real(kind=4)类型的向量元素拷贝操作

real(kind=4) a;

返回值

- 1、v4real(kind=8) vb

- 2、v4real(kind=4) vb

功能说明

浮点向量元素拷贝操作，将 a 中浮点数据，复制成 4 个相同元素，组成新的浮点向量返回给 vb。

函数名

simd_vcond —— v4real(kind=8)类型的向量元素拼接操作

simd_vcons ——v4real(kind=4)类型的向量元素拼接操作

参数说明

- 1、v4real(kind=8)类型的向量元素拼接操作

v4real(kind=8) va;

v4real(kind=8) vb;

integer(kind=8) addr;

2、 v4real(kind=4)类型的向量元素拼接操作

```
v4real(kind=4) va;
v4real(kind=4) vb;
integer(kind=8) addr;
```

返回值

- 1、 v4real(kind=8) vd
- 2、 v4real(kind=4) vd

功能说明

浮点向量元素拼接操作，以 64 位分量为单位，va 存放需拼接的浮点向量低段，vb 存放需拼接的浮点向量高段，addr 存放不对界访存地址。根据不对界访存地址，将 vb 的低段数据放入目标向量的高段，将 va 的高段数据放入目标向量的低段，拼接后形成新的浮点向量，返回给 vd。

对于 v4real(kind=8)类型，addr 存放不对界访存地址低 5 位（忽略其低 3 位），对于 v4real(kind=4)类型，addr 存放不对界访存地址低 4 位（忽略其低 2 位）

C.4.3 从核内部函数接口

C.4.3.1 装入/存储函数接口

表 35 Fortran 从核装入/存储函数接口

宏定义	操作	参数说明	
		VRav	Rbv
call simd_loade()	装入并扩展	v32real(kind=2)	real(kind=2)*
call simd_load()	装入	扩展类型	real(kind=2/4/8)*
call simd_loadu()	不对界装入		
call simd_load_u()			
call simd_store()	存储		
call simd_storeu()	不对界存储		

说明：当 Rbv 为 real(kind=2)*时，VRav 只能是 v32real(kind=2)类型；当 Rbv 为 real(kind=4)*时，VRav 只能是 v8real(kind=4)类型；Rbv 为 real(kind=8)*类型时，VRav 只能是 v8real(kind=8)类型。如果上述类型不匹配，编译器将报错。

函数名

call **simd_loade** ——半精度浮点类型装入并扩展

参数说明

```
v32real(kind=2) va;
real(kind=2) *addr;
```

返回值：

无

功能说明

半精度浮点类型的装入并扩展，将 2 字节长度的数据从连续内存区域中装入到一个向量变量的低 16 位，并且将该数据扩展到高 496 位，使向量寄存器的 32 个半精度分量具有相同的值。从核仅支持半精度浮点向量的装入并扩展。**不允许访问 LDM 共享空间**，要求地址满足 2 字节对界要求。

函数名

call **simd_load** ——扩展类型装入

参数说明

- 1、v8real(kind=4)类型的装入

```
v8real(kind=4) va;  
real(kind=4) *addr;
```

- 2、v8real(kind=8)类型的装入

```
v8real(kind=8) va;  
real(kind=8) *addr;
```

- 3、v32real(kind=2)类型的装入

```
v32real(kind=2) va;  
real(kind=2) *addr;
```

返回值:

无

功能说明

扩展类型的装入，将 64 字节（v8real(kind=4)类型是 32 字节）长度的数据从连续内存区域中装入到一个向量变量中。一般要求内存地址满足 64 字节（v8real(kind=4)类型是 32 字节）对界要求。

函数名

call **simd_loadu** ——扩展类型不对界装入

参数说明

- 1、v8real (kind=4)类型的不对界装入

```
v8real (kind=4) va;  
real(kind=4) *addr;
```

- 2、v8real (kind=8)类型的不对界装入

```
v8real (kind=8) va;  
real(kind=8) *addr;
```

- 3、v32real(kind=2)类型的不对界装入

```
v32real(kind=2) va;  
real(kind=2) *addr;
```

返回值:

无

功能说明

扩展类型的不对界装入，将 64 字节（v8real(kind=4)类型的装入是 32 字节）长度的数据，从任意地址开始的内存区域中装入到一个向量变量中。一般要求内存地址满足 8 字节（v8real(kind=4)/ v32real(kind=2)类型是 4 字节）对界要求。

函数名

call **simd_load_u** —— 扩展类型不对界装入

参数说明

- 1、v8real(kind=4)类型的不对界装入
v8real(kind=4) va;
real(kind=4) *addr;
- 2、v8real(kind=8)类型的不对界装入
v8real(kind=8) va;
real(kind=8) *addr;
- 3、v32real(kind=2)类型的不对界装入
v32real(kind=2) va;
real(kind=2) *addr;

返回值:

无

功能说明

扩展类型的不对界装入，将不对界地址 addr 低位清零形成强制对界地址，将 64 字节（v8real(kind=4)类型是 32 字节）长度的数据，从强制对界地址开始的连续内存区域中装入到一个向量变量中。

函数名

call **simd_store** —— 扩展类型存储

参数说明

- 1、v8real(kind=4)类型的存储
v8real(kind=4) va;
real(kind=4) *addr;
- 2、v8real(kind=8)类型的存储
v8real(kind=8) va;
real(kind=8) *addr;
- 3、v32real(kind=2)类型的存储
v32real(kind=2) va;
real(kind=2) *addr;

返回值:

无

功能说明

扩展类型的存储, 将一个向量变量中的数据存储在 64 字节 (v8real(kind=4) 类型的存储是 32 字节) 连续内存区域中。一般要求内存地址满足 64 字节 (v8real(kind=4) 类型是 32 字节) 对界要求。

函数名

call **simd_storeu** —— 扩展类型不对界存储

参数说明

1、v8real(kind=4) 类型的不对界存储

v8real(kind=4) va;

real(kind=4) *addr;

2、v8real(kind=8) 类型的不对界存储

v8real(kind=8) va;

real(kind=8) *addr;

3、v32real(kind=2) 类型的不对界存储

v32real(kind=2) va;

real(kind=2) *addr;

返回值:

无

功能说明

扩展类型的存储, 将一个向量变量中的数据存储在 64 字节 (v8real(kind=4) 类型的存储是 32 字节) 连续内存区域中。一般要求内存地址满足 8 字节 (v8real(kind=4)/ v32real(kind=2) 类型是 4 字节) 对界要求。

C.4.3.2 浮点向量运算函数接口

表 36 Fortran 浮点向量运算函数接口

内部函数	操作	参数说明			
		VRc（返回值）	VRav	VRbv	VRcv
simd_vadds	+	v8real (kind=4/8)	v8real (kind=4/8)	v8real(ki nd=4/8)	—
simd_vaddd					
simd_vsubs	-				
simd_vsubd					
simd_vmuls	*				
simd_vmuld					

simd_vdivs	/				
simd_vdivd					
simd_vmas	乘加				v8real(kind=4/8)
simd_vmad					
simd_vmss	乘减				
simd_vmsd					
simd_vnmas	负乘加				
simd_vnmad					
simd_vnmss	负乘减				
simd_vnmsd					
simd_vfcmpeqs	== .EQ.				
simd_vfcmpeqd					
simd_vfcmpnes	/=				
simd_vfcmpned					
simd_vfcmple	<= .LE.				
simd_vfcmpld					
simd_vfcmplt	< .LT.				
simd_vfcmpltd					
simd_vfcmpge	>= .GE.				
simd_vfcmpged					
simd_vfcmpgt	> .GT.				
simd_vfcmpgtd					
simd_vfcmpun	无序比较				
simd_vfcmpund					
simd_vfseleq	等于 0 条件选择				
simd_vfseleqd					
simd_vfsell	小于 0 条件选择				v8real(kind=4/8)
simd_vfselltd					
simd_vfselle	小于等于 0 条件选择				
simd_vfselled					
simd_vcpyss	拷贝符号				
simd_vcpysd					
simd_vcpyses	拷贝符号和指数				
simd_vcpysed					
simd_vcpysns	拷贝符号的反码				
simd_vcpysnd					
simd_vfrecs simd_vfrecd	近似倒数			—	
simd_vfrecps	精确倒数				

simd_vfrecpd					
simd_vsqrts simd_vsqrtd	平方根				
simd_vrsqrts simd_vrsqrtd	平方根倒数				
simd_vaddh	+			v32real(kind=2)	—
simd_vsubh	-				
simd_vmulh	*				
simd_vdivh	/				
simd_vsqrth	平方根			—	
simd_vmah	乘加				
simd_vmsh	乘减				
simd_vnmah	负乘加				
simd_vnmsh	负乘减				
simd_vseleqh	条件选择	v32real(kind=2)	v32real(kind =2)	v32real(kind=2)	v32real(kind=2)
simd_vsellth					
simd_vselleh					
simd_vcpysh	符号拷贝			v32real(kind=2)	
simd_vcpyseh					
simd_vcpysnh					
simd_vfcmpeqh	== / .EQ.	v32real(kind=2)	v32real(kind =2)	v32real(kind=2)	—
simd_vfcmpneh	!= / .NE.				
simd_vfcmpleh	<= / .LE.				
simd_vfcmplth	< / .LT.				
simd_vfcmpunh	无序比较				
simd_smaxh	大值比较选择				
simd_sminh	小值比较选择				
simd_vfcvtsh	向量转换	v32real(kind=2)	v8real (kind=4)	lit2	
simd_vfcvths	向量转换	v8real(kind=4)	v32real (kind=2)	lit2	

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效
- 3、加法、减法、乘法、除法、乘加类运算、比较类操作（除无序比较外）支持运算符操作。

函数名

simd_vadddd —— v8real(kind=8)类型的加法运算

simd_vadds —— v8real(kind=4)类型的加法运算

参数说明

- 1、v8real(kind=8)类型的加法运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、v8real(kind=4)类型的加法运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

- 1、v8real(kind=8)

- 2、v8real(kind=4)

功能说明

浮点向量加法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行加法运算。可以用“+”符号替换。

函数名

simd_vsubd —— v8real(kind=8)类型的减法运算

simd_vsubs —— v8real(kind=4)类型的减法运算

参数说明

- 1、v8real(kind=8)类型的减法运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、v8real(kind=4)类型的减法运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

- 1、v8real(kind=8)

- 2、v8real(kind=4)

功能说明

浮点向量减法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行减法运算。可以用“-”符号替换。

函数名

simd_vmuld —— v8real(kind=8)类型的乘法运算

simd_vmuls —— v8real(kind=4)类型的乘法运算

参数说明

- 1、v8real(kind=8)类型的乘法运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、v8real(kind=4)类型的乘法运算

```
v8real(kind=4) va;  
v8real(kind=4) vb;
```

返回值

- 1、 v8real(kind=8)
- 2、 v8real(kind=4)

功能说明

浮点向量乘法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行乘法运算。可以用“*”符号替换。

函数名

simd_vdivd —— v8real(kind=8)类型的除法运算

simd_vdivs —— v8real(kind=4)类型的除法运算

参数说明

- 1、 v8real(kind=8)类型的除法运算
v8real(kind=8) va;
v8real(kind=8) vb;
- 2、 v8real(kind=4)类型的除法运算
v8real(kind=4) va;
v8real(kind=4) vb;

返回值

- 1、 v8real(kind=8)
- 2、 v8real(kind=4)

功能说明

浮点向量除法运算，将 va 中的 8 个浮点数分别与 vb 中的 8 个浮点数进行除法运算。可以用“/”符号替换。

函数名

simd_vmad —— v8real(kind=8)类型的乘加运算

simd_vmas —— v8real(kind=4)类型的乘加运算

参数说明

- 1、 v8real(kind=8)类型的乘加运算
v8real(kind=8) va;
v8real(kind=8) vb;
v8real(kind=8) vc;
- 2、 v8real(kind=4)类型的乘加运算
v8real(kind=4) va;
v8real(kind=4) vb;
v8real(kind=4) vc;

返回值

- 1、v8real(kind=8)
- 2、v8real(kind=4)

功能说明

浮点向量乘加运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行乘加运算。可以用“*”和“+”符号替换。

$vd = va * vb + vc;$

函数名

simd_vmsd —— v8real(kind=8)类型的乘减运算

simd_vmss ——v8real(kind=4)类型的乘减运算

参数说明

- 1、v8real(kind=8)类型的乘减运算

v8real(kind=8) va;

v8real(kind=8) vb;

v8real(kind=8) vc;

- 2、v8real(kind=4)类型的乘减运算

v8real(kind=4) va;

v8real(kind=4) vb;

v8real(kind=4) vc;

返回值

- 1、v8real(kind=8)
- 2、v8real(kind=4)

功能说明

浮点向量乘减运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行乘减运算。可以用“*”和“-”符号替换。

$vd = va * vb - vc;$

函数名

simd_vnmad —— v8real(kind=8)类型的负乘加运算

simd_vnmas ——v8real(kind=4)类型的负乘加运算

参数说明

- 1、v8real(kind=8)类型的负乘加运算

v8real(kind=8) va;

v8real(kind=8) vb;

v8real(kind=8) vc;

- 2、v8real(kind=4)类型的负乘加运算

v8real(kind=4) va;


```
v8real(kind=4) vb;  
v8real(kind=4) vc;
```

返回值

- 1、 v8real(kind=8)
- 2、 v8real(kind=4)

功能说明

浮点向量负乘加运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。

```
vd = -va*vb + vc;
```

函数名

simd_vnmsd —— v8real(kind=8)类型的负乘减运算
simd_vnmss ——v8real(kind=4)类型的负乘减运算

参数说明

- 1、 v8real(kind=8)类型的负乘减运算
v8real(kind=8) va;
v8real(kind=8) vb;
v8real(kind=8) vc;
- 2、 v8real(kind=4)类型的负乘减运算
v8real(kind=4) va;
v8real(kind=4) vb;
v8real(kind=4) vc;

返回值

- 1、 v8real(kind=8)
- 2、 v8real(kind=4)

功能说明

浮点向量负乘减运算，将 va 中的 8 个浮点数和 vb 中的 8 个浮点数以及 vc 中的 8 个浮点数分别进行负乘减运算。可以用“*”和“-”符号替换。

```
vd = -va*vb - vc;
```

函数名

simd_vfcmpeqd —— v8real(kind=8)类型的等于比较运算
simd_vfcmpeqs ——v8real(kind=4)类型的等于比较运算

参数说明

- 1、 v8real(kind=8)类型的等于比较运算
v8real(kind=8) va;
v8real(kind=8) vb;
- 2、 v8real(kind=4)类型的等于比较运算

`v8real(kind=4) va;`

`v8real(kind=4) vb;`

返回值

1、 `v8real(kind=8) vc`

2、 `v8real(kind=4) vc`

功能说明

浮点向量等于比较运算，将 `va` 和 `vb` 中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 `vc` 的对应位置，否则将真“0”写入 `vc` 的对应位置。可以用“==”或者“.EQ.”符号替换。

函数名

`simd_vfcmpned` —— `v8real(kind=8)`类型的不等于比较运算

`simd_vfcmpnes` —— `v8real(kind=4)`类型的不等于比较运算

参数说明

1、 `v8real(kind=8)`类型的不等于比较运算

`v8real(kind=8) va;`

`v8real(kind=8) vb;`

2、 `v8real(kind=4)`类型的不等于比较运算

`v8real(kind=4) va;`

`v8real(kind=4) vb;`

返回值

1、 `v8real(kind=8) vc`

2、 `v8real(kind=4) vc`

功能说明

浮点向量不等于比较运算，将 `va` 和 `vb` 中的对应浮点向量元素进行不等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 `vc` 的对应位置，否则将真“0”写入 `vc` 的对应位置。可以用“/=”或者“.NE.”符号替换。

函数名

`simd_vfcmpged` —— `v8real(kind=8)`类型的大于等于比较运算

`simd_vfcmpges` —— `v8real(kind=4)`类型的大于等于比较运算

参数说明

1、 `v8real(kind=8)`类型的大于等于比较运算

`v8real(kind=8) va;`

`v8real(kind=8) vb;`

2、 `v8real(kind=4)`类型的大于等于比较运算

`v8real(kind=4) va;`

`v8real(kind=4) vb;`

返回值

- 1、 v8real(kind=8) vc
- 2、 v8real(kind=4) vc

功能说明

浮点向量大于等于比较运算，将 va 和 vb 中的对应浮点向量元素进行大于等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“>=”或者“.GE.”符号替换。

函数名

simd_vfcmpgtd —— v8real(kind=8)类型的大于比较运算

simd_vfcmpgts ——v8real(kind=4)类型的大于比较运算

参数说明

- 1、 v8real(kind=8)类型的大于比较运算
v8real(kind=8) va;
v8real(kind=8) vb;
- 2、 v8real(kind=4)类型的大于比较运算
v8real(kind=4) va;
v8real(kind=4) vb;

返回值

- 1、 v8real(kind=8) vc
- 2、 v8real(kind=4) vc

功能说明

浮点向量大于比较运算，将 va 和 vb 中的对应浮点向量元素进行大于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“>”或者“.GT.”符号替换。

函数名

simd_vfcmpltd —— v8real(kind=8)类型的小于比较运算

simd_vfcmplts ——v8real(kind=4)类型的小于比较运算

参数说明

- 1、 v8real(kind=8)类型的小于比较运算
v8real(kind=8) va;
v8real(kind=8) vb;
- 2、 v8real(kind=4)类型的小于比较运算
v8real(kind=4) va;
v8real(kind=4) vb;

返回值

- 1、 v8real(kind=8) vc

2、 v8real(kind=4) vc

功能说明

浮点向量小于比较运算，将 va 和 vb 中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“<”或者“.LT.”符号替换。

函数名

simd_vfcmpled —— v8real(kind=8)类型的小于等于比较运算

simd_vfcmples ——v8real(kind=4)类型的小于等于比较运算

参数说明

1、 v8real(kind=8)类型的小于等于比较运算

v8real(kind=8) va;

v8real(kind=8) vb;

2、 v8real(kind=4)类型的小于等于比较运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

1、 v8real(kind=8) vc

2、 v8real(kind=4) vc

功能说明

浮点向量小于等于比较运算，将 va 和 vb 中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。可以用“<=”或者“.LE.”符号替换。

函数名

simd_vfcmpund —— v8real(kind=8)类型的无序比较运算

simd_vfcmpuns ——v8real(kind=4)类型的无序比较运算

参数说明

1、 v8real(kind=8)类型的无序比较运算

v8real(kind=8) va;

v8real(kind=8) vb;

2、 v8real(kind=4)类型的无序比较运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

1、 v8real(kind=8) vc

2、 v8real(kind=4) vc

功能说明

浮点向量无序比较运算，将 **va** 和 **vb** 中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（2.0）写入 **vc** 的对应位置，否则将真“0”写入 **vc** 的对应位置。

函数名

simd_vcpysd —— **v8real(kind=8)**类型的拷贝符号运算

simd_vcpyss —— **v8real(kind=4)**类型的拷贝符号运算

参数说明

- 1、 **v8real(kind=8)**类型的拷贝符号运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、 **v8real(kind=4)**类型的拷贝符号运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

- 1、 **v8real(kind=8) vc**

- 2、 **v8real(kind=4) vc**

功能说明

浮点向量拷贝符号运算，对**va**的每个浮点向量元素分别取其符号位，与向量**vb**对应浮点向量元素的指数和尾数拼接起来组成新的浮点向量元素写入**vc**。

函数名

simd_vcpysed —— **v8real(kind=8)**类型的拷贝符号和指数运算

simd_vcpyses —— **v8real(kind=4)**类型的拷贝符号和指数运算

参数说明

- 1、 **v8real(kind=8)**类型的拷贝符号和指数运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、 **v8real(kind=4)**类型的拷贝符号和指数运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

- 1、 **v8real(kind=8) vc**

- 2、 **v8real(kind=4) vc**

功能说明

浮点向量拷贝符号和指数运算，对**va**的每个浮点向量元素分别取其符号位和指数，与向量**vb**对应浮点向量元素的尾数拼接起来组成新的浮点向量元素写入**vc**。

函数名

simd_vcpysnd —— v8real(kind=8)类型的拷贝符号反码运算

simd_vcpysns ——v8real(kind=4)类型的拷贝符号反码运算

参数说明

- 1、 v8real(kind=8)类型的拷贝符号反码运算

v8real(kind=8) va;

v8real(kind=8) vb;

- 2、 v8real(kind=4)类型的拷贝符号反码运算

v8real(kind=4) va;

v8real(kind=4) vb;

返回值

- 1、 v8real(kind=8) vc

- 2、 v8real(kind=4) vc

功能说明

浮点向量拷贝符号反码运算，对va的每个浮点向量元素分别取其符号位的反码作为符号位，与向量vb对应浮点向量元素的指数和尾数拼接起来组成新的浮点向量元素写入vc。

函数名

simd_vfseleqd —— v8real(kind=8)类型的等于 0 条件选择运算

simd_vfseleqs ——v8real(kind=4)类型的等于 0 条件选择运算

参数说明

- 1、 v8real(kind=8)类型的等于 0 条件选择运算

v8real(kind=8) va;

v8real(kind=8) vb;

v8real(kind=8) vc;

- 2、 v8real(kind=4)类型的等于 0 条件选择运算

v8real(kind=4) va;

v8real(kind=4) vb;

v8real(kind=4) vc;

返回值

- 1、 v8real(kind=8)

- 2、 v8real(kind=4)

功能说明

浮点向量等于 0 条件选择运算，对 va 的每个浮点向量元素分别进行等于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vfselltd —— v8real(kind=8)类型的小于 0 条件选择运算

simd_vfsellts ——v8real(kind=4)类型的小于 0 条件选择运算

参数说明

- 1、v8real(kind=8)类型的小于 0 条件选择运算

v8real(kind=8) va;

v8real(kind=8) vb;

v8real(kind=8) vc;

- 2、v8real(kind=4)类型的小于 0 条件选择运算

v8real(kind=4) va;

v8real(kind=4) vb;

v8real(kind=4) vc;

返回值

- 1、v8real(kind=8)

- 2、v8real(kind=4)

功能说明

浮点向量小于 0 条件选择运算，对 va 的每个浮点向量元素分别进行小于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vfselled —— v8real(kind=8)类型的小于等于 0 条件选择运算

simd_vfselles ——v8real(kind=4)类型的小于等于 0 条件选择运算

参数说明

- 1、v8real(kind=8)类型的小于等于 0 条件选择运算

v8real(kind=8) va;

v8real(kind=8) vb;

v8real(kind=8) vc;

- 2、v8real(kind=4)类型的小于等于 0 条件选择运算

v8real(kind=4) va;

v8real(kind=4) vb;

v8real(kind=4) vc;

返回值

- 1、v8real(kind=8)

- 2、v8real(kind=4)

功能说明

浮点向量小于等于 0 条件选择运算，对 va 的每个浮点向量元素分别进行小

于等于 0 测试，若条件成立，则将向量 vb 对应浮点向量元素写入 vd，否则将向量 vc 对应浮点向量元素数据写入 vd。

函数名

simd_vfrecs —— v8real(kind=4)类型的近似倒数运算

simd_vfrecd ——v8real(kind=8)类型的近似倒数运算

参数说明

- 1、v8real(kind=4)类型的近似倒数运算

v8real(kind=4) va;

- 2、v8real(kind=8)类型的近似倒数运算

v8real(kind=8) va;

返回值

- 1、v8real(kind=4)

- 2、v8real(kind=8)

功能说明

浮点向量求近似倒数运算，对 va 的每个浮点向量元素分别求近似倒数。

函数名

simd_vfrecps —— v8real(kind=4)类型的精确倒数运算

simd_vfrecpd ——v8real(kind=8)类型的精确倒数运算

参数说明

- 1、v8real(kind=4)类型的精确倒数运算

v8real(kind=4) va;

- 2、v8real(kind=8)类型的精确倒数运算

v8real(kind=8) va;

返回值

- 1、v8real(kind=4)

- 2、v8real(kind=8)

功能说明

浮点向量求精确倒数运算，对 va 的每个浮点向量元素分别求精确倒数。

函数名

simd_vsqrts —— v8real(kind=4)类型的平方根运算

simd_vsqrtd ——v8real(kind=8)类型的平方根运算

参数说明

- 1、v8real(kind=4)类型的平方根运算

v8real(kind=4) va;

- 2、v8real(kind=8)类型的平方根运算

`v8real(kind=8) va;`

返回值

- 1、 `v8real(kind=4)`
- 2、 `v8real(kind=8)`

功能说明

浮点向量求平方根运算，对 `va` 的每个浮点向量元素分别求平方根。

函数名

`simd_vrsqrts` —— `v8real(kind=4)`类型的平方根倒数运算

`simd_vrsqrtd` —— `v8real(kind=8)`类型的平方根倒数运算

参数说明

- 1、 `v8real(kind=4)`类型的平方根倒数运算
`v8real(kind=4) va;`
- 2、 `v8real(kind=8)`类型的平方根倒数运算
`v8real(kind=8) va;`

返回值

- 1、 `v8real(kind=4)`
- 2、 `v8real(kind=8)`

功能说明

浮点向量求平方根倒数运算，对 `va` 的每个浮点向量元素分别求平方根倒数。

函数名

`simd_vaddh` —— `v32real(kind=2)`的加法运算

参数说明

`v32real(kind=2) va;`
`v32real(kind=2) vb;`

返回值

`v32real(kind=2)`

功能说明

`v32real(kind=2)`类型的加法运算，将 `va` 中的 32 个 `real(kind=2)`浮点数分别与 `vb` 中的 32 个 `real(kind=2)`浮点数进行加法运算。可以用“+”符号替换。

函数名

`simd_vsubh` —— `v32real(kind=2)`的减法运算

参数说明

`v32real(kind=2) va;`
`v32real(kind=2) vb;`

返回值

v32real(kind=2)

功能说明

v32real(kind=2)类型的减法运算，将 va 中的 32 个 real(kind=2)浮点数分别与 vb 中的 32 个 real(kind=2)浮点数进行减法运算。可以用“-”符号替换。

函数名

simd_vmulh —— v32real(kind=2)的乘法运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2)

功能说明

v32real(kind=2)类型的乘法运算，将 va 中的 32 个 real(kind=2)浮点数分别与 vb 中的 32 个 real(kind=2)浮点数进行乘法运算。可以用“*”符号替换。

函数名

simd_vdivh —— v32real(kind=2)的除法

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

除法运算，将 va 中的 32 个 real(kind=2)浮点数分别与 vb 中的 32 个 real(kind=2)浮点数进行除法运算。可以用“/”符号替换。

函数名

simd_vsqrth —— v32real(kind=2)的平方根

参数说明

v32real(kind=2) va;

返回值

v32real(kind=2) vc

功能说明

求平方根运算，将 va 中的 32 个 real(kind=2)浮点数分别求平方根。

函数名

simd_vmah —— v32real(kind=2)的乘加运算

参数说明

```
v32real(kind=2) va;  
v32real(kind=2) vb;  
v32real(kind=2) vc;
```

返回值

```
v32real(kind=2)
```

功能说明

v32real(kind=2)类型的乘加运算，将 va 中的 32 个 real(kind=2)浮点数和 vb 中的 32 个 real(kind=2)浮点数以及 vc 中的 32 个 real(kind=2)浮点数分别进行乘加运算。可以用“*”和“+”符号替换。

```
ret = va*vb + vc;
```

函数名

simd_vmsh —— v32real(kind=2)的乘减运算

参数说明

```
v32real(kind=2) va;  
v32real(kind=2) vb;  
v32real(kind=2) vc;
```

返回值

```
v32real(kind=2)
```

功能说明

v32real(kind=2)类型的乘减运算，将 va 中的 32 个 real(kind=2)浮点数和 vb 中的 32 个 real(kind=2)浮点数以及 vc 中的 32 个 real(kind=2)浮点数分别进行乘减运算。可以用“*”和“-”符号替换。

```
ret=va*vb-vc;
```

函数名

simd_vnmah —— v32real(kind=2)的负乘加运算

参数说明

```
v32real(kind=2) va;  
v32real(kind=2) vb;  
v32real(kind=2) vc;
```

返回值

```
v32real(kind=2)
```

功能说明

v32real(kind=2)类型的负乘加运算，将 va 中的 32 个 real(kind=2)浮点数和 vb 中的 32 个 real(kind=2)浮点数以及 vc 中的 32 个 real(kind=2)浮点数分别进行负乘加运算。可以用“*”和“+”和“-”符号替换。

ret = -va*vb + vc;

函数名

simd_vnmsh —— v32real(kind=2)的负乘减运算

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;
v32real(kind=2) vc;

返回值

v32real(kind=2)

功能说明

v32real(kind=2)类型的负乘减运算，将 va 中的 32 个 real(kind=2)浮点数和 vb 中的 32 个 real(kind=2)浮点数以及 vc 中的 32 个 real(kind=2)浮点数分别进行负乘减运算。可以用“*”和“+”和“-”符号替换。

ret = -va*vb - vc;

函数名

simd_vseleqh —— v32real(kind=2)的等于选择运算

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;
v32real(kind=2) vc;

返回值

v32real(kind=2)

功能说明

对 v32real(kind=2)类型的参数 va 中的对应浮点向量元素进行比较，如果等于 0，则返回 vb 中对应浮点向量的值；否则返回 vc 中对应浮点向量的值。

函数名

simd_vsellth —— v32real(kind=2)的小于选择运算

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;
v32real(kind=2) vc;

返回值

v32real(kind=2)

功能说明

对 v32real(kind=2)类型的参数 va 中的对应浮点向量元素进行比较，如果小

于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。

函数名

simd_vselleh —— v32real(kind=2)的小于等于选择运算

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;
v32real(kind=2) vc;

返回值

v32real(kind=2)

功能说明

对 v32real(kind=2)类型的参数 **va** 中的对应浮点向量元素进行比较，如果小于等于 0，则返回 **vb** 中对应浮点向量的值；否则返回 **vc** 中对应浮点向量的值。

函数名

simd_vcpysh —— v32real(kind=2)的拷贝符号

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;

返回值

v32real(kind=2)

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 **va** 中对应浮点向量的符号位，返回值中每个浮点向量的指数位和尾数位是 **vb** 中对应浮点向量的指数位和尾数位。

函数名

simd_vcpyseh —— v32real(kind=2)的拷贝符号和指数

参数说明

v32real(kind=2) va;
v32real(kind=2) vb;

返回值

v32real(kind=2)

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位和指数位是 **va** 中对应浮点向量的符号位和指数位，返回值中每个浮点向量的尾数位是 **vb** 中对应浮点向量的尾数位。

函数名

simd_vcpysnh—— v32real(kind=2)的拷贝符号反码

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2)

功能说明

浮点拷贝符号，返回值中每个浮点向量的符号位是 va 中对应浮点向量的符号位的反码，返回值中每个浮点向量的指数位和尾数位是 vb 中对应浮点向量的指数位和尾数位。

函数名

simd_vfcmpeqh —— v32real(kind=2)的等于比较运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_vfcmpneh —— v32real(kind=2)的不等比较运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行不等比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_vfcmpleh —— v32real(kind=2)的小于等于比较运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于等于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_vfcmplth —— v32real(kind=2)的小于比较运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行小于比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_vfcmpunh —— v32real(kind=2)的无序比较运算

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

将 va 和 vb 中的对应浮点向量元素进行无序比较，如果条件成立，则将非“0”浮点值（2.0）写入 vc 的对应位置，否则将真“0”写入 vc 的对应位置。

函数名

simd_smaxh ——v32real(kind=2)的大值比较选择

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

返回值

v32real(kind=2) vc

功能说明

对 va 与 vb 中每个浮点元素进行大于比较测试，若条件成立，则将向量 va

对应的浮点元素写入 `vc` 的对应位置，否则将向量 `vb` 对应的浮点元素写入 `vc` 的对应位置。

函数名

`simd_sminh` —— `v32real(kind=2)` 的小值比较选择

参数说明

`v32real(kind=2) va;`

`v32real(kind=2) vb;`

返回值

`v32real(kind=2) vc`

功能说明

对 `va` 与 `vb` 中每个浮点元素进行小于比较测试，若条件成立，则将向量 `va` 对应的浮点元素写入 `vc` 的对应位置，否则将向量 `vb` 对应的浮点元素写入 `vc` 的对应位置。

函数名

`simd_vfcvtsh` —— `v8real(kind=4)` 向量到 `v32real(kind=2)` 向量的转换

参数说明

`v8real(kind=4) va;`

`lit2 b;`

返回值

`v32real(kind=2) vc`

功能说明

`va` 中的 8 个 64 位单精度扩展格式浮点数，转换成 8 个 16 位半精度格式浮点数，然后根据立即数 `b` 的值写入 `vc` 指定位置，并将 `vc` 其它位置清 0。

例如： `v8real(kind=4) :: va`

`v32real(kind=2) :: vc`

`real(kind=4)::a(8)`

`DATA a /1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0/`

`call simd_load(va,a(1))`

`vc = simd_vfcvtsh(va, 1)`

`vc` 的结果为： `[0.0, 0.0, 8.0, 0.0, 0.0, 0.0, 7.0, 0.0, 0.0, 0.0, 6.0, 0.0, 0.0, 0.0, 5.0, 0.0, 0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 3.0, 0.0, 0.0, 0.0, 2.0, 0.0, 0.0, 0.0, 1.0, 0.0]`

函数名

`simd_vfcvths` —— `v32real(kind=2)` 向量到 `v8real(kind=4)` 向量的转换

参数说明

`v32real(kind=2) va;`

`lit2 b;`

返回值

v8real(kind=4) vc

功能说明

根据立即数#b 的值，从 va 的每个 64 位中选择 1 个 16 位半精度浮点数分量，转换成 64 位的单精度浮点数扩展格式，将转换后的 8 个 64 位 SP 浮点数写入 vc。

例如： v32real(kind=2) :: va
v8real(kind=4) :: vc
real(kind=2)::a(32)
DATA a /0.0,1.0,2.0,3.0,4.0,5.0,6.0,7.0,
8.0,9.0,10.0,11.0,12.0,13.0,14.0,15.0,
16.0,17.0,18.0,19.0,20.0,21.0,22.0,23.0,
24.0,25.0,26.0,27.0,28.0,29.0,30.0,31.0/
vc = simd_vfcvths(va, 1);
vc 的结果为： [29.0, 25.0, 21.0, 17.0, 13.0, 9.0, 5.0, 1.0]

C.4.3.3 数据整理函数接口

表 37 Fortran 从核数据整理函数接口

内部函数	操作	参数说明			
		VRc（返回值）	VRav	VRbv	VRcv
simd_vinsfs	向量元素插入	v8real(kind=4/8)	real(kind=4/8)	v8real(kind=4/8)	lit6
simd_vinsfd					
simd_vextfs	向量元素提取	real(kind=4/8)	v8real(kind=4/8)	lit6	—
simd_vextfd					
simd_vcpyfs	向量元素拷贝	v8real(kind=4/8)	real(kind=4/8)	—	integer(kind=8)
simd_vcpyfd					
simd_vcons	向量元素拼接		v8real(kind=4/8)	v8real(kind=4/8)	integer(kind=8)
simd_vcond					
simd_vshfs	向量元素混洗				integer(kind=8)
simd_vshd					
simd_vinsh	半精度向量插入	v32real(kind=2)	real(kind=2)	v32real(kind=2)	lit6
simd_vexth	半精度向量提取	real(kind=2)	v32real(kind=2)	lit6	/
simd_vcpyh	半精度向量拷贝	v32real(kind=2)	real(kind=2)	/	/
simd_vshfh	半精度向量混洗	v32real(kind=2)	v32real(kind=2)	v32real(kind=2)	integer(kind=8)

说明：

- 1、VRav、VRbv、VRcv 分别为第一、第二、第三个参数，VRc 为返回值；
- 2、“—”表示对应接口的该参数无效

函数名

simd_vinsfd —— v8real(kind=8)类型的向量元素插入操作

simd_vinsfs ——v8real(kind=4)类型的向量元素插入操作

参数说明

- 1、 v8real(kind=8)类型的向量元素插入操作

```
real(kind=8) a;  
v8real(kind=8) vb;  
lit6 c;
```

- 2、 v8real(kind=4)类型的向量元素插入操作

```
real(kind=4) a;  
v8real(kind=4) vb;  
lit6 c;
```

返回值

- 1、 v8real(kind=8) vd

- 2、 v8real(kind=4) vd

功能说明

浮点向量元素插入操作，将浮点变量 a 的 64 位浮点数据替代 vb 中由 c（有效值为 0~7，只有最低三位有效）指定的 64 位浮点元素，组成并返回新的浮点向量。比如：

```
vd=simd_vinsfd(a, vb, 2)
```

表示 a 的 64 位浮点数据替代 vb 的[191:128]这 64 位的浮点元素，组成新的 vb 返回给 vd。

函数名

simd_vextfd —— v8real(kind=8)类型的向量元素提取操作

simd_vextfs ——v8real(kind=4)类型的向量元素提取操作

参数说明

- 1、 v8real(kind=8)类型的向量元素提取操作

```
v8real(kind=8) va;  
lit6 b
```

- 2、 v8real(kind=4)类型的向量元素提取操作

```
v8real(kind=4) va;  
lit6 b
```

返回值

- 1、 real(kind=8) c

- 2、 real(kind=4) c

功能说明

浮点向量元素提取操作，将向量 va 中由 b（有效值为 0~7，只有最低三位

有效) 指定的 64 位浮点元素提取出来, 返回给 c。比如:

```
c = simd_vextfd(va, 1)
```

表示将 va 中[127:64]这 64 位浮点元素赋值给 c 作为返回结果。

函数名

simd_vcopyfd —— v8real(kind=8)类型的向量元素拷贝操作

simd_vcopyfs ——v8real(kind=4)类型的向量元素拷贝操作

参数说明

- 1、v8real(kind=8)类型的向量元素拷贝操作

```
real(kind=8) a;
```

- 2、v8real(kind=4)类型的向量元素拷贝操作

```
real(kind=4) a;
```

返回值

- 1、v8real(kind=8) vb

- 2、v8real(kind=4) vb

功能说明

浮点向量元素拷贝操作, 将 a 中浮点数据, 复制成 8 个相同元素, 组成新的浮点向量返回给 vb。

函数名

simd_vcond —— v8real(kind=8)类型的向量元素拼接操作

simd_vcons ——v8real(kind=4)类型的向量元素拼接操作

参数说明

- 1、v8real(kind=8)类型的向量元素拼接操作

```
v8real(kind=8) va;
```

```
v8real(kind=8) vb;
```

```
integer(kind=8) addr;
```

- 2、v8real(kind=4)类型的向量元素拼接操作

```
v8real(kind=4) va;
```

```
v8real(kind=4) vb;
```

```
integer(kind=8) addr;
```

返回值

- 1、v8real(kind=8) vd

- 2、v8real(kind=4) vd

功能说明

浮点向量元素拼接操作, 以 64 位分量为单位, va 存放需拼接的浮点向量低段, vb 存放需拼接的浮点向量高段, addr 存放不对界访存地址低 5 位 (忽略其低 2 位)。根据不对界访存地址, 将 vb 的低段数据放入目标向量的高段, 将 va

的高段数据放入目标向量的低段，拼接后形成新的浮点向量，返回给 `vd`。

函数名

`simd_vshfd`——`v8real(kind=8)`向量全混洗

`simd_vshfs`——`v8real(kind=4)`向量全混洗

参数说明

- 1、`v8real(kind=8)`类型的向量混洗

`v8real(kind=8) va;`

`v8real(kind=8) vb;`

`integer(kind=8) vc(8);`

- 2、`v8real(kind=4)`类型的向量混洗

`v8real(kind=4) va;`

`v8real(kind=4) vb;`

`integer(kind=8) vc(8);`

返回值

- 1、`v8real(kind=8) vd`

- 2、`v8real(kind=4) vd`

功能说明

操作数寄存器为 `va`、`vb`、`vc` 和 `vd`。硬件没有浮点向量混洗指令，所以此时我们不能简单的将向量 `va`、`vb` 和 `vd` 看成 8 个 64 位浮点标量组成的向量，而需要将其看成由 16 个 32 位标量组成的向量。`vd` 的 16 个 32bit 的数由 `va` 和 `vb` 装填，具体如何装填由 `vc` 说明。数组 `vc` 会首先被向量加载指令加载到一个 512 位的向量寄存器中，然后：

`vc` 的<79:75>指出 `vd` 的<511:480>来自 `va` 或 `vb` 的位置

`vc` 的<74:70>指出 `vd` 的<479:448>来自 `va` 或 `vb` 的位置

.....

`vc` 的<9:5>指出 `vd` 的<63:32>来自 `va` 或 `vb` 的位置

`vc` 的<4:0>指出 `vd` 的<31:0>来自 `va` 或 `vb` 的位置

`vc` 的[79:0]位，拆分成 16 个 5bit 位置分量。每个 5bit 位置分量的最高位为 0 时，`vd` 相应的数来自 `va`；为 1 时，`vd` 相应的数来自 `vb`。5bit 位置的低 4bit（0~15）含义为：位置为 0 表示第 0 个 32bit（[31:0]），依次类推。

`vc` 的[511:80]位无意义。

附加说明

由于单双精度浮点向量混洗接口借助字向量混洗指令实现，混洗的时候是以字（32 位）为单位进行，而非浮点数据在寄存器中占用的位数（64 位），故使用时若以浮点向量（8 个分量元素）为视角，则需要 `vc` 以每 10bit 为一组：

vc 的<4:0>决定浮点向量第一个元素来自 va 或 vb 的位置（此时需将 va 或 vb 当成 16 个 32 位数来看待，下同），vc 的<9:5>的值为 vc 的<4:0>的值加 1；

vc 的<14:10>决定向量第二个元素来自 va 或 vb 的位置，vc 的<19:15>的值为 vc 的<14:10>的值加 1；

.....

vc 的<74:70>决定向量第八个元素来自 va 或 vb 的位置，vc 的<79:75>的值为 vc 的<74:70>的值加 1。

使用时须对 vc 取地址，如：

```
vd = simd_vshfs(va,vb,loc(vc(1)))
```

数组 vc 的首地址需要满足 64 字节对界。用户使用时，需要根据希望实现的混洗方式，对数组 vc 的最低 2 个元素赋以适当的值。

例子

请参考 B.5.3.4 中 c 语言的例子。如果需要间隔取 va 和 vb 的数：

```
program main
v8real(kind=4) :: va=(0,1,2,3,4,5,6,7)
v8real(kind=4) :: vb=(999,999,999,999,999,999,999,999)
v8real(kind=4) :: vd

integer(kind=8),volatile :: vc(8)

vc=[-4203204006882735568,31675,0,0,0,0,0,0]

vd = simd_vshfs(va,vb,loc(vc(1)))
write(*,*) 'vd = ', vd
end
```

输出为：

```
vd=[7.000000000,999.0000000,5.000000000,999.0000000,3.000000000,999.0000000,1.000000000,999.0000000]
```

函数名

simd_vinsh—— 半精度浮点向量元素插入运算

参数说明

```
real(kind=2) a;
v32real(kind=2) vb;
lit6 c;
```

返回值

```
v32real(kind=2)
```

功能说明

将16位半精度浮点数a替代vb中由c（有效值为0～31）指定的16位半精度浮点元素，组成并返回新的半精度浮点数向量。

请参考 B.5.3.4 中 c 语言的例子。

函数名

simd_vexth——半精度浮点向量元素提取运算

参数说明

v32real(kind=2) va;

lit6 b;

返回值

real(kind=2) a

功能说明

将va中由b（有效值为0～31）指定的16位半精度浮点元素，变为16位寄存器格式的半精度浮点数写入目的寄存器a。

函数名

simd_vcpyh——半精度浮点向量元素拷贝运算

参数说明

real(kind=2) a;

返回值

v32real(kind=2)

功能说明

将16位半精度浮点数a复制成32个相同元素组成新的半精度浮点数向量写入目的寄存器。

函数名

simd_vshfh——半精度向量全混洗

参数说明

v32real(kind=2) va;

v32real(kind=2) vb;

integer(kind=8) vc(8);

返回值

v32real(kind=2) vd

功能说明

操作数寄存器为 va、vb、vc 和 vd。vd 的 32 个 16bit 的数由 va 和 vb 装填，具体如何装填由 vc 说明。数组 vc 会首先被向量加载指令加载到一个 512 位的向量寄存器中，然后：

vc 的<191:186>指出 vd 的<511:496>来自 va 或 vb 的位置

vc 的<185:180>指出 vd 的<495:480>来自 va 或 vb 的位置

.....

vc 的<11:6>指出 vd 的<31:16>来自 va 或 vb 的位置

vc 的<5:0>指出 vd 的<15:0>来自 va 或 vb 的位置

vc 的[191:0]位，拆分成 32 个 6bit 位置分量。每个 6bit 位置分量的最高位为 0 时，vd 相应的数来自 va；为 1 时，vd 相应的数来自 vb。6bit 位置的低 5bit (0~31) 含义为：位置为 0 表示第 0 个 16bit ([15:0])，依次类推。

vc 的[511:192]位无意义。

附加说明

使用时须对 vc 取地址，如：

vd = simd_vshfh(va,vb,loc(vc(1)))

数组 vc 的首地址需要满足 64 字节对界。用户使用时，需要根据希望实现的混洗方式，对数组 vc 的最低 3 个元素赋以适当的值。

例子：

请参考 B.5.3.4 中 c 语言的例子。如果需要取 vb 的前十个数，其他取 va：

```
program main
integer :: i
v32real(kind=2) :: va
v32real(kind=2) :: vb
v32real(kind=2) :: vd
real(kind=2) :: a(32)
real(kind=2) :: b(32)
real(kind=2) :: d(32)
!(kind=2) can't print, so trans them to (kind=4)
real(kind=4) :: p(32)
integer(kind=8),volatile :: vc(8)
vc=[750023513452980320,0,0,0,0,0,0,0]

do i=1, 32
    a(i)=0
    b(i)=i
enddo
call simd_load(va,a(1))
call simd_load(vb,b(1))

vd = simd_vshfh(va,vb,loc(vc(1)))

call simd_store(vd, d(1))
```

```
do i=1, 32
    p(i)=d(i)
enddo
do i=1, 32
    write(*,*)p(i)
enddo
end
```

注意 fortran 中半精度打印需转换成单精度打印

附录 D: Fortran 语言支持的运行时 athread 接口

函数名	功能
int athread_init()	初始化 athread 接口功能
int athread_init_all()	
int athread_init_cgs()	
int athread_spawn(start_routine *func, void *arg)	创建/等待从核线程组
int athread_spawn_noflush (start_routine *func, void *arg)	
int athread_join ()	
int athread_spawn_cgmask (int cgmask, start_routine *func, void *arg)	按核组位图创建/等待从核线程组
int athread_spawn_cgmask_noflush (int cgmask, start_routine *func, void *arg)	
int athread_join_cgmask (int cgmask)	
int athread_spawn_all (void* func , void * arg)	创建/等待大共享模式从核线程组
int athread_spawn_all_noflush (void* func , void * arg)	
int athread_join_all ()	
int athread_spawn_cgs (void* func , void * arg)	
int athread_spawn_cgs_noflush (void* func , void * arg)	
int athread_join_cgs ()	
int SET_TO_SWCG (int cpu_mask)	绑定主核线程到指定核组
char athread_get_tid (void)	获取核组内从核号
char athread_get_rid (void)	获取核组内从核行号
char athread_get_cid (void)	获取核组内从核列号
char athread_get_cgn (void)	从核获取当前核组号
void athread_sync_node (void)	同步
void athread_sync_master_spe (int tid)	
void athread_sync_master_array (void)	
void athread_ssync_array (void)	
void athread_ssync_sldm (void)	
void athread_ssync_master_spe (void)	
void athread_ssync_master_array (void)	
athread_ssync_node (void)	
int athread_mutex_lock_node (void)	锁
int athread_mutex_trylock_node (void)	
int athread_mutex_unlock_node (void)	
int athread_mutex_lock (volatile int64_t *lock)	
int athread_mutex_trylock (volatile int64_t *lock)	
int athread_mutex_unlock (volatile int64_t *lock)	
int athread_lock (long * lock_var)	

int athread_unlock (long * lock_var)	
int athread_dma_get (void *dst, void *src, int len)	DMA
int athread_dma_get_stride (void *dst, void *src, int len, int bsize, int stride)	
int athread_get(dma_mode mode, void *src, void *dest, int len, void *reply, char mask,int stride, int bsize)	
int athread_dma_iget (void *dst, void *src, int len, athread_rply_t *rply)	
int athread_dma_iget_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply)	
int athread_dma_wait_value (athread_rply_t *rply, int value)	
int athread_dma_put (void *dst, void *src, int len)	
int athread_dma_put_stride (void *dst, void *src,int len,int bsize, int stride)	
int athread_dma_iput (void *dst, void *src, int len,athread_rply_t *rply)	
int athread_dma_iput_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply)	
int athread_dma_ibcast (void *dst, void *src, int len, athread_rply_t *rply)	
int athread_dma_ibcast_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply)	
int athread_dma_bcast_coll (void *dst, void *src, int len)	
int athread_dma_bcast_stride_coll (void *dst, void *src,int len, int bsize, int stride)	
int athread_dma_row_ibcast (void *dst, void *src, int len, athread_rply_t *rply)	
int athread_dma_row_ibcast_stride (void *dst, void *src, int len, int bsize, int stride, athread_rply_t *rply)	
int athread_dma_row_bcast_coll (void *dst, void *src, int len)	
int athread_dma_row_bcast_stride_coll (void *dst, void *src, int len, int bsize, int stride)	
int athread_dma_row_bcast_coll (void *dst, void *src, int len)	
int athread_dma_row_bcast_stride_coll (void *dst, void *src, int len, int bsize, int stride)	
int athread_dma_col_ibcast (void *dst, void *src, int len, athread_rply_t *rply)	
int athread_dma_col_ibcast_stride (void *dst, void *src, int len,int bsize, int stride, athread_rply_t *rply)	
int athread_dma_col_bcast_coll (void *dst, void *src, int len)	
int athread_dma_col_bcast_stride_coll (void *dst, void *src, int len, int bsize, int stride)	
int athread_memcpy_sldm (void *dst, void *src, int len, int direction)	
int athread_dma_barrier (void)	
int athread_dma_all_barrier (void)	

int athread_rma_get (void *l_addr, int len, int r_tid,void *r_addr, athread_rply_t *r_rply)	RMA
int athread_rma_iget (void *l_addr, athread_rply_t *l_rply, int len,int r_tid, void *r_addr, athread_rply_t *r_rply)	
int athread_rma_wait_value (athread_rply_t *rply, int value)	
int athread_rma_put (void *l_addr, int len,int r_tid, void *r_addr, athread_rply_t *r_rply)	
int athread_rma_iput (void *l_addr, athread_rply_t *l_rply, int len,int r_tid, void *r_addr, athread_rply_t *r_rply)	
int athread_rma_bcast (void *dst, void *src, int len,athread_rply_t *r_rply)	
int athread_rma_ibcast (void *dst, void *src, athread_rply_t *l_rply,int len, athread_rply_t *r_rply)	
int athread_rma_bcast_coll (void *dst, void *src, int len,int root)	
int athread_rma_row_bcast (void *dst, void *src, int len, athread_rply_t *r_rply)	
int athread_rma_row_ibcast (void *dst, void *src, int len,athread_rply_t *l_rply, athread_rply_t *r_rply)	
int athread_rma_row_bcast_coll (void *dst, void *src, int len, int root)	
int athread_rma_row_mcast (void *dst, void *src, int len,unsigned char mask, athread_rply_t *r_rply)	
int athread_rma_row_imcast (void *dst, void *src, int len,athread_rply_t *l_rply, unsigned char mask, athread_rply_t *r_rply)	
int athread_rma_col_bcast (void *dst, void *src, int len, athread_rply_t *r_rply)	
int athread_rma_col_ibcast (void *dst, void *src, int len,athread_rply_t *l_rply, athread_rply_t *r_rply)	
int athread_rma_col_bcast_coll (void *dst, void *src, int len, int root)	
int athread_rma_col_mcast (void *dst, void *src, int len,unsigned char mask, athread_rply_t *r_rply)	
int athread_rma_col_imcast (void *dst, void *src, int len,athread_rply_t *l_rply, unsigned char mask, athread_rply_t *r_rply)	
int athread_sldm_get (void *dst, void *src, int len)	
int athread_rma_barrier (void)	
int athread_rma_all_barrier (void)	
unsigned long athread_time_cycle (void)	计时
unsigned long athread_stime_cycle (void)	
int athread_alltoall(void *src_addr, void *dest_addr, int units_size)	集合通信
int athread_alltoall_inplace(void *src_addr, int units_size, void *buf, int buf_item)	
int athread_redurt(void *src_addr, void *dest_addr, int units, int dtype, int optype, void *buf, int buf_item)	

修订记录

时间	版本	修订内容	修改人
2021.01.05	0.1	完成基础版本章节架构的设计，完成第一章与第二章内容	吴伟
2021.01.07	0.11	完成 3.1 节、3.2 节内容	钱宏
2021.01.09	0.12	完成 3.3 节内容	杨涛
2021.01.11	0.13	完成 3.4 节内容	管茂林
2021.01.13	0.14	完成 3.5 节与第四章内容	朱琪
2021.01.14	0.15	统一文本格式，增加图表目录	樊行健，
2021.01.16	0.2	完善架构设计，调整部分章节	吴伟
2021.01.18	0.21	完善部分描述	钱宏
2021.01.19	0.3	完成附录 A 内容	朱琪
2021.01.20	0.31	完成附录 B、附录 C 内容	管茂林
2021.01.21	0.32	完成附录 D 内容	杨涛
2021.01.21	0.33	统一文本格式	樊行健
2021.01.21	0.34	添加 4.14 节内容	朱琪
2021.01.22	0.4	完善整体内容，2.3.1 节添加 LDM 连续共享描述	吴伟
2021.01.26	0.41	修改 3.3.2 节函数说明	杨涛
2021.01.27	0.42	修改 4.14 节，增加使用限制的说明；删除文档的批注部分；完善部分章节格式	吴伟
2021.02.02	0.43	完善 3.3.15.8 节，增加从核 cache 私有段置无效的说明	管茂林
2021.02.02	0.44	修改 3.3.12.1 节 bsub -crossmalloc 说明	钱宏
2021.02.03	0.45	修改 3.3.6.3 节及附录 D 函数名	杨涛
2021.02.24	0.46	3.3.8 节添加说明，3.3.12 增加可自定义对界的 libc_set_aligned_malloc	杨涛
2021.03.02	0.47	修改 3.3.15.10 从核指针设成可 cache 访问和示例	钱宏
2021.03.02	0.48	3.3.7 节新增节点锁 和示例	杨涛
2021.03.04	0.49	3.3.7.3 修改函数说明	杨涛
2021.03.05	0.50	增加 intv16 的乘法和除法，对应编译器 1227	杨涛
2021.03.11	0.51	增加 2.3.2	杨涛
2021.03.17	0.52	3.3.12.1 增加_sw_xfree 的描述	杨涛
2021.03.19	0.53	修改 2.3.2 交叉段默认为 0，对应 swrun-106	杨涛
2021.03.26	0.54	增加 athread_put	杨涛

2021.03.31	0.55	修改 B2.1 示例 2 的 simd_vconw 参数	钱宏
2021.04.02	0.56	为向量混洗增加例子	樊行健
2021.04.01	0.57	修改 athread_put 接口参数描述错误	杨涛
2021.04.12	0.58	修改 athread_ssync 描述，增加示例	樊行健
2021.06.08	0.60	修改 3.3.15.11 关于指针可 cache、不可 cache 访问的示例	杨涛
2021.08.26	0.61	4.14 修改示例图	杨涛
2022.07.18	0.62	修改 B.4.2.1 例 6、例 7	杨涛